Sentinel

A Hands-On Preparation for the Fullstack .NET / Angular Skill Test — Building an Account-Security Portal with Clean Architecture, CQRS, Vertical Slices, EF Core and Angular

Benjamin Fadina

October 2026

Contents

About this book

A word about the test, and about honesty
Who this book is for
How to use this book
Getting the code

Part I · Orientation

Chapter 1 · Reading the Skill Test Like an Engineer

1.1 What the guide actually says
1.2 The marking scheme, scenario by scenario
1.3 Ten minutes in Qualified
1.4 A timetable for each scenario
1.5 What this chapter asks you to do

Chapter 2 · The Sentinel Domain

2.1 What Sentinel is
2.2 The actors
2.3 The vocabulary
2.4 The access rules
2.5 The compromise rules
2.6 The people in the seed data
2.7 What we are deliberately not building

Chapter 3 · The Architecture

3.1 The dependency rule
3.2 Why the domain has no dependencies
3.3 CQRS, and what it means here
3.4 Vertical slices inside the layers
3.5 SOLID, concretely
3.6 The dispatcher, and a note on MediatR
3.7 A note on repositories
3.8 What is not here, and when it would be

Part II · The Back End

Chapter 4 · Environment and Solution Setup

4.1 What to install
4.2 Creating the projects
4.3 Central build configuration
4.4 Central package management
4.5 The project files
4.6 The EF Core tool
4.7 Verifying the build

Chapter 5 · Domain Building Blocks

5.1 Errors are values, exceptions are for bugs
5.2 Result: success or failure, never both
5.3 Entities and aggregate roots
5.4 Value objects, briefly

Chapter 6 · Users, Roles and Permissions

6.1 Permission: a value object with one interesting method
6.2 Role
6.3 User
6.4 AccessSubject: flattening a user for the rules

Chapter 7 · The Page Access Evaluator

7.1 The process, before any code
7.2 The supporting types
7.3 The evaluator
7.4 The tests
7.5 The same problem in 40 minutes

Chapter 8 · Sign-ins and the Compromise Detector

8.1 Where a sign-in came from
8.2 The sign-in attempt
8.3 Findings, severity and the assessment
8.4 The detector
8.5 The tests
8.6 Doing it in the browser instead

Chapter 9 · The Application Layer

9.1 The messages
9.2 The pipeline
9.3 The dispatcher
9.4 Validation without a library
9.5 The behaviours
9.6 The ports
9.7 Registering everything

Chapter 10 · Feature Slices

10.1 A command slice: CreateUser
10.2 Commands that change an aggregate
10.3 The access slices
10.4 The security slices
10.5 The remaining slices

Chapter 11 · Infrastructure and Persistence

11.1 The DbContext
11.2 Configurations
11.3 Services
11.4 Seeding the stories
11.5 Registration and migrations

Chapter 12 · The API Surface

12.1 The composition root
12.2 Results become HTTP in one place
12.3 Development authentication
12.4 Guarding endpoints with permissions
12.5 The endpoint modules
12.6 Trying it by hand

Chapter 13 · Testing the Back End

13.1 Domain tests
13.2 Handlers against SQLite
13.3 The API in process

13.4 Architecture tests
13.5 What not to test

Part III · The Angular Front End

Chapter 14 · Angular and TypeScript for the Test
14.1 The ES2015+ you will actually use
14.2 TypeScript in one table
14.3 Angular in one page
14.4 RxJS: the six operators that cover most code
14.5 Using AI in the Angular scenario, properly

Chapter 15 · The Angular Workspace and the Core Layer
15.1 Creating the workspace
15.2 Talking to the API through a proxy
15.3 The models
15.4 Errors, once
15.5 Identity on every request
15.6 The session
15.7 Wiring it together
15.8 Global styles

Chapter 16 · The Shell, Navigation and the Route Guard
16.1 The shell
16.2 The user switcher: two-way binding by hand
16.3 The menu: a recursive component
16.4 The routes
16.5 The guard
16.6 Explaining a denial

Chapter 17 · Passing Data Between Components: the Account Security Page
17.1 The component tree
17.2 The presentational children
17.3 The container
17.4 Every way to pass information, and when to use each
17.5 The edge-case checklist for this page

Chapter 18 · Displaying and Formatting Dates
18.1 Three rules
18.2 The implementation
18.3 The pipes
18.4 The clock
18.5 Pitfalls checklist

Chapter 19 · The Dashboard and the Admin Screens
19.1 The dashboard: reusing a child with a different parent
19.2 The admin API service
19.3 Search as you type
19.4 The access explorer: the second scenario, made visible

Chapter 20 · Testing the Angular Application
20.1 Pure functions first
20.2 Presentational components: set inputs, read the DOM, catch outputs
20.3 The container against a fake back end
20.4 The guard and the interceptor
20.5 What to test in 25 minutes

Part IV · Debugging with Limited Information

Chapter 21 · A Method for Debugging with Limited Information

21.1 The five questions the reviewer is asking
21.2 The loop
21.3 Tools for gathering evidence in .NET
21.4 Research, and citing it
21.5 A 35-minute timetable
21.6 The templates

Chapter 22 · Lab 1 — SUP-4172: "My sign-in history shows the wrong times"

22.1 Setting up
22.2 Reading the ticket
22.3 Reproducing
22.4 Hypotheses
22.5 Finding the change
22.6 The regression tests, then the fix
22.7 And the London sign-in?
22.8 A follow-up the investigation uncovered
22.9 The write-ups

Chapter 23 · Lab 2 — SUP-4219: "Security page says all clear, but I got a lockout email"

23.1 Reading the ticket
23.2 Reproducing, and the first clue
23.3 Hypotheses
23.4 The cause
23.5 The regression test, then the fix
23.6 What made this bug possible
23.7 Follow-ups

Chapter 24 · Writing It Up for Engineers and for Everyone Else

24.1 Two audiences, two questions
24.2 Worked example: SUP-4172 for engineers
24.3 Worked example: SUP-4172 for the customer
24.4 Worked example: SUP-4219, both audiences
24.5 Habits that make this fast
24.6 The templates

Part V · Rehearsal

Chapter 25 · Katas and Timed Drills

25.1 The exercises
25.2 Kata 1: which pages may each user open?
25.3 Kata 2: summarise an access log
25.4 The Angular drill
25.5 The rubric

Chapter 26 · Test Day

26.1 The day before
26.2 Before each scenario
26.3 Inside the authorisation scenario (40 min)
26.4 Inside the debugging scenario (35 min)
26.5 Inside the Angular scenario (25 min)
26.6 Citing, in practice
26.7 After

Appendix A · C# Fundamentals for the Test

A.1 Iteration

A.2 Inspecting values

A.3 Classes and objects

A.4 Collections

A.5 Converting between types

A.6 Null handling

Appendix B · TypeScript and JavaScript Fundamentals for the Test

B.1 Iteration

B.2 Inspecting values

B.3 Classes and objects

B.4 Arrays, maps, objects as dictionaries

B.5 Converting between types

B.6 Equality and truthiness

Appendix C · The Same Code in Older Angular

Appendix D · Troubleshooting

Resetting everything

About this book

Visual Studio 2026 · .NET 10 (LTS) · C# 14 · ASP.NET Core minimal APIs · EF Core 10 · SQL Server · Angular 22 · TypeScript 6 · Vitest

Copyright © 2026 Benjamin Fadina. All rights reserved. Every source code listing is released under the MIT licence and may be used freely.

Microsoft, Visual Studio, .NET, Entity Framework Core and SQL Server are trademarks of the Microsoft group of companies. Angular is a trademark of Google LLC. Woven, Qualified and Andela are the names of their respective owners. No endorsement by any of them is implied, and nothing in this book was supplied by them.

First edition, October 2026.

A word about the test, and about honesty

This book prepares you for the **Fullstack .NET Engineer / Angular skill test** described in the Woven prep guide used by Andela. That guide lists three timed scenarios:

Scenario	Time	AI	Language
Debugging a problem with limited information	35 min	**not allowed**	C#
Check which pages users are authorized to access	40 min	**not allowed**	C#
Show a user when their account may be compromised	25 min	allowed (cite it)	Angular

The guide is explicit that the scenarios are confidential, that using someone else's solution is unacceptable, and that sharing them is not allowed. This book respects that completely. **It does not contain the test's scenarios, does not guess at them, and contains no "answers".** What it contains is a real application, Sentinel, whose features happen to sit in the same three areas, and a method for each area that you can rehearse until it is a habit. When you sit the test, every line you write will be your own.

Two of the three scenarios forbid AI assistance. That matters for how you use this book: type the code yourself, run the labs without a chatbot, and build the reflexes that the test will measure. The third scenario encourages AI and asks you to cite it; Chapter 26 shows how.

Who this book is for

A developer who can already write C# and has met Angular, and who wants to walk into a timed assessment rehearsed rather than hopeful. If you are newer to programming, Appendices A and B cover the fundamentals the prep guide lists (iteration, classes and objects, arrays, lists and dictionaries, converting between types) in the context of this project.

The application is built the way an enterprise system should be built: Clean Architecture, CQRS, vertical slices and SOLID. You do not need any of that to pass a 40-minute exercise. You do need what it teaches: small functions with clear names, decisions written down, edge cases handled on purpose, and

tests that prove it. The architecture is the long version of the same discipline the test rewards in miniature.

How to use this book

Every listing in this book is complete and is the file in the repository, inserted at build time; there are no `// ... rest of the class` elisions in production code. File paths are given as a header above each listing, relative to the repository root, like this:

src/Sentinel.Domain/Access/PermissionEffect.cs

```
namespace Sentinel.Domain.Access;

/// <summary>Whether a user-level override adds a permission or takes it away.</summary>
public enum PermissionEffect
{
    Allow,
    Deny
}
```

Type the code. Reading code produces the feeling of understanding; typing code produces understanding. If you clone the repository instead, at least pause at the end of each chapter and rebuild from memory the shape of what you just read.

The book has five parts:

- **Part I (Chapters 1–3)** reads the test like an engineer, introduces the Sentinel domain, and lays out the architecture.
- **Part II (Chapters 4–13)** builds the C# back end. Chapter 7 is the heart of the second scenario; Chapter 8 is the server side of the third.
- **Part III (Chapters 14–20)** builds the Angular front end. Chapters 17 and 18 are the heart of the third scenario: passing information between components, and displaying dates.
- **Part IV (Chapters 21–24)** is the first scenario: a method for debugging with limited information, two reproducible incidents to practise it on, and how to write up what you found for engineers and for people who are not.
- **Part V (Chapters 25–26)** is rehearsal: timed katas and drills, a self-review rubric built from the test's own evaluation criteria, and a test-day checklist.

If your test is in three days, read Chapters 1, 7, 17, 18, 21, 22 and 26, and do the katas in Chapter 25. If it is in three weeks, read everything, in order.

Getting the code

```
git clone <your copy of this repository> sentinel
cd sentinel
git branch -a        # main, two lab branches, one drill branch
```

The README at the root explains how to run everything. Chapter 4 does it step by step.

Part I · Orientation

Chapter 1 · Reading the Skill Test Like an Engineer

Most candidates read a prep guide once, skim the bold text, and open the first scenario. That is like reading the acceptance criteria of a ticket after you have written the code. This chapter reads the guide the way you would read a specification, and turns each sentence into a habit you can practise.

1.1 What the guide actually says

Strip the guide to its load-bearing sentences and you get this:

1. **Three scenarios, each with its own timer**, inside a 48-hour window. The timers are short (35, 40 and 25 minutes) and they are per scenario. You can rest between scenarios; you cannot rest inside one.
2. **Research is allowed and should be cited.** Looking things up is "not a negative"; using a snippet without a link is.
3. **AI is forbidden in two scenarios and encouraged in one.** Breaking the rule disqualifies you.
4. **The environment is Qualified**, an online IDE. The guide recommends ten minutes trying it first.
5. **Each scenario lists what reviewers look for.** Those lists are the marking scheme. Treat them as such.

The guide also says something about the coding scenario that is easy to skim past: if a more advanced concept is beyond you in the time, "use what you know and leave a comment for the reviewer". That advice tells you the reviewer is a person reading your code with empathy, not a test runner counting green ticks. A working simple solution with a clear comment beats an ambitious broken one, every time.

1.2 The marking scheme, scenario by scenario

Debugging a problem with limited information (35 minutes, C#, no AI)

You receive a bug report from customer support and "will not have all of the information you need". Reviewers look at:

Criterion	The habit that satisfies it	Where this book practises it
How do you gather information about a problem in code?	Reproduce first; read the code path end to end; use logs and the debugger before you form opinions	Chapter 21, both labs
How do you research issues?	Search precisely, read primary documentation, cite every source	§21.6, §22.5
How do you document your learning so others can benefit?	An investigation log written as you go	`docs/templates/investigation-log.md`
How do you communicate technical concepts to other engineers?	A blameless write-up: cause, fix, why it was missed, follow-ups	Chapter 24
How do you communicate technical concepts to non-engineers?	A reply that answers the reader's actual worry, first, in their words	Chapter 24

Notice that **three of the five criteria are about writing**. Candidates who find the bug in twenty minutes and spend the last fifteen on a two-line note leave most of the marks on the table. Chapter 21 gives you a timetable for the 35 minutes that protects the writing time.

Check which pages users are authorized to access (40 minutes, C#, no AI)

A coding scenario with provided tests. Reviewers look at:

Criterion	The habit
Does your code address the core requirements?	Restate the requirements as a checklist in a comment before coding
Did you handle edge cases?	List them *before* coding; decide each one; write a test per decision
Are your variable and function names clear?	Name by intent: `CanOpen(page)`, not `Check(p)`
Is the structure good? Functions right-sized? Anything hard-coded?	One public entry point, small private helpers, constants named
Did you add helpful tests beyond the ones provided?	Every edge-case decision gets a test
Do you leave helpful comments explaining trade-offs?	A short "Decisions" comment at the top of the class

The domain is authorisation: users, pages, and who may open what. Sentinel is built around exactly that problem, at production scale (Chapters 6 and 7), and Kata 1 in Chapter 25 is a timed exercise in the same shape.

Show a user when their account may be compromised (25 minutes, Angular, AI allowed)

Reviewers look at:

Criterion	The habit
Core requirements	Get something correct on screen in the first ten minutes
How do you pass information between components?	Inputs down, outputs up; a container that owns data and side effects
Edge cases	Empty, loading, error, unknown values, the boundary of every rule
How do you display and format dates?	UTC on the wire; parse strictly; format with `Intl` in the user's zone; relative time with an injectable "now"
Names, structure, comments	As above

Twenty-five minutes is very little. The guide says the scenario "does not require expert-level knowledge of Angular", and the environment is preconfigured. The candidates who do well are the ones who do not have to think about *how* to pass data to a child component or how to format a date in a zone, because they have done it twenty times. Chapters 17 and 18 exist so that you have.

1.3 Ten minutes in Qualified

Do what the guide says: open the Qualified practice environment before the real thing and spend ten minutes in it. Specifically, find out:

- how to run the tests, and how long a run takes;
- where console output appears (you will want `Console.WriteLine` or `console.log` when a test fails mysteriously);
- whether you can add a test file or only edit the provided ones;
- which language version and framework version it runs. **Look for the Angular version** in particular: if it predates Angular 17 you will be writing `*ngIf` and `@Input()` rather than `@if` and `input()`. Appendix C translates everything in this book into the older syntax.

1.4 A timetable for each scenario

Timers make people skip the parts that earn marks. Decide your timetable in advance and keep to it.

Debugging (35 min): 3 read the report and write down what you know and don't · 7 reproduce · 12 investigate and fix · 3 regression test · 10 write-ups (engineer + non-engineer). If you have not found the cause at minute 22, stop and write up what you have ruled out: a clear record of an unfinished investigation is worth more than a silent half-fix.

Authorisation (40 min): 5 read, restate requirements, list edge cases in a comment · 20 implement · 10 add tests for each edge case · 5 tidy names and write the "Decisions" comment.

Angular (25 min): 3 read and sketch the component tree · 10 get the data on screen, correctly · 7 edge cases (empty, unknown, date formatting) · 5 names, comments, AI citation.

1.5 What this chapter asks you to do

Nothing yet, except decide when you will sit the test and work backwards. The rest of the book is built so that by test day each of the habits above is something you have done, not something you have read about.

Chapter 2 · The Sentinel Domain

Before you can model a domain you must understand it. This chapter has almost no code. It is the chapter the others depend on, because every decision in the code is downstream of the vocabulary set here.

2.1 What Sentinel is

Sentinel is the internal portal of a mid-sized company. Employees sign in to it to reach reports, billing, the support desk and administration screens. It answers two questions, constantly:

1. **May this person open this page?** — the *access* half.
2. **Is this person really this person?** — the *account security* half.

Each half has a rule engine in the domain layer. `PageAccessEvaluator` answers the first (Chapter 7); `CompromiseDetector` answers the second (Chapter 8). Everything else in the system exists to feed those two engines data and to show their verdicts to people.

2.2 The actors

Actor	What they do in Sentinel
Employee	Signs in, uses the pages their roles allow, checks their own account security
Administrator	Manages users, roles and pages; investigates "why can't X see Y?"
Auditor	May look at users and every report; may change nothing
Support agent	Fields tickets from employees; forwards bugs to engineering
Identity provider	An external system that performs the actual sign-in and reports each attempt to Sentinel

The last actor is important and easily forgotten: Sentinel does not check passwords. It *records* sign-in attempts reported by an identity provider (Azure AD, Okta, Keycloak…) and analyses them. In development, you play the identity provider with the `.http` file or `curl`.

2.3 The vocabulary

Permission. A named capability, written as dot-separated lower-case segments: `reports.finance`, `admin.users.manage`. A *grant* may end in a wildcard: `reports.*` covers every permission beneath `reports`, and `*` covers everything. A page always *requires* a concrete permission, never a wildcard.

Role. A named bundle of permissions: *Finance Manager* grants `reports.view`, `reports.finance`, `reports.finance.*` and `billing.*`.

User. A person, with an email address, a display name, an IANA time zone such as `Africa/Lagos`, a status (*Active* or *Suspended*), a set of roles, and optional per-user **overrides**: a direct *Allow* or *Deny* of one permission.

Page. A screen in the portal, identified by its path (`/reports/finance`). Pages form a tree: every page except the roots has a parent. A page may require a permission, require only that you are signed in, or be **public**.

Sign-in attempt. One attempt to sign in, successful or failed, with its time (UTC), IP address, approximate location and device.

Finding. One reason to think an account may be compromised: a brute-force burst, impossible travel, a new country, a new device. Each has a **severity** (Low, Medium, High).

Assessment. The findings for one account and the worst of their severities, the **risk level**.

Acknowledgement. The user's "that was me" on a finding. An acknowledged finding no longer raises the risk level.

2.4 The access rules

These are the rules the business agreed. They are written as numbered statements because that is how you should write rules you are about to implement in a timed test, too.

1. A **public** page may be opened by anyone, signed in or not.
2. Nobody else may open anything unless signed in **and active**. A suspended user sees public pages only.
3. A page may be opened only if its **parent** may be opened. You cannot reach a room through a locked door.
4. A page with **no required permission** may be opened by any active user (subject to rule 3).
5. A page that requires a permission may be opened if any of the user's grants **covers** it (rule 7), and
6. **no deny covers it.** An explicit deny beats every allow, from any role.
7. A grant `x` covers `x`. A grant `x.*` covers anything that starts `x.`, at any depth, but not `x` itself. A grant `*` covers everything.
8. Bad data fails **closed**: a page whose parent does not exist, or whose ancestry loops back on itself, may not be opened by anyone.

Rule 8 is the one people forget, and the one a reviewer looks for. Data that "cannot happen" happens: an import goes wrong, a test builds a tree by hand, a future cache serves a stale parent. The access rules must never throw and must never let someone in by accident.

2.5 The compromise rules

Agreed with the security team, with thresholds that can be tuned:

Finding	Rule	Severity
Brute force	At least 5 failed attempts within any 15-minute window	Medium; **High** if a success follows within the window
Impossible travel	Two consecutive *successful* sign-ins more than 500 km apart, implying a speed above 900 km/h	High
New country	A successful sign-in from a country this user has never signed in from successfully	Medium
New device	A successful sign-in from a device this user has never used	Low

And the conditions around them:

- Findings are reported only for the last **30 days**, but the previous **90 days** count as "seen before".
- A user's **first ever** successful sign-in is never "new" anything; there is nothing to compare it with.
- A **failed** sign-in from a new country does not make that country "known".
- Attempts **after now** (clock skew upstream) are ignored.
- The risk level is the highest severity among **unacknowledged** findings. *Medium* and above are "compromise suspected": the UI interrupts the user.

2.6 The people in the seed data

Every developer database is seeded with eight people, each of whom tells one story. You will meet them throughout the book; this table is worth a bookmark.

User	Role	Zone	Story	Expected risk
Ada Okafor	Administrator	Africa/Lagos	Ordinary; one new phone	Low
Bola Adeyemi	Finance Manager	Africa/Lagos	Lagos, then London 40 minutes later	High
Chen Wei	Operations	Asia/Singapore	8 failures from Frankfurt in 9 minutes, none succeeded	Medium
Dana Smith	Support Agent	America/New_York	A real trip to London on a new laptop	Medium
Emeka Nwosu	Finance Manager + **deny** `reports.finance.export`	Europe/London	A contractor	None
Femi Bello	Auditor	Africa/Lagos	Reads users, changes nothing	None
Grace Eze	Support Agent, **suspended**	Africa/Lagos	Left the company	None
Tunde Bakare	Operations	Africa/Lagos	500+ scripted sign-ins; then a successful attack from Amsterdam	High

2.7 What we are deliberately not building

Real identity platforms do much more. Sentinel does not authenticate passwords, send email, lock accounts, or keep an audit log. It uses a development-only authentication scheme (Chapter 12). The pages behind the menu (reports, billing) are placeholders. Every one of those omissions is a place where the book would grow without teaching anything new about the three scenarios.

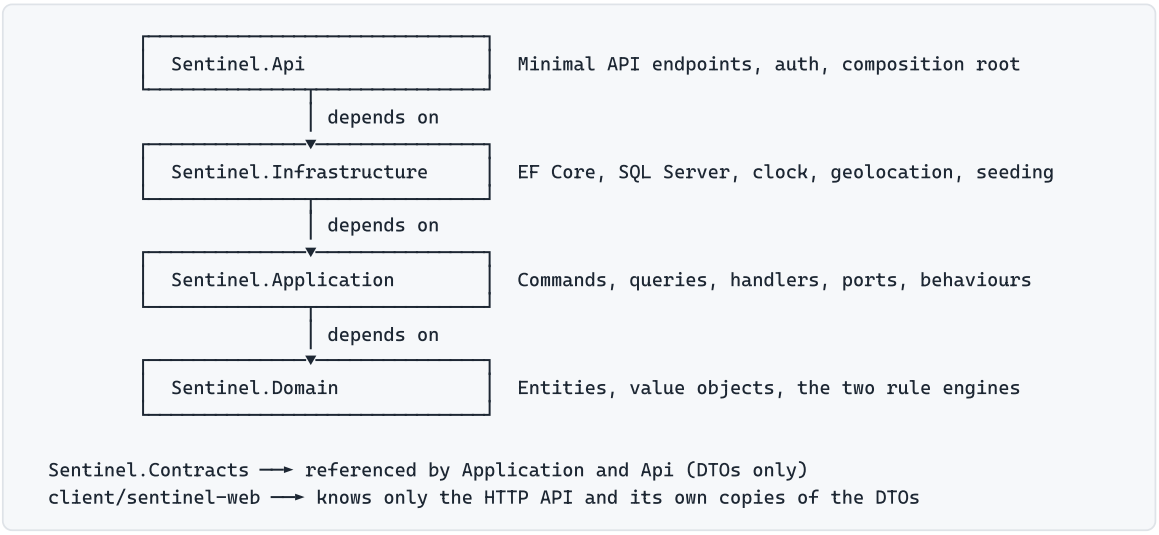
Chapter 3 · The Architecture

3.1 The dependency rule

Clean Architecture is usually taught as concentric circles, which is memorable and nearly useless, because a picture does not tell you what to do on a Tuesday afternoon. It reduces to one mechanical rule:

> Source code dependencies point only inwards. An inner layer never names a type defined in an outer layer.

Everything else (ports, adapters, dependency inversion) is technique for obeying that rule when control has to flow the other way. Sentinel has four production layers and a contracts assembly:



Sentinel.Domain references nothing but the base class library. No EF Core, no ASP.NET, no NuGet packages at all; an architecture test (Chapter 13) fails the build if one appears.

Sentinel.Application references Domain and Contracts. It defines the *ports*, interfaces such as `IApplicationDbContext`, `IClock` and `IGeoLocator`, and contains every command, query and handler. It does not know SQL Server exists.

Sentinel.Infrastructure implements the ports: the EF Core context, the system clock, a geolocation table.

Sentinel.Api is the composition root. It references everything and is referenced by nothing.

Sentinel.Contracts is a deliberately anaemic assembly of request and response records: the wire format. The Angular client mirrors it by hand.

3.2 Why the domain has no dependencies

The two rule engines are the most valuable code in the system and the longest-lived. Every dependency you add to them is a constraint on how they can evolve and a cost on every test. `PageAccessEvaluator` can be unit tested with nothing but `new`: no database, no web host, no mocks. That is not purity for its own sake; it is why Chapter 7 has thirty tests that run in under a second.

It is also the single most transferable lesson for the second scenario. In a 40-minute test, write the rules as a pure function of their inputs, then test that function. Do not let I/O anywhere near it.

3.3 CQRS, and what it means here

CQRS (Command Query Responsibility Segregation) is often described as needing two databases and an event bus. It needs neither. Its essence is:

> The model you use to change state need not be the model you use to read it.

Sentinel's version, sometimes called "CQRS-lite", is the right default for a system of this size:

- **Commands** load an aggregate with change tracking, call a method on it, and save. `AssignRoleCommandHandler` loads a `User`, calls `user.AssignRole(roleId, now)`, and lets the aggregate decide whether that is allowed. The handler contains almost no logic.
- **Queries** never modify anything. They read with `AsNoTracking()` and shape a DTO for the screen. `GetAccountSecurityStatusQueryHandler` loads history, runs the detector, and returns exactly what the page needs.

Both kinds are messages (`record` types) dispatched to exactly one handler. That uniformity is what lets cross-cutting behaviour (validation, logging) be written once (Chapter 9).

3.4 Vertical slices inside the layers

Inside Application, code is organised by **feature, then use case**, never by technical kind:

```
src/Sentinel.Application/
├── Abstractions/        Messaging/, Validation/, Ports/
├── Behaviors/           ValidationBehavior, LoggingBehavior
├── Access/HasPermission/
├── Pages/
│   ├── CheckPageAccess/     CheckPageAccessQuery.cs, CheckPageAccessQueryHandler.cs
│   ├── GetNavigation/
│   ├── GetPageAccess/
│   └── ...
├── Security/
│   ├── GetAccountSecurityStatus/
│   ├── AcknowledgeFinding/
│   └── ...
├── Roles/   └── Users/
```

A developer asked to change how findings are acknowledged opens one folder. The alternative, top-level `Commands/`, `Handlers/` and `Validators/` folders, scatters each use case across three places and gets worse with every feature. An architecture test (Chapter 13) enforces the folder convention.

Vertical slices and Clean Architecture are sometimes presented as rivals. They are not: the layers decide *which direction dependencies point*; the slices decide *how code is grouped within a layer*. Sentinel uses both.

3.5 SOLID, concretely

SOLID is easy to recite and hard to recognise. Here is where each principle lives in this codebase, so you can point at it in an interview.

Principle	Where you can see it
Single responsibility	Each handler does one use case. `PageAccessEvaluator` decides access; it does not load data or build menus (`PageAccessMapping` does that).
Open/closed	Adding a use case adds a folder; nothing existing changes, because handlers and validators are discovered by `AddApplication()`. Adding a pipeline behaviour wraps every handler without editing any.
Liskov substitution	`Result<T>` is usable anywhere a `Result` is expected; `ValidationError` anywhere an `Error` is. The kata tests run one test class against two implementations of the same interface.
Interface segregation	`IClock` has one member. `IGeoLocator` has one. The Angular `Session` exposes read-only signals, not its writable state.
Dependency inversion	Application owns `IApplicationDbContext`; Infrastructure implements it. The arrow of the source dependency points inwards even though control flows outwards.

3.6 The dispatcher, and a note on MediatR

Most CQRS samples in .NET reach for MediatR. Since version 13 MediatR is commercially licensed above a revenue threshold. We write our own dispatcher instead: about sixty lines, no licensing question, and writing it teaches exactly what MediatR does (Chapter 9). Swapping to MediatR or Wolverine later is mechanical.

3.7 A note on repositories

We do not use the repository pattern. `DbContext` already *is* a unit of work with repositories hanging off it; wrapping it yields either a leaky abstraction that exposes `IQueryable` (and abstracts nothing) or a growing interface with a method per query. The Application layer depends on `IApplicationDbContext`, an interface it owns, which satisfies the dependency rule while keeping LINQ.

3.8 What is not here, and when it would be

Domain events. Nothing in Sentinel reacts to a change in another aggregate yet. When something does (say, "suspending a user acknowledges nothing and emails security"), raise an event from the aggregate and dispatch it after `SaveChanges`. Until then, an event bus with no subscribers is ceremony.

A read model. The page access queries load the whole page tree on every call. With tens of pages that costs microseconds. With tens of thousands, cache the tree and invalidate it when a page changes. Measure first.

Part II · The Back End

Chapter 4 · Environment and Solution Setup

4.1 What to install

- **.NET 10 SDK** (10.0.3xx or later). Check with `dotnet --list-sdks`.
- **Visual Studio 2026** with the *ASP.NET and web development* workload, or VS Code with C# Dev Kit, or Rider. Nothing in this book needs a particular IDE.
- **SQL Server**: LocalDB (installed with Visual Studio) is enough. Any edition from 2019 works; set the connection string in `src/Sentinel.Api/appsettings.json` or with user secrets.
- **Node.js 22 or later** (24 LTS recommended) and npm 10+, for Angular.
- **Git**.
- Optionally **pandoc** and Chrome or Edge, to build this book.

Verify:

```
dotnet --list-sdks
node --version
sqllocaldb info
```

4.2 Creating the projects

```
mkdir sentinel; cd sentinel
git init -b main
dotnet new sln -n Sentinel           # creates Sentinel.slnx (the XML solution format)

foreach ($p in 'Domain','Contracts','Application','Infrastructure') {
    dotnet new classlib -n "Sentinel.$p" -o "src/Sentinel.$p" -f net10.0
}
dotnet new web -n Sentinel.Api -o src/Sentinel.Api -f net10.0

foreach ($t in 'Domain','Application','Api','Architecture') {
    dotnet new xunit -n "Sentinel.$t.Tests" -o "tests/Sentinel.$t.Tests" -f net10.0
}

dotnet sln Sentinel.slnx add (Get-ChildItem src,tests -Recurse -Filter *.csproj).FullName
```

Delete the `Class1.cs` and `UnitTest1.cs` files the templates create. The solution file is short enough to read:

`Sentinel.slnx`

```
<Solution>
  <Folder Name="/src/">
    <Project Path="src/Sentinel.Api/Sentinel.Api.csproj" />
    <Project Path="src/Sentinel.Application/Sentinel.Application.csproj" />
    <Project Path="src/Sentinel.Contracts/Sentinel.Contracts.csproj" />
    <Project Path="src/Sentinel.Domain/Sentinel.Domain.csproj" />
    <Project Path="src/Sentinel.Infrastructure/Sentinel.Infrastructure.csproj" />
  </Folder>
  <Folder Name="/tests/">
    <Project Path="tests/Sentinel.Api.Tests/Sentinel.Api.Tests.csproj" />
    <Project Path="tests/Sentinel.Application.Tests/Sentinel.Application.Tests.csproj" />
    <Project Path="tests/Sentinel.Architecture.Tests/Sentinel.Architecture.Tests.csproj"
/>
    <Project Path="tests/Sentinel.Domain.Tests/Sentinel.Domain.Tests.csproj" />
```

```
  </Folder>
</Solution>
```

4.3 Central build configuration

Settings every project shares go in one file at the root. MSBuild finds `Directory.Build.props` by walking up from each project:

Directory.Build.props

```
<Project>
  <!-- Applies to every project under this folder. Keep it boring: anything
       project-specific belongs in that project's .csproj. -->
  <PropertyGroup>
    <TargetFramework>net10.0</TargetFramework>
    <LangVersion>latest</LangVersion>
    <Nullable>enable</Nullable>
    <ImplicitUsings>enable</ImplicitUsings>
    <TreatWarningsAsErrors>true</TreatWarningsAsErrors>
    <!-- CS1591: missing XML comment. We document intent, not every member. -->
    <NoWarn>$(NoWarn);CS1591</NoWarn>
    <ManagePackageVersionsCentrally>true</ManagePackageVersionsCentrally>
  </PropertyGroup>
</Project>
```

`TreatWarningsAsErrors` is worth its occasional annoyance. A nullable warning ignored today is a `NullReferenceException` reported by a customer next month, and a debugging lab in this book.

Pin the SDK so that everyone builds with the same compiler:

global.json

```
{
  "sdk": {
    "version": "10.0.301",
    "rollForward": "latestFeature"
  }
}
```

4.4 Central package management

Versions are declared once, in `Directory.Packages.props`; project files name packages without versions.

Directory.Packages.props

```
<Project>
  <!-- Central package management: every version lives here, once. -->
  <PropertyGroup>
    <!-- Lets us pin a vulnerable transitive dependency without referencing it directly. -
->
    <CentralPackageTransitivePinningEnabled>true</CentralPackageTransitivePinningEnabled>
  </PropertyGroup>
  <ItemGroup>
    <PackageVersion Include="Microsoft.EntityFrameworkCore" Version="10.0.13" />
    <PackageVersion Include="Microsoft.EntityFrameworkCore.Relational" Version="10.0.13"
/>
    <PackageVersion Include="Microsoft.EntityFrameworkCore.SqlServer" Version="10.0.13" />
```

```
    <PackageVersion Include="Microsoft.EntityFrameworkCore.Sqlite" Version="10.0.13" />
    <PackageVersion Include="Microsoft.EntityFrameworkCore.Design" Version="10.0.13" />
    <PackageVersion Include="Microsoft.Extensions.DependencyInjection.Abstractions"
Version="10.0.13" />
    <PackageVersion Include="Microsoft.Extensions.Logging.Abstractions" Version="10.0.13"
/>
    <PackageVersion Include="Microsoft.AspNetCore.OpenApi" Version="10.0.13" />
    <PackageVersion Include="Microsoft.AspNetCore.Mvc.Testing" Version="10.0.13" />
    <!-- Transitive pin: Microsoft.Data.SqlClient brings 9.0.x, which has published
advisories. -->
    <PackageVersion Include="System.Security.Cryptography.Pkcs" Version="10.0.13" />
    <!-- Test tooling -->
    <PackageVersion Include="Microsoft.NET.Test.Sdk" Version="17.14.1" />
    <PackageVersion Include="xunit" Version="2.9.3" />
    <PackageVersion Include="xunit.runner.visualstudio" Version="3.1.4" />
    <PackageVersion Include="coverlet.collector" Version="6.0.4" />
  </ItemGroup>
</Project>
```

Note the transitive pin. `Microsoft.Data.SqlClient` (pulled in by the EF Core SQL Server provider) depended on a version of `System.Security.Cryptography.Pkcs` with published advisories, and with warnings as errors the build failed with `NU1903`. Transitive pinning lets us raise that one package without referencing it from any project. This is a real incident from writing this book: the fix took two minutes because the error message named the package and the advisory.

4.5 The project files

Each project's file states its references and nothing else. The Domain:

`src/Sentinel.Domain/Sentinel.Domain.csproj`

```
<Project Sdk="Microsoft.NET.Sdk">

  <!-- The Domain references nothing but the base class library.
       If you are about to add a PackageReference here, stop: the concept
       you want almost certainly belongs in Application as a port. -->
  <PropertyGroup>
    <RootNamespace>Sentinel.Domain</RootNamespace>
  </PropertyGroup>

  <ItemGroup>
    <InternalsVisibleTo Include="Sentinel.Domain.Tests" />
  </ItemGroup>

</Project>
```

The Application layer references EF Core for `DbSet<T>` and LINQ, but not a provider:

`src/Sentinel.Application/Sentinel.Application.csproj`

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <RootNamespace>Sentinel.Application</RootNamespace>
  </PropertyGroup>

  <ItemGroup>
    <ProjectReference Include="..\Sentinel.Domain\Sentinel.Domain.csproj" />
    <ProjectReference Include="..\Sentinel.Contracts\Sentinel.Contracts.csproj" />
  </ItemGroup>

  <ItemGroup>
```

```
    <!-- EF Core for DbSet<T> and the LINQ extensions only. The Application
         layer never learns which database provider is behind them. -->
    <PackageReference Include="Microsoft.EntityFrameworkCore" />
    <PackageReference Include="Microsoft.Extensions.DependencyInjection.Abstractions" />
    <PackageReference Include="Microsoft.Extensions.Logging.Abstractions" />
  </ItemGroup>

  <ItemGroup>
    <InternalsVisibleTo Include="Sentinel.Application.Tests" />
  </ItemGroup>

</Project>
```

Infrastructure brings the provider and the design-time tools:

src/Sentinel.Infrastructure/Sentinel.Infrastructure.csproj

```
<Project Sdk="Microsoft.NET.Sdk">

  <PropertyGroup>
    <RootNamespace>Sentinel.Infrastructure</RootNamespace>
  </PropertyGroup>

  <ItemGroup>
    <ProjectReference Include="..\Sentinel.Application\Sentinel.Application.csproj" />
  </ItemGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.EntityFrameworkCore.SqlServer" />
    <PackageReference Include="Microsoft.EntityFrameworkCore.Design">
      <PrivateAssets>all</PrivateAssets>
      <IncludeAssets>runtime; build; native; contentfiles; analyzers;
buildtransitive</IncludeAssets>
    </PackageReference>
  </ItemGroup>

  <ItemGroup>
    <InternalsVisibleTo Include="Sentinel.Application.Tests" />
    <InternalsVisibleTo Include="Sentinel.Api.Tests" />
  </ItemGroup>

</Project>
```

The API composes them:

src/Sentinel.Api/Sentinel.Api.csproj

```
<Project Sdk="Microsoft.NET.Sdk.Web">

  <PropertyGroup>
    <RootNamespace>Sentinel.Api</RootNamespace>
    <UserSecretsId>sentinel-api-6f1d3c2e</UserSecretsId>
  </PropertyGroup>

  <ItemGroup>
    <ProjectReference Include="..\Sentinel.Application\Sentinel.Application.csproj" />
    <ProjectReference Include="..\Sentinel.Infrastructure\Sentinel.Infrastructure.csproj"
/>
  </ItemGroup>

  <ItemGroup>
    <PackageReference Include="Microsoft.AspNetCore.OpenApi" />
  </ItemGroup>

</Project>
```

4.6 The EF Core tool

The `dotnet ef` commands come from a local tool, recorded in the repository so every clone uses the same version:

```
dotnet new tool-manifest
dotnet tool install dotnet-ef --version 10.0.13
```

`dotnet-tools.json`

```
{
  "version": 1,
  "isRoot": true,
  "tools": {
    "dotnet-ef": {
      "version": "10.0.13",
      "commands": [
        "dotnet-ef"
      ],
      "rollForward": false
    }
  }
}
```

4.7 Verifying the build

```
dotnet build
dotnet test
```

Both must be clean before you go on. From here on, every chapter ends in the same state: it builds, and the tests pass.

Chapter 5 · Domain Building Blocks

Four small types carry the whole domain: an error, a result, an entity and an aggregate root. They are worth getting exactly right, because every other class leans on them.

5.1 Errors are values, exceptions are for bugs

There are two kinds of failure. “That email is already registered” happens every day and is part of the business. “The database connection string is null” should never happen and means the program is wrong. Mixing them, by throwing exceptions for both, makes every caller wrap every call in `try/catch` and guess which is which.

Sentinel’s rule: **business failures are returned as `Error` values; exceptions are reserved for programming errors.**

`src/Sentinel.Domain/Common/ErrorType.cs`

```
namespace Sentinel.Domain.Common;

/// <summary>
/// The category of a failure. The API layer maps each category to exactly one
/// HTTP status code, so adding a member here is an API change.
/// </summary>
public enum ErrorType
{
    Validation,
    NotFound,
    Conflict,
    Forbidden
}
```

`src/Sentinel.Domain/Common/Error.cs`

```
namespace Sentinel.Domain.Common;

/// <summary>
/// A failure that is part of the normal flow of the business — "that email is
/// already registered", "that role does not exist". Exceptions are reserved for
/// things that should never happen; errors are things that happen every day.
/// Not sealed: the Application layer derives a ValidationError that carries
/// one message per field.
/// </summary>
public record Error(string Code, string Message, ErrorType Type)
{
    public static Error Validation(string code, string message) => new(code, message,
ErrorType.Validation);
    public static Error NotFound(string code, string message) => new(code, message,
ErrorType.NotFound);
    public static Error Conflict(string code, string message) => new(code, message,
ErrorType.Conflict);
    public static Error Forbidden(string code, string message) => new(code, message,
ErrorType.Forbidden);

    public override string ToString() => $"{Code}: {Message}";
}
```

An error has a stable `Code` (`User.DuplicateEmail`) that clients and tests can rely on, a human `Message`, and a `Type` that the API maps to a status code in exactly one place (Chapter 12). The record is not sealed because the Application layer derives a `ValidationError` that carries one message per field.

5.2 Result: success or failure, never both

`src/Sentinel.Domain/Common/Result.cs`

```
namespace Sentinel.Domain.Common;

/// <summary>
/// The outcome of an operation that can fail for a business reason.
/// A result is either a success or a failure, never both and never neither:
/// the constructor enforces it, so no caller has to.
/// </summary>
public class Result
{
    protected Result(bool isSuccess, Error? error)
    {
        if (isSuccess && error is not null)
            throw new InvalidOperationException("A successful result cannot carry an
error.");
        if (!isSuccess && error is null)
            throw new InvalidOperationException("A failed result must carry an error.");

        IsSuccess = isSuccess;
        Error = error;
    }

    public bool IsSuccess { get; }
    public bool IsFailure => !IsSuccess;

    /// <summary>Null when <see cref="IsSuccess"/> is true.</summary>
    public Error? Error { get; }

    public static Result Success() => new(true, null);
    public static Result Failure(Error error) => new(false, error);

    public static Result<T> Success<T>(T value) => Result<T>.Success(value);
    public static Result<T> Failure<T>(Error error) => Result<T>.Failure(error);

    public static implicit operator Result(Error error) => Failure(error);
}

/// <summary>A <see cref="Result"/> that carries a value when it succeeds.</summary>
public sealed class Result<T> : Result
{
    private readonly T? _value;

    private Result(T? value, bool isSuccess, Error? error) : base(isSuccess, error) =>
_value = value;

    /// <summary>
    /// The value of a successful result. Reading it from a failure is a
    /// programming error, so it throws rather than returning a default.
    /// </summary>
    public T Value => IsSuccess
        ? _value!
        : throw new InvalidOperationException($"Cannot read the value of a failed result
({Error}).");

    public static Result<T> Success(T value) => new(value, true, null);
    public static new Result<T> Failure(Error error) => new(default, false, error);

    public static implicit operator Result<T>(T value) => Success(value);
```

```
    public static implicit operator Result<T>(Error error) => Failure(error);
}
```

Points worth noticing:

- The constructor enforces the invariant. A `Result` that is a success with an error, or a failure without one, cannot be constructed, so no caller ever has to check for it.
- Reading `Value` from a failure **throws**. Returning `default` would turn a forgotten `if (result.IsFailure)` into a silent zero or null somewhere far away.
- The implicit conversions make handlers read naturally: `return UserErrors.NotFound(id);` and `return user.Id;` both compile in a method returning `Result<Guid>`.

> **On the test —** In a 40-minute coding scenario you will not build a `Result` type. You will still face the decision it encodes: what does my function return when the input is bad? Decide explicitly (an empty list, `null`, an exception), say so in a comment, and test it.

5.3 Entities and aggregate roots

`src/Sentinel.Domain/Common/Entity.cs`

```
namespace Sentinel.Domain.Common;

/// <summary>
/// Something with an identity that outlives changes to its attributes.
/// Two entities are the same entity when their ids are equal.
/// </summary>
public abstract class Entity
{
    protected Entity(Guid id) => Id = id;

    // For EF Core materialisation only.
    protected Entity() { }

    public Guid Id { get; protected init; }

    public override bool Equals(object? obj) =>
        obj is Entity other && other.GetType() == GetType() && other.Id == Id;

    public override int GetHashCode() => HashCode.Combine(GetType(), Id);
}
```

Equality by id, and by type: a `User` and a `Role` that happened to share a Guid are not equal. The protected parameterless constructor exists for EF Core only.

`src/Sentinel.Domain/Common/AggregateRoot.cs`

```
namespace Sentinel.Domain.Common;

/// <summary>
/// The entry point to a cluster of objects that must change together.
/// Outside code holds references to the root only, and every change goes
/// through a method on the root, which is where the invariants are checked.
/// </summary>
/// <remarks>
/// There are no domain events here. Nothing in Sentinel reacts to a change
/// in another aggregate yet, and an event bus with no subscribers is
/// ceremony. Chapter 3 explains when to add one.
```

```
        /// </remarks>
        public abstract class AggregateRoot : Entity
        {
            protected AggregateRoot(Guid id) : base(id) { }
            protected AggregateRoot() { }
        }
```

An aggregate root is the only entry point to a cluster of objects that must change together. `User` is a root; `UserRole` and `PermissionOverride` live inside it and are only changed through `User`'s methods. That is what lets `User` guarantee, for example, that a role is never assigned twice.

5.4 Value objects, briefly

A value object has no identity: two `EmailAddress` instances with the same text *are* the same email. Sentinel's value objects (`EmailAddress`, `Permission`, `GeoLocation`) share a pattern:

1. A private constructor.
2. A static `Create` that validates, normalises and returns a `Result`.
3. Equality by value (a `record`, or `IEquatable<T>` written out).

`src/Sentinel.Domain/Users/EmailAddress.cs`

```
        using System.Net.Mail;
        using Sentinel.Domain.Common;

        namespace Sentinel.Domain.Users;

        /// <summary>
        /// An email address, trimmed and lower-cased so that "Ada@Example.com" and
        /// "ada@example.com " are the same person. Validation is deliberately
        /// shallow: the only real test of an address is sending mail to it.
        /// </summary>
        public sealed record EmailAddress
        {
            public const int MaxLength = 254; // RFC 5321 path limit

            private EmailAddress(string value) => Value = value;

            public string Value { get; }

            public static Result<EmailAddress> Create(string? value)
            {
                if (string.IsNullOrWhiteSpace(value))
                    return UserErrors.EmailRequired;

                var normalised = value.Trim().ToLowerInvariant();

                if (normalised.Length > MaxLength)
                    return UserErrors.EmailInvalid(value);

                // MailAddress accepts "Display Name <a@b>", so insist that what it
                // parsed is exactly what we were given.
                return MailAddress.TryCreate(normalised, out var parsed) && parsed.Address ==
normalised && normalised.Contains('.')
                    ? new EmailAddress(normalised)
                    : UserErrors.EmailInvalid(value);
            }

            public override string ToString() => Value;
        }
```

The validation is deliberately shallow. The only true test of an email address is delivering mail to it; everything else is a heuristic, and a strict regex rejects real addresses. Note the comment about `MailAddress.TryCreate` accepting a display name: that is the kind of library behaviour you only discover by testing edge cases, and the kind of discovery worth a comment.

Chapter 6 · Users, Roles and Permissions

6.1 Permission: a value object with one interesting method

`src/Sentinel.Domain/Access/Permission.cs`

```
        using System.Text.RegularExpressions;
        using Sentinel.Domain.Common;

        namespace Sentinel.Domain.Access;

        /// <summary>
        /// A named capability such as <c>reports.finance</c>. Permissions are
        /// dot-separated, lower-case segments. A grant may end in a wildcard:
        /// <c>reports.*</c> covers every permission beneath <c>reports</c>, and
        /// <c>*</c> covers everything.
        /// </summary>
        /// <remarks>
        /// Wildcards are only meaningful on the granting side. A page *requires*
        /// a concrete permission; a role may *grant* a pattern. <c>reports.*</c>
        /// deliberately does not cover <c>reports</c> itself: "everything under
        /// reports" is not the same claim as "the reports landing page".
        /// </remarks>
        public sealed partial class Permission : IEquatable<Permission>
        {
            public const string Wildcard = "*";

            private Permission(string value) => Value = value;

            public string Value { get; }

            public bool IsWildcard => Value == Wildcard || Value.EndsWith(".*",
StringComparison.Ordinal);

            public static Result<Permission> Create(string? value)
            {
                if (string.IsNullOrWhiteSpace(value))
                    return PermissionErrors.Blank;

                var normalised = value.Trim().ToLowerInvariant();

                return normalised == Wildcard || PermissionPattern().IsMatch(normalised)
                    ? new Permission(normalised)
                    : PermissionErrors.Malformed(value);
            }

            /// <summary>
            /// For values already validated: seed data and database rows. Throws on
            /// bad input because bad input here is a bug, not a user mistake.
            /// </summary>
            public static Permission From(string value)
            {
                var result = Create(value);
                return result.IsSuccess
                    ? result.Value
                    : throw new ArgumentException(result.Error!.Message, nameof(value));
            }

            /// <summary>Does this grant (or deny) cover the <paramref name="required"/>
permission?</summary>
            public bool Covers(Permission required)
            {
                ArgumentNullException.ThrowIfNull(required);

                if (Value == Wildcard)
```

```
                    return true;

                if (!IsWildcard)
                    return Value == required.Value;

                // "reports.*" -> the prefix "reports." must start the required value.
                var prefix = Value[..^1];
                return required.Value.StartsWith(prefix, StringComparison.Ordinal);
            }

            public bool Equals(Permission? other) => other is not null && other.Value == Value;
            public override bool Equals(object? obj) => Equals(obj as Permission);
            public override int GetHashCode() => Value.GetHashCode(StringComparison.Ordinal);
            public override string ToString() => Value;

            // segment(.segment)* optionally followed by ".*"
            [GeneratedRegex(@"^[a-z0-9][a-z0-9_-]*(\.[a-z0-9][a-z0-9_-]*)*(\.\*)?$")]
            private static partial Regex PermissionPattern();
        }

        public static class PermissionErrors
        {
            public static readonly Error Blank =
                Error.Validation("Permission.Blank", "A permission name is required.");

            public static Error Malformed(string value) =>
                Error.Validation("Permission.Malformed",
                    $"'{value}' is not a valid permission. Use dot-separated segments such as
'reports.finance' or 'reports.*'.");
        }
```

`Create` normalises (trim, lower-case) and validates against one regular expression. The pattern reads: one or more segments of `[a-z0-9_-]`, separated by dots, optionally ending in `.*`; or the single wildcard `*`. `[GeneratedRegex]` compiles it at build time, so there is no start-up cost and no risk of a typo surfacing only at run time.

`Covers` is the rule from §2.4, rule 7, and nothing else:

Grant	Required	Covers?	Why
`reports.finance`	`reports.finance`	yes	exact match
`reports.*`	`reports.finance.export`	yes	any depth
`reports.*`	`reports`	**no**	"everything under" ≠ "the thing itself"
`reports.*`	`reportsx.finance`	**no**	the prefix includes the dot
`reports.finance`	`reports.finance.export`	**no**	a concrete grant has no children
`*`	anything	yes	superuser

The fourth row is the one that catches people. Implementing the wildcard as `required.StartsWith("reports")` would let `reports.*` cover `reportsx`. Keeping the trailing dot in the prefix (`Value[..^1]` drops only the `*`) makes the comparison segment-aware.

> **Tip —** When a rule has a table like this, put the table in your tests as a `[Theory]`. `PermissionTests.Covers` in Chapter 13 is exactly this table.

6.2 Role

`src/Sentinel.Domain/Roles/Role.cs`

```
using Sentinel.Domain.Access;
using Sentinel.Domain.Common;

namespace Sentinel.Domain.Roles;

/// <summary>A named bundle of permissions, such as "Finance Analyst".</summary>
public sealed class Role : AggregateRoot
{
    public const int MaxNameLength = 64;

    // Not readonly: EF Core assigns a fresh list when it reads the column.
    private List<Permission> _permissions = [];

    private Role(Guid id, string name, string description) : base(id)
    {
        Name = name;
        Description = description;
    }

    // For EF Core.
    private Role() { }

    public string Name { get; private set; } = null!;
    public string Description { get; private set; } = null!;

    public IReadOnlyCollection<Permission> Permissions => _permissions.AsReadOnly();

    public static Result<Role> Create(string name, string? description = null, Guid? id =
null)
    {
        if (string.IsNullOrWhiteSpace(name))
            return RoleErrors.NameRequired;

        if (name.Trim().Length > MaxNameLength)
            return RoleErrors.NameTooLong;

        return new Role(id ?? Guid.CreateVersion7(), name.Trim(), description?.Trim() ??
string.Empty);
    }

    public Result Grant(Permission permission)
    {
        ArgumentNullException.ThrowIfNull(permission);

        if (_permissions.Contains(permission))
            return RoleErrors.AlreadyGranted(Name, permission);

        _permissions.Add(permission);
        return Result.Success();
    }

    public Result Revoke(Permission permission)
    {
        ArgumentNullException.ThrowIfNull(permission);

        return _permissions.Remove(permission)
            ? Result.Success()
            : RoleErrors.NotGranted(Name, permission);
    }
}

public static class RoleErrors
{
    public static readonly Error NameRequired =
        Error.Validation("Role.NameRequired", "A role name is required.");
```

```
            public static readonly Error NameTooLong =
                Error.Validation("Role.NameTooLong", $"A role name cannot exceed
{Role.MaxNameLength} characters.");

            public static Error DuplicateName(string name) =>
                Error.Conflict("Role.DuplicateName", $"A role named '{name}' already exists.");

            public static Error NotFound(Guid id) =>
                Error.NotFound("Role.NotFound", $"Role {id} does not exist.");

            public static Error AlreadyGranted(string role, Permission permission) =>
                Error.Conflict("Role.AlreadyGranted", $"Role '{role}' already grants
'{permission}'.");

            public static Error NotGranted(string role, Permission permission) =>
                Error.NotFound("Role.NotGranted", $"Role '{role}' does not grant
'{permission}'.");
        }
```

A role grants each permission at most once; granting twice is a `Conflict`, revoking something not granted is `NotFound`. The field `_permissions` is not `readonly` because EF Core assigns a new list when it reads the column (Chapter 11). The public `Permissions` property is a read-only view, so nothing outside the aggregate can add to the list behind its back.

6.3 User

`src/Sentinel.Domain/Users/User.cs`

```
        using Sentinel.Domain.Access;
        using Sentinel.Domain.Common;

        namespace Sentinel.Domain.Users;

        public enum UserStatus
        {
            Active,
            Suspended
        }

        /// <summary>A role held by a user. Part of the <see cref="User"/> aggregate.</summary>
        public sealed class UserRole
        {
            internal UserRole(Guid roleId, DateTime assignedAtUtc)
            {
                RoleId = roleId;
                AssignedAtUtc = assignedAtUtc;
            }

            // For EF Core.
            private UserRole() { }

            public Guid RoleId { get; private set; }
            public DateTime AssignedAtUtc { get; private set; }
        }

        /// <summary>
        /// A permission granted or denied to one user directly, regardless of role.
        /// Denies exist for the "everyone in Finance except the contractor" case.
        /// </summary>
        public sealed class PermissionOverride
        {
            internal PermissionOverride(Permission permission, PermissionEffect effect)
            {
                Permission = permission;
                Effect = effect;
```

```
            }

            // For EF Core.
            private PermissionOverride() { }

            public Permission Permission { get; private set; } = null!;
            public PermissionEffect Effect { get; private set; }
        }

        /// <summary>A person who signs in to the portal.</summary>
        public sealed class User : AggregateRoot
        {
            public const int MaxDisplayNameLength = 100;

            private readonly List<UserRole> _roles = [];
            private readonly List<PermissionOverride> _overrides = [];

            private User(Guid id, EmailAddress email, string displayName, string timeZoneId,
DateTime createdAtUtc) : base(id)
            {
                Email = email;
                DisplayName = displayName;
                TimeZoneId = timeZoneId;
                CreatedAtUtc = createdAtUtc;
                Status = UserStatus.Active;
            }

            // For EF Core.
            private User() { }

            public EmailAddress Email { get; private set; } = null!;
            public string DisplayName { get; private set; } = null!;

            /// <summary>
            /// An IANA zone such as <c>Africa/Lagos</c>. Timestamps are stored in UTC
            /// and converted to this zone only at the moment they are shown.
            /// </summary>
            public string TimeZoneId { get; private set; } = null!;

            public UserStatus Status { get; private set; }
            public string? SuspensionReason { get; private set; }
            public DateTime CreatedAtUtc { get; private set; }

            public IReadOnlyCollection<UserRole> Roles => _roles.AsReadOnly();
            public IEnumerable<Guid> RoleIds => _roles.Select(r => r.RoleId);
            public IReadOnlyCollection<PermissionOverride> Overrides => _overrides.AsReadOnly();

            public static Result<User> Create(EmailAddress email, string displayName, string
timeZoneId, DateTime nowUtc, Guid? id = null)
            {
                ArgumentNullException.ThrowIfNull(email);

                if (string.IsNullOrWhiteSpace(displayName))
                    return UserErrors.DisplayNameRequired;

                if (displayName.Trim().Length > MaxDisplayNameLength)
                    return UserErrors.DisplayNameTooLong;

                if (!IsKnownTimeZone(timeZoneId))
                    return UserErrors.UnknownTimeZone(timeZoneId);

                return new User(id ?? Guid.CreateVersion7(), email, displayName.Trim(),
timeZoneId, EnsureUtc(nowUtc));
            }

            public Result AssignRole(Guid roleId, DateTime nowUtc)
            {
                if (_roles.Any(r => r.RoleId == roleId))
                    return UserErrors.RoleAlreadyAssigned(roleId);

                _roles.Add(new UserRole(roleId, EnsureUtc(nowUtc)));
                return Result.Success();
```

```
            }

            public Result RevokeRole(Guid roleId)
            {
                var held = _roles.FirstOrDefault(r => r.RoleId == roleId);
                if (held is null)
                    return UserErrors.RoleNotAssigned(roleId);

                _roles.Remove(held);
                return Result.Success();
            }

            /// <summary>
            /// Adds or replaces the user's override for <paramref name="permission"/>.
            /// A user has at most one override per permission: allowing and denying
            /// the same thing at once has no meaning.
            /// </summary>
            public Result SetOverride(Permission permission, PermissionEffect effect)
            {
                ArgumentNullException.ThrowIfNull(permission);

                _overrides.RemoveAll(o => o.Permission == permission);
                _overrides.Add(new PermissionOverride(permission, effect));
                return Result.Success();
            }

            public Result ClearOverride(Permission permission)
            {
                ArgumentNullException.ThrowIfNull(permission);

                return _overrides.RemoveAll(o => o.Permission == permission) > 0
                    ? Result.Success()
                    : UserErrors.OverrideNotFound(permission);
            }

            public Result Suspend(string reason)
            {
                if (Status == UserStatus.Suspended)
                    return UserErrors.AlreadySuspended;

                if (string.IsNullOrWhiteSpace(reason))
                    return UserErrors.SuspensionReasonRequired;

                Status = UserStatus.Suspended;
                SuspensionReason = reason.Trim();
                return Result.Success();
            }

            public Result Reactivate()
            {
                if (Status == UserStatus.Active)
                    return UserErrors.NotSuspended;

                Status = UserStatus.Active;
                SuspensionReason = null;
                return Result.Success();
            }

            private static bool IsKnownTimeZone(string? timeZoneId) =>
                !string.IsNullOrWhiteSpace(timeZoneId) &&
TimeZoneInfo.TryFindSystemTimeZoneById(timeZoneId, out _);

            private static DateTime EnsureUtc(DateTime value) =>
                value.Kind == DateTimeKind.Utc
                    ? value
                    : throw new ArgumentException("Timestamps must be UTC.", nameof(value));
        }

        public static class UserErrors
        {
            public static readonly Error EmailRequired =
                Error.Validation("User.EmailRequired", "An email address is required.");
```

```
            public static Error EmailInvalid(string? email) =>
                Error.Validation("User.EmailInvalid", $"'{email}' is not a valid email address.");

            public static readonly Error DisplayNameRequired =
                Error.Validation("User.DisplayNameRequired", "A display name is required.");

            public static readonly Error DisplayNameTooLong =
                Error.Validation("User.DisplayNameTooLong", $"A display name cannot exceed
{User.MaxDisplayNameLength} characters.");

            public static Error UnknownTimeZone(string? zone) =>
                Error.Validation("User.UnknownTimeZone", $"'{zone}' is not a recognised time zone.
Use an IANA name such as 'Africa/Lagos'.");

            public static Error DuplicateEmail(EmailAddress email) =>
                Error.Conflict("User.DuplicateEmail", $"A user with email '{email}' already
exists.");

            public static Error NotFound(Guid id) =>
                Error.NotFound("User.NotFound", $"User {id} does not exist.");

            public static Error RoleAlreadyAssigned(Guid roleId) =>
                Error.Conflict("User.RoleAlreadyAssigned", $"The user already holds role
{roleId}.");

            public static Error RoleNotAssigned(Guid roleId) =>
                Error.NotFound("User.RoleNotAssigned", $"The user does not hold role {roleId}.");

            public static Error OverrideNotFound(Permission permission) =>
                Error.NotFound("User.OverrideNotFound", $"The user has no override for
'{permission}'.");

            public static readonly Error AlreadySuspended =
                Error.Conflict("User.AlreadySuspended", "The user is already suspended.");

            public static readonly Error NotSuspended =
                Error.Conflict("User.NotSuspended", "The user is not suspended.");

            public static readonly Error SuspensionReasonRequired =
                Error.Validation("User.SuspensionReasonRequired", "A reason is required to suspend
a user.");
        }
```

Points worth studying:

- **Time zones are validated at creation** with `TimeZoneInfo.TryFindSystemTimeZoneById`, which on .NET 6+ accepts IANA names (`Africa/Lagos`) on every operating system. A user with a bad zone would break every date on their security page; reject it at the door.
- **Every timestamp must be UTC.** `EnsureUtc` throws for any other `Kind`, because a local time slipping into the domain is a bug, not a user error. You will see what happens when that promise breaks on the way *out* of the database in Chapter 22.
- **One override per permission.** `SetOverride` replaces rather than adds: allowing and denying the same thing at once has no meaning.
- **Suspension carries a reason.** Suspending twice is a conflict, not a silent success, so an administrator who thinks they are suspending an active user learns that someone else already did.

6.4 AccessSubject: flattening a user for the rules

The access rules do not need a `User`. They need three facts: is this person active, what do they hold, what is denied to them. `AccessSubject` is that, and nothing more:

`src/Sentinel.Domain/Access/AccessSubject.cs`

```
        using Sentinel.Domain.Roles;
        using Sentinel.Domain.Users;

        namespace Sentinel.Domain.Access;

        public enum SubjectStatus
        {
            Anonymous,
            Active,
            Suspended
        }

        /// <summary>
        /// Everything the access rules need to know about the person asking, and
        /// nothing else. Flattening the user and their roles into this shape is what
        /// lets <see cref="PageAccessEvaluator"/> stay a pure function: no database,
        /// no clock, no HTTP context.
        /// </summary>
        public sealed class AccessSubject
        {
            private AccessSubject(SubjectStatus status, IReadOnlyCollection<Permission> granted,
IReadOnlyCollection<Permission> denied)
            {
                Status = status;
                Granted = granted;
                Denied = denied;
            }

            public SubjectStatus Status { get; }
            public IReadOnlyCollection<Permission> Granted { get; }
            public IReadOnlyCollection<Permission> Denied { get; }

            public static AccessSubject Anonymous { get; } = new(SubjectStatus.Anonymous, [], []);

            public static AccessSubject Create(SubjectStatus status, IEnumerable<Permission>
granted, IEnumerable<Permission> denied) =>
                new(status, granted.Distinct().ToList(), denied.Distinct().ToList());

            /// <summary>
            /// Builds the subject for <paramref name="user"/>. <paramref name="roles"/> may
            /// contain roles the user does not hold (only the user's own roles count),
            /// so callers can pass every role without filtering first.
            /// </summary>
            public static AccessSubject For(User user, IEnumerable<Role> roles)
            {
                ArgumentNullException.ThrowIfNull(user);
                ArgumentNullException.ThrowIfNull(roles);

                var heldRoleIds = user.RoleIds.ToHashSet();

                var granted = roles
                    .Where(role => heldRoleIds.Contains(role.Id))
                    .SelectMany(role => role.Permissions)
                    .Concat(user.Overrides.Where(o => o.Effect == PermissionEffect.Allow).Select(o
=> o.Permission));

                var denied = user.Overrides
                    .Where(o => o.Effect == PermissionEffect.Deny)
                    .Select(o => o.Permission);

                var status = user.Status == UserStatus.Active ? SubjectStatus.Active :
SubjectStatus.Suspended;
                return Create(status, granted, denied);
            }

            public bool IsDenied(Permission required) => Denied.Any(d => d.Covers(required));

            public bool IsGranted(Permission required) => Granted.Any(g => g.Covers(required));

            /// <summary>Deny always wins over allow, whichever role the allow came from.
</summary>
```

```
        public bool Has(Permission required) =>
            Status == SubjectStatus.Active && !IsDenied(required) && IsGranted(required);
    }
```

`For(user, roles)` accepts *every* role and keeps only the user's own. That makes the caller simpler (pass the list you have) and puts the filtering rule, the one thing that must not be got wrong, in one tested place.

`Has` encodes "deny wins": it is `!IsDenied && IsGranted`, in that order, and it is false for anyone who is not active.

`src/Sentinel.Domain/Access/PermissionEffect.cs`

```
namespace Sentinel.Domain.Access;

/// <summary>Whether a user-level override adds a permission or takes it away.</summary>
public enum PermissionEffect
{
    Allow,
    Deny
}
```

Chapter 7 · The Page Access Evaluator

This chapter is the long version of the skill test's second scenario. Read it twice: once for the code, once for the *process*, because the process is what you will reproduce in 40 minutes.

7.1 The process, before any code

Step 1 — restate the requirements as a numbered list. You have seen it already: §2.4, rules 1 to 8. In the test, write that list as a comment at the top of your class before writing anything else. It costs two minutes, and it is the first thing a reviewer reads.

Step 2 — list the edge cases and decide each one. For page access:

Edge case	Decision	Why
Page's parent does not exist	Deny, reason `OrphanedPage`	Fail closed
Parent cycle (`/a` → `/b` → `/a`)	Deny pages on the cycle (`CyclicHierarchy`), and pages below it as `ParentDenied`	Fail closed; no infinite loop
A page is its own parent	Impossible to create (`Page.Create` / `MoveUnder` refuse) **and** handled if it arrives anyway	Belt and braces
Two pages share an id	Throw `ArgumentException`	A programming error in the caller, not bad business data
Public page under a private parent	Allow	Public means public; the menu promotes it
Path with different case or a trailing slash	Normalise before comparing	`/Admin/` and `/admin` are one page
Unknown path	Return `null`	"No such page" is not "denied"
Very deep tree	Must not overflow or go quadratic beyond reason	Memoise decisions
Empty page list	Empty result	Not an error

Step 3 — pick the shape. A pure function: subject and pages in, decisions out. Every decision carries a *reason*, because "denied" with no reason becomes a support ticket (Chapter 19 builds the screen that shows them).

Step 4 — write it, then test each decision from step 2.

7.2 The supporting types

`src/Sentinel.Domain/Access/AccessReason.cs`

```
namespace Sentinel.Domain.Access;

/// <summary>
/// Why a page was allowed or denied. Every decision carries one, because
/// "you can't see this" without a reason is tomorrow's support ticket.
```

```
        /// </summary>
        public enum AccessReason
        {
            // Allowed
            PublicPage,
            AuthenticatedUser,
            PermissionGranted,

            // Denied
            NotAuthenticated,
            UserSuspended,
            MissingPermission,
            ExplicitlyDenied,
            ParentDenied,
            OrphanedPage,
            CyclicHierarchy
        }
```

`src/Sentinel.Domain/Access/PageAccessDecision.cs`

```
        using Sentinel.Domain.Pages;

        namespace Sentinel.Domain.Access;

        public sealed record PageAccessDecision(Page Page, bool IsAllowed, AccessReason Reason);
```

The page itself, and the path normalisation it relies on:

`src/Sentinel.Domain/Pages/PagePath.cs`

```
        using System.Text.RegularExpressions;

        namespace Sentinel.Domain.Pages;

        /// <summary>
        /// The canonical form of a page route: leading slash, lower case, no
        /// trailing slash (except the root itself), no empty segments.
        /// <c>" /Admin/Users/ "</c> becomes <c>"/admin/users"</c>.
        /// </summary>
        public static partial class PagePath
        {
            public const string Root = "/";

            /// <summary>The normalised path, or null when <paramref name="raw"/> cannot be a page
path.</summary>
            public static string? TryNormalise(string? raw)
            {
                if (string.IsNullOrWhiteSpace(raw))
                    return null;

                var path = raw.Trim().ToLowerInvariant();

                if (!path.StartsWith('/'))
                    return null;

                if (path.Length > 1)
                    path = path.TrimEnd('/');

                if (path.Length == 0)
                    return Root; // the input was "///"

                return path == Root || SegmentedPath().IsMatch(path) ? path : null;
            }

            /// <summary>"/admin/users" -> "/admin"; "/admin" -> "/"; "/" -> null.</summary>
            public static string? ParentOf(string normalisedPath)
```

```
            {
                if (normalisedPath == Root)
                    return null;

                var lastSlash = normalisedPath.LastIndexOf('/');
                return lastSlash == 0 ? Root : normalisedPath[..lastSlash];
            }

            // One or more "/segment" where a segment is [a-z0-9-]+ and has no empty parts.
            [GeneratedRegex(@"^(/[a-z0-9][a-z0-9-]*)+$")]
            private static partial Regex SegmentedPath();
        }
```

src/Sentinel.Domain/Pages/Page.cs

```
        using Sentinel.Domain.Access;
        using Sentinel.Domain.Common;

        namespace Sentinel.Domain.Pages;

        /// <summary>
        /// A screen in the portal. Pages form a tree through <see cref="ParentId"/>;
        /// the tree is the navigation menu, and it is also the access model, because
        /// a child page is only reachable through its parent.
        /// </summary>
        public sealed class Page : AggregateRoot
        {
            public const int MaxTitleLength = 100;

            private Page(Guid id, string path, string title, Guid? parentId, Permission?
requiredPermission, bool isPublic, int sortOrder)
                : base(id)
            {
                Path = path;
                Title = title;
                ParentId = parentId;
                RequiredPermission = requiredPermission;
                IsPublic = isPublic;
                SortOrder = sortOrder;
            }

            // For EF Core.
            private Page() { }

            /// <summary>Always normalised. See <see cref="PagePath"/>.</summary>
            public string Path { get; private set; } = null!;

            public string Title { get; private set; } = null!;

            public Guid? ParentId { get; private set; }

            /// <summary>Null means "any signed-in user"; it does not mean "anyone".</summary>
            public Permission? RequiredPermission { get; private set; }

            /// <summary>Visible without signing in. A public page cannot also require a
permission.</summary>
            public bool IsPublic { get; private set; }

            public int SortOrder { get; private set; }

            public static Result<Page> Create(
                string path,
                string title,
                Guid? parentId = null,
                Permission? requiredPermission = null,
                bool isPublic = false,
                int sortOrder = 0,
                Guid? id = null)
            {
```

```
                var normalisedPath = PagePath.TryNormalise(path);
                if (normalisedPath is null)
                    return PageErrors.InvalidPath(path);

                if (string.IsNullOrWhiteSpace(title))
                    return PageErrors.TitleRequired;

                if (title.Trim().Length > MaxTitleLength)
                    return PageErrors.TitleTooLong;

                if (isPublic && requiredPermission is not null)
                    return PageErrors.PublicPageWithPermission;

                var pageId = id ?? Guid.CreateVersion7();
                if (parentId == pageId)
                    return PageErrors.OwnParent;

                return new Page(pageId, normalisedPath, title.Trim(), parentId,
requiredPermission, isPublic, sortOrder);
            }

            /// <summary>
            /// Re-parents the page. Only the trivial cycle (a page under itself) is
            /// caught here; longer cycles need the whole tree, so the evaluator
            /// treats them as corrupt data rather than trusting this method alone.
            /// </summary>
            public Result MoveUnder(Guid? parentId)
            {
                if (parentId == Id)
                    return PageErrors.OwnParent;

                ParentId = parentId;
                return Result.Success();
            }

            public Result RequirePermission(Permission? permission)
            {
                if (IsPublic && permission is not null)
                    return PageErrors.PublicPageWithPermission;

                RequiredPermission = permission;
                return Result.Success();
            }
        }

        public static class PageErrors
        {
            public static Error InvalidPath(string? path) =>
                Error.Validation("Page.InvalidPath", $"'{path}' is not a valid page path. Paths
look like '/reports/finance'.");

            public static readonly Error TitleRequired =
                Error.Validation("Page.TitleRequired", "A page title is required.");

            public static readonly Error TitleTooLong =
                Error.Validation("Page.TitleTooLong", $"A page title cannot exceed
{Page.MaxTitleLength} characters.");

            public static readonly Error PublicPageWithPermission =
                Error.Validation("Page.PublicPageWithPermission", "A public page cannot require a
permission.");

            public static readonly Error OwnParent =
                Error.Validation("Page.OwnParent", "A page cannot be its own parent.");

            public static Error DuplicatePath(string path) =>
                Error.Conflict("Page.DuplicatePath", $"A page already exists at '{path}'.");

            public static Error ParentNotFound(Guid parentId) =>
                Error.NotFound("Page.ParentNotFound", $"Parent page {parentId} does not exist.");

            public static Error NotFound(string path) =>
```

```
                Error.NotFound("Page.NotFound", $"No page exists at '{path}'.");
        }
```

7.3 The evaluator

src/Sentinel.Domain/Access/PageAccessEvaluator.cs

```
        using Sentinel.Domain.Pages;

        namespace Sentinel.Domain.Access;

        /// <summary>
        /// Decides which pages a subject may open. The rules, in the order they apply:
        /// <list type="number">
        ///   <item>A page that is part of a parent cycle is denied (the data is corrupt; fail
closed).</item>
        ///   <item>A public page is allowed for everyone, signed in or not.</item>
        ///   <item>Anonymous and suspended subjects are denied everything else.</item>
        ///   <item>A page whose parent is missing is denied (fail closed).</item>
        ///   <item>A page whose parent is denied is denied: you cannot reach a room through a
locked door.</item>
        ///   <item>A page with no required permission is allowed for any active user.</item>
        ///   <item>An explicit deny covering the required permission wins.</item>
        ///   <item>Otherwise the page is allowed if any grant covers the required permission.
</item>
        /// </list>
        /// </summary>
        /// <remarks>
        /// The evaluator is a pure function of its inputs. It never throws on bad
        /// data from the database (orphans and cycles become denials with a reason)
        /// because a navigation menu that crashes is worse than one that hides a page.
        /// </remarks>
        public sealed class PageAccessEvaluator
        {
            private readonly AccessSubject _subject;
            private readonly Dictionary<Guid, Page> _pagesById = [];
            private readonly Dictionary<Guid, PageAccessDecision> _decisions = [];

            private PageAccessEvaluator(AccessSubject subject, IEnumerable<Page> pages)
            {
                _subject = subject;
                foreach (var page in pages)
                {
                    if (!_pagesById.TryAdd(page.Id, page))
                        throw new ArgumentException($"Page id {page.Id} appears more than once.",
nameof(pages));
                }
            }

            /// <summary>One decision per page, in the order the pages were supplied.</summary>
            public static IReadOnlyList<PageAccessDecision> EvaluateAll(AccessSubject subject,
IEnumerable<Page> pages)
            {
                ArgumentNullException.ThrowIfNull(subject);
                ArgumentNullException.ThrowIfNull(pages);

                var pageList = pages.ToList();
                var evaluator = new PageAccessEvaluator(subject, pageList);
                return pageList.Select(evaluator.Decide).ToList();
            }

            /// <summary>
            /// The decision for the page at <paramref name="path"/>, or null when no
            /// such page exists. Paths are compared after normalisation, so
            /// <c>/Admin/Users/</c> and <c>/admin/users</c> are the same page.
            /// </summary>
            public static PageAccessDecision? EvaluatePath(AccessSubject subject,
```

```
IEnumerable<Page> pages, string path)
            {
                ArgumentNullException.ThrowIfNull(subject);
                ArgumentNullException.ThrowIfNull(pages);

                var normalised = PagePath.TryNormalise(path);
                if (normalised is null)
                    return null;

                var pageList = pages.ToList();
                var target = pageList.FirstOrDefault(p => p.Path == normalised);
                return target is null ? null : new PageAccessEvaluator(subject,
pageList).Decide(target);
            }

            private PageAccessDecision Decide(Page page)
            {
                if (_decisions.TryGetValue(page.Id, out var known))
                    return known;

                var decision = DecideUncached(page);
                _decisions[page.Id] = decision;
                return decision;
            }

            private PageAccessDecision DecideUncached(Page page)
            {
                if (IsOnCycle(page))
                    return Deny(page, AccessReason.CyclicHierarchy);

                if (page.IsPublic)
                    return Allow(page, AccessReason.PublicPage);

                switch (_subject.Status)
                {
                    case SubjectStatus.Anonymous: return Deny(page,
AccessReason.NotAuthenticated);
                    case SubjectStatus.Suspended: return Deny(page, AccessReason.UserSuspended);
                }

                if (page.ParentId is { } parentId)
                {
                    if (!_pagesById.TryGetValue(parentId, out var parent))
                        return Deny(page, AccessReason.OrphanedPage);

                    // Safe to recurse: this page is not on a cycle, so walking up
                    // either reaches a root or reaches a page that *is* on a cycle,
                    // which returns above without recursing further.
                    if (!Decide(parent).IsAllowed)
                        return Deny(page, AccessReason.ParentDenied);
                }

                if (page.RequiredPermission is not { } required)
                    return Allow(page, AccessReason.AuthenticatedUser);

                if (_subject.IsDenied(required))
                    return Deny(page, AccessReason.ExplicitlyDenied);

                return _subject.IsGranted(required)
                    ? Allow(page, AccessReason.PermissionGranted)
                    : Deny(page, AccessReason.MissingPermission);
            }

            /// <summary>True when following parents from <paramref name="page"/> leads back to
it.</summary>
            private bool IsOnCycle(Page page)
            {
                var seen = new HashSet<Guid>();
                var current = page;

                while (current.ParentId is { } parentId && _pagesById.TryGetValue(parentId, out
var parent))
```

```
                {
                    if (parent.Id == page.Id)
                        return true;

                    // A cycle further up that does not include this page: the page
                    // hangs below the cycle rather than sitting on it. ParentDenied
                    // will deal with it when its ancestors are decided.
                    if (!seen.Add(parent.Id))
                        return false;

                    current = parent;
                }

                return false;
            }

            private static PageAccessDecision Allow(Page page, AccessReason reason) => new(page,
true, reason);
            private static PageAccessDecision Deny(Page page, AccessReason reason) => new(page,
false, reason);
        }
```

Read `DecideUncached` from top to bottom and compare it with the rules in §2.4. They are in the same order. That is not an accident; it is the cheapest way to make a reviewer trust your code.

Why memoise?

`Decide` caches each decision in `_decisions`. Without the cache, deciding every page in a tree means deciding each ancestor again for every descendant: a ten-level-deep branch decides its root ten times. With it, each page is decided once. The test `A_very_deep_tree_does_not_overflow_the_stack` builds a 2,000-level chain to prove the recursion depth is fine for any realistic menu.

Why is recursion safe here?

Recursion over user data is a classic way to hang a server: a cycle in the data becomes infinite recursion. The evaluator checks `IsOnCycle(page)` *before* recursing into the parent. If the page is on a cycle it returns immediately. If it is not, walking up must reach either a root or a page that *is* on a cycle, and that page returns without recursing. Either way the recursion terminates. The comment in the code says exactly this, because the next reader will wonder.

Why does `IsOnCycle` distinguish “on” from “below”?

Given `/a → /b → /a` and `/c → /a`, page `/c` is not *on* the cycle, it hangs *below* it. Reporting `/c` as `CyclicHierarchy` would send whoever investigates to the wrong page. Reporting it as `ParentDenied` points at its parent, which points at the cycle. Good reasons lead you to the cause.

Why throw for duplicate ids but not for orphans?

An orphan is bad *data*: it came from the database and the evaluator must cope. Duplicate ids mean the *caller* built a wrong list, which is a bug in our code. Bugs should be loud; data problems should be safe.

7.4 The tests

The test data builders keep each test focused on what is different about it:

`tests/Sentinel.Domain.Tests/TestData.cs`

```
        using Sentinel.Domain.Access;
        using Sentinel.Domain.Pages;
        using Sentinel.Domain.Security;

        namespace Sentinel.Domain.Tests;

        /// <summary>Small, readable builders. A test should say what is different about its data,
and nothing else.</summary>
        internal static class Places
        {
            public static readonly GeoLocation Lagos = GeoLocation.Create("NG", "Lagos", 6.5244,
3.3792);
            public static readonly GeoLocation Ikeja = GeoLocation.Create("NG", "Ikeja", 6.6018,
3.3515); // ~9 km from Lagos
            public static readonly GeoLocation Abuja = GeoLocation.Create("NG", "Abuja", 9.0765,
7.3986);
            public static readonly GeoLocation London = GeoLocation.Create("GB", "London",
51.5072, -0.1276);
            public static readonly GeoLocation NewYork = GeoLocation.Create("US", "New York",
40.7128, -74.0060);
            public static readonly GeoLocation Frankfurt = GeoLocation.Create("DE", "Frankfurt",
50.1109, 8.6821);
        }

        internal static class Agents
        {
            public const string Laptop = "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36";
            public const string Phone = "Mozilla/5.0 (iPhone; CPU iPhone OS 18_6 like Mac OS X)
AppleWebKit/605.1.15 (KHTML, like Gecko) Version/18.6 Mobile/15E148 Safari/604.1";
        }

        /// <summary>Builds one user's sign-in history in time order, relative to a fixed "now".
</summary>
        internal sealed class History
        {
            public static readonly DateTime Now = new(2026, 10, 25, 12, 0, 0, DateTimeKind.Utc);

            private readonly List<SignInAttempt> _attempts = [];
            private readonly Guid _userId = Guid.NewGuid();

            public IReadOnlyList<SignInAttempt> Attempts => _attempts;

            public SignInAttempt Last => _attempts[^1];

            public History Success(DateTime atUtc, GeoLocation? where = null, string agent =
Agents.Laptop) =>
                Add(atUtc, where ?? Places.Lagos, agent, succeeded: true);

            public History Failure(DateTime atUtc, GeoLocation? where = null, string agent =
Agents.Laptop) =>
                Add(atUtc, where ?? Places.Lagos, agent, succeeded: false);

            /// <summary>A month of ordinary daily sign-ins from Lagos on the laptop, ending a
week ago.</summary>
            public History WithOrdinaryBaseline(int days = 30)
            {
                for (var day = days + 7; day > 7; day--)
                    Success(Now.AddDays(-day).AddHours(-3));
                return this;
            }

            public History Failures(DateTime startUtc, int count, TimeSpan spacing, GeoLocation?
where = null)
            {
                for (var i = 0; i < count; i++)
                    Failure(startUtc + spacing * i, where);
                return this;
            }

            private History Add(DateTime atUtc, GeoLocation where, string agent, bool succeeded)
```

```
            {
                _attempts.Add(SignInAttempt.Record(
                    _userId, atUtc, "102.89.34.10", where, agent, succeeded,
                    succeeded ? null : SignInFailureReason.WrongPassword).Value);
                return this;
            }
        }

        /// <summary>Builds a page tree with readable names instead of Guids.</summary>
        internal sealed class PageTree
        {
            private readonly Dictionary<string, Page> _byPath = [];

            public IReadOnlyList<Page> Pages => _byPath.Values.ToList();

            public Page this[string path] => _byPath[path];

            public PageTree Add(string path, string? parentPath = null, string? permission = null,
bool isPublic = false)
            {
                var parentId = parentPath is null ? (Guid?)null : _byPath[parentPath].Id;
                var title = path == "/" ? "Home" : path.TrimStart('/');
                var page = Page.Create(path, title, parentId,
                    permission is null ? null : Permission.From(permission), isPublic).Value;
                _byPath[page.Path] = page;
                return this;
            }

            /// <summary>The standard tree most tests share: / > reports > finance > export, plus
public /help.</summary>
            public static PageTree Standard() => new PageTree()
                .Add("/")
                .Add("/reports", "/", "reports.view")
                .Add("/reports/finance", "/reports", "reports.finance")
                .Add("/reports/finance/export", "/reports/finance", "reports.finance.export")
                .Add("/reports/operations", "/reports", "reports.operations")
                .Add("/account", "/")
                .Add("/help", isPublic: true);
        }

        internal static class Subjects
        {
            public static AccessSubject Active(params string[] granted) =>
                AccessSubject.Create(SubjectStatus.Active, granted.Select(Permission.From), []);

            public static AccessSubject ActiveWithDenies(string[] granted, string[] denied) =>
                AccessSubject.Create(SubjectStatus.Active, granted.Select(Permission.From),
denied.Select(Permission.From));

            public static AccessSubject Suspended(params string[] granted) =>
                AccessSubject.Create(SubjectStatus.Suspended, granted.Select(Permission.From),
[]);
        }
```

And the tests themselves. Read the names first: together they are the specification.

`tests/Sentinel.Domain.Tests/Access/PageAccessEvaluatorTests.cs`

```
        using Sentinel.Domain.Access;
        using Sentinel.Domain.Pages;

        namespace Sentinel.Domain.Tests.Access;

        public class PageAccessEvaluatorTests
        {
            private static PageAccessDecision Decide(AccessSubject subject, PageTree tree, string
path) =>
                PageAccessEvaluator.EvaluatePath(subject, tree.Pages, path)
                ?? throw new InvalidOperationException($"No page at {path}");
```

```
            // ---- The happy paths ----

            [Fact]
            public void Page_without_a_permission_is_allowed_for_any_active_user()
            {
                var decision = Decide(Subjects.Active(), PageTree.Standard(), "/account");

                Assert.True(decision.IsAllowed);
                Assert.Equal(AccessReason.AuthenticatedUser, decision.Reason);
            }

            [Fact]
            public void Page_is_allowed_when_a_grant_covers_its_permission()
            {
                var decision = Decide(Subjects.Active("reports.view", "reports.finance"),
PageTree.Standard(), "/reports/finance");

                Assert.True(decision.IsAllowed);
                Assert.Equal(AccessReason.PermissionGranted, decision.Reason);
            }

            [Fact]
            public void Wildcard_grant_covers_every_page_beneath_it()
            {
                var subject = Subjects.Active("reports.view", "reports.*");

                Assert.True(Decide(subject, PageTree.Standard(), "/reports/finance").IsAllowed);
                Assert.True(Decide(subject, PageTree.Standard(),
"/reports/finance/export").IsAllowed);
                Assert.True(Decide(subject, PageTree.Standard(),
"/reports/operations").IsAllowed);
            }

            [Fact]
            public void Wildcard_grant_does_not_cover_the_parent_permission_itself()
            {
                // A landing page that requires "reports" itself. The grant covers
                // everything beneath reports, but not reports, and the children sit
                // behind it. (In the standard tree the landing page requires
                // "reports.view", which reports.* does cover. Writing this test the
                // first time against that tree is how we learnt the difference.)
                var tree = new PageTree()
                    .Add("/reports", permission: "reports")
                    .Add("/reports/finance", "/reports", "reports.finance");
                var subject = Subjects.Active("reports.*");

                Assert.Equal(AccessReason.MissingPermission, Decide(subject, tree,
"/reports").Reason);
                Assert.Equal(AccessReason.ParentDenied, Decide(subject, tree,
"/reports/finance").Reason);
            }

            [Fact]
            public void Superuser_wildcard_allows_everything()
            {
                var decisions = PageAccessEvaluator.EvaluateAll(Subjects.Active("*"),
PageTree.Standard().Pages);

                Assert.All(decisions, d => Assert.True(d.IsAllowed));
            }

            // ---- Denials, each with the right reason ----

            [Fact]
            public void Page_is_denied_when_no_grant_covers_its_permission()
            {
                var decision = Decide(Subjects.Active("reports.view"), PageTree.Standard(),
"/reports/finance");

                Assert.False(decision.IsAllowed);
                Assert.Equal(AccessReason.MissingPermission, decision.Reason);
```

```
            }

            [Fact]
            public void Explicit_deny_beats_a_role_grant()
            {
                var subject = Subjects.ActiveWithDenies(
                    granted: ["reports.view", "reports.finance", "reports.finance.*"],
                    denied: ["reports.finance.export"]);

                Assert.True(Decide(subject, PageTree.Standard(), "/reports/finance").IsAllowed);
                Assert.Equal(AccessReason.ExplicitlyDenied, Decide(subject, PageTree.Standard(),
"/reports/finance/export").Reason);
            }

            [Fact]
            public void Wildcard_deny_beats_a_concrete_grant()
            {
                var subject = Subjects.ActiveWithDenies(granted: ["reports.view",
"reports.finance"], denied: ["reports.*"]);

                // reports.* covers reports.view too, so the deny bites at the top of
                // the branch and everything beneath it follows.
                Assert.Equal(AccessReason.ExplicitlyDenied, Decide(subject, PageTree.Standard(),
"/reports").Reason);
                Assert.Equal(AccessReason.ParentDenied, Decide(subject, PageTree.Standard(),
"/reports/finance").Reason);
            }

            [Fact]
            public void
Child_is_denied_when_its_parent_is_denied_even_if_the_child_permission_is_held()
            {
                // Holds the finance permission but not reports.view.
                var decision = Decide(Subjects.Active("reports.finance"), PageTree.Standard(),
"/reports/finance");

                Assert.False(decision.IsAllowed);
                Assert.Equal(AccessReason.ParentDenied, decision.Reason);
            }

            [Fact]
            public void Denial_propagates_down_every_level()
            {
                var decision = Decide(Subjects.Active("reports.finance",
"reports.finance.export"), PageTree.Standard(), "/reports/finance/export");

                Assert.Equal(AccessReason.ParentDenied, decision.Reason);
            }

            // ---- Who is asking ----

            [Fact]
            public void Anonymous_subject_sees_public_pages_only()
            {
                var decisions = PageAccessEvaluator.EvaluateAll(AccessSubject.Anonymous,
PageTree.Standard().Pages);

                var allowed = decisions.Where(d => d.IsAllowed).Select(d => d.Page.Path);
                Assert.Equal(["/help"], allowed);
                Assert.All(decisions.Where(d => !d.IsAllowed), d =>
Assert.Equal(AccessReason.NotAuthenticated, d.Reason));
            }

            [Fact]
            public void
Suspended_subject_loses_everything_but_public_pages_whatever_they_were_granted()
            {
                var decisions = PageAccessEvaluator.EvaluateAll(Subjects.Suspended("*"),
PageTree.Standard().Pages);

                Assert.Equal(["/help"], decisions.Where(d => d.IsAllowed).Select(d =>
d.Page.Path));
```

```
                Assert.All(decisions.Where(d => !d.IsAllowed), d =>
Assert.Equal(AccessReason.UserSuspended, d.Reason));
            }

            [Fact]
            public void Public_page_under_a_denied_parent_is_still_allowed()
            {
                var tree = PageTree.Standard().Add("/reports/glossary", "/reports", isPublic:
true);

                var decision = Decide(AccessSubject.Anonymous, tree, "/reports/glossary");

                Assert.True(decision.IsAllowed);
                Assert.Equal(AccessReason.PublicPage, decision.Reason);
            }

            // ---- Bad data: fail closed, never throw ----

            [Fact]
            public void Orphaned_page_is_denied()
            {
                var orphan = Page.Create("/lost", "Lost", parentId: Guid.NewGuid()).Value;

                var decision = PageAccessEvaluator.EvaluateAll(Subjects.Active("*"),
[orphan]).Single();

                Assert.False(decision.IsAllowed);
                Assert.Equal(AccessReason.OrphanedPage, decision.Reason);
            }

            [Fact]
            public void
Pages_on_a_parent_cycle_are_denied_and_pages_below_it_are_denied_as_children()
            {
                var a = Page.Create("/a", "A").Value;
                var b = Page.Create("/b", "B", parentId: a.Id).Value;
                var c = Page.Create("/c", "C", parentId: b.Id).Value;
                a.MoveUnder(b.Id); // a -> b -> a

                var decisions = PageAccessEvaluator.EvaluateAll(Subjects.Active("*"), [a, b, c])
                    .ToDictionary(d => d.Page.Path, d => d.Reason);

                Assert.Equal(AccessReason.CyclicHierarchy, decisions["/a"]);
                Assert.Equal(AccessReason.CyclicHierarchy, decisions["/b"]);
                Assert.Equal(AccessReason.ParentDenied, decisions["/c"]);
            }

            [Fact]
            public void A_page_cannot_be_made_its_own_parent()
            {
                var page = Page.Create("/a", "A").Value;

                Assert.True(page.MoveUnder(page.Id).IsFailure);
                Assert.Null(page.ParentId);
            }

            [Fact]
            public void Duplicate_page_ids_are_a_programming_error()
            {
                var page = Page.Create("/a", "A").Value;

                Assert.Throws<ArgumentException>(() =>
PageAccessEvaluator.EvaluateAll(Subjects.Active(), [page, page]));
            }

            [Fact]
            public void A_very_deep_tree_does_not_overflow_the_stack()
            {
                var tree = new PageTree().Add("/l0");
                for (var i = 1; i < 2_000; i++)
                    tree.Add($"/l{i}", $"/l{i - 1}");
```

```
            var decisions = PageAccessEvaluator.EvaluateAll(Subjects.Active(), tree.Pages);

            Assert.Equal(2_000, decisions.Count);
            Assert.All(decisions, d => Assert.True(d.IsAllowed));
        }

        // ---- The API's contract ----

        [Fact]
        public void EvaluateAll_returns_one_decision_per_page_in_input_order()
        {
            var pages = PageTree.Standard().Pages.Reverse().ToList();

            var decisions = PageAccessEvaluator.EvaluateAll(Subjects.Active(), pages);

            Assert.Equal(pages.Select(p => p.Id), decisions.Select(d => d.Page.Id));
        }

        [Fact]
        public void EvaluateAll_with_no_pages_returns_nothing()
        {
            Assert.Empty(PageAccessEvaluator.EvaluateAll(Subjects.Active("*"), []));
        }

        [Theory]
        [InlineData("/Reports/Finance")]
        [InlineData("/reports/finance/")]
        [InlineData("  /reports/finance  ")]
        public void EvaluatePath_normalises_the_path(string path)
        {
            var decision = PageAccessEvaluator.EvaluatePath(Subjects.Active("reports.view",
"reports.finance"), PageTree.Standard().Pages, path);

            Assert.NotNull(decision);
            Assert.Equal("/reports/finance", decision.Page.Path);
        }

        [Theory]
        [InlineData("/does-not-exist")]
        [InlineData("reports")]    // not a path
        [InlineData("")]
        [InlineData("/a//b")]
        public void EvaluatePath_returns_null_for_unknown_or_invalid_paths(string path)
        {
            Assert.Null(PageAccessEvaluator.EvaluatePath(Subjects.Active("*"),
PageTree.Standard().Pages, path));
        }
    }
```

Two of these tests were wrong the first time they were written, and the evaluator was right. The first draft of `Wildcard_grant_does_not_cover_the_parent_permission_itself` used the standard tree, where `/reports` requires `reports.view`, and asserted that `reports.*` would be refused there. But `reports.*` *does* cover `reports.view`. The failing test was a misunderstanding of my own rule, caught in seconds because the rule was written down and the test was precise. The comment in the test records it. This happens in timed tests too: when a test you wrote fails, re-read the requirement before you touch the code.

7.5 The same problem in 40 minutes

In the test you will not have `Result`, `Permission` or a page tree class. You will have whatever input types the scenario gives you. The *shape* of the answer transfers directly:

1. A comment with the requirements and your edge-case decisions.
2. Index the input once (`Dictionary<string, Page>` by normalised path).

3. One small function per rule (`ParentAllows`, `RolesAllow`).
4. Memoise per user, guard against cycles with a `visiting` set.
5. A test per decision.

Kata 1 (`katas/PageAccessKata`, Chapter 25) is exactly that, with a model answer of about a hundred lines. Do it against the clock at least twice.

Chapter 8 · Sign-ins and the Compromise Detector

The skill test's third scenario is an Angular one: *show* a user when their account may be compromised. Showing is only honest if the verdict is right, so Sentinel computes it on the server, in the domain, where it can be tested exhaustively. This chapter builds that verdict. Chapters 17 and 18 show it.

8.1 Where a sign-in came from

`src/Sentinel.Domain/Security/GeoLocation.cs`

```
        namespace Sentinel.Domain.Security;

        /// <summary>
        /// Where a sign-in appeared to come from, as resolved from its IP address.
        /// IP geolocation is approximate: city-level at best, and wrong for VPNs.
        /// Treat it as evidence, never as proof.
        /// </summary>
        public sealed record GeoLocation
        {
            public const string UnknownCountryCode = "ZZ"; // ISO 3166 user-assigned "unknown"
            private const double EarthRadiusKm = 6371.0;

            private GeoLocation(string countryCode, string city, double? latitude, double?
longitude)
            {
                CountryCode = countryCode;
                City = city;
                Latitude = latitude;
                Longitude = longitude;
            }

            public string CountryCode { get; }
            public string City { get; }
            public double? Latitude { get; }
            public double? Longitude { get; }

            public bool IsKnown => CountryCode != UnknownCountryCode;
            public bool HasCoordinates => Latitude.HasValue && Longitude.HasValue;

            public static GeoLocation Unknown { get; } = new(UnknownCountryCode, "Unknown", null,
null);

            public static GeoLocation Create(string countryCode, string city, double latitude,
double longitude)
            {
                if (string.IsNullOrWhiteSpace(countryCode) || countryCode.Trim().Length != 2)
                    throw new ArgumentException("Country code must be ISO 3166-1 alpha-2.",
nameof(countryCode));
                if (latitude is < -90 or > 90)
                    throw new ArgumentOutOfRangeException(nameof(latitude));
                if (longitude is < -180 or > 180)
                    throw new ArgumentOutOfRangeException(nameof(longitude));

                return new GeoLocation(countryCode.Trim().ToUpperInvariant(), city.Trim(),
latitude, longitude);
            }

            /// <summary>Rehydrates a stored location. Null coordinates mean "unknown".</summary>
            public static GeoLocation Restore(string countryCode, string city, double? latitude,
double? longitude) =>
                countryCode == UnknownCountryCode || latitude is null || longitude is null
                    ? Unknown
                    : Create(countryCode, city, latitude.Value, longitude.Value);
```

```
        /// <summary>
        /// Great-circle distance by the haversine formula, or null when either
        /// end has no coordinates. Accurate to about 0.5%, which is far better
        /// than the IP lookup that produced the coordinates.
        /// </summary>
        public double? DistanceKmTo(GeoLocation other)
        {
            if (!HasCoordinates || !other.HasCoordinates)
                return null;

            var lat1 = ToRadians(Latitude!.Value);
            var lat2 = ToRadians(other.Latitude!.Value);
            var dLat = lat2 - lat1;
            var dLon = ToRadians(other.Longitude!.Value - Longitude!.Value);

            var a = Math.Pow(Math.Sin(dLat / 2), 2) +
                    Math.Cos(lat1) * Math.Cos(lat2) * Math.Pow(Math.Sin(dLon / 2), 2);

            return 2 * EarthRadiusKm * Math.Asin(Math.Sqrt(a));
        }

        public override string ToString() => IsKnown ? ${"{City}, {CountryCode}" : "Unknown
location";

        private static double ToRadians(double degrees) => degrees * Math.PI / 180.0;
    }
```

Three decisions are encoded here:

- **Unknown is a value**, not `null`. `GeoLocation.Unknown` (country `ZZ`, the ISO "user-assigned" code) can be passed around, displayed and compared without null checks, and it has no coordinates, so distance calculations involving it return `null` instead of measuring from the Gulf of Guinea at latitude 0, longitude 0.
- **Distance is the haversine formula.** Accurate to about half a percent, which is far better than IP geolocation itself.
- **The summary comment says what the data is worth**: evidence, never proof. A VPN puts a Lagos user in Amsterdam.

The device is identified by hashing the user agent:

`src/Sentinel.Domain/Security/DeviceFingerprint.cs`

```
using System.Security.Cryptography;
using System.Text;

namespace Sentinel.Domain.Security;

/// <summary>
/// A short, stable identifier for "the same browser on the same machine".
/// Real products combine many signals; we hash the user agent, which is
/// enough to demonstrate the rule and honest about being weak evidence.
/// </summary>
public static class DeviceFingerprint
{
    public static string FromUserAgent(string? userAgent)
    {
        var normalised = (userAgent ?? string.Empty).Trim().ToLowerInvariant();
        var hash = SHA256.HashData(Encoding.UTF8.GetBytes(normalised));
        return Convert.ToHexStringLower(hash)[..16];
    }

    /// <summary>Just enough of a user agent to be recognisable in a sentence.</summary>
    public static string Describe(string userAgent)
    {
        if (string.IsNullOrWhiteSpace(userAgent))
```

```
                    return "unknown browser";

                string[] known = ["Edg", "Chrome", "Firefox", "Safari", "curl", "python-
requests"];
                var browser = known.FirstOrDefault(k => userAgent.Contains(k,
StringComparison.OrdinalIgnoreCase));
                var os = userAgent.Contains("Windows", StringComparison.OrdinalIgnoreCase) ?
"Windows"
                       : userAgent.Contains("Android", StringComparison.OrdinalIgnoreCase) ?
"Android"
                       : userAgent.Contains("iPhone", StringComparison.OrdinalIgnoreCase) ?
"iPhone"
                       : userAgent.Contains("Mac OS", StringComparison.OrdinalIgnoreCase) ?
"macOS"
                       : userAgent.Contains("Linux", StringComparison.OrdinalIgnoreCase) ? "Linux"
                       : null;

                return (browser == "Edg" ? "Edge" : browser, os) switch
                {
                    (null, null) => "unrecognised client",
                    (null, { } o) => o,
                    ({ } b, null) => b,
                    ({ } b, { } o) => $"{b} on {o}"
                };
            }
        }
```

8.2 The sign-in attempt

`src/Sentinel.Domain/Security/SignInAttempt.cs`

```
        using Sentinel.Domain.Common;

        namespace Sentinel.Domain.Security;

        public enum SignInFailureReason
        {
            WrongPassword,
            WrongSecondFactor,
            AccountLocked,
            AccountSuspended
        }

        /// <summary>
        /// One attempt to sign in, successful or not. Attempts are immutable facts:
        /// once recorded they are never edited, only read.
        /// </summary>
        public sealed class SignInAttempt : AggregateRoot
        {
            public const int MaxUserAgentLength = 512;

            private SignInAttempt(
                Guid id, Guid userId, DateTime occurredAtUtc, string ipAddress, GeoLocation
location,
                string userAgent, bool succeeded, SignInFailureReason? failureReason) : base(id)
            {
                UserId = userId;
                OccurredAtUtc = occurredAtUtc;
                IpAddress = ipAddress;
                CountryCode = location.CountryCode;
                City = location.City;
                Latitude = location.Latitude;
                Longitude = location.Longitude;
                UserAgent = userAgent;
                DeviceId = DeviceFingerprint.FromUserAgent(userAgent);
                Succeeded = succeeded;
                FailureReason = failureReason;
```

```
            }

            // For EF Core.
            private SignInAttempt() { }

            public Guid UserId { get; private set; }

            /// <summary>
            /// Always <see cref="DateTimeKind.Utc"/>. Infrastructure owns the promise
            /// that this survives a round trip through the database; see
            /// <c>UtcDateTimeConverter</c> and Chapter 22 for what happens when it does not.
            /// </summary>
            public DateTime OccurredAtUtc { get; private set; }

            public string IpAddress { get; private set; } = null!;

            // Stored flat so that EF Core maps four plain columns. The value object
            // is rebuilt on read; it has no identity, so nothing is lost.
            public string CountryCode { get; private set; } = null!;
            public string City { get; private set; } = null!;
            public double? Latitude { get; private set; }
            public double? Longitude { get; private set; }

            public GeoLocation Location => GeoLocation.Restore(CountryCode, City, Latitude,
Longitude);

            public string UserAgent { get; private set; } = null!;
            public string DeviceId { get; private set; } = null!;
            public bool Succeeded { get; private set; }
            public SignInFailureReason? FailureReason { get; private set; }

            public static Result<SignInAttempt> Record(
                Guid userId,
                DateTime occurredAtUtc,
                string ipAddress,
                GeoLocation location,
                string? userAgent,
                bool succeeded,
                SignInFailureReason? failureReason = null,
                Guid? id = null)
            {
                ArgumentNullException.ThrowIfNull(location);

                if (occurredAtUtc.Kind != DateTimeKind.Utc)
                    return SignInAttemptErrors.TimestampNotUtc;

                if (string.IsNullOrWhiteSpace(ipAddress) ||
!System.Net.IPAddress.TryParse(ipAddress.Trim(), out _))
                    return SignInAttemptErrors.InvalidIpAddress(ipAddress);

                if (succeeded && failureReason is not null)
                    return SignInAttemptErrors.SuccessWithFailureReason;

                if (!succeeded && failureReason is null)
                    return SignInAttemptErrors.FailureWithoutReason;

                var agent = (userAgent ?? string.Empty).Trim();
                if (agent.Length > MaxUserAgentLength)
                    agent = agent[..MaxUserAgentLength];

                return new SignInAttempt(
                    id ?? Guid.CreateVersion7(), userId, occurredAtUtc, ipAddress.Trim(),
location,
                    agent, succeeded, failureReason);
            }
        }

        public static class SignInAttemptErrors
        {
            public static readonly Error TimestampNotUtc =
                Error.Validation("SignIn.TimestampNotUtc", "Sign-in timestamps must be UTC (end
the value with 'Z' or an offset).");
```

```
            public static Error InvalidIpAddress(string? ip) =>
                Error.Validation("SignIn.InvalidIpAddress", $"'{ip}' is not a valid IP address.");

            public static readonly Error SuccessWithFailureReason =
                Error.Validation("SignIn.SuccessWithFailureReason", "A successful sign-in cannot
have a failure reason.");

            public static readonly Error FailureWithoutReason =
                Error.Validation("SignIn.FailureWithoutReason", "A failed sign-in must say why it
failed.");
        }
```

An attempt is an immutable fact: there are no methods that change it. Its invariants are checked once, in `Record`: the timestamp is UTC, the IP parses, and a success has no failure reason while a failure must have one. The location is stored as four flat columns (Chapter 11 explains why) and rebuilt as a value object on read.

8.3 Findings, severity and the assessment

`src/Sentinel.Domain/Security/SecurityFinding.cs`

```
        namespace Sentinel.Domain.Security;

        public enum FindingKind
        {
            BruteForce,
            ImpossibleTravel,
            NewCountry,
            NewDevice
        }

        /// <summary>Ordered: a higher value is more serious, so <c>Max()</c> is meaningful.
</summary>
        public enum Severity
        {
            None = 0,
            Low = 1,
            Medium = 2,
            High = 3
        }

        /// <summary>
        /// One reason to think the account may be compromised.
        /// </summary>
        /// <param name="Key">
        /// Stable across recomputation (it is built from attempt ids, not from a
        /// counter), so the user can acknowledge it and the acknowledgement sticks.
        /// </param>
        /// <param name="OccurredAtUtc">When the suspicious attempt happened (the latest one, for
a burst).</param>
        /// <param name="AttemptIds">The sign-in attempts that are the evidence for this finding.
</param>
        public sealed record SecurityFinding(
            string Key,
            FindingKind Kind,
            Severity Severity,
            DateTime OccurredAtUtc,
            IReadOnlyList<Guid> AttemptIds,
            string Summary);

        /// <summary>The verdict for an account: the findings, and the worst of them.</summary>
        public sealed record SecurityAssessment(Severity RiskLevel, IReadOnlyList<SecurityFinding>
Findings)
        {
            public static SecurityAssessment Clear { get; } = new(Severity.None, []);
```

```
        /// <summary>Medium or above: worth interrupting the user about.</summary>
        public bool IsCompromiseSuspected => RiskLevel >= Severity.Medium;
    }
```

`Severity` has explicit numeric values so that `Max()` means "worst". A finding's `Key` is built from the id of the attempt that triggered it, so recomputing the assessment tomorrow produces the same key, and the user's "that was me" sticks.

`src/Sentinel.Domain/Security/DetectionPolicy.cs`

```
namespace Sentinel.Domain.Security;

/// <summary>
/// The thresholds the detector uses. They are data, not constants buried in
/// the rules, because the first thing a security team asks after launch is
/// "can we make it less noisy?", and the answer should be a config change.
/// </summary>
/// <param name="ReportingWindow">Only attempts newer than this produce findings.</param>
/// <param name="FailedAttemptThreshold">Failures within <paramref
name="FailedAttemptWindow"/> that count as a brute-force burst.</param>
/// <param name="FailedAttemptWindow">The sliding window for counting failures.</param>
/// <param name="MaxPlausibleSpeedKmh">Faster than this between two sign-ins is
"impossible travel". Airliners cruise near 900 km/h.</param>
/// <param name="MinTravelDistanceKm">Ignore jumps shorter than this; IP geolocation is
too coarse to judge them.</param>
public sealed record DetectionPolicy(
    TimeSpan ReportingWindow,
    int FailedAttemptThreshold,
    TimeSpan FailedAttemptWindow,
    double MaxPlausibleSpeedKmh,
    double MinTravelDistanceKm)
{
    public static DetectionPolicy Default { get; } = new(
        ReportingWindow: TimeSpan.FromDays(30),
        FailedAttemptThreshold: 5,
        FailedAttemptWindow: TimeSpan.FromMinutes(15),
        MaxPlausibleSpeedKmh: 900,
        MinTravelDistanceKm: 500);
}
```

Thresholds are data. The security team's first question after launch will be "can it be less noisy?", and the answer should be a configuration change, not a code review.

8.4 The detector

`src/Sentinel.Domain/Security/CompromiseDetector.cs`

```
namespace Sentinel.Domain.Security;

/// <summary>
/// Looks at one user's sign-in history and says whether the account may be
/// compromised. Four rules, each independent:
/// <list type="bullet">
///   <item><b>Brute force</b>: at least N failures inside a short window. High if a
success follows the burst.</item>
///   <item><b>Impossible travel</b>: two successful sign-ins further apart than anyone
could travel in the time between them. High.</item>
///   <item><b>New country</b>: a successful sign-in from a country this user has never
signed in from. Medium.</item>
///   <item><b>New device</b>: a successful sign-in from a device this user has never
used. Low.</item>
/// </list>
```

```
        /// </summary>
        /// <remarks>
        /// <para>
        /// The history passed in is both the evidence and the baseline. Attempts older
        /// than the reporting window never produce findings, but they still count as
        /// "seen before" for the new-country and new-device rules. Callers should
        /// therefore pass more history than the reporting window: the application
        /// layer passes ninety days against a thirty-day window.
        /// </para>
        /// <para>
        /// The detector sorts its input itself and does not trust the caller's order.
        /// It cannot, however, notice that the caller sent the wrong <i>subset</i> of
        /// history; Chapter 23 is about exactly that.
        /// </para>
        /// </remarks>
        public static class CompromiseDetector
        {
            public static SecurityAssessment Assess(
                IEnumerable<SignInAttempt> history,
                DateTime nowUtc,
                IReadOnlySet<string>? acknowledgedKeys = null,
                DetectionPolicy? policy = null)
            {
                ArgumentNullException.ThrowIfNull(history);
                if (nowUtc.Kind != DateTimeKind.Utc)
                    throw new ArgumentException("nowUtc must be UTC.", nameof(nowUtc));

                policy ??= DetectionPolicy.Default;
                acknowledgedKeys ??= new HashSet<string>();

                // Oldest first; ties broken by id so the output is deterministic.
                // Attempts "from the future" (clock skew upstream) are ignored rather
                // than allowed to poison the baseline.
                var ordered = history
                    .Where(a => a.OccurredAtUtc <= nowUtc)
                    .OrderBy(a => a.OccurredAtUtc)
                    .ThenBy(a => a.Id)
                    .ToList();

                if (ordered.Count == 0)
                    return SecurityAssessment.Clear;

                var reportFrom = nowUtc - policy.ReportingWindow;

                var findings = DetectBruteForce(ordered, policy)
                    .Concat(DetectImpossibleTravel(ordered, policy))
                    .Concat(DetectNewCountries(ordered))
                    .Concat(DetectNewDevices(ordered))
                    .Where(f => f.OccurredAtUtc >= reportFrom)
                    .Where(f => !acknowledgedKeys.Contains(f.Key))
                    .OrderByDescending(f => f.Severity)
                    .ThenByDescending(f => f.OccurredAtUtc)
                    .ToList();

                var riskLevel = findings.Count == 0 ? Severity.None : findings.Max(f =>
f.Severity);
                return new SecurityAssessment(riskLevel, findings);
            }

            /// <summary>
            /// Groups failures into bursts (a gap longer than the window ends a burst),
            /// then reports a burst when some window-sized slice of it holds at least
            /// the threshold number of failures.
            /// </summary>
            private static IEnumerable<SecurityFinding> DetectBruteForce(List<SignInAttempt>
ordered, DetectionPolicy policy)
            {
                var failures = ordered.Where(a => !a.Succeeded).ToList();
                var burst = new List<SignInAttempt>();

                foreach (var failure in failures)
                {
```

```
                    if (burst.Count > 0 && failure.OccurredAtUtc - burst[^1].OccurredAtUtc >
policy.FailedAttemptWindow)
                    {
                        if (TryReportBurst(burst, ordered, policy) is { } finding)
                            yield return finding;
                        burst = [];
                    }

                    burst.Add(failure);
                }

                if (TryReportBurst(burst, ordered, policy) is { } last)
                    yield return last;
            }

            private static SecurityFinding? TryReportBurst(List<SignInAttempt> burst,
List<SignInAttempt> ordered, DetectionPolicy policy)
            {
                if (burst.Count < policy.FailedAttemptThreshold || MaxInAnyWindow(burst,
policy.FailedAttemptWindow) < policy.FailedAttemptThreshold)
                    return null;

                var first = burst[0];
                var last = burst[^1];

                // Did the attacker get in? A success shortly after the burst is the
                // difference between "someone is trying" and "someone succeeded".
                var successAfter = ordered.FirstOrDefault(a =>
                    a.Succeeded &&
                    a.OccurredAtUtc > first.OccurredAtUtc &&
                    a.OccurredAtUtc <= last.OccurredAtUtc + policy.FailedAttemptWindow);

                var evidence = burst.Select(a => a.Id).ToList();
                if (successAfter is not null)
                    evidence.Add(successAfter.Id);

                var countries = burst.Select(a => a.Location).Where(l => l.IsKnown).Select(l =>
l.CountryCode).Distinct().ToList();
                var from = countries.Count == 0 ? "an unknown location" : string.Join(", ",
countries);

                return new SecurityFinding(
                    Key: $"brute-force:{first.Id:N}",
                    Kind: FindingKind.BruteForce,
                    Severity: successAfter is null ? Severity.Medium : Severity.High,
                    OccurredAtUtc: successAfter?.OccurredAtUtc ?? last.OccurredAtUtc,
                    AttemptIds: evidence,
                    Summary: successAfter is null
                        ? $"{burst.Count} failed sign-in attempts from {from} in a short period."
                        : $"{burst.Count} failed sign-in attempts from {from}, followed by a
successful sign-in.");
            }

            /// <summary>Two-pointer sweep: the most failures that fit inside any one window.
</summary>
            private static int MaxInAnyWindow(List<SignInAttempt> burst, TimeSpan window)
            {
                var best = 0;
                var start = 0;
                for (var end = 0; end < burst.Count; end++)
                {
                    while (burst[end].OccurredAtUtc - burst[start].OccurredAtUtc > window)
                        start++;
                    best = Math.Max(best, end - start + 1);
                }
                return best;
            }

            private static IEnumerable<SecurityFinding> DetectImpossibleTravel(List<SignInAttempt>
ordered, DetectionPolicy policy)
            {
                SignInAttempt? previous = null;
```

```
                foreach (var current in ordered.Where(a => a.Succeeded &&
a.Location.HasCoordinates))
                {
                    if (previous is not null && previous.Location.DistanceKmTo(current.Location)
is { } km && km >= policy.MinTravelDistanceKm)
                    {
                        var hours = (current.OccurredAtUtc - previous.OccurredAtUtc).TotalHours;

                        // Zero elapsed time over 500 km is the clearest case of all;
                        // guard the division rather than letting it produce infinity.
                        var speedKmh = hours <= 0 ? double.PositiveInfinity : km / hours;

                        if (speedKmh > policy.MaxPlausibleSpeedKmh)
                        {
                            yield return new SecurityFinding(
                                Key: $"impossible-travel:{current.Id:N}",
                                Kind: FindingKind.ImpossibleTravel,
                                Severity: Severity.High,
                                OccurredAtUtc: current.OccurredAtUtc,
                                AttemptIds: [previous.Id, current.Id],
                                Summary: $"Signed in from {previous.Location} and then
{current.Location} " +
                                         $"{FormatElapsed(current.OccurredAtUtc -
previous.OccurredAtUtc)} later ({km:N0} km apart).");
                        }
                    }

                    previous = current;
                }
            }

            private static IEnumerable<SecurityFinding> DetectNewCountries(List<SignInAttempt>
ordered)
            {
                var seen = new HashSet<string>(StringComparer.Ordinal);
                var isFirstSuccess = true;

                foreach (var attempt in ordered.Where(a => a.Succeeded && a.Location.IsKnown))
                {
                    // The very first success sets the baseline. Without history there
                    // is nothing to compare against, and flagging every new account
                    // as "new country" would train users to ignore the banner.
                    if (seen.Add(attempt.Location.CountryCode) && !isFirstSuccess)
                    {
                        yield return new SecurityFinding(
                            Key: $"new-country:{attempt.Id:N}",
                            Kind: FindingKind.NewCountry,
                            Severity: Severity.Medium,
                            OccurredAtUtc: attempt.OccurredAtUtc,
                            AttemptIds: [attempt.Id],
                            Summary: $"First sign-in from {attempt.Location}.");
                    }

                    isFirstSuccess = false;
                }
            }

            private static IEnumerable<SecurityFinding> DetectNewDevices(List<SignInAttempt>
ordered)
            {
                var seen = new HashSet<string>(StringComparer.Ordinal);
                var isFirstSuccess = true;

                foreach (var attempt in ordered.Where(a => a.Succeeded))
                {
                    if (seen.Add(attempt.DeviceId) && !isFirstSuccess)
                    {
                        yield return new SecurityFinding(
                            Key: $"new-device:{attempt.Id:N}",
                            Kind: FindingKind.NewDevice,
                            Severity: Severity.Low,
```

```
                            OccurredAtUtc: attempt.OccurredAtUtc,
                            AttemptIds: [attempt.Id],
                            Summary: $"Sign-in from a device not seen before
({DeviceFingerprint.Describe(attempt.UserAgent)}).");
                    }

                    isFirstSuccess = false;
                }
            }

            private static string FormatElapsed(TimeSpan elapsed) => elapsed.TotalMinutes < 60
                ? $"{Math.Max(0, (int)elapsed.TotalMinutes)} minutes"
                : $"{elapsed.TotalHours:0.#} hours";
        }
```

The brute-force rule, carefully

"At least five failures within fifteen minutes" sounds simple and hides two traps.

Trap one: what counts as "within"? Five failures ten minutes apart are within forty minutes of each other but never more than two inside any fifteen-minute window. The detector first groups failures into bursts (a gap longer than the window ends a burst), then uses a two-pointer sweep, `MaxInAnyWindow`, to find the most failures inside any window-sized slice.
`Failures_spread_more_thinly_than_the_window_allows_are_not_a_burst` pins that behaviour.

Trap two: did the attacker get in? Failures alone mean someone is trying. A success shortly after them means someone may have succeeded. The detector looks for a success after the first failure and within a window of the last, raises the severity to High, and adds the success to the evidence, so the table highlights it.

The impossible-travel rule

Only successful attempts with coordinates are compared, each with the one before it. Two guards matter:

- **Short hops are ignored** (below 500 km). IP geolocation cannot tell Lagos from Ikeja, so "travelling" 9 km in a minute is noise.
- **Zero elapsed time is the clearest case, not a division by zero.** The speed is set to infinity explicitly, rather than letting `km / 0.0` produce it by accident.

New country and new device

Both walk successes in order with a `HashSet` of what has been seen. The first success sets the baseline and is never reported: flagging every new account would teach users to ignore the banner, and a security warning that is ignored is worse than none.

What the detector refuses to trust

It sorts its input itself (oldest first, ties by id, so the output is deterministic), ignores attempts dated after `now`, and requires `now` to be UTC. What it *cannot* do is notice that it has been handed the wrong subset of history. Remember that sentence; Chapter 23 is about it.

8.5 The tests

`tests/Sentinel.Domain.Tests/Security/CompromiseDetectorTests.cs`

```
        using Sentinel.Domain.Security;

        namespace Sentinel.Domain.Tests.Security;

        public class CompromiseDetectorTests
        {
            private static readonly DateTime Now = History.Now;

            private static SecurityAssessment Assess(History history, IReadOnlySet<string>?
acknowledged = null) =>
                CompromiseDetector.Assess(history.Attempts, Now, acknowledged);

            // ---- Nothing to see ----

            [Fact]
            public void No_history_is_clear()
            {
                var assessment = CompromiseDetector.Assess([], Now);

                Assert.Equal(Severity.None, assessment.RiskLevel);
                Assert.Empty(assessment.Findings);
                Assert.False(assessment.IsCompromiseSuspected);
            }

            [Fact]
            public void An_ordinary_history_is_clear()
            {
                var assessment = Assess(new
History().WithOrdinaryBaseline().Success(Now.AddHours(-1)));

                Assert.Equal(Severity.None, assessment.RiskLevel);
            }

            [Fact]
            public void The_first_ever_sign_in_is_not_new_anything()
            {
                // No baseline yet: flagging it would flag every new account.
                var assessment = Assess(new History().Success(Now.AddHours(-1), Places.London,
Agents.Phone));

                Assert.Empty(assessment.Findings);
            }

            // ---- Brute force ----

            [Fact]
            public void Failures_below_the_threshold_are_not_a_burst()
            {
                var history = new History().WithOrdinaryBaseline()
                    .Failures(Now.AddHours(-2), count: 4, spacing: TimeSpan.FromMinutes(1));

                Assert.DoesNotContain(Assess(history).Findings, f => f.Kind ==
FindingKind.BruteForce);
            }

            [Fact]
            public void Threshold_failures_inside_the_window_are_a_medium_finding()
            {
                var history = new History().WithOrdinaryBaseline()
                    .Failures(Now.AddHours(-2), count: 5, spacing: TimeSpan.FromMinutes(2), where:
Places.Frankfurt);

                var finding = Assert.Single(Assess(history).Findings);

                Assert.Equal(FindingKind.BruteForce, finding.Kind);
                Assert.Equal(Severity.Medium, finding.Severity);
                Assert.Equal(5, finding.AttemptIds.Count);
                Assert.Contains("DE", finding.Summary);
            }

            [Fact]
```

```
            public void Failures_spread_more_thinly_than_the_window_allows_are_not_a_burst()
            {
                // Ten minutes apart: never more than two inside any fifteen minutes.
                var history = new History().WithOrdinaryBaseline()
                    .Failures(Now.AddHours(-3), count: 8, spacing: TimeSpan.FromMinutes(10));

                Assert.DoesNotContain(Assess(history).Findings, f => f.Kind ==
FindingKind.BruteForce);
            }

            [Fact]
            public void A_success_straight_after_the_burst_makes_it_high_and_cites_the_success()
            {
                var history = new History().WithOrdinaryBaseline()
                    .Failures(Now.AddHours(-2), count: 6, spacing: TimeSpan.FromSeconds(30))
                    .Success(Now.AddHours(-2).AddMinutes(4));

                var finding = Assert.Single(Assess(history).Findings, f => f.Kind ==
FindingKind.BruteForce);

                Assert.Equal(Severity.High, finding.Severity);
                Assert.Contains(history.Last.Id, finding.AttemptIds);
                Assert.Equal(history.Last.OccurredAtUtc, finding.OccurredAtUtc);
            }

            [Fact]
            public void Two_separate_bursts_are_two_findings()
            {
                var history = new History().WithOrdinaryBaseline()
                    .Failures(Now.AddDays(-3), count: 5, spacing: TimeSpan.FromMinutes(1))
                    .Failures(Now.AddDays(-1), count: 5, spacing: TimeSpan.FromMinutes(1));

                Assert.Equal(2, Assess(history).Findings.Count(f => f.Kind ==
FindingKind.BruteForce));
            }

            // ---- Impossible travel ----

            [Fact]
            public void Lagos_then_London_forty_minutes_later_is_impossible_travel()
            {
                var history = new History().WithOrdinaryBaseline()
                    .Success(Now.AddHours(-5), Places.Lagos)
                    .Success(Now.AddHours(-5).AddMinutes(40), Places.London);

                var finding = Assert.Single(Assess(history).Findings, f => f.Kind ==
FindingKind.ImpossibleTravel);

                Assert.Equal(Severity.High, finding.Severity);
                Assert.Equal(2, finding.AttemptIds.Count);
                Assert.Contains("40 minutes", finding.Summary);
            }

            [Fact]
            public void New_York_then_London_a_day_later_is_plausible()
            {
                var history = new History().WithOrdinaryBaseline()
                    .Success(Now.AddDays(-2), Places.NewYork)
                    .Success(Now.AddDays(-1), Places.London);

                Assert.DoesNotContain(Assess(history).Findings, f => f.Kind ==
FindingKind.ImpossibleTravel);
            }

            [Fact]
            public void Short_hops_are_ignored_because_ip_geolocation_is_too_coarse()
            {
                var history = new History().WithOrdinaryBaseline()
                    .Success(Now.AddHours(-1), Places.Lagos)
                    .Success(Now.AddHours(-1).AddMinutes(1), Places.Ikeja);

                Assert.DoesNotContain(Assess(history).Findings, f => f.Kind ==
```

```
FindingKind.ImpossibleTravel);
        }

        [Fact]
        public void Simultaneous_sign_ins_far_apart_do_not_divide_by_zero()
        {
            var at = Now.AddHours(-1);
            var history = new History().WithOrdinaryBaseline()
                .Success(at, Places.Lagos)
                .Success(at, Places.London);

            Assert.Contains(Assess(history).Findings, f => f.Kind ==
FindingKind.ImpossibleTravel);
        }

        [Fact]
        public void Unknown_locations_are_skipped_rather_than_treated_as_zero_zero()
        {
            var history = new History().WithOrdinaryBaseline()
                .Success(Now.AddHours(-2), GeoLocation.Unknown)
                .Success(Now.AddHours(-1), Places.Lagos);

            Assert.DoesNotContain(Assess(history).Findings, f => f.Kind ==
FindingKind.ImpossibleTravel);
        }

        [Fact]
        public void Failed_attempts_do_not_count_as_travel()
        {
            var history = new History().WithOrdinaryBaseline()
                .Success(Now.AddHours(-2), Places.Lagos)
                .Failure(Now.AddHours(-2).AddMinutes(5), Places.London);

            Assert.DoesNotContain(Assess(history).Findings, f => f.Kind ==
FindingKind.ImpossibleTravel);
        }

        // ---- New country, new device ----

        [Fact]
        public void A_success_from_a_new_country_is_medium()
        {
            var history = new History().WithOrdinaryBaseline().Success(Now.AddDays(-1),
Places.London);

            var finding = Assert.Single(Assess(history).Findings, f => f.Kind ==
FindingKind.NewCountry);

            Assert.Equal(Severity.Medium, finding.Severity);
            Assert.Contains("London, GB", finding.Summary);
        }

        [Fact]
        public void Only_the_first_sign_in_from_a_country_is_new()
        {
            var history = new History().WithOrdinaryBaseline()
                .Success(Now.AddDays(-3), Places.London)
                .Success(Now.AddDays(-2), Places.London);

            Assert.Single(Assess(history).Findings, f => f.Kind == FindingKind.NewCountry);
        }

        [Fact]
        public void A_failure_from_a_new_country_neither_raises_nor_establishes_it()
        {
            var history = new History().WithOrdinaryBaseline()
                .Failure(Now.AddDays(-2), Places.Frankfurt)
                .Success(Now.AddDays(-1), Places.Frankfurt);

            // The later success is still the first *successful* sign-in from DE.
            var finding = Assert.Single(Assess(history).Findings, f => f.Kind ==
FindingKind.NewCountry);
```

```
                Assert.Equal(history.Last.Id, Assert.Single(finding.AttemptIds));
            }

            [Fact]
            public void A_new_device_is_low()
            {
                var history = new History().WithOrdinaryBaseline().Success(Now.AddDays(-1), agent:
Agents.Phone);

                var finding = Assert.Single(Assess(history).Findings);

                Assert.Equal(FindingKind.NewDevice, finding.Kind);
                Assert.Equal(Severity.Low, finding.Severity);
                Assert.Contains("Safari on iPhone", finding.Summary);
                Assert.False(Assess(history).IsCompromiseSuspected);
            }

            // ---- Windows, acknowledgements, robustness ----

            [Fact]
            public void
Findings_older_than_the_reporting_window_are_not_reported_but_still_count_as_baseline()
            {
                var history = new History()
                    .Success(Now.AddDays(-80), Places.Lagos)
                    .Success(Now.AddDays(-60), Places.London)  // new country, but 60 days ago
                    .Success(Now.AddDays(-1), Places.London);   // London is no longer new

                Assert.Empty(Assess(history).Findings);
            }

            [Fact]
            public void Acknowledged_findings_are_excluded_and_the_risk_level_drops()
            {
                var history = new History().WithOrdinaryBaseline()
                    .Success(Now.AddDays(-1), Places.London, Agents.Phone);
                var before = Assess(history);
                var newCountry = before.Findings.Single(f => f.Kind == FindingKind.NewCountry);

                var after = Assess(history, new HashSet<string> { newCountry.Key });

                Assert.Equal(Severity.Medium, before.RiskLevel);
                Assert.Equal(Severity.Low, after.RiskLevel);
                Assert.DoesNotContain(after.Findings, f => f.Key == newCountry.Key);
            }

            [Fact]
            public void Finding_keys_are_stable_across_assessments()
            {
                var history = new History().WithOrdinaryBaseline().Success(Now.AddDays(-1),
Places.London);

                Assert.Equal(
                    Assess(history).Findings.Select(f => f.Key),
                    Assess(history).Findings.Select(f => f.Key));
            }

            [Fact]
            public void Input_order_does_not_matter()
            {
                var history = new History().WithOrdinaryBaseline()
                    .Success(Now.AddHours(-5), Places.Lagos)
                    .Success(Now.AddHours(-5).AddMinutes(40), Places.London);

                var shuffled = history.Attempts.OrderBy(_ => Guid.NewGuid()).ToList();

                Assert.Equal(
                    Assess(history).Findings.Select(f => f.Key),
                    CompromiseDetector.Assess(shuffled, Now).Findings.Select(f => f.Key));
            }

            [Fact]
```

```
            public void Attempts_after_now_are_ignored()
            {
                var history = new History().WithOrdinaryBaseline().Success(Now.AddHours(1),
Places.London);

                Assert.Empty(Assess(history).Findings);
            }

            [Fact]
            public void Findings_are_ordered_most_severe_first_then_newest_first()
            {
                var history = new History().WithOrdinaryBaseline()
                    .Success(Now.AddDays(-3), agent: Agents.Phone)            // low
                    .Success(Now.AddDays(-2), Places.Lagos)
                    .Success(Now.AddDays(-2).AddMinutes(30), Places.London);  // high + medium

                var severities = Assess(history).Findings.Select(f => f.Severity).ToList();

                Assert.Equal(severities.OrderByDescending(s => s), severities);
                Assert.Equal(Severity.High, severities[0]);
            }

            [Fact]
            public void Now_must_be_utc()
            {
                Assert.Throws<ArgumentException>(() => CompromiseDetector.Assess([],
DateTime.Now));
            }
        }
```

These tests are organised by rule, and each rule has tests for both sides of its boundary: four failures and five, ten minutes apart and two, 9 km and 5,000 km. That habit (test just inside and just outside every threshold) is the single most reliable way to “handle edge cases” in a timed test.

8.6 Doing it in the browser instead

In the test, you may be given raw login events in Angular and asked to decide what looks suspicious there. The same rules translate directly to TypeScript: sort a copy (`[...events].sort(...)`, never sort the input in place), walk it once, keep `Set`s of seen countries and devices, and return findings rather than booleans so the template can explain itself. Chapter 17 shows where such a function belongs in a component tree (a pure function in its own file, called by the container) and Chapter 20 how to test it.

Chapter 9 · The Application Layer

The Application layer turns requests into calls on the domain. Its machinery (messages, handlers, a dispatcher, a pipeline) is small, and writing it yourself is the fastest way to understand what MediatR and its relatives do.

9.1 The messages

`src/Sentinel.Application/Abstractions/Messaging/Requests.cs`

```
        using Sentinel.Domain.Common;

        namespace Sentinel.Application.Abstractions.Messaging;

        /// <summary>
        /// Anything the dispatcher can send. Every response is a <see cref="Result"/>,
        /// so a caller always has to look at the outcome before it can use a value.
        /// </summary>
        public interface IRequest<TResponse> where TResponse : Result;

        /// <summary>Changes state and returns nothing but success or failure.</summary>
        public interface ICommand : IRequest<Result>;

        /// <summary>Changes state and returns a value, typically the id of something created.
</summary>
        public interface ICommand<TValue> : IRequest<Result<TValue>>;

        /// <summary>Reads state. A query must never change anything.</summary>
        public interface IQuery<TValue> : IRequest<Result<TValue>>;
```

Every request declares its response type, and every response is a `Result`. The compiler therefore knows that `dispatcher.Send(new GetUserByIdQuery(id))` returns `Result<UserDetailDto>`; no casts anywhere.

`src/Sentinel.Application/Abstractions/Messaging/Handlers.cs`

```
        using Sentinel.Domain.Common;

        namespace Sentinel.Application.Abstractions.Messaging;

        /// <summary>
        /// The one interface the dispatcher knows about. The three below exist so a
        /// handler's declaration says what kind of request it serves.
        /// </summary>
        public interface IRequestHandler<in TRequest, TResponse>
            where TRequest : IRequest<TResponse>
            where TResponse : Result
        {
            Task<TResponse> Handle(TRequest request, CancellationToken cancellationToken);
        }

        public interface ICommandHandler<in TCommand> : IRequestHandler<TCommand, Result>
            where TCommand : ICommand;

        public interface ICommandHandler<in TCommand, TValue> : IRequestHandler<TCommand,
Result<TValue>>
            where TCommand : ICommand<TValue>;

        public interface IQueryHandler<in TQuery, TValue> : IRequestHandler<TQuery,
```

```
Result<TValue>>
            where TQuery : IQuery<TValue>;
```

The dispatcher only knows `IRequestHandler<TRequest, TResponse>`. The three specialised interfaces exist for the reader: a class that implements `IQueryHandler<,>` tells you at a glance that it must not change state.

9.2 The pipeline

`src/Sentinel.Application/Abstractions/Messaging/IPipelineBehavior.cs`

```
        using Sentinel.Domain.Common;

        namespace Sentinel.Application.Abstractions.Messaging;

        /// <summary>Calls the next behaviour in the pipeline, or the handler itself at the end.
</summary>
        public delegate Task<TResponse> RequestHandlerDelegate<TResponse>();

        /// <summary>
        /// Code that wraps every handler: validation, logging, timing. Behaviours are
        /// the Decorator pattern applied by the dispatcher, which is how cross-cutting
        /// concerns stay out of the handlers (the S and O of SOLID in one interface).
        /// </summary>
        public interface IPipelineBehavior<in TRequest, TResponse>
            where TRequest : IRequest<TResponse>
            where TResponse : Result
        {
            Task<TResponse> Handle(TRequest request, RequestHandlerDelegate<TResponse> next,
CancellationToken cancellationToken);
        }
```

A behaviour receives the request and a delegate for "the rest of the pipeline". It can act before, after, or instead of calling it. That is the Decorator pattern, applied uniformly to every handler.

9.3 The dispatcher

`src/Sentinel.Application/Abstractions/Messaging/Dispatcher.cs`

```
        using System.Collections.Concurrent;
        using Microsoft.Extensions.DependencyInjection;
        using Sentinel.Domain.Common;

        namespace Sentinel.Application.Abstractions.Messaging;

        /// <summary>Sends a request to its single handler, through the pipeline.</summary>
        public interface IDispatcher
        {
            Task<TResponse> Send<TResponse>(IRequest<TResponse> request, CancellationToken
cancellationToken = default)
                where TResponse : Result;
        }

        /// <summary>
        /// Our replacement for MediatR's <c>ISender</c>. The only clever part is the
        /// wrapper: <see cref="Send{TResponse}"/> knows the response type statically
        /// but the request type only at run time, and DI needs both to resolve
        /// <c>IRequestHandler&lt;TRequest, TResponse&gt;</c>. A closed generic wrapper,
        /// built once per request type with reflection and cached, bridges the gap so
        /// that every call after the first is ordinary, strongly-typed code.
        /// </summary>
```

```
        public sealed class Dispatcher(IServiceProvider services) : IDispatcher
        {
            private static readonly ConcurrentDictionary<Type, object> Wrappers = new();

            public Task<TResponse> Send<TResponse>(IRequest<TResponse> request, CancellationToken
cancellationToken = default)
                where TResponse : Result
            {
                ArgumentNullException.ThrowIfNull(request);

                var wrapper = (RequestHandlerWrapper<TResponse>)Wrappers.GetOrAdd(
                    request.GetType(),
                    requestType => Activator.CreateInstance(
                        typeof(RequestHandlerWrapper<,>).MakeGenericType(requestType,
typeof(TResponse)))!);

                return wrapper.Handle(request, services, cancellationToken);
            }
        }

        internal abstract class RequestHandlerWrapper<TResponse> where TResponse : Result
        {
            public abstract Task<TResponse> Handle(IRequest<TResponse> request, IServiceProvider
services, CancellationToken cancellationToken);
        }

        internal sealed class RequestHandlerWrapper<TRequest, TResponse> :
RequestHandlerWrapper<TResponse>
            where TRequest : IRequest<TResponse>
            where TResponse : Result
        {
            public override Task<TResponse> Handle(IRequest<TResponse> request, IServiceProvider
services, CancellationToken cancellationToken)
            {
                var typedRequest = (TRequest)request;
                var handler = services.GetRequiredService<IRequestHandler<TRequest, TResponse>>();

                RequestHandlerDelegate<TResponse> pipeline = () => handler.Handle(typedRequest,
cancellationToken);

                // Registered first runs outermost: wrap in reverse so that
                // AddBehavior(A); AddBehavior(B) gives A(B(handler)).
                foreach (var behavior in services.GetServices<IPipelineBehavior<TRequest,
TResponse>>().Reverse())
                {
                    var next = pipeline;
                    pipeline = () => behavior.Handle(typedRequest, next, cancellationToken);
                }

                return pipeline();
            }
        }
```

The interesting problem is generic type resolution. `Send<TResponse>` knows the response type at compile time, but the concrete request type (`GetUserByIdQuery`) only at run time, and DI needs both to resolve `IRequestHandler<GetUserByIdQuery, Result<UserDetailDto>>`. The solution:

1. On the first `Send` for a request type, build the closed generic type `RequestHandlerWrapper<GetUserByIdQuery, Result<UserDetailDto>>` with `MakeGenericType` and create one instance.
2. Cache it in a `ConcurrentDictionary` keyed by request type.
3. From then on, call it through the abstract base `RequestHandlerWrapper<TResponse>`, whose `Handle` is ordinary, strongly-typed code.

Reflection runs once per request type for the life of the process.

Behaviours are wrapped in reverse registration order so that the first one registered runs outermost: register logging then validation, and a rejected request is still logged.

9.4 Validation without a library

`src/Sentinel.Application/Abstractions/Validation/Validation.cs`

```
        using Sentinel.Domain.Common;

        namespace Sentinel.Application.Abstractions.Validation;

        public sealed record ValidationFailure(string Property, string Message);

        /// <summary>
        /// An <see cref="Error"/> carrying one message per offending field, which the
        /// API turns into an RFC 9457 validation problem response.
        /// </summary>
        public sealed record ValidationError(IReadOnlyList<ValidationFailure> Failures)
            : Error("Validation.Failed", "One or more fields are invalid.", ErrorType.Validation);

        /// <summary>
        /// Checks the <i>shape</i> of a request: required fields, lengths, ranges.
        /// Rules that need the database ("is this email taken?") belong in the
        /// handler; rules about the business belong in the domain.
        /// </summary>
        public interface IValidator<in TRequest>
        {
            IEnumerable<ValidationFailure> Validate(TRequest request);
        }

        /// <summary>
        /// A tiny base class in place of FluentValidation. It is not a framework:
        /// it is three helpers that make a validator read like a checklist.
        /// </summary>
        public abstract class Validator<TRequest> : IValidator<TRequest>
        {
            public abstract IEnumerable<ValidationFailure> Validate(TRequest request);

            protected static IEnumerable<ValidationFailure> Required(string? value, string
property)
            {
                if (string.IsNullOrWhiteSpace(value))
                    yield return new ValidationFailure(property, $"{property} is required.");
            }

            protected static IEnumerable<ValidationFailure> MaxLength(string? value, int max,
string property)
            {
                if (value is not null && value.Trim().Length > max)
                    yield return new ValidationFailure(property, $"{property} cannot exceed {max}
characters.");
            }

            protected static IEnumerable<ValidationFailure> InRange(int value, int min, int max,
string property)
            {
                if (value < min || value > max)
                    yield return new ValidationFailure(property, $"{property} must be between
{min} and {max}.");
            }
        }
```

FluentValidation is excellent; it is also a dependency for something that, in this application, needs three helpers. Writing them shows the boundary that matters more than the library:

Kind of rule	Example	Where it lives
Shape	“Email is required”, “PageSize between 1 and 100”	Validator
Needs the database	“That email is already registered”	Handler
Business invariant	“A role cannot be granted twice”	Domain

A validator that queries the database, or a handler that checks required fields, is a sign the layers have blurred.

9.5 The behaviours

`src/Sentinel.Application/Behaviors/ValidationBehavior.cs`

```
        using Sentinel.Application.Abstractions.Messaging;
        using Sentinel.Application.Abstractions.Validation;
        using Sentinel.Domain.Common;

        namespace Sentinel.Application.Behaviors;

        /// <summary>
        /// Runs every validator registered for the request and short-circuits with a
        /// <see cref="ValidationError"/> if any of them complain. The handler is
        /// never called with a malformed request, so it never has to check.
        /// </summary>
        internal sealed class ValidationBehavior<TRequest, TResponse>
(IEnumerable<IValidator<TRequest>> validators)
            : IPipelineBehavior<TRequest, TResponse>
            where TRequest : IRequest<TResponse>
            where TResponse : Result
        {
            public Task<TResponse> Handle(TRequest request, RequestHandlerDelegate<TResponse>
next, CancellationToken cancellationToken)
            {
                var failures = validators
                    .SelectMany(v => v.Validate(request))
                    .ToList();

                return failures.Count == 0
                    ? next()
                    : Task.FromResult(ResultFactory.Failure<TResponse>(new
ValidationError(failures)));
            }
        }
```

When validation fails, the behaviour must return a failed `TResponse` without knowing what `TResponse` is beyond “some `Result`”:

`src/Sentinel.Application/Behaviors/ResultFactory.cs`

```
        using System.Collections.Concurrent;
        using System.Reflection;
        using Sentinel.Domain.Common;

        namespace Sentinel.Application.Behaviors;

        /// <summary>
        /// Builds a failed <typeparamref name="TResponse"/> when all a behaviour
        /// knows is that TResponse is "some kind of Result". For <c>Result</c> that
        /// is a direct call; for <c>Result&lt;T&gt;</c> the T is only known at run
        /// time, so the factory method is found by reflection once and cached.
```

```
        /// </summary>
        internal static class ResultFactory
        {
            private static readonly ConcurrentDictionary<Type, Func<Error, Result>> Factories =
new();

            public static TResponse Failure<TResponse>(Error error) where TResponse : Result
            {
                if (typeof(TResponse) == typeof(Result))
                    return (TResponse)Result.Failure(error);

                var factory = Factories.GetOrAdd(typeof(TResponse), static type =>
                {
                    // DeclaredOnly: Result<T>.Failure hides Result.Failure, and we want the one
on Result<T>.
                    var method = type.GetMethod(
                        nameof(Result.Failure),
                        BindingFlags.Public | BindingFlags.Static | BindingFlags.DeclaredOnly,
                        [typeof(Error)])
                        ?? throw new InvalidOperationException($"{type} has no static
Failure(Error) method.");

                    return method.CreateDelegate<Func<Error, Result>>();
                });

                return (TResponse)factory(error);
            }
        }
```

`BindingFlags.DeclaredOnly` matters: `Result<T>` declares its own static `Failure` that hides the base one, and we want the one that returns `Result<T>`.

`src/Sentinel.Application/Behaviors/LoggingBehavior.cs`

```
        using System.Diagnostics;
        using Microsoft.Extensions.Logging;
        using Sentinel.Application.Abstractions.Messaging;
        using Sentinel.Domain.Common;

        namespace Sentinel.Application.Behaviors;

        /// <summary>
        /// Logs every request's name, outcome and duration. When a support ticket
        /// arrives with nothing but a time and a user, this log line is usually the
        /// first evidence you have (Chapter 21).
        /// </summary>
        internal sealed partial class LoggingBehavior<TRequest, TResponse>
(ILogger<LoggingBehavior<TRequest, TResponse>> logger)
            : IPipelineBehavior<TRequest, TResponse>
            where TRequest : IRequest<TResponse>
            where TResponse : Result
        {
            public async Task<TResponse> Handle(TRequest request,
RequestHandlerDelegate<TResponse> next, CancellationToken cancellationToken)
            {
                var requestName = typeof(TRequest).Name;
                var started = Stopwatch.GetTimestamp();

                var response = await next();

                var elapsedMs = Stopwatch.GetElapsedTime(started).TotalMilliseconds;
                if (response.IsSuccess)
                    LogSucceeded(requestName, elapsedMs);
                else
                    LogFailed(requestName, response.Error!.Code, response.Error.Message,
elapsedMs);

                return response;
```

```
            }

            [LoggerMessage(Level = LogLevel.Information, Message = "{RequestName} succeeded in
{ElapsedMs:0.0} ms")]
            private partial void LogSucceeded(string requestName, double elapsedMs);

            [LoggerMessage(Level = LogLevel.Warning, Message = "{RequestName} failed with
{ErrorCode} ({ErrorMessage}) in {ElapsedMs:0.0} ms")]
            private partial void LogFailed(string requestName, string errorCode, string
errorMessage, double elapsedMs);
        }
```

`[LoggerMessage]` generates the logging code at compile time: no boxing, no parsing the template on every call, and structured properties (`RequestName`, `ErrorCode`) that you can filter on. When a ticket arrives with nothing but "it failed around 10 this morning", these lines are your first evidence (Chapter 21).

9.6 The ports

`src/Sentinel.Application/Abstractions/Ports/Ports.cs`

```
        using Microsoft.EntityFrameworkCore;
        using Sentinel.Domain.Pages;
        using Sentinel.Domain.Roles;
        using Sentinel.Domain.Security;
        using Sentinel.Domain.Users;

        namespace Sentinel.Application.Abstractions.Ports;

        /// <summary>
        /// The persistence port. Infrastructure implements it with EF Core; handlers
        /// use it with LINQ. There is no repository layer on top (Chapter 3 explains
        /// why), and this interface is the reason none is needed: it is owned by
        /// Application, so the dependency rule holds.
        /// </summary>
        public interface IApplicationDbContext
        {
            DbSet<User> Users { get; }
            DbSet<Role> Roles { get; }
            DbSet<Page> Pages { get; }
            DbSet<SignInAttempt> SignInAttempts { get; }
            DbSet<FindingAcknowledgement> FindingAcknowledgements { get; }

            Task<int> SaveChangesAsync(CancellationToken cancellationToken = default);
        }

        /// <summary>
        /// The current time, injectable. A handler that calls <c>DateTime.UtcNow</c>
        /// directly cannot be tested at "thirty-one days later".
        /// </summary>
        public interface IClock
        {
            DateTime UtcNow { get; }
        }

        /// <summary>Resolves an IP address to an approximate location.</summary>
        public interface IGeoLocator
        {
            GeoLocation Locate(string ipAddress);
        }
```

`IClock` deserves a word. A handler that calls `DateTime.UtcNow` directly cannot be tested at "thirty-one days later", and every time-based rule in this system (reporting windows, clock skew, relative times) needs exactly that. Every "now" in Sentinel comes from an `IClock`, on the server, and from a `Clock`

service in Angular.

9.7 Registering everything

`src/Sentinel.Application/DependencyInjection.cs`

```
        using System.Reflection;
        using Microsoft.Extensions.DependencyInjection;
        using Sentinel.Application.Abstractions.Messaging;
        using Sentinel.Application.Abstractions.Validation;
        using Sentinel.Application.Behaviors;
        using Sentinel.Application.Security;
        using Sentinel.Domain.Security;

        namespace Sentinel.Application;

        public static class DependencyInjection
        {
            /// <summary>
            /// Registers the dispatcher, the pipeline, and every handler and validator
            /// in this assembly. Adding a slice never means editing this file.
            /// </summary>
            public static IServiceCollection AddApplication(this IServiceCollection services,
DetectionPolicy? detectionPolicy = null)
            {
                services.AddScoped<IDispatcher, Dispatcher>();

                // Order matters: logging wraps validation, so rejected requests are logged too.
                services.AddScoped(typeof(IPipelineBehavior<,>), typeof(LoggingBehavior<,>));
                services.AddScoped(typeof(IPipelineBehavior<,>), typeof(ValidationBehavior<,>));

                var assembly = Assembly.GetExecutingAssembly();
                RegisterClosedImplementations(services, assembly, typeof(IRequestHandler<,>));
                RegisterClosedImplementations(services, assembly, typeof(IValidator<>));

                services.AddSingleton(detectionPolicy ?? DetectionPolicy.Default);
                services.AddScoped<AccountAssessor>();

                return services;
            }

            /// <summary>
            /// For every concrete class implementing a closed form of
            /// <paramref name="openInterface"/>, register it against that closed form.
            /// This is the twenty lines of assembly scanning that Scrutor would
            /// otherwise provide.
            /// </summary>
            private static void RegisterClosedImplementations(IServiceCollection services,
Assembly assembly, Type openInterface)
            {
                var registrations =
                    from type in assembly.GetTypes()
                    where type is { IsClass: true, IsAbstract: false, IsGenericTypeDefinition:
false }
                    from implemented in type.GetInterfaces()
                    where implemented.IsGenericType && implemented.GetGenericTypeDefinition() ==
openInterface
                    select (Service: implemented, Implementation: type);

                foreach (var (service, implementation) in registrations)
                    services.AddScoped(service, implementation);
            }
        }
```

Adding a slice never touches this file: the scan finds every closed implementation of `IRequestHandler<,>` and `IValidator<>`. That is the open/closed principle doing visible work.

Chapter 10 · Feature Slices

Each use case is a folder with a request (and its validator) in one file and its handler in another. This chapter walks through the slices that matter for the skill test, and lists the rest.

10.1 A command slice: CreateUser

`src/Sentinel.Application/Users/CreateUser/CreateUserCommand.cs`

```
        using Sentinel.Application.Abstractions.Messaging;
        using Sentinel.Application.Abstractions.Validation;
        using Sentinel.Domain.Users;

        namespace Sentinel.Application.Users.CreateUser;

        public sealed record CreateUserCommand(string Email, string DisplayName, string
TimeZoneId) : ICommand<Guid>;

        internal sealed class CreateUserCommandValidator : Validator<CreateUserCommand>
        {
            public override IEnumerable<ValidationFailure> Validate(CreateUserCommand request) =>
                Required(request.Email, nameof(request.Email))
                    .Concat(MaxLength(request.Email, EmailAddress.MaxLength,
nameof(request.Email)))
                    .Concat(Required(request.DisplayName, nameof(request.DisplayName)))
                    .Concat(MaxLength(request.DisplayName, User.MaxDisplayNameLength,
nameof(request.DisplayName)))
                    .Concat(Required(request.TimeZoneId, nameof(request.TimeZoneId)));
        }
```

`src/Sentinel.Application/Users/CreateUser/CreateUserCommandHandler.cs`

```
        using Microsoft.EntityFrameworkCore;
        using Sentinel.Application.Abstractions.Messaging;
        using Sentinel.Application.Abstractions.Ports;
        using Sentinel.Domain.Common;
        using Sentinel.Domain.Users;

        namespace Sentinel.Application.Users.CreateUser;

        internal sealed class CreateUserCommandHandler(IApplicationDbContext db, IClock clock)
            : ICommandHandler<CreateUserCommand, Guid>
        {
            public async Task<Result<Guid>> Handle(CreateUserCommand request, CancellationToken
cancellationToken)
            {
                var email = EmailAddress.Create(request.Email);
                if (email.IsFailure)
                    return email.Error!;

                // The unique index is the real guarantee; this check exists to turn
                // the common case into a friendly 409 instead of a database exception.
                if (await db.Users.AnyAsync(u => u.Email == email.Value, cancellationToken))
                    return UserErrors.DuplicateEmail(email.Value);

                var user = User.Create(email.Value, request.DisplayName, request.TimeZoneId,
clock.UtcNow);
                if (user.IsFailure)
                    return user.Error!;

                db.Users.Add(user.Value);
                await db.SaveChangesAsync(cancellationToken);
```

```
            return user.Value.Id;
        }
    }
```

The shape every command handler follows:

1. Turn primitives into domain values (`EmailAddress.Create`); return the error if that fails.
2. Check what needs the database (duplicate email).
3. Ask the domain to do the work (`User.Create`).
4. Save; return the id.

The comment about the unique index matters. The `AnyAsync` check is a courtesy that turns the common case into a friendly 409; under concurrency, two requests can both pass it, and only the database's unique index stops the second. Never let an application-level check be the only guarantee of uniqueness.

10.2 Commands that change an aggregate

`src/Sentinel.Application/Users/RoleAssignment/RoleAssignmentHandlers.cs`

```
    using Microsoft.EntityFrameworkCore;
    using Sentinel.Application.Abstractions.Messaging;
    using Sentinel.Application.Abstractions.Ports;
    using Sentinel.Domain.Common;
    using Sentinel.Domain.Roles;
    using Sentinel.Domain.Users;

    namespace Sentinel.Application.Users.RoleAssignment;

    // Assign and revoke are two halves of one use case and change for the same
    // reasons, so they share a slice. Two handlers, still one responsibility each.

    internal sealed class AssignRoleCommandHandler(IApplicationDbContext db, IClock clock) :
ICommandHandler<AssignRoleCommand>
    {
        public async Task<Result> Handle(AssignRoleCommand request, CancellationToken
cancellationToken)
        {
            var user = await db.Users.FirstOrDefaultAsync(u => u.Id == request.UserId,
cancellationToken);
            if (user is null)
                return UserErrors.NotFound(request.UserId);

            if (!await db.Roles.AnyAsync(r => r.Id == request.RoleId, cancellationToken))
                return RoleErrors.NotFound(request.RoleId);

            var result = user.AssignRole(request.RoleId, clock.UtcNow);
            if (result.IsFailure)
                return result;

            await db.SaveChangesAsync(cancellationToken);
            return Result.Success();
        }
    }

    internal sealed class RevokeRoleCommandHandler(IApplicationDbContext db) :
ICommandHandler<RevokeRoleCommand>
    {
        public async Task<Result> Handle(RevokeRoleCommand request, CancellationToken
cancellationToken)
        {
            var user = await db.Users.FirstOrDefaultAsync(u => u.Id == request.UserId,
cancellationToken);
```

```
            if (user is null)
                return UserErrors.NotFound(request.UserId);

            var result = user.RevokeRole(request.RoleId);
            if (result.IsFailure)
                return result;

            await db.SaveChangesAsync(cancellationToken);
            return Result.Success();
        }
    }
```

Load, call one method, save. All the rules are in `User.AssignRole`; the handler only checks the thing the aggregate cannot know (whether the role exists).

10.3 The access slices

All three share one helper, so that "how a user becomes an `AccessSubject`" cannot drift between screens:

`src/Sentinel.Application/Access/AccessSubjectLoader.cs`

```
    using Microsoft.EntityFrameworkCore;
    using Sentinel.Application.Abstractions.Ports;
    using Sentinel.Domain.Access;

    namespace Sentinel.Application.Access;

    /// <summary>
    /// Loads what <see cref="PageAccessEvaluator"/> needs to know about a user.
    /// Shared by every slice that answers "what may this person see?", which is
    /// the one place sharing between slices is worth its coupling: the rule
    /// "how a user becomes an AccessSubject" must not drift between screens.
    /// </summary>
    internal static class AccessSubjectLoader
    {
        /// <returns>The subject, or null when <paramref name="userId"/> names no user.
</returns>
        public static async Task<AccessSubject?> LoadAccessSubjectAsync(
            this IApplicationDbContext db, Guid? userId, CancellationToken cancellationToken)
        {
            if (userId is null)
                return AccessSubject.Anonymous;

            var user = await db.Users
                .AsNoTracking()
                .FirstOrDefaultAsync(u => u.Id == userId, cancellationToken);

            if (user is null)
                return null;

            var roleIds = user.RoleIds.ToList();
            var roles = await db.Roles
                .AsNoTracking()
                .Where(r => roleIds.Contains(r.Id))
                .ToListAsync(cancellationToken);

            return AccessSubject.For(user, roles);
        }
    }
```

Check one page (used by the Angular route guard on every navigation):

`src/Sentinel.Application/Pages/CheckPageAccess/CheckPageAccessQuery.cs`

```
        using Sentinel.Application.Abstractions.Messaging;
        using Sentinel.Application.Abstractions.Validation;
        using Sentinel.Contracts;

        namespace Sentinel.Application.Pages.CheckPageAccess;

        /// <summary>
        /// May this user open this one page? The Angular route guard asks this
        /// before every navigation, so a hand-typed URL is checked as carefully as
        /// a click on the menu.
        /// </summary>
        public sealed record CheckPageAccessQuery(Guid? UserId, string Path) :
IQuery<PageAccessDto>;

        internal sealed class CheckPageAccessQueryValidator : Validator<CheckPageAccessQuery>
        {
            public override IEnumerable<ValidationFailure> Validate(CheckPageAccessQuery request)
=>
                Required(request.Path, nameof(request.Path))
                    .Concat(MaxLength(request.Path, 200, nameof(request.Path)));
        }
```

src/Sentinel.Application/Pages/CheckPageAccess/CheckPageAccessQueryHandler.cs

```
        using Microsoft.EntityFrameworkCore;
        using Sentinel.Application.Abstractions.Messaging;
        using Sentinel.Application.Abstractions.Ports;
        using Sentinel.Application.Access;
        using Sentinel.Contracts;
        using Sentinel.Domain.Access;
        using Sentinel.Domain.Common;
        using Sentinel.Domain.Pages;

        namespace Sentinel.Application.Pages.CheckPageAccess;

        internal sealed class CheckPageAccessQueryHandler(IApplicationDbContext db) :
IQueryHandler<CheckPageAccessQuery, PageAccessDto>
        {
            public async Task<Result<PageAccessDto>> Handle(CheckPageAccessQuery request,
CancellationToken cancellationToken)
            {
                var subject = await db.LoadAccessSubjectAsync(request.UserId, cancellationToken)
?? AccessSubject.Anonymous;

                // The whole tree is needed even for one page: the answer depends on
                // every ancestor. Sentinel has tens of pages, not thousands; Chapter 10
                // discusses what to do when that stops being true.
                var pages = await db.Pages.AsNoTracking().ToListAsync(cancellationToken);

                var decision = PageAccessEvaluator.EvaluatePath(subject, pages, request.Path);
                if (decision is null)
                    return PageErrors.NotFound(request.Path);

                return decision.ToDto(pages.ToDictionary(p => p.Id));
            }
        }
```

The menu for the current user:

src/Sentinel.Application/Pages/GetNavigation/GetNavigationQueryHandler.cs

```
        using Microsoft.EntityFrameworkCore;
        using Sentinel.Application.Abstractions.Messaging;
        using Sentinel.Application.Abstractions.Ports;
        using Sentinel.Application.Access;
        using Sentinel.Contracts;
```

```
        using Sentinel.Domain.Access;
        using Sentinel.Domain.Common;

        namespace Sentinel.Application.Pages.GetNavigation;

        internal sealed class GetNavigationQueryHandler(IApplicationDbContext db) :
IQueryHandler<GetNavigationQuery, IReadOnlyList<NavigationNodeDto>>>
        {
            public async Task<Result<IReadOnlyList<NavigationNodeDto>>> Handle(GetNavigationQuery
request, CancellationToken cancellationToken)
            {
                // An id that names no user (deleted since the token was issued) is
                // treated as anonymous rather than as an error: the menu should
                // degrade to the public pages, not fail to render.
                var subject = await db.LoadAccessSubjectAsync(request.UserId, cancellationToken)
?? AccessSubject.Anonymous;

                var pages = await db.Pages.AsNoTracking().ToListAsync(cancellationToken);

                return Result.Success(PageAccessEvaluator.EvaluateAll(subject,
pages).ToNavigationTree());
            }
        }
```

Note the edge case in the comment: a user id that no longer exists (deleted since the token was issued) degrades to the public menu rather than failing.

Every page with its verdict for one user, for the admin access explorer:

`src/Sentinel.Application/Pages/GetPageAccess/GetPageAccessQueryHandler.cs`

```
        using Microsoft.EntityFrameworkCore;
        using Sentinel.Application.Abstractions.Messaging;
        using Sentinel.Application.Abstractions.Ports;
        using Sentinel.Application.Access;
        using Sentinel.Contracts;
        using Sentinel.Domain.Access;
        using Sentinel.Domain.Common;
        using Sentinel.Domain.Users;

        namespace Sentinel.Application.Pages.GetPageAccess;

        internal sealed class GetPageAccessQueryHandler(IApplicationDbContext db) :
IQueryHandler<GetPageAccessQuery, IReadOnlyList<PageAccessDto>>
        {
            public async Task<Result<IReadOnlyList<PageAccessDto>>> Handle(GetPageAccessQuery
request, CancellationToken cancellationToken)
            {
                var subject = await db.LoadAccessSubjectAsync(request.UserId, cancellationToken);
                if (subject is null)
                    return UserErrors.NotFound(request.UserId);

                var pages = await db.Pages.AsNoTracking().OrderBy(p =>
p.Path).ToListAsync(cancellationToken);
                var pagesById = pages.ToDictionary(p => p.Id);

                IReadOnlyList<PageAccessDto> result = PageAccessEvaluator
                    .EvaluateAll(subject, pages)
                    .Select(d => d.ToDto(pagesById))
                    .ToList();

                return Result.Success(result);
            }
        }
```

The mapping from decisions to DTOs, including building the menu tree:

src/Sentinel.Application/Pages/PageAccessMapping.cs

```
        using Sentinel.Contracts;
        using Sentinel.Domain.Access;
        using Sentinel.Domain.Pages;

        namespace Sentinel.Application.Pages;

        /// <summary>Turns evaluator decisions into the shapes the three page-access slices
return.</summary>
        internal static class PageAccessMapping
        {
            public static PageAccessDto ToDto(this PageAccessDecision decision,
IReadOnlyDictionary<Guid, Page> pagesById)
            {
                var page = decision.Page;
                var parentPath = page.ParentId is { } parentId && pagesById.TryGetValue(parentId,
out var parent)
                    ? parent.Path
                    : null;

                return new PageAccessDto(
                    page.Path,
                    page.Title,
                    parentPath,
                    page.RequiredPermission?.Value,
                    decision.IsAllowed,
                    decision.Reason.ToString());
            }

            /// <summary>
            /// Builds the menu from the allowed pages. A public page whose parent is
            /// hidden is promoted to the top level rather than dropped: the evaluator
            /// said the user may open it, so the menu must offer a way to get there.
            /// </summary>
            public static IReadOnlyList<NavigationNodeDto> ToNavigationTree(this
IEnumerable<PageAccessDecision> decisions)
            {
                var allowed = decisions.Where(d => d.IsAllowed).Select(d => d.Page).ToList();
                var allowedIds = allowed.Select(p => p.Id).ToHashSet();

                var childrenByParent = allowed
                    .Where(p => p.ParentId is { } id && allowedIds.Contains(id))
                    .ToLookup(p => p.ParentId!.Value);

                var roots = allowed.Where(p => p.ParentId is not { } id ||
!allowedIds.Contains(id));

                return Build(roots, childrenByParent);
            }

            private static List<NavigationNodeDto> Build(IEnumerable<Page> pages, ILookup<Guid,
Page> childrenByParent) =>
                pages
                    .OrderBy(p => p.SortOrder)
                    .ThenBy(p => p.Title, StringComparer.OrdinalIgnoreCase)
                    .Select(p => new NavigationNodeDto(p.Path, p.Title,
Build(childrenByParent[p.Id], childrenByParent)))
                    .ToList();
        }
```

`ToNavigationTree` handles the edge case from §7.1: a public page under a hidden parent is promoted to the top level. The evaluator said the user may open it, so the menu must offer a way to get there.

Guarding endpoints with the same rules:

src/Sentinel.Application/Access/HasPermission/HasPermissionQueryHandler.cs

```
        using Sentinel.Application.Abstractions.Messaging;
        using Sentinel.Application.Abstractions.Ports;
        using Sentinel.Domain.Access;
        using Sentinel.Domain.Common;
        using Sentinel.Domain.Users;

        namespace Sentinel.Application.Access.HasPermission;

        internal sealed class HasPermissionQueryHandler(IApplicationDbContext db) :
IQueryHandler<HasPermissionQuery, bool>
        {
            public async Task<Result<bool>> Handle(HasPermissionQuery request, CancellationToken
cancellationToken)
            {
                var permission = Permission.Create(request.Permission);
                if (permission.IsFailure)
                    return permission.Error!;

                var subject = await db.LoadAccessSubjectAsync(request.UserId, cancellationToken);
                if (subject is null)
                    return UserErrors.NotFound(request.UserId);

                return subject.Has(permission.Value);
            }
        }
```

10.4 The security slices

The status query, the history query and the acknowledge command must agree on what the findings are, so they share an assessor:

`src/Sentinel.Application/Security/AccountAssessor.cs`

```
        using Microsoft.EntityFrameworkCore;
        using Sentinel.Application.Abstractions.Ports;
        using Sentinel.Domain.Security;

        namespace Sentinel.Application.Security;

        internal sealed record AccountAssessment(
            SecurityAssessment Assessment,
            IReadOnlyList<SignInAttempt> History,
            DateTime AssessedAtUtc)
        {
            /// <summary>Ids of every attempt cited as evidence by an unacknowledged finding.
</summary>
            public IReadOnlySet<Guid> FlaggedAttemptIds { get; } =
                Assessment.Findings.SelectMany(f => f.AttemptIds).ToHashSet();
        }

        /// <summary>
        /// Loads a user's history and runs the detector over it. Shared by the
        /// status query, the history query and the acknowledge command, because all
        /// three must agree on what the findings are: a user cannot acknowledge a
        /// finding the status page did not show them.
        /// </summary>
        internal sealed class AccountAssessor(IApplicationDbContext db, IClock clock,
DetectionPolicy policy)
        {
            /// <summary>History older than the reporting window, used as the baseline for "new"
countries and devices.</summary>
            public static readonly TimeSpan BaselineWindow = TimeSpan.FromDays(90);

            /// <summary>
            /// A ceiling on how much history one assessment loads. An integration
            /// account that signs in every minute must not turn the security page
```

```
            /// into a full table scan.
            /// </summary>
            public const int MaxHistory = 500;

            public async Task<AccountAssessment> AssessAsync(Guid userId, CancellationToken
cancellationToken)
            {
                var now = clock.UtcNow;
                var since = now - BaselineWindow;

                // Newest first, THEN take: when the ceiling bites, what we drop must
                // be the oldest baseline, never the most recent evidence. The order
                // of these two calls is the whole of Chapter 23.
                var history = await db.SignInAttempts
                    .AsNoTracking()
                    .Where(a => a.UserId == userId && a.OccurredAtUtc >= since)
                    .OrderByDescending(a => a.OccurredAtUtc)
                    .Take(MaxHistory)
                    .ToListAsync(cancellationToken);

                var acknowledged = await db.FindingAcknowledgements
                    .AsNoTracking()
                    .Where(a => a.UserId == userId)
                    .Select(a => a.FindingKey)
                    .ToListAsync(cancellationToken);

                var assessment = CompromiseDetector.Assess(history, now,
acknowledged.ToHashSet(StringComparer.Ordinal), policy);
                return new AccountAssessment(assessment, history, now);
            }
        }
```

Read the comment above the query twice. It is the most important comment in the back end, and Chapter 23 explains why.

`src/Sentinel.Application/Security/GetAccountSecurityStatus/GetAccountSecurityStatusQueryHandler.cs`

```
        using Microsoft.EntityFrameworkCore;
        using Sentinel.Application.Abstractions.Messaging;
        using Sentinel.Application.Abstractions.Ports;
        using Sentinel.Contracts;
        using Sentinel.Domain.Common;
        using Sentinel.Domain.Users;

        namespace Sentinel.Application.Security.GetAccountSecurityStatus;

        internal sealed class GetAccountSecurityStatusQueryHandler(IApplicationDbContext db,
AccountAssessor assessor)
            : IQueryHandler<GetAccountSecurityStatusQuery, AccountSecurityStatusDto>
        {
            /// <summary>How many sign-ins the page shows under the banner. The full list is paged
separately.</summary>
            public const int RecentSignInCount = 20;

            public async Task<Result<AccountSecurityStatusDto>>
Handle(GetAccountSecurityStatusQuery request, CancellationToken cancellationToken)
            {
                var timeZoneId = await db.Users
                    .AsNoTracking()
                    .Where(u => u.Id == request.UserId)
                    .Select(u => u.TimeZoneId)
                    .FirstOrDefaultAsync(cancellationToken);

                if (timeZoneId is null)
                    return UserErrors.NotFound(request.UserId);

                var result = await assessor.AssessAsync(request.UserId, cancellationToken);
```

```
                var recent = result.History
                    .OrderByDescending(a => a.OccurredAtUtc)
                    .Take(RecentSignInCount)
                    .Select(a => a.ToDto(result.FlaggedAttemptIds))
                    .ToList();

                return new AccountSecurityStatusDto(
                    result.Assessment.RiskLevel.ToString(),
                    result.Assessment.IsCompromiseSuspected,
                    result.AssessedAtUtc,
                    timeZoneId,
                    result.Assessment.Findings.Select(f => f.ToDto()).ToList(),
                    recent);
            }
        }
```

`src/Sentinel.Application/Security/SecurityMapping.cs`

```
        using Sentinel.Contracts;
        using Sentinel.Domain.Security;

        namespace Sentinel.Application.Security;

        internal static class SecurityMapping
        {
            public static SignInAttemptDto ToDto(this SignInAttempt attempt, IReadOnlySet<Guid>
flaggedIds) => new(
                attempt.Id,
                attempt.OccurredAtUtc,
                attempt.IpAddress,
                attempt.Location.CountryCode,
                attempt.Location.City,
                DeviceFingerprint.Describe(attempt.UserAgent),
                attempt.Succeeded,
                attempt.FailureReason?.ToString(),
                flaggedIds.Contains(attempt.Id));

            public static SecurityFindingDto ToDto(this SecurityFinding finding) => new(
                finding.Key,
                finding.Kind.ToString(),
                finding.Severity.ToString(),
                finding.OccurredAtUtc,
                finding.Summary,
                finding.AttemptIds);
        }
```

Acknowledging a finding:

`src/Sentinel.Application/Security/AcknowledgeFinding/AcknowledgeFindingCommandHandler.cs`

```
        using Microsoft.EntityFrameworkCore;
        using Sentinel.Application.Abstractions.Messaging;
        using Sentinel.Application.Abstractions.Ports;
        using Sentinel.Domain.Common;
        using Sentinel.Domain.Security;

        namespace Sentinel.Application.Security.AcknowledgeFinding;

        internal sealed class AcknowledgeFindingCommandHandler(IApplicationDbContext db,
AccountAssessor assessor, IClock clock)
            : ICommandHandler<AcknowledgeFindingCommand>
        {
            public async Task<Result> Handle(AcknowledgeFindingCommand request, CancellationToken
cancellationToken)
            {
                var alreadyAcknowledged = await db.FindingAcknowledgements
```

```
                    .AnyAsync(a => a.UserId == request.UserId && a.FindingKey ==
request.FindingKey, cancellationToken);

                if (alreadyAcknowledged)
                    return Result.Success();

                // Only findings the user can currently see may be acknowledged. This
                // stops a client from pre-acknowledging keys it has guessed, which
                // would let a compromised session hide its own tracks in advance.
                var current = await assessor.AssessAsync(request.UserId, cancellationToken);
                if (current.Assessment.Findings.All(f => f.Key != request.FindingKey))
                    return Error.NotFound("Finding.NotFound", "That finding does not exist or no
longer applies.");

                var acknowledgement = FindingAcknowledgement.Create(request.UserId,
request.FindingKey, clock.UtcNow);
                if (acknowledgement.IsFailure)
                    return acknowledgement.Error!;

                db.FindingAcknowledgements.Add(acknowledgement.Value);
                await db.SaveChangesAsync(cancellationToken);
                return Result.Success();
            }
        }
```

Two edge cases are handled on purpose. Acknowledging twice succeeds (idempotency: a double-click or a retried request must not produce an error). Acknowledging a key the user cannot currently see fails, so a compromised session cannot pre-acknowledge findings it is about to cause.

Recording a sign-in, the endpoint the identity provider calls:

`src/Sentinel.Application/Security/RecordSignIn/RecordSignInCommand.cs`

```
        using Sentinel.Application.Abstractions.Messaging;
        using Sentinel.Application.Abstractions.Validation;
        using Sentinel.Domain.Security;

        namespace Sentinel.Application.Security.RecordSignIn;

        /// <summary>
        /// Records one sign-in attempt. In production the identity provider calls
        /// this; in development the seeder and the "simulate" buttons do.
        /// <c>OccurredAt</c> is a <see cref="DateTimeOffset"/> on purpose: a bare
        /// DateTime from the wire cannot say which zone it is in.
        /// </summary>
        public sealed record RecordSignInCommand(
            string Email,
            DateTimeOffset OccurredAt,
            string IpAddress,
            string? UserAgent,
            bool Succeeded,
            string? FailureReason) : ICommand<Guid>;

        internal sealed class RecordSignInCommandValidator : Validator<RecordSignInCommand>
        {
            public override IEnumerable<ValidationFailure> Validate(RecordSignInCommand request)
            {
                foreach (var failure in Required(request.Email,
nameof(request.Email)).Concat(Required(request.IpAddress, nameof(request.IpAddress))))
                    yield return failure;

                if (request.FailureReason is not null && !Enum.TryParse<SignInFailureReason>
(request.FailureReason, ignoreCase: true, out _))
                {
                    yield return new ValidationFailure(nameof(request.FailureReason),
                        $"FailureReason must be one of: {string.Join(", ",
Enum.GetNames<SignInFailureReason>())}.");
                }
```

```
                if (request.OccurredAt == default)
                    yield return new ValidationFailure(nameof(request.OccurredAt), "OccurredAt is
required.");
            }
        }
```

`src/Sentinel.Application/Security/RecordSignIn/RecordSignInCommandHandler.cs`

```
        using Microsoft.EntityFrameworkCore;
        using Sentinel.Application.Abstractions.Messaging;
        using Sentinel.Application.Abstractions.Ports;
        using Sentinel.Domain.Common;
        using Sentinel.Domain.Security;
        using Sentinel.Domain.Users;

        namespace Sentinel.Application.Security.RecordSignIn;

        internal sealed class RecordSignInCommandHandler(IApplicationDbContext db, IGeoLocator
geoLocator, IClock clock)
            : ICommandHandler<RecordSignInCommand, Guid>
        {
            /// <summary>Clocks disagree. Allow a little skew, but not "next Tuesday".</summary>
            public static readonly TimeSpan MaxClockSkew = TimeSpan.FromMinutes(5);

            public async Task<Result<Guid>> Handle(RecordSignInCommand request, CancellationToken
cancellationToken)
            {
                var email = EmailAddress.Create(request.Email);
                if (email.IsFailure)
                    return email.Error!;

                var occurredAtUtc = request.OccurredAt.UtcDateTime; // Kind is Utc by definition
                if (occurredAtUtc > clock.UtcNow + MaxClockSkew)
                    return Error.Validation("SignIn.InFuture", "A sign-in cannot be recorded in
the future.");

                var userId = await db.Users
                    .Where(u => u.Email == email.Value)
                    .Select(u => (Guid?)u.Id)
                    .FirstOrDefaultAsync(cancellationToken);

                if (userId is null)
                    return Error.NotFound("User.EmailNotFound", $"No user has the email
'{email.Value}'.");

                SignInFailureReason? reason = request.FailureReason is null
                    ? null
                    : Enum.Parse<SignInFailureReason>(request.FailureReason, ignoreCase: true);

                var attempt = SignInAttempt.Record(
                    userId.Value,
                    occurredAtUtc,
                    request.IpAddress,
                    geoLocator.Locate(request.IpAddress),
                    request.UserAgent,
                    request.Succeeded,
                    reason);

                if (attempt.IsFailure)
                    return attempt.Error!;

                db.SignInAttempts.Add(attempt.Value);
                await db.SaveChangesAsync(cancellationToken);
                return attempt.Value.Id;
            }
        }
```

The request carries a `DateTimeOffset`, not a `DateTime`. A `DateTime` from the wire cannot say which zone it is in; a `DateTimeOffset` must. (It still accepts a string with no offset and assumes server-local time, which is a trap Chapter 22 returns to.)

10.5 The remaining slices

They follow the same patterns and are in the repository:

Feature	Slices
Users	`GetUsers` (paged, search), `GetUserById`, `GetCurrentUser`, `GetDemoUsers`, `FindUserByEmail`, `Suspension`, `PermissionOverrides`
Roles	`CreateRole`, `RolePermissions` (grant, revoke), `GetRoles`
Pages	`CreatePage`, `GetPages`
Security	`GetSignInHistory`

One worth opening is `GetUsersQueryHandler`: its comment explains why the search matches part of a display name but only a *whole* email address. Email is stored through a value converter, which EF Core can compare but not search inside. That is a trade-off, written down, which is what the reviewer of your skill test wants to see whenever you choose the simpler path.

Chapter 11 · Infrastructure and Persistence

11.1 The DbContext

`src/Sentinel.Infrastructure/Persistence/SentinelDbContext.cs`

```
        using Microsoft.EntityFrameworkCore;
        using Sentinel.Application.Abstractions.Ports;
        using Sentinel.Domain.Pages;
        using Sentinel.Domain.Roles;
        using Sentinel.Domain.Security;
        using Sentinel.Domain.Users;
        using Sentinel.Infrastructure.Persistence.Converters;

        namespace Sentinel.Infrastructure.Persistence;

        public sealed class SentinelDbContext(DbContextOptions<SentinelDbContext> options)
            : DbContext(options), IApplicationDbContext
        {
            public DbSet<User> Users => Set<User>();
            public DbSet<Role> Roles => Set<Role>();
            public DbSet<Page> Pages => Set<Page>();
            public DbSet<SignInAttempt> SignInAttempts => Set<SignInAttempt>();
            public DbSet<FindingAcknowledgement> FindingAcknowledgements =>
Set<FindingAcknowledgement>();

            protected override void OnModelCreating(ModelBuilder modelBuilder) =>
                modelBuilder.ApplyConfigurationsFromAssembly(typeof(SentinelDbContext).Assembly);

            protected override void ConfigureConventions(ModelConfigurationBuilder
configurationBuilder)
            {
                // Every DateTime in this model is UTC. SQL Server's datetime2 has no
                // notion of a zone, so without this line every value read back has
                // DateTimeKind.Unspecified, serialises without its 'Z', and is then
                // read by every browser on earth as local time. Chapter 22 is the
                // post-mortem of deleting this line.
                configurationBuilder.Properties<DateTime>().HaveConversion<UtcDateTimeConverter>
();
            }
        }
```

The context implements `IApplicationDbContext` and applies one configuration class per aggregate. `ConfigureConventions` contains the most consequential line in the project. SQL Server's `datetime2` stores an instant with no notion of a zone. Without a converter, every `DateTime` EF Core reads back has `DateTimeKind.Unspecified`, `System.Text.Json` serialises it without a trailing `Z`, and every browser on earth then reads it as *local* time. One line prevents that everywhere:

`src/Sentinel.Infrastructure/Persistence/Converters/UtcDateTimeConverter.cs`

```
        using Microsoft.EntityFrameworkCore.Storage.ValueConversion;

        namespace Sentinel.Infrastructure.Persistence.Converters;

        /// <summary>
        /// Writes DateTimes as they are (they are UTC by domain rule) and stamps
        /// <see cref="DateTimeKind.Utc"/> on everything read back. The database
        /// stores the instant; this converter restores the knowledge of what kind
        /// of instant it is.
        /// </summary>
        internal sealed class UtcDateTimeConverter() : ValueConverter<DateTime, DateTime>(
```

```
            toDatabase => RequireUtc(toDatabase),
            fromDatabase => DateTime.SpecifyKind(fromDatabase, DateTimeKind.Utc))
        {
            // A method, not an inline throw: converters are expression trees, and
            // expression trees cannot contain throw expressions.
            private static DateTime RequireUtc(DateTime value) =>
                value.Kind == DateTimeKind.Utc
                    ? value
                    : throw new InvalidOperationException($"Refusing to store a {value.Kind}
DateTime; the model is UTC-only.");
        }
```

It also refuses to *write* a non-UTC `DateTime`, so a local time cannot sneak into the database. (Converters are expression trees, which cannot contain `throw` expressions; hence the helper method.)

11.2 Configurations

`src/Sentinel.Infrastructure/Persistence/Configurations/UserConfiguration.cs`

```
        using Microsoft.EntityFrameworkCore;
        using Microsoft.EntityFrameworkCore.Metadata.Builders;
        using Sentinel.Domain.Users;
        using Sentinel.Infrastructure.Persistence.Converters;

        namespace Sentinel.Infrastructure.Persistence.Configurations;

        internal sealed class UserConfiguration : IEntityTypeConfiguration<User>
        {
            public void Configure(EntityTypeBuilder<User> builder)
            {
                builder.ToTable("Users");
                builder.HasKey(u => u.Id);
                builder.Property(u => u.Id).ValueGeneratedNever();

                builder.Property(u => u.Email)
                    .HasConversion(email => email.Value, value =>
EmailAddress.Create(value).Value)
                    .HasMaxLength(EmailAddress.MaxLength)
                    .IsRequired();
                builder.HasIndex(u => u.Email).IsUnique();

                builder.Property(u =>
u.DisplayName).HasMaxLength(User.MaxDisplayNameLength).IsRequired();
                builder.Property(u => u.TimeZoneId).HasMaxLength(64).IsRequired();
                builder.Property(u => u.Status).HasConversion<string>().HasMaxLength(16);
                builder.Property(u => u.SuspensionReason).HasMaxLength(500);

                // Roles and overrides are part of the aggregate: they have no life
                // outside a user, so they are owned rather than separate entities.
                builder.OwnsMany(u => u.Roles, roles =>
                {
                    roles.ToTable("UserRoles");
                    roles.WithOwner().HasForeignKey("UserId");
                    roles.HasKey("UserId", nameof(UserRole.RoleId));
                });
                builder.Navigation(u =>
u.Roles).HasField("_roles").UsePropertyAccessMode(PropertyAccessMode.Field);

                builder.OwnsMany(u => u.Overrides, overrides =>
                {
                    overrides.ToTable("UserPermissionOverrides");
                    overrides.WithOwner().HasForeignKey("UserId");
                    overrides.Property(o => o.Permission).HasConversion(new
PermissionConverter()).HasMaxLength(128);
                    overrides.Property(o => o.Effect).HasConversion<string>().HasMaxLength(8);
                    overrides.HasKey("UserId", nameof(PermissionOverride.Permission));
                });
```

```
                builder.Navigation(u =>
u.Overrides).HasField("_overrides").UsePropertyAccessMode(PropertyAccessMode.Field);

                // RoleIds is computed from Roles; it is not a column.
                builder.Ignore(u => u.RoleIds);
            }
        }
```

Roles and overrides are *owned* collections: they have no life outside a user, so they have no `DbSet` of their own. Navigations are mapped to the private fields so that EF Core can populate them while the public API stays read-only.

src/Sentinel.Infrastructure/Persistence/Converters/PermissionConverters.cs

```
        using Microsoft.EntityFrameworkCore.ChangeTracking;
        using Microsoft.EntityFrameworkCore.Storage.ValueConversion;
        using Sentinel.Domain.Access;

        namespace Sentinel.Infrastructure.Persistence.Converters;

        internal sealed class PermissionConverter() : ValueConverter<Permission, string>(
            permission => permission.Value,
            value => Permission.From(value));

        /// <summary>
        /// Stores a role's permissions as one space-separated column. A space can
        /// never appear inside a permission (the pattern forbids it), so splitting is
        /// unambiguous. A child table would be more "relational", but nobody ever
        /// queries "which roles grant X" in SQL; they ask the application.
        /// </summary>
        internal sealed class PermissionListConverter() : ValueConverter<List<Permission>, string>
(
            permissions => string.Join(' ', permissions.Select(p => p.Value)),
            value => value.Split(' ',
StringSplitOptions.RemoveEmptyEntries).Select(Permission.From).ToList());

        /// <summary>
        /// Without a comparer EF Core compares the list by reference and never
        /// notices that <c>Grant</c> added an item, so the change is silently lost.
        /// </summary>
        internal sealed class PermissionListComparer() : ValueComparer<List<Permission>>(
            (left, right) => left!.SequenceEqual(right!),
            list => list.Aggregate(0, (hash, p) => HashCode.Combine(hash, p.GetHashCode())),
            list => list.ToList());
```

src/Sentinel.Infrastructure/Persistence/Configurations/RoleConfiguration.cs

```
        using Microsoft.EntityFrameworkCore;
        using Microsoft.EntityFrameworkCore.Metadata.Builders;
        using Sentinel.Domain.Access;
        using Sentinel.Domain.Roles;
        using Sentinel.Infrastructure.Persistence.Converters;

        namespace Sentinel.Infrastructure.Persistence.Configurations;

        internal sealed class RoleConfiguration : IEntityTypeConfiguration<Role>
        {
            public void Configure(EntityTypeBuilder<Role> builder)
            {
                builder.ToTable("Roles");
                builder.HasKey(r => r.Id);
                builder.Property(r => r.Id).ValueGeneratedNever();

                builder.Property(r => r.Name).HasMaxLength(Role.MaxNameLength).IsRequired();
                builder.HasIndex(r => r.Name).IsUnique();
```

```
                builder.Property(r => r.Description).HasMaxLength(500);

                // The public Permissions property is a read-only view; map the
                // private list behind it, as one column.
                builder.Ignore(r => r.Permissions);
                builder.Property<List<Permission>>("_permissions")
                    .HasColumnName("Permissions")
                    .HasConversion(new PermissionListConverter(), new PermissionListComparer())
                    .HasMaxLength(2000)
                    .IsRequired();
            }
        }
```

The value comparer is not optional. Without it EF Core compares the list by reference, never notices that `Grant` added to it, and silently does not save. The test `Granting_a_role_permission_is_saved_and_changes_access` (Chapter 13) exists because this is exactly the bug you get.

src/Sentinel.Infrastructure/Persistence/Configurations/PageConfiguration.cs

```
        using Microsoft.EntityFrameworkCore;
        using Microsoft.EntityFrameworkCore.Metadata.Builders;
        using Microsoft.EntityFrameworkCore.Storage.ValueConversion;
        using Sentinel.Domain.Pages;
        using Sentinel.Infrastructure.Persistence.Converters;

        namespace Sentinel.Infrastructure.Persistence.Configurations;

        internal sealed class PageConfiguration : IEntityTypeConfiguration<Page>
        {
            public void Configure(EntityTypeBuilder<Page> builder)
            {
                builder.ToTable("Pages");
                builder.HasKey(p => p.Id);
                builder.Property(p => p.Id).ValueGeneratedNever();

                builder.Property(p => p.Path).HasMaxLength(200).IsRequired();
                builder.HasIndex(p => p.Path).IsUnique();
                builder.Property(p => p.Title).HasMaxLength(Page.MaxTitleLength).IsRequired();
                // The cast picks the non-generic overload; the generic one is fussy about
                // the nullability of Permission? versus Permission.
                builder.Property(p => p.RequiredPermission).HasConversion((ValueConverter)new
PermissionConverter()).HasMaxLength(128);

                // A real foreign key: the database refuses orphans. The evaluator
                // still copes with them, because data arrives from places other than
                // this database (an import, a test, a future cache) and the menu must
                // never crash. Belt and braces, on purpose.
                builder.HasOne<Page>()
                    .WithMany()
                    .HasForeignKey(p => p.ParentId)
                    .OnDelete(DeleteBehavior.Restrict);
            }
        }
```

src/Sentinel.Infrastructure/Persistence/Configurations/SecurityConfigurations.cs

```
        using Microsoft.EntityFrameworkCore;
        using Microsoft.EntityFrameworkCore.Metadata.Builders;
        using Sentinel.Domain.Security;
        using Sentinel.Domain.Users;

        namespace Sentinel.Infrastructure.Persistence.Configurations;

        internal sealed class SignInAttemptConfiguration : IEntityTypeConfiguration<SignInAttempt>
```

```
        {
            public void Configure(EntityTypeBuilder<SignInAttempt> builder)
            {
                builder.ToTable("SignInAttempts");
                builder.HasKey(a => a.Id);
                builder.Property(a => a.Id).ValueGeneratedNever();

                builder.HasOne<User>().WithMany().HasForeignKey(a =>
a.UserId).OnDelete(DeleteBehavior.Cascade);

                builder.Property(a => a.IpAddress).HasMaxLength(45).IsRequired(); // longest IPv6
text form
                builder.Property(a => a.CountryCode).HasMaxLength(2).IsFixedLength().IsRequired();
                builder.Property(a => a.City).HasMaxLength(100).IsRequired();
                builder.Property(a =>
a.UserAgent).HasMaxLength(SignInAttempt.MaxUserAgentLength).IsRequired();
                builder.Property(a => a.DeviceId).HasMaxLength(16).IsRequired();
                builder.Property(a => a.FailureReason).HasConversion<string>().HasMaxLength(32);
                builder.Ignore(a => a.Location);

                // Every read is "this user's attempts, newest first".
                builder.HasIndex(a => new { a.UserId, a.OccurredAtUtc }).IsDescending(false,
true);
            }
        }

        internal sealed class FindingAcknowledgementConfiguration :
IEntityTypeConfiguration<FindingAcknowledgement>
        {
            public void Configure(EntityTypeBuilder<FindingAcknowledgement> builder)
            {
                builder.ToTable("FindingAcknowledgements");
                builder.HasKey(a => a.Id);
                builder.Property(a => a.Id).ValueGeneratedNever();

                builder.HasOne<User>().WithMany().HasForeignKey(a =>
a.UserId).OnDelete(DeleteBehavior.Cascade);
                builder.Property(a =>
a.FindingKey).HasMaxLength(FindingAcknowledgement.MaxKeyLength).IsRequired();
                builder.HasIndex(a => new { a.UserId, a.FindingKey }).IsUnique();
            }
        }
```

The composite index on `(UserId, OccurredAtUtc DESC)` matches the one query that matters: “this user’s attempts, newest first”.

11.3 Services

`src/Sentinel.Infrastructure/Services/SystemClock.cs`

```
        using Sentinel.Application.Abstractions.Ports;

        namespace Sentinel.Infrastructure.Services;

        internal sealed class SystemClock(TimeProvider timeProvider) : IClock
        {
            public DateTime UtcNow => timeProvider.GetUtcNow().UtcDateTime;
        }
```

`TimeProvider` is .NET’s own clock abstraction (since .NET 8). Wrapping it in `IClock` keeps the Application layer independent of even that.

`src/Sentinel.Infrastructure/Services/StaticGeoLocator.cs`

```
using System.Net;
using Sentinel.Application.Abstractions.Ports;
using Sentinel.Domain.Security;

namespace Sentinel.Infrastructure.Services;

/// <summary>
/// A stand-in for a real geolocation database (MaxMind GeoLite2, ipinfo...).
/// The prefixes are illustrative: they make the demo data tell a story, and
/// they are not a claim about who owns those addresses.
/// </summary>
internal sealed class StaticGeoLocator : IGeoLocator
{
    private static readonly (string Prefix, GeoLocation Location)[] Table =
    [
        ("102.89.", GeoLocation.Create("NG", "Lagos", 6.5244, 3.3792)),
        ("105.112.", GeoLocation.Create("NG", "Abuja", 9.0765, 7.3986)),
        ("81.2.", GeoLocation.Create("GB", "London", 51.5072, -0.1276)),
        ("185.220.", GeoLocation.Create("DE", "Frankfurt", 50.1109, 8.6821)),
        ("145.53.", GeoLocation.Create("NL", "Amsterdam", 52.3676, 4.9041)),
        ("116.88.", GeoLocation.Create("SG", "Singapore", 1.3521, 103.8198)),
        ("72.229.", GeoLocation.Create("US", "New York", 40.7128, -74.0060)),
        ("177.38.", GeoLocation.Create("BR", "São Paulo", -23.5558, -46.6396)),
    ];

    public GeoLocation Locate(string ipAddress)
    {
        if (!IPAddress.TryParse(ipAddress, out var parsed) || IsPrivateOrLoopback(parsed))
            return GeoLocation.Unknown;

        var text = parsed.ToString();
        foreach (var (prefix, location) in Table)
        {
            if (text.StartsWith(prefix, StringComparison.Ordinal))
                return location;
        }

        return GeoLocation.Unknown;
    }

    private static bool IsPrivateOrLoopback(IPAddress address)
    {
        if (IPAddress.IsLoopback(address))
            return true;

        var bytes = address.GetAddressBytes();
        return bytes.Length == 4 && (bytes[0] == 10 ||
                                     (bytes[0] == 172 && bytes[1] is >= 16 and <= 31) ||
                                     (bytes[0] == 192 && bytes[1] == 168));
    }
}
```

11.4 Seeding the stories

`src/Sentinel.Infrastructure/Persistence/Seeding/DevelopmentDataSeeder.cs`

```
using Microsoft.EntityFrameworkCore;
using Sentinel.Application.Abstractions.Ports;
using Sentinel.Domain.Access;
using Sentinel.Domain.Pages;
using Sentinel.Domain.Roles;
using Sentinel.Domain.Security;
using Sentinel.Domain.Users;

namespace Sentinel.Infrastructure.Persistence.Seeding;

/// <summary>
```

```
        /// Fills an empty development database with a small company whose sign-in
        /// histories each tell one story the security page should recognise. All
        /// times are relative to "now", so the demo looks fresh whenever it runs.
        /// </summary>
        /// <remarks>
        /// The seeder uses the domain factories, not raw inserts, so seed data obeys
        /// the same invariants as real data. A seeder that bypasses the domain is how
        /// a development database ends up containing states production never can.
        /// </remarks>
        public sealed class DevelopmentDataSeeder(SentinelDbContext db, IClock clock, IGeoLocator
geoLocator)
        {
            private static class Agents
            {
                public const string ChromeWindows = "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36";
                public const string EdgeWindows = "Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36 Edg/141.0.0.0";
                public const string ChromeMac = "Mozilla/5.0 (Macintosh; Intel Mac OS X 10_15_7)
AppleWebKit/537.36 (KHTML, like Gecko) Chrome/141.0.0.0 Safari/537.36";
                public const string SafariIphone = "Mozilla/5.0 (iPhone; CPU iPhone OS 18_6 like
Mac OS X) AppleWebKit/605.1.15 (KHTML, like Gecko) Version/18.6 Mobile/15E148 Safari/604.1";
                public const string FirefoxLinux = "Mozilla/5.0 (X11; Linux x86_64; rv:143.0)
Gecko/20100101 Firefox/143.0";
                public const string PythonRequests = "python-requests/2.32.3";
            }

            private static class Ips
            {
                public const string Lagos = "102.89.34.";
                public const string Abuja = "105.112.18.";
                public const string London = "81.2.69.";
                public const string Frankfurt = "185.220.101.";
                public const string Amsterdam = "145.53.12.";
                public const string Singapore = "116.88.40.";
                public const string NewYork = "72.229.28.";
            }

            private readonly List<SignInAttempt> _attempts = [];
            private DateTime _now;

            public async Task SeedAsync(CancellationToken cancellationToken = default)
            {
                if (await db.Users.AnyAsync(cancellationToken))
                    return;

                // Whole minutes: the demo reads better without microseconds.
                var now = clock.UtcNow;
                _now = new DateTime(now.Year, now.Month, now.Day, now.Hour, now.Minute, 0,
DateTimeKind.Utc);

                var roles = SeedRoles();
                SeedPages();
                SeedUsers(roles);

                db.SignInAttempts.AddRange(_attempts);
                await db.SaveChangesAsync(cancellationToken);
            }

            private Dictionary<string, Role> SeedRoles()
            {
                var definitions = new (string Name, string Description, string[] Permissions)[]
                {
                    ("Administrator", "Full access to everything.", ["*"]),
                    ("Finance Manager", "Finance reports, ledger export and billing.",
["reports.view", "reports.finance", "reports.finance.*", "billing.*"]),
                    ("Finance Analyst", "Finance reports and billing, read only.",
["reports.view", "reports.finance", "billing.view"]),
                    ("Operations", "Operational reporting.", ["reports.view",
"reports.operations"]),
                    ("Support Agent", "The support desk.", ["support.*"]),
                    ("Auditor", "Every report, and read access to user administration.",
```

```
["reports.view", "reports.*", "admin.access", "admin.users"]),
                };

                var roles = new Dictionary<string, Role>();
                foreach (var (name, description, permissions) in definitions)
                {
                    var role = Role.Create(name, description).Value;
                    foreach (var permission in permissions)
                        role.Grant(Permission.From(permission));

                    roles[name] = role;
                    db.Roles.Add(role);
                }

                return roles;
            }

            private void SeedPages()
            {
                Page Add(string path, string title, Page? parent = null, string? permission =
null, bool isPublic = false, int order = 0)
                {
                    var page = Page.Create(
                        path, title, parent?.Id,
                        permission is null ? null : Permission.From(permission),
                        isPublic, order).Value;
                    db.Pages.Add(page);
                    return page;
                }

                var home = Add("/", "Dashboard", order: 0);

                var account = Add("/account", "My account", home, order: 10);
                Add("/account/security", "Account security", account, order: 0);

                var reports = Add("/reports", "Reports", home, "reports.view", order: 20);
                var finance = Add("/reports/finance", "Finance", reports, "reports.finance",
order: 0);
                Add("/reports/finance/export", "Ledger export", finance, "reports.finance.export",
order: 0);
                Add("/reports/operations", "Operations", reports, "reports.operations", order:
10);

                Add("/billing", "Billing", home, "billing.view", order: 30);
                Add("/support", "Support desk", home, "support.tickets", order: 40);

                var admin = Add("/admin", "Administration", home, "admin.access", order: 90);
                Add("/admin/users", "Users", admin, "admin.users", order: 0);
                Add("/admin/roles", "Roles", admin, "admin.roles", order: 10);
                Add("/admin/access-explorer", "Access explorer", admin, "admin.users", order: 20);

                // Public pages sit outside the tree: they must work signed out.
                Add("/help", "Help centre", isPublic: true, order: 100);
                Add("/status", "System status", isPublic: true, order: 110);
            }

            private void SeedUsers(Dictionary<string, Role> roles)
            {
                User Add(string email, string name, string zone, params string[] roleNames)
                {
                    var user = User.Create(EmailAddress.Create(email).Value, name, zone,
_now.AddDays(-120)).Value;
                    foreach (var roleName in roleNames)
                        user.AssignRole(roles[roleName].Id, _now.AddDays(-120));
                    db.Users.Add(user);
                    return user;
                }

                // Ada: an administrator with an ordinary history and one new phone. Low.
                var ada = Add("ada.okafor@sentinel.dev", "Ada Okafor", "Africa/Lagos",
"Administrator");
                Workdays(ada, Ips.Lagos, Agents.ChromeWindows, days: 60, localHour: 8.5);
```

```
                At(ada, _now.AddDays(-3).AddHours(-2), Ips.Lagos, Agents.SafariIphone);
                At(ada, _now.AddHours(-2), Ips.Lagos, Agents.ChromeWindows);

                // Bola: Lagos, then London forty minutes later. Impossible travel. High.
                var bola = Add("bola.adeyemi@sentinel.dev", "Bola Adeyemi", "Africa/Lagos",
"Finance Manager");
                Workdays(bola, Ips.Lagos, Agents.ChromeWindows, days: 60, localHour: 9,
skipLastDays: 2);
                var lagosSignIn = _now.AddDays(-2).Date.AddHours(8); // 09:00 in Lagos
                At(bola, lagosSignIn, Ips.Lagos, Agents.ChromeWindows);
                At(bola, lagosSignIn.AddMinutes(40), Ips.London, Agents.SafariIphone);

                // Chen: eight failures from Frankfurt in nine minutes, none succeeded. Medium.
                var chen = Add("chen.wei@sentinel.dev", "Chen Wei", "Asia/Singapore",
"Operations");
                Workdays(chen, Ips.Singapore, Agents.ChromeMac, days: 60, localHour: 9);
                Burst(chen, _now.AddDays(-1).Date.AddHours(3).AddMinutes(10), Ips.Frankfurt,
Agents.FirefoxLinux, failures: 8, spacing: TimeSpan.FromSeconds(70));

                // Dana: a genuine trip from New York to London, new laptop too. Medium + Low.
                var dana = Add("dana.smith@sentinel.dev", "Dana Smith", "America/New_York",
"Support Agent");
                Workdays(dana, Ips.NewYork, Agents.FirefoxLinux, days: 60, localHour: 9,
skipLastDays: 5);
                for (var day = 5; day >= 1; day--)
                    At(dana, _now.AddDays(-day).Date.AddHours(9), Ips.London, Agents.EdgeWindows);

                // Emeka: a contractor. Finance Manager, but explicitly denied the ledger export.
Clean.
                var emeka = Add("emeka.nwosu@sentinel.dev", "Emeka Nwosu", "Europe/London",
"Finance Manager");
                emeka.SetOverride(Permission.From("reports.finance.export"),
PermissionEffect.Deny);
                Workdays(emeka, Ips.London, Agents.ChromeWindows, days: 60, localHour: 9);

                // Femi: an auditor in Abuja. Clean.
                var femi = Add("femi.bello@sentinel.dev", "Femi Bello", "Africa/Lagos",
"Auditor");
                Workdays(femi, Ips.Abuja, Agents.EdgeWindows, days: 60, localHour: 10);

                // Grace: left the company. Suspended; sees public pages only.
                var grace = Add("grace.eze@sentinel.dev", "Grace Eze", "Africa/Lagos", "Support
Agent");
                Workdays(grace, Ips.Lagos, Agents.ChromeWindows, days: 60, localHour: 8,
skipLastDays: 30);
                grace.Suspend("Left the company.");

                // Tunde: runs integration scripts every four hours, so has more than
                // 500 sign-ins in ninety days. Yesterday someone in Amsterdam guessed
                // the password. High. This is the account in ticket SUP-4219.
                var tunde = Add("tunde.bakare@sentinel.dev", "Tunde Bakare", "Africa/Lagos",
"Operations");
                Every(tunde, Ips.Lagos, Agents.PythonRequests, days: 90, interval:
TimeSpan.FromHours(4));
                var attack = _now.AddDays(-1).Date.AddHours(21).AddMinutes(5);
                Burst(tunde, attack, Ips.Amsterdam, Agents.ChromeWindows, failures: 7, spacing:
TimeSpan.FromSeconds(50));
                At(tunde, attack.AddMinutes(6.5), Ips.Amsterdam, Agents.ChromeWindows);
            }

            /// <summary>One successful sign-in per weekday at <paramref name="localHour"/> in the
user's own zone.</summary>
            private void Workdays(User user, string ipPrefix, string agent, int days, double
localHour, int skipLastDays = 0)
            {
                var zone = TimeZoneInfo.FindSystemTimeZoneById(user.TimeZoneId);
                // string.GetHashCode is randomised per process; seed from the characters
                // instead so the demo data is the same on every run.
                var random = new Random(user.Email.Value.Sum(c => c));

                for (var day = days; day > skipLastDays; day--)
                {
```

```
                    var localDate = TimeZoneInfo.ConvertTimeFromUtc(_now, zone).Date.AddDays(-
day);
                    if (localDate.DayOfWeek is DayOfWeek.Saturday or DayOfWeek.Sunday)
                        continue;

                    var local = localDate.AddHours(localHour).AddMinutes(random.Next(-20, 25));
                    At(user, TimeZoneInfo.ConvertTimeToUtc(DateTime.SpecifyKind(local,
DateTimeKind.Unspecified), zone), ipPrefix, agent, random);
                }
            }

            private void Every(User user, string ipPrefix, string agent, int days, TimeSpan
interval)
            {
                for (var at = _now.AddDays(-days); at < _now; at += interval)
                    At(user, at, ipPrefix, agent);
            }

            private void Burst(User user, DateTime startUtc, string ipPrefix, string agent, int
failures, TimeSpan spacing)
            {
                for (var i = 0; i < failures; i++)
                    At(user, startUtc + spacing * i, ipPrefix, agent, succeeded: false);
            }

            private void At(User user, DateTime occurredAtUtc, string ipPrefix, string agent,
Random? random = null, bool succeeded = true)
            {
                if (occurredAtUtc >= _now)
                    return;

                var ip = ipPrefix + ((random?.Next(2, 250)) ?? 17);
                var attempt = SignInAttempt.Record(
                    user.Id,
                    DateTime.SpecifyKind(occurredAtUtc, DateTimeKind.Utc),
                    ip,
                    geoLocator.Locate(ip),
                    agent,
                    succeeded,
                    succeeded ? null : SignInFailureReason.WrongPassword).Value;

                _attempts.Add(attempt);
            }
        }
```

Two details are worth stealing:

- **The seeder uses the domain factories**, so seed data obeys the same rules as real data.
- **Its random numbers are seeded from the email's characters**, not `string.GetHashCode()`, which in .NET is randomised per process. The first draft used `GetHashCode` and produced different demo data on every run, which is exactly the kind of non-determinism that turns a debugging session into a ghost hunt.

11.5 Registration and migrations

`src/Sentinel.Infrastructure/DependencyInjection.cs`

```
        using Microsoft.EntityFrameworkCore;
        using Microsoft.Extensions.Configuration;
        using Microsoft.Extensions.DependencyInjection;
        using Sentinel.Application.Abstractions.Ports;
        using Sentinel.Infrastructure.Persistence;
        using Sentinel.Infrastructure.Persistence.Seeding;
        using Sentinel.Infrastructure.Services;
```

```
namespace Sentinel.Infrastructure;

public static class DependencyInjection
{
    public const string ConnectionStringName = "Sentinel";

    public static IServiceCollection AddInfrastructure(this IServiceCollection services,
IConfiguration configuration)
    {
        var connectionString = configuration.GetConnectionString(ConnectionStringName)
            ?? throw new InvalidOperationException(
                $"Connection string '{ConnectionStringName}' is missing. Set
ConnectionStrings:{ConnectionStringName} in appsettings or user secrets.");

        services.AddDbContext<SentinelDbContext>(options =>
            options.UseSqlServer(connectionString, sql => sql.EnableRetryOnFailure()));

        // One context instance per request, reachable through the port.
        services.AddScoped<IApplicationDbContext>(sp =>
sp.GetRequiredService<SentinelDbContext>());

        services.AddSingleton(TimeProvider.System);
        services.AddSingleton<IClock, SystemClock>();
        services.AddSingleton<IGeoLocator, StaticGeoLocator>();
        services.AddScoped<DevelopmentDataSeeder>();

        return services;
    }
}
```

`src/Sentinel.Infrastructure/Persistence/DesignTimeDbContextFactory.cs`

```
using Microsoft.EntityFrameworkCore;
using Microsoft.EntityFrameworkCore.Design;

namespace Sentinel.Infrastructure.Persistence;

/// <summary>
/// Lets <c>dotnet ef migrations add</c> build a context without starting the
/// API. The connection string is only used for commands that touch a real
/// database (<c>database update</c>); adding a migration never connects.
/// </summary>
internal sealed class DesignTimeDbContextFactory :
IDesignTimeDbContextFactory<SentinelDbContext>
{
    public SentinelDbContext CreateDbContext(string[] args)
    {
        var connectionString = Environment.GetEnvironmentVariable("SENTINEL_CONNECTION")
            ?? "Server=
(localdb)\\MSSQLLocalDB;Database=Sentinel;Trusted_Connection=True;TrustServerCertificate=True";

        var options = new DbContextOptionsBuilder<SentinelDbContext>()
            .UseSqlServer(connectionString)
            .Options;

        return new SentinelDbContext(options);
    }
}
```

```
dotnet ef migrations add InitialCreate `
  --project src/Sentinel.Infrastructure --startup-project src/Sentinel.Infrastructure `
  --output-dir Persistence/Migrations
```

The API applies migrations and seeds on start-up in development (Chapter 12). In production, apply migrations from the pipeline with `dotnet ef migrations bundle`, never from the running application.

Chapter 12 · The API Surface

12.1 The composition root

src/Sentinel.Api/Program.cs

```
using Microsoft.AspNetCore.Authentication;
using Microsoft.EntityFrameworkCore;
using Sentinel.Api.Authentication;
using Sentinel.Api.Endpoints;
using Sentinel.Application;
using Sentinel.Infrastructure;
using Sentinel.Infrastructure.Persistence;
using Sentinel.Infrastructure.Persistence.Seeding;

var builder = WebApplication.CreateBuilder(args);

// ---- Composition root: the only place that knows every layer. ----
builder.Services.AddApplication();
builder.Services.AddInfrastructure(builder.Configuration);

builder.Services
    .AddAuthentication(DemoUserAuthenticationHandler.SchemeName)
    .AddScheme<AuthenticationSchemeOptions, DemoUserAuthenticationHandler>
(DemoUserAuthenticationHandler.SchemeName, null);
builder.Services.AddAuthorization();

builder.Services.AddProblemDetails();
builder.Services.AddOpenApi();

var app = builder.Build();

if (!app.Environment.IsDevelopment())
{
    // The demo scheme trusts a header. That is acceptable on a laptop and
    // nowhere else, so fail loudly rather than run insecurely by accident.
    throw new InvalidOperationException(
        "Sentinel is configured with development-only authentication. Replace
DemoUserAuthenticationHandler before deploying.");
}

app.UseExceptionHandler();
app.UseStatusCodePages();

app.UseAuthentication();
app.UseAuthorization();

app.MapOpenApi();
app.MapMeEndpoints();
app.MapUserEndpoints();
app.MapRoleEndpoints();
app.MapPageEndpoints();
app.MapSignInEndpoints();
app.MapDevelopmentEndpoints();
app.MapGet("/health", () => TypedResults.Ok(new { status = "ok" })).WithTags("Health");

if (app.Configuration.GetValue("Database:MigrateAndSeedOnStartup", true))
{
    await using var scope = app.Services.CreateAsyncScope();
    var db = scope.ServiceProvider.GetRequiredService<SentinelDbContext>();
    await db.Database.MigrateAsync();
    await scope.ServiceProvider.GetRequiredService<DevelopmentDataSeeder>().SeedAsync();
}

await app.RunAsync();
```

```
        /// <summary>Exposed so WebApplicationFactory&lt;Program&gt; can host the API in tests.
</summary>
        public partial class Program;
```

Program.cs refuses to start outside Development. The authentication scheme below trusts a header; that is acceptable on a laptop and nowhere else, so the application fails loudly rather than running insecurely by accident.

`src/Sentinel.Api/appsettings.json`

```
        {
          "Logging": {
            "LogLevel": {
              "Default": "Information",
              "Microsoft.AspNetCore": "Warning",
              "Microsoft.EntityFrameworkCore.Database.Command": "Warning"
            }
          },
          "AllowedHosts": "*",
          "ConnectionStrings": {
            "Sentinel": "Server=
(localdb)\\MSSQLLocalDB;Database=Sentinel;Trusted_Connection=True;TrustServerCertificate=True"
          },
          "Database": {
            "MigrateAndSeedOnStartup": true
          }
        }
```

12.2 Results become HTTP in one place

`src/Sentinel.Api/Http/ResultHttpExtensions.cs`

```
        using Sentinel.Application.Abstractions.Validation;
        using Sentinel.Domain.Common;

        namespace Sentinel.Api.Http;

        /// <summary>
        /// The single place where a business outcome becomes an HTTP response.
        /// Every endpoint ends in one of these calls, so the mapping from
        /// <see cref="ErrorType"/> to status code cannot differ between endpoints.
        /// </summary>
        public static class ResultHttpExtensions
        {
            public static IResult ToNoContent(this Result result) =>
                result.IsSuccess ? TypedResults.NoContent() : result.Error!.ToProblem();

            public static IResult ToOk<T>(this Result<T> result) =>
                result.IsSuccess ? TypedResults.Ok(result.Value) : result.Error!.ToProblem();

            public static IResult ToCreated(this Result<Guid> result, Func<Guid, string> location)
=>
                result.IsSuccess
                    ? TypedResults.Created(location(result.Value), new
Sentinel.Contracts.CreatedResponse(result.Value))
                    : result.Error!.ToProblem();

            public static IResult ToProblem(this Error error)
            {
                if (error is ValidationError validation)
                {
                    var errors = validation.Failures
                        .GroupBy(f => f.Property)
```

```
                        .ToDictionary(g => g.Key, g => g.Select(f => f.Message).ToArray());

                    return TypedResults.ValidationProblem(errors, title: validation.Message, type:
validation.Code);
                }

                var status = error.Type switch
                {
                    ErrorType.Validation => StatusCodes.Status400BadRequest,
                    ErrorType.NotFound => StatusCodes.Status404NotFound,
                    ErrorType.Conflict => StatusCodes.Status409Conflict,
                    ErrorType.Forbidden => StatusCodes.Status403Forbidden,
                    _ => StatusCodes.Status500InternalServerError
                };

                return TypedResults.Problem(
                    statusCode: status,
                    title: error.Code,
                    detail: error.Message,
                    extensions: new Dictionary<string, object?> { ["code"] = error.Code });
            }
        }
```

Every endpoint ends in `.ToOk()`, `.ToNoContent()` or `.ToCreated(...)`. The mapping from `ErrorType` to status code exists once, so `NotFound` can never be a 404 in one endpoint and a 400 in another. Responses are RFC 9457 problem details, with the stable error `code` as an extension member that clients can switch on.

12.3 Development authentication

`src/Sentinel.Api/Authentication/DemoUserAuthenticationHandler.cs`

```
        using System.Security.Claims;
        using System.Text.Encodings.Web;
        using Microsoft.AspNetCore.Authentication;
        using Microsoft.Extensions.Options;
        using Sentinel.Application.Abstractions.Messaging;
        using Sentinel.Application.Users.FindUserByEmail;

        namespace Sentinel.Api.Authentication;

        /// <summary>
        /// Development-only authentication: the caller says who they are in an
        /// <c>X-Demo-User</c> header and we believe them. It exists so the book can
        /// switch between eight users in one click without an identity provider.
        /// </summary>
        /// <remarks>
        /// In production, replace this scheme with JWT bearer or OpenID Connect.
        /// Nothing else changes: endpoints read the user id from the
        /// <see cref="SentinelClaims.UserId"/> claim, whoever put it there.
        /// Program.cs refuses to start with this scheme outside Development.
        /// </remarks>
        public sealed class DemoUserAuthenticationHandler(
            IOptionsMonitor<AuthenticationSchemeOptions> options,
            ILoggerFactory logger,
            UrlEncoder encoder,
            IDispatcher dispatcher)
            : AuthenticationHandler<AuthenticationSchemeOptions>(options, logger, encoder)
        {
            public const string SchemeName = "Demo";
            public const string HeaderName = "X-Demo-User";

            protected override async Task<AuthenticateResult> HandleAuthenticateAsync()
            {
                if (!Request.Headers.TryGetValue(HeaderName, out var values) ||
string.IsNullOrWhiteSpace(values.ToString()))
```

```
                    return AuthenticateResult.NoResult(); // anonymous, not a failure

                var email = values.ToString().Trim();
                var userId = await dispatcher.Send(new FindUserByEmailQuery(email),
Context.RequestAborted);
                if (userId.IsFailure)
                    return AuthenticateResult.Fail($"Unknown demo user '{email}'.");

                var identity = new ClaimsIdentity(
                    [
                        new Claim(SentinelClaims.UserId, userId.Value.ToString()),
                        new Claim(ClaimTypes.Email, email)
                    ],
                    SchemeName);

                return AuthenticateResult.Success(new AuthenticationTicket(new
ClaimsPrincipal(identity), SchemeName));
            }
        }
```

`src/Sentinel.Api/Authentication/SentinelClaims.cs`

```
        using System.Security.Claims;

        namespace Sentinel.Api.Authentication;

        public static class SentinelClaims
        {
            public const string UserId = "sentinel:user_id";

            /// <summary>The signed-in user's id, or null for an anonymous request.</summary>
            public static Guid? GetUserId(this ClaimsPrincipal principal) =>
                Guid.TryParse(principal.FindFirstValue(UserId), out var id) ? id : null;

            /// <summary>For endpoints that already require authentication.</summary>
            public static Guid GetRequiredUserId(this ClaimsPrincipal principal) =>
                principal.GetUserId() ?? throw new InvalidOperationException("The endpoint
requires authentication but no user id claim is present.");
        }
```

The handler distinguishes *no header* (`NoResult`: anonymous, which some endpoints allow) from *a header naming nobody* (`Fail`: 401). Endpoints read only the `sentinel:user_id` claim, so replacing this scheme with JWT bearer changes nothing else.

12.4 Guarding endpoints with permissions

`src/Sentinel.Api/Authentication/PermissionFilter.cs`

```
        using Sentinel.Application.Abstractions.Messaging;
        using Sentinel.Application.Access.HasPermission;

        namespace Sentinel.Api.Authentication;

        /// <summary>
        /// Guards an endpoint with a permission, evaluated by the same domain rules
        /// that build the menu. "Hidden in the UI" is not security; this is.
        /// </summary>
        public static class PermissionFilter
        {
            public static TBuilder RequirePermission<TBuilder>(this TBuilder builder, string
permission)
                where TBuilder : IEndpointConventionBuilder
            {
                builder.RequireAuthorization();
```

```
                builder.AddEndpointFilter(async (context, next) =>
                {
                    var http = context.HttpContext;
                    var userId = http.User.GetRequiredUserId();
                    var dispatcher = http.RequestServices.GetRequiredService<IDispatcher>();

                    var allowed = await dispatcher.Send(new HasPermissionQuery(userId,
permission), http.RequestAborted);

                    return allowed is { IsSuccess: true, Value: true }
                        ? await next(context)
                        : TypedResults.Problem(
                            statusCode: StatusCodes.Status403Forbidden,
                            title: "Forbidden",
                            detail: $"This action requires the '{permission}' permission.");
                });

                return builder;
            }
        }
```

Hiding a menu item is user experience. This filter is security: it asks the same domain rules, through the dispatcher, on every request.

12.5 The endpoint modules

One static class per feature, one `Map...` method each:

src/Sentinel.Api/Endpoints/MeEndpoints.cs

```
        using System.Security.Claims;
        using Sentinel.Api.Authentication;
        using Sentinel.Api.Http;
        using Sentinel.Application.Abstractions.Messaging;
        using Sentinel.Application.Pages.CheckPageAccess;
        using Sentinel.Application.Pages.GetNavigation;
        using Sentinel.Application.Security.AcknowledgeFinding;
        using Sentinel.Application.Security.GetAccountSecurityStatus;
        using Sentinel.Application.Security.GetSignInHistory;
        using Sentinel.Application.Users.GetCurrentUser;

        namespace Sentinel.Api.Endpoints;

        /// <summary>Endpoints about the caller themselves. No admin permission involved.
</summary>
        public static class MeEndpoints
        {
            public static IEndpointRouteBuilder MapMeEndpoints(this IEndpointRouteBuilder app)
            {
                var me = app.MapGroup("/api/me").WithTags("Me");

                me.MapGet("/", async (ClaimsPrincipal user, IDispatcher dispatcher,
CancellationToken ct) =>
                        (await dispatcher.Send(new GetCurrentUserQuery(user.GetRequiredUserId()),
ct)).ToOk())
                    .RequireAuthorization();

                // The two access endpoints allow anonymous callers: a signed-out
                // visitor still needs a menu (the public pages) and a guard verdict.
                me.MapGet("/navigation", async (ClaimsPrincipal user, IDispatcher dispatcher,
CancellationToken ct) =>
                    (await dispatcher.Send(new GetNavigationQuery(user.GetUserId()), ct)).ToOk());

                me.MapGet("/page-access", async (string path, ClaimsPrincipal user, IDispatcher
dispatcher, CancellationToken ct) =>
                    (await dispatcher.Send(new CheckPageAccessQuery(user.GetUserId(), path),
```

```
ct)).ToOk());

                me.MapGet("/security", async (ClaimsPrincipal user, IDispatcher dispatcher,
CancellationToken ct) =>
                        (await dispatcher.Send(new
GetAccountSecurityStatusQuery(user.GetRequiredUserId()), ct)).ToOk())
                    .RequireAuthorization();

                me.MapGet("/sign-ins", async (ClaimsPrincipal user, IDispatcher dispatcher,
CancellationToken ct, int page = 1, int pageSize = 50) =>
                        (await dispatcher.Send(new GetSignInHistoryQuery(user.GetRequiredUserId(),
page, pageSize), ct)).ToOk())
                    .RequireAuthorization();

                me.MapPost("/security/findings/{key}/acknowledge", async (string key,
ClaimsPrincipal user, IDispatcher dispatcher, CancellationToken ct) =>
                        (await dispatcher.Send(new
AcknowledgeFindingCommand(user.GetRequiredUserId(), key), ct)).ToNoContent())
                    .RequireAuthorization();

                return app;
            }
        }
```

`src/Sentinel.Api/Endpoints/UserEndpoints.cs`

```
        using Sentinel.Api.Authentication;
        using Sentinel.Api.Http;
        using Sentinel.Application.Abstractions.Messaging;
        using Sentinel.Application.Pages.GetPageAccess;
        using Sentinel.Application.Security.GetAccountSecurityStatus;
        using Sentinel.Application.Users.CreateUser;
        using Sentinel.Application.Users.GetUserById;
        using Sentinel.Application.Users.GetUsers;
        using Sentinel.Application.Users.PermissionOverrides;
        using Sentinel.Application.Users.RoleAssignment;
        using Sentinel.Application.Users.Suspension;
        using Sentinel.Contracts;

        namespace Sentinel.Api.Endpoints;

        /// <summary>
        /// User administration. Reading needs <c>admin.users</c>; changing needs
        /// <c>admin.users.manage</c>, so an auditor can look without touching.
        /// </summary>
        public static class UserEndpoints
        {
            public const string ReadPermission = "admin.users";
            public const string ManagePermission = "admin.users.manage";

            public static IEndpointRouteBuilder MapUserEndpoints(this IEndpointRouteBuilder app)
            {
                var users = app.MapGroup("/api/users").WithTags("Users");

                users.MapGet("/", async (IDispatcher dispatcher, CancellationToken ct, string?
search, int page = 1, int pageSize = 25) =>
                        (await dispatcher.Send(new GetUsersQuery(search, page, pageSize),
ct)).ToOk())
                    .RequirePermission(ReadPermission);

                users.MapGet("/{id:guid}", async (Guid id, IDispatcher dispatcher,
CancellationToken ct) =>
                        (await dispatcher.Send(new GetUserByIdQuery(id), ct)).ToOk())
                    .RequirePermission(ReadPermission);

                users.MapGet("/{id:guid}/page-access", async (Guid id, IDispatcher dispatcher,
CancellationToken ct) =>
                        (await dispatcher.Send(new GetPageAccessQuery(id), ct)).ToOk())
                    .RequirePermission(ReadPermission);
```

```
                users.MapGet("/{id:guid}/security", async (Guid id, IDispatcher dispatcher,
CancellationToken ct) =>
                        (await dispatcher.Send(new GetAccountSecurityStatusQuery(id), ct)).ToOk())
                    .RequirePermission(ReadPermission);

                users.MapPost("/", async (CreateUserRequest body, IDispatcher dispatcher,
CancellationToken ct) =>
                        (await dispatcher.Send(new CreateUserCommand(body.Email, body.DisplayName,
body.TimeZoneId), ct))
                            .ToCreated(id => $"/api/users/{id}"))
                    .RequirePermission(ManagePermission);

                users.MapPost("/{id:guid}/roles", async (Guid id, AssignRoleRequest body,
IDispatcher dispatcher, CancellationToken ct) =>
                        (await dispatcher.Send(new AssignRoleCommand(id, body.RoleId),
ct)).ToNoContent())
                    .RequirePermission(ManagePermission);

                users.MapDelete("/{id:guid}/roles/{roleId:guid}", async (Guid id, Guid roleId,
IDispatcher dispatcher, CancellationToken ct) =>
                        (await dispatcher.Send(new RevokeRoleCommand(id, roleId),
ct)).ToNoContent())
                    .RequirePermission(ManagePermission);

                users.MapPost("/{id:guid}/suspend", async (Guid id, SuspendUserRequest body,
IDispatcher dispatcher, CancellationToken ct) =>
                        (await dispatcher.Send(new SuspendUserCommand(id, body.Reason),
ct)).ToNoContent())
                    .RequirePermission(ManagePermission);

                users.MapPost("/{id:guid}/reactivate", async (Guid id, IDispatcher dispatcher,
CancellationToken ct) =>
                        (await dispatcher.Send(new ReactivateUserCommand(id), ct)).ToNoContent())
                    .RequirePermission(ManagePermission);

                users.MapPut("/{id:guid}/overrides", async (Guid id, SetPermissionOverrideRequest
body, IDispatcher dispatcher, CancellationToken ct) =>
                        (await dispatcher.Send(new SetPermissionOverrideCommand(id,
body.Permission, body.Effect), ct)).ToNoContent())
                    .RequirePermission(ManagePermission);

                users.MapDelete("/{id:guid}/overrides/{permission}", async (Guid id, string
permission, IDispatcher dispatcher, CancellationToken ct) =>
                        (await dispatcher.Send(new ClearPermissionOverrideCommand(id, permission),
ct)).ToNoContent())
                    .RequirePermission(ManagePermission);

                return app;
            }
        }
```

The roles, pages and sign-in modules follow the same pattern and are in the repository. Read and change permissions are separate (`admin.users` versus `admin.users.manage`), which is why the auditor can open the user list but gets a 403 when creating a user.

12.6 Trying it by hand

`src/Sentinel.Api/Sentinel.http`

```
# Requests for the Sentinel API. Open in Visual Studio, Rider or VS Code
# (REST Client extension) and click "Send Request" above each one.
# The X-Demo-User header is the development-only identity (Chapter 12).

@api = http://localhost:5080
@admin = ada.okafor@sentinel.dev
```

```
### Who am I?
GET {{api}}/api/me
X-Demo-User: {{admin}}

### My menu (try with and without the header)
GET {{api}}/api/me/navigation
X-Demo-User: emeka.nwosu@sentinel.dev

### May Emeka open the ledger export? (no: explicitly denied)
GET {{api}}/api/me/page-access?path=/reports/finance/export
X-Demo-User: emeka.nwosu@sentinel.dev

### Is my account safe?
GET {{api}}/api/me/security
X-Demo-User: tunde.bakare@sentinel.dev

### Simulate the identity provider: a failed sign-in from Frankfurt
POST {{api}}/api/sign-ins
X-Demo-User: {{admin}}
Content-Type: application/json

{
  "email": "femi.bello@sentinel.dev",
  "occurredAt": "{{$datetime iso8601}}",
  "ipAddress": "185.220.101.9",
  "userAgent": "curl/8.9.1",
  "succeeded": false,
  "failureReason": "WrongPassword"
}

### A timestamp without a zone is accepted by DateTimeOffset as LOCAL time
### on the server. Compare with the request above and read Chapter 22.
POST {{api}}/api/sign-ins
X-Demo-User: {{admin}}
Content-Type: application/json

{
  "email": "femi.bello@sentinel.dev",
  "occurredAt": "2026-10-25T09:00:00",
  "ipAddress": "185.220.101.9",
  "succeeded": true
}

### Validation problem (400 with one entry per field)
POST {{api}}/api/users
X-Demo-User: {{admin}}
Content-Type: application/json

{ "email": "", "displayName": "", "timeZoneId": "Africa/Lagos" }

### Forbidden: an auditor may read users but not create them
POST {{api}}/api/users
X-Demo-User: femi.bello@sentinel.dev
Content-Type: application/json

{ "email": "new.person@sentinel.dev", "displayName": "New Person", "timeZoneId": "Africa/Lagos" }
```

Run these with the API started. The second sign-in request is a preview of Chapter 22: send a timestamp with no zone and see what the server does with it.

Chapter 13 · Testing the Back End

Sentinel has four test projects, one per kind of question.

Project	Question it answers	Speed
Domain.Tests	Are the rules right?	~1 s for 127 tests
Application.Tests	Do the slices do the right thing with a real database?	~15 s
Api.Tests	Does HTTP behave: status codes, auth, the wire format?	~15 s
Architecture.Tests	Does the code still obey its own structure?	~3 s

13.1 Domain tests

You have seen the evaluator and detector tests in Chapters 7 and 8. The remaining domain tests follow the same habits: a `[Theory]` for every rule that is a table, and a test on each side of every boundary.

`tests/Sentinel.Domain.Tests/Access/PermissionTests.cs`

```
using Sentinel.Domain.Access;

namespace Sentinel.Domain.Tests.Access;

public class PermissionTests
{
    [Theory]
    [InlineData("reports.finance", "reports.finance")]
    [InlineData("  Reports.Finance ", "reports.finance")]
    [InlineData("reports.*", "reports.*")]
    [InlineData("*", "*")]
    [InlineData("a", "a")]
    [InlineData("billing.view-all", "billing.view-all")]
    public void Create_normalises_valid_names(string input, string expected)
    {
        var result = Permission.Create(input);

        Assert.True(result.IsSuccess);
        Assert.Equal(expected, result.Value.Value);
    }

    [Theory]
    [InlineData(null)]
    [InlineData("")]
    [InlineData("   ")]
    [InlineData("reports.")]
    [InlineData(".reports")]
    [InlineData("reports..finance")]
    [InlineData("reports finance")]
    [InlineData("reports.*.finance")] // wildcard only at the end
    [InlineData("*.reports")]
    [InlineData("reports*")]
    public void Create_rejects_malformed_names(string? input)
    {
        var result = Permission.Create(input);

        Assert.True(result.IsFailure);
        Assert.Equal(Common.ErrorType.Validation, result.Error!.Type);
```

```
            }

            [Theory]
            [InlineData("reports.finance", "reports.finance", true)]
            [InlineData("reports.finance", "reports.operations", false)]
            [InlineData("reports.*", "reports.finance", true)]
            [InlineData("reports.*", "reports.finance.export", true)]  // any depth
            [InlineData("reports.*", "reports", false)]                // not the parent itself
            [InlineData("reports.*", "reportsx.finance", false)]       // prefix is segment-aware
            [InlineData("*", "anything.at.all", true)]
            [InlineData("reports.finance", "reports.finance.export", false)] // a concrete grant
has no children
            public void Covers(string grant, string required, bool expected)
            {
                Assert.Equal(expected, Permission.From(grant).Covers(Permission.From(required)));
            }

            [Fact]
            public void Equality_is_by_value()
            {
                Assert.Equal(Permission.From("a.b"), Permission.From("A.B"));
                Assert.Equal(Permission.From("a.b").GetHashCode(),
Permission.From("A.B").GetHashCode());
            }
        }
```

tests/Sentinel.Domain.Tests/Users/UserTests.cs

```
        using Sentinel.Domain.Access;
        using Sentinel.Domain.Common;
        using Sentinel.Domain.Users;

        namespace Sentinel.Domain.Tests.Users;

        public class UserTests
        {
            private static readonly DateTime Now = History.Now;

            private static User NewUser() =>
                User.Create(EmailAddress.Create("ada@example.com").Value, "Ada", "Africa/Lagos",
Now).Value;

            [Theory]
            [InlineData(" Ada@Example.COM ", "ada@example.com")]
            [InlineData("first.last+tag@sub.example.org", "first.last+tag@sub.example.org")]
            public void Email_is_normalised(string input, string expected)
            {
                Assert.Equal(expected, EmailAddress.Create(input).Value.Value);
            }

            [Theory]
            [InlineData("")]
            [InlineData("ada")]
            [InlineData("ada@localhost")]
            [InlineData("Ada <ada@example.com>")]
            public void Invalid_emails_are_rejected(string input)
            {
                Assert.True(EmailAddress.Create(input).IsFailure);
            }

            [Fact]
            public void Unknown_time_zones_are_rejected()
            {
                var result = User.Create(EmailAddress.Create("a@b.com").Value, "A",
"Mars/Olympus_Mons", Now);

                Assert.Equal("User.UnknownTimeZone", result.Error!.Code);
            }
```

```
            [Fact]
            public void A_new_user_is_active_and_trimmed()
            {
                var user = User.Create(EmailAddress.Create("a@b.com").Value, "  Ada  ",
"Europe/London", Now).Value;

                Assert.Equal(UserStatus.Active, user.Status);
                Assert.Equal("Ada", user.DisplayName);
            }

            [Fact]
            public void A_role_cannot_be_assigned_twice()
            {
                var user = NewUser();
                var roleId = Guid.NewGuid();

                Assert.True(user.AssignRole(roleId, Now).IsSuccess);
                Assert.Equal(ErrorType.Conflict, user.AssignRole(roleId, Now).Error!.Type);
                Assert.Single(user.Roles);
            }

            [Fact]
            public void Revoking_a_role_the_user_does_not_hold_is_not_found()
            {
                Assert.Equal(ErrorType.NotFound,
NewUser().RevokeRole(Guid.NewGuid()).Error!.Type);
            }

            [Fact]
            public void Setting_an_override_replaces_any_existing_one_for_that_permission()
            {
                var user = NewUser();
                var export = Permission.From("reports.finance.export");

                user.SetOverride(export, PermissionEffect.Allow);
                user.SetOverride(export, PermissionEffect.Deny);

                var only = Assert.Single(user.Overrides);
                Assert.Equal(PermissionEffect.Deny, only.Effect);
            }

            [Fact]
            public void Suspension_needs_a_reason_and_cannot_be_repeated()
            {
                var user = NewUser();

                Assert.True(user.Suspend(" ").IsFailure);
                Assert.True(user.Suspend("Left the company").IsSuccess);
                Assert.Equal(UserErrors.AlreadySuspended, user.Suspend("Again").Error);
                Assert.Equal("Left the company", user.SuspensionReason);
            }

            [Fact]
            public void Reactivation_clears_the_reason()
            {
                var user = NewUser();
                user.Suspend("Left");

                Assert.True(user.Reactivate().IsSuccess);
                Assert.Null(user.SuspensionReason);
                Assert.Equal(UserErrors.NotSuspended, user.Reactivate().Error);
            }
        }

        public class ResultTests
        {
            [Fact]
            public void Reading_the_value_of_a_failure_throws()
            {
                Result<int> failure = Error.NotFound("X", "missing");

                Assert.Throws<InvalidOperationException>(() => failure.Value);
```

```
    }

    [Fact]
    public void Values_and_errors_convert_implicitly()
    {
        Result<int> success = 42;
        Result<int> failure = Error.Conflict("X", "taken");

        Assert.Equal(42, success.Value);
        Assert.Equal(ErrorType.Conflict, failure.Error!.Type);
    }
}
```

13.2 Handlers against SQLite

`tests/Sentinel.Application.Tests/TestHost.cs`

```
using Microsoft.Data.Sqlite;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.DependencyInjection;
using Sentinel.Application;
using Sentinel.Application.Abstractions.Messaging;
using Sentinel.Application.Abstractions.Ports;
using Sentinel.Domain.Common;
using Sentinel.Infrastructure.Persistence;
using Sentinel.Infrastructure.Persistence.Seeding;
using Sentinel.Infrastructure.Services;

namespace Sentinel.Application.Tests;

public sealed class FakeClock(DateTime utcNow) : IClock
{
    public DateTime UtcNow { get; set; } = utcNow;
}

/// <summary>
/// The real Application layer and the real EF Core model, over an in-memory
/// SQLite database that lives as long as this object. Handlers run through
/// the real dispatcher and pipeline, so a test here exercises exactly what an
/// endpoint would, minus HTTP.
/// </summary>
/// <remarks>
/// SQLite is not SQL Server. It is close enough for the queries in this book,
/// and the differences that matter (case sensitivity, mainly) are handled
/// explicitly in the handlers. Run the API tests against SQL Server in CI if
/// your queries grow provider-specific.
/// </remarks>
public sealed class TestHost : IAsyncDisposable
{
    public static readonly DateTime Now = new(2026, 10, 25, 12, 0, 0, DateTimeKind.Utc);

    private readonly SqliteConnection _connection;
    private readonly ServiceProvider _services;

    private TestHost(SqliteConnection connection, ServiceProvider services)
    {
        _connection = connection;
        _services = services;
    }

    public FakeClock Clock => (FakeClock)_services.GetRequiredService<IClock>();

    public static async Task<TestHost> CreateAsync(bool seed = false)
    {
        // The database exists for as long as one connection to it is open.
        var connection = new SqliteConnection("DataSource=:memory:");
        await connection.OpenAsync();
```

```
                var services = new ServiceCollection();
                services.AddLogging();
                services.AddApplication();
                services.AddDbContext<SentinelDbContext>(o => o.UseSqlite(connection));
                services.AddScoped<IApplicationDbContext>(sp =>
sp.GetRequiredService<SentinelDbContext>());
                services.AddSingleton<IClock>(new FakeClock(Now));
                services.AddSingleton<IGeoLocator, StaticGeoLocator>();
                services.AddScoped<DevelopmentDataSeeder>();

                var host = new TestHost(connection, services.BuildServiceProvider(validateScopes:
true));

                await using var scope = host._services.CreateAsyncScope();
                var db = scope.ServiceProvider.GetRequiredService<SentinelDbContext>();
                await db.Database.EnsureCreatedAsync();
                if (seed)
                    await scope.ServiceProvider.GetRequiredService<DevelopmentDataSeeder>
().SeedAsync();

                return host;
            }

            /// <summary>Sends a request in its own scope, like one HTTP request would.</summary>
            public async Task<TResponse> Send<TResponse>(IRequest<TResponse> request) where
TResponse : Result
            {
                await using var scope = _services.CreateAsyncScope();
                return await scope.ServiceProvider.GetRequiredService<IDispatcher>
().Send(request);
            }

            /// <summary>A fresh context, so assertions read what was saved rather than what is
cached.</summary>
            public async Task<T> Query<T>(Func<SentinelDbContext, Task<T>> query)
            {
                await using var scope = _services.CreateAsyncScope();
                return await query(scope.ServiceProvider.GetRequiredService<SentinelDbContext>());
            }

            public async Task<Guid> UserIdAsync(string email) =>
                await Query(db => db.Users.Where(u => u.Email ==
Domain.Users.EmailAddress.Create(email).Value).Select(u => u.Id).SingleAsync());

            public async ValueTask DisposeAsync()
            {
                await _services.DisposeAsync();
                await _connection.DisposeAsync();
            }
        }
```

The host runs the real dispatcher, the real pipeline and the real EF Core model, over an in-memory SQLite database that lives as long as the connection is open. Each `Send` gets its own DI scope, as an HTTP request would, so a test cannot accidentally pass because two operations shared a change tracker.

tests/Sentinel.Application.Tests/SecuritySliceTests.cs

```
        using Microsoft.EntityFrameworkCore;
        using Sentinel.Application.Security;
        using Sentinel.Application.Security.AcknowledgeFinding;
        using Sentinel.Application.Security.GetAccountSecurityStatus;
        using Sentinel.Application.Security.GetSignInHistory;
        using Sentinel.Application.Security.RecordSignIn;
        using Sentinel.Application.Users.CreateUser;
        using Sentinel.Domain.Common;

        namespace Sentinel.Application.Tests;
```

```
        public class SecuritySliceTests
        {
            [Theory]
            [InlineData("ada.okafor@sentinel.dev", "Low")]
            [InlineData("bola.adeyemi@sentinel.dev", "High")]
            [InlineData("chen.wei@sentinel.dev", "Medium")]
            [InlineData("dana.smith@sentinel.dev", "Medium")]
            [InlineData("emeka.nwosu@sentinel.dev", "None")]
            [InlineData("tunde.bakare@sentinel.dev", "High")]
            public async Task Each_seeded_story_produces_its_risk_level(string email, string
expected)
            {
                await using var host = await TestHost.CreateAsync(seed: true);

                var status = (await host.Send(new GetAccountSecurityStatusQuery(await
host.UserIdAsync(email)))).Value;

                Assert.Equal(expected, status.RiskLevel);
            }

            /// <summary>
            /// Regression test for SUP-4219 (Chapter 23). Tunde has more sign-ins than
            /// the assessor loads. If the query takes the oldest rows instead of the
            /// newest, yesterday's attack is never seen and the page says "all clear".
            /// </summary>
            [Fact]
            public async Task When_history_exceeds_the_cap_the_newest_attempts_are_assessed()
            {
                await using var host = await TestHost.CreateAsync(seed: true);
                var tunde = await host.UserIdAsync("tunde.bakare@sentinel.dev");
                var total = await host.Query(db => db.SignInAttempts.CountAsync(a => a.UserId ==
tunde));
                Assert.True(total > AccountAssessor.MaxHistory, "The seed data must exceed the cap
for this test to mean anything.");

                var status = (await host.Send(new GetAccountSecurityStatusQuery(tunde))).Value;

                Assert.Contains(status.Findings, f => f.Kind == "BruteForce" && f.Severity ==
"High");
                Assert.Equal(status.RecentSignIns.Max(s => s.OccurredAtUtc),
status.RecentSignIns[0].OccurredAtUtc);
            }

            /// <summary>Regression test for SUP-4172 (Chapter 22): what comes back from the
database is UTC.</summary>
            [Fact]
            public async Task Sign_in_times_come_back_from_the_database_as_utc()
            {
                await using var host = await TestHost.CreateAsync(seed: true);

                var status = (await host.Send(new GetAccountSecurityStatusQuery(await
host.UserIdAsync("ada.okafor@sentinel.dev")))).Value;

                Assert.All(status.RecentSignIns, s => Assert.Equal(DateTimeKind.Utc,
s.OccurredAtUtc.Kind));
                Assert.All(status.Findings, f => Assert.Equal(DateTimeKind.Utc,
f.OccurredAtUtc.Kind));
            }

            [Fact]
            public async Task Flagged_rows_are_exactly_the_evidence_for_the_findings()
            {
                await using var host = await TestHost.CreateAsync(seed: true);

                var status = (await host.Send(new GetAccountSecurityStatusQuery(await
host.UserIdAsync("bola.adeyemi@sentinel.dev")))).Value;

                var evidence = status.Findings.SelectMany(f => f.AttemptIds).ToHashSet();
                Assert.All(status.RecentSignIns, s => Assert.Equal(evidence.Contains(s.Id),
s.IsFlagged));
                Assert.Contains(status.RecentSignIns, s => s.IsFlagged);
```

```
    }

    [Fact]
    public async Task Acknowledging_a_finding_lowers_the_risk_and_is_idempotent()
    {
        await using var host = await TestHost.CreateAsync(seed: true);
        var dana = await host.UserIdAsync("dana.smith@sentinel.dev");
        var before = (await host.Send(new GetAccountSecurityStatusQuery(dana))).Value;
        var newCountry = before.Findings.Single(f => f.Kind == "NewCountry");

        Assert.True((await host.Send(new AcknowledgeFindingCommand(dana,
newCountry.Key))).IsSuccess);
        Assert.True((await host.Send(new AcknowledgeFindingCommand(dana,
newCountry.Key))).IsSuccess);

        var after = (await host.Send(new GetAccountSecurityStatusQuery(dana))).Value;
        Assert.Equal("Medium", before.RiskLevel);
        Assert.Equal("Low", after.RiskLevel);
        Assert.False(after.IsCompromiseSuspected);
    }

    [Fact]
    public async Task A_finding_the_user_cannot_see_cannot_be_acknowledged()
    {
        await using var host = await TestHost.CreateAsync(seed: true);
        var dana = await host.UserIdAsync("dana.smith@sentinel.dev");

        var result = await host.Send(new AcknowledgeFindingCommand(dana, "brute-
force:made-up"));

        Assert.Equal(ErrorType.NotFound, result.Error!.Type);
    }

    [Fact]
    public async Task Recording_a_sign_in_resolves_the_location_and_stores_utc()
    {
        await using var host = await TestHost.CreateAsync();
        await host.Send(new CreateUserCommand("ada@example.com", "Ada", "Africa/Lagos"));

        // 13:00 in Lagos is 12:00 UTC.
        var lagosTime = new DateTimeOffset(2026, 10, 25, 13, 0, 0, TimeSpan.FromHours(1));
        var id = (await host.Send(new RecordSignInCommand("ada@example.com", lagosTime,
"102.89.34.7", "curl/8", true, null))).Value;

        var saved = await host.Query(db => db.SignInAttempts.SingleAsync(a => a.Id ==
id));
        Assert.Equal(new DateTime(2026, 10, 25, 12, 0, 0, DateTimeKind.Utc),
saved.OccurredAtUtc);
        Assert.Equal(DateTimeKind.Utc, saved.OccurredAtUtc.Kind);
        Assert.Equal("Lagos, NG", saved.Location.ToString());
    }

    [Fact]
    public async Task Sign_ins_from_the_future_and_for_unknown_users_are_rejected()
    {
        await using var host = await TestHost.CreateAsync();
        await host.Send(new CreateUserCommand("ada@example.com", "Ada", "Africa/Lagos"));
        var tomorrow = new DateTimeOffset(TestHost.Now.AddDays(1));

        Assert.Equal("SignIn.InFuture", (await host.Send(new
RecordSignInCommand("ada@example.com", tomorrow, "1.1.1.1", null, true, null))).Error!.Code);
        Assert.Equal(ErrorType.NotFound, (await host.Send(new
RecordSignInCommand("who@example.com", TestHost.Now, "1.1.1.1", null, true, null))).Error!.Type);
        Assert.Equal(ErrorType.Validation, (await host.Send(new
RecordSignInCommand("ada@example.com", TestHost.Now, "1.1.1.1", null, false,
"Bananas"))).Error!.Type);
    }

    [Fact]
    public async Task History_is_paged_newest_first()
    {
        await using var host = await TestHost.CreateAsync(seed: true);
```

```
                var tunde = await host.UserIdAsync("tunde.bakare@sentinel.dev");

                var first = (await host.Send(new GetSignInHistoryQuery(tunde, 1, 10))).Value;
                var second = (await host.Send(new GetSignInHistoryQuery(tunde, 2, 10))).Value;

                Assert.Equal(10, first.Items.Count);
                Assert.True(first.Items[^1].OccurredAtUtc >= second.Items[0].OccurredAtUtc);
                Assert.Equal(first.Items.OrderByDescending(i => i.OccurredAtUtc).Select(i =>
i.Id), first.Items.Select(i => i.Id));
                Assert.True(first.TotalCount > 500);
            }
        }
```

Two tests here are labelled as regression tests for the incidents in Part IV. On the lab branches they are deleted, because the point of a lab is that the bug shipped: the test did not exist.

`tests/Sentinel.Application.Tests/PageAccessSliceTests.cs`

```
        using Sentinel.Application.Pages.CheckPageAccess;
        using Sentinel.Application.Pages.CreatePage;
        using Sentinel.Application.Pages.GetNavigation;
        using Sentinel.Application.Pages.GetPageAccess;
        using Sentinel.Application.Roles.GetRoles;
        using Sentinel.Application.Roles.RolePermissions;
        using Sentinel.Application.Users.RoleAssignment;
        using Sentinel.Contracts;
        using Sentinel.Domain.Common;

        namespace Sentinel.Application.Tests;

        public class PageAccessSliceTests
        {
            private static IEnumerable<string> Flatten(IEnumerable<NavigationNodeDto> nodes) =>
                nodes.SelectMany(n => Flatten(n.Children).Prepend(n.Path));

            [Fact]
            public async Task The_menu_contains_exactly_the_allowed_pages()
            {
                await using var host = await TestHost.CreateAsync(seed: true);
                var emeka = await host.UserIdAsync("emeka.nwosu@sentinel.dev");

                var menu = (await host.Send(new GetNavigationQuery(emeka))).Value;

                Assert.Equal(
                    ["/", "/account", "/account/security", "/reports", "/reports/finance",
"/billing", "/help", "/status"],
                    Flatten(menu));
            }

            [Fact]
            public async Task Anonymous_visitors_get_the_public_pages()
            {
                await using var host = await TestHost.CreateAsync(seed: true);

                var menu = (await host.Send(new GetNavigationQuery(null))).Value;

                Assert.Equal(["/help", "/status"], Flatten(menu));
            }

            [Fact]
            public async Task An_explicit_deny_is_explained()
            {
                await using var host = await TestHost.CreateAsync(seed: true);
                var emeka = await host.UserIdAsync("emeka.nwosu@sentinel.dev");

                var verdict = (await host.Send(new CheckPageAccessQuery(emeka,
"/reports/finance/export"))).Value;
```

```
                Assert.False(verdict.IsAllowed);
                Assert.Equal("ExplicitlyDenied", verdict.Reason);
                Assert.Equal("/reports/finance", verdict.ParentPath);
            }

            [Fact]
            public async Task Checking_an_unknown_page_is_not_found()
            {
                await using var host = await TestHost.CreateAsync(seed: true);

                var result = await host.Send(new CheckPageAccessQuery(null, "/nowhere"));

                Assert.Equal(ErrorType.NotFound, result.Error!.Type);
            }

            [Fact]
            public async Task The_access_explorer_lists_every_page_with_a_reason()
            {
                await using var host = await TestHost.CreateAsync(seed: true);
                var grace = await host.UserIdAsync("grace.eze@sentinel.dev"); // suspended

                var rows = (await host.Send(new GetPageAccessQuery(grace))).Value;

                Assert.Equal(15, rows.Count);
                Assert.All(rows.Where(r => !r.IsAllowed), r => Assert.Equal("UserSuspended",
r.Reason));
                Assert.Equal(["/help", "/status"], rows.Where(r => r.IsAllowed).Select(r =>
r.Path));
            }

            [Fact]
            public async Task Granting_a_role_permission_is_saved_and_changes_access()
            {
                await using var host = await TestHost.CreateAsync(seed: true);
                var chen = await host.UserIdAsync("chen.wei@sentinel.dev");
                var operations = (await host.Send(new GetRolesQuery())).Value.Single(r => r.Name
== "Operations");

                Assert.False((await host.Send(new CheckPageAccessQuery(chen,
"/billing"))).Value.IsAllowed);

                Assert.True((await host.Send(new GrantPermissionCommand(operations.Id,
"billing.view"))).IsSuccess);

                // A fresh scope reads from the database: proves the value comparer
                // noticed the list changed and EF Core actually wrote it.
                Assert.True((await host.Send(new CheckPageAccessQuery(chen,
"/billing"))).Value.IsAllowed);
            }

            [Fact]
            public async Task Revoking_a_role_removes_its_pages()
            {
                await using var host = await TestHost.CreateAsync(seed: true);
                var femi = await host.UserIdAsync("femi.bello@sentinel.dev");
                var auditor = (await host.Send(new GetRolesQuery())).Value.Single(r => r.Name ==
"Auditor");

                await host.Send(new RevokeRoleCommand(femi, auditor.Id));

                var menu = Flatten((await host.Send(new
GetNavigationQuery(femi))).Value).ToList();
                Assert.DoesNotContain("/admin", menu);
                Assert.DoesNotContain("/reports", menu);
            }

            [Fact]
            public async Task Pages_cannot_require_a_wildcard()
            {
                await using var host = await TestHost.CreateAsync(seed: true);

                var result = await host.Send(new CreatePageCommand("/reports/all", "All",
```

```
"/reports", "reports.*", false, 0));

                Assert.Equal("Page.WildcardPermission", result.Error!.Code);
            }

            [Fact]
            public async Task A_new_page_must_have_an_existing_parent_and_a_unique_path()
            {
                await using var host = await TestHost.CreateAsync(seed: true);

                Assert.Equal(ErrorType.NotFound, (await host.Send(new CreatePageCommand("/x/y",
"Y", "/x", null, false, 0))).Error!.Type);
                Assert.Equal(ErrorType.Conflict, (await host.Send(new
CreatePageCommand("/Billing/", "Again", "/", null, false, 0))).Error!.Type);
                Assert.True((await host.Send(new CreatePageCommand("/billing/invoices",
"Invoices", "/billing", "billing.view", false, 0))).IsSuccess);
            }
        }
```

13.3 The API in process

`tests/Sentinel.Api.Tests/SentinelApiFactory.cs`

```
        using System.Net.Http.Headers;
        using Microsoft.AspNetCore.Hosting;
        using Microsoft.AspNetCore.Mvc.Testing;
        using Microsoft.Data.Sqlite;
        using Microsoft.EntityFrameworkCore;
        using Microsoft.EntityFrameworkCore.Infrastructure;
        using Microsoft.Extensions.DependencyInjection;
        using Microsoft.Extensions.DependencyInjection.Extensions;
        using Microsoft.Extensions.Hosting;
        using Sentinel.Api.Authentication;
        using Sentinel.Infrastructure.Persistence;
        using Sentinel.Infrastructure.Persistence.Seeding;

        namespace Sentinel.Api.Tests;

        /// <summary>
        /// The whole API, in process, over an in-memory SQLite database seeded with
        /// the development data. One factory per test class (xUnit's class fixture),
        /// so tests in a class share a database: write tests that do not depend on
        /// each other's changes.
        /// </summary>
        public sealed class SentinelApiFactory : WebApplicationFactory<Program>
        {
            private readonly SqliteConnection _connection = new("DataSource=:memory:");

            protected override void ConfigureWebHost(IWebHostBuilder builder)
            {
                _connection.Open();

                builder.UseEnvironment("Development");

                // The migrations are SQL Server's. Tests build the schema from the model instead.
                builder.UseSetting("Database:MigrateAndSeedOnStartup", "false");

                builder.ConfigureServices(services =>
                {
                    services.RemoveAll<DbContextOptions<SentinelDbContext>>();
                    services.RemoveAll(typeof(IDbContextOptionsConfiguration<SentinelDbContext>));
                    services.AddDbContext<SentinelDbContext>(o => o.UseSqlite(_connection));
                });
            }

            protected override IHost CreateHost(IHostBuilder builder)
            {
```

```
                var host = base.CreateHost(builder);

                using var scope = host.Services.CreateScope();
                scope.ServiceProvider.GetRequiredService<SentinelDbContext>
().Database.EnsureCreated();
                scope.ServiceProvider.GetRequiredService<DevelopmentDataSeeder>
().SeedAsync().GetAwaiter().GetResult();

                return host;
            }

            /// <summary>A client signed in as <paramref name="email"/>, or anonymous when null.
</summary>
            public HttpClient ClientFor(string? email)
            {
                var client = CreateClient();
                if (email is not null)
                    client.DefaultRequestHeaders.Add(DemoUserAuthenticationHandler.HeaderName,
email);
                client.DefaultRequestHeaders.Accept.Add(new
MediaTypeWithQualityHeaderValue("application/json"));
                return client;
            }

            protected override void Dispose(bool disposing)
            {
                base.Dispose(disposing);
                if (disposing)
                    _connection.Dispose();
            }
        }
```

`WebApplicationFactory<Program>` hosts the real API in memory. The factory swaps SQL Server for SQLite (removing both the options and EF Core 9+'s `IDbContextOptionsConfiguration<T>` registrations; removing only the first leaves SQL Server configured), turns off start-up migration, and seeds.

tests/Sentinel.Api.Tests/ApiTests.cs

```
        using System.Net;
        using System.Net.Http.Json;
        using System.Text.Json;
        using System.Text.RegularExpressions;
        using Sentinel.Contracts;

        namespace Sentinel.Api.Tests;

        public partial class ApiTests(SentinelApiFactory factory) :
IClassFixture<SentinelApiFactory>
        {
            private const string Ada = "ada.okafor@sentinel.dev";      // Administrator
            private const string Femi = "femi.bello@sentinel.dev";     // Auditor: admin.users,
not .manage
            private const string Chen = "chen.wei@sentinel.dev";       // Operations
            private const string Dana = "dana.smith@sentinel.dev";

            // ---- Authentication ----

            [Fact]
            public async Task Me_requires_authentication()
            {
                var response = await factory.ClientFor(null).GetAsync("/api/me");

                Assert.Equal(HttpStatusCode.Unauthorized, response.StatusCode);
            }

            [Fact]
            public async Task An_unknown_demo_user_is_not_authenticated()
```

```
        {
            var response = await factory.ClientFor("nobody@example.com").GetAsync("/api/me");

            Assert.Equal(HttpStatusCode.Unauthorized, response.StatusCode);
        }

        [Fact]
        public async Task Me_returns_the_signed_in_user()
        {
            var me = await factory.ClientFor(Ada).GetFromJsonAsync<CurrentUserDto>("/api/me");

            Assert.Equal("Ada Okafor", me!.DisplayName);
            Assert.Equal("Africa/Lagos", me.TimeZoneId);
            Assert.Equal(["*"], me.Permissions);
        }

        // ---- Authorisation: the endpoint, not just the menu ----

        [Theory]
        [InlineData(Ada, "/api/users", HttpStatusCode.OK)]
        [InlineData(Femi, "/api/users", HttpStatusCode.OK)]
        [InlineData(Chen, "/api/users", HttpStatusCode.Forbidden)]
        [InlineData(Femi, "/api/roles", HttpStatusCode.Forbidden)] // auditors see users, not
roles
        public async Task Admin_reads_are_guarded_by_permission(string user, string url,
HttpStatusCode expected)
        {
            var response = await factory.ClientFor(user).GetAsync(url);

            Assert.Equal(expected, response.StatusCode);
        }

        [Fact]
        public async Task Reading_a_resource_does_not_imply_changing_it()
        {
            var response = await factory.ClientFor(Femi).PostAsJsonAsync("/api/users",
                new CreateUserRequest("new@sentinel.dev", "New", "Africa/Lagos"));

            Assert.Equal(HttpStatusCode.Forbidden, response.StatusCode);
        }

        // ---- Results to HTTP ----

        [Fact]
        public async Task Validation_errors_are_a_400_problem_with_one_entry_per_field()
        {
            var response = await factory.ClientFor(Ada).PostAsJsonAsync("/api/users", new
CreateUserRequest("", "", "Africa/Lagos"));

            Assert.Equal(HttpStatusCode.BadRequest, response.StatusCode);
            using var problem = JsonDocument.Parse(await
response.Content.ReadAsStringAsync());
            var errors = problem.RootElement.GetProperty("errors");
            Assert.True(errors.TryGetProperty("Email", out _));
            Assert.True(errors.TryGetProperty("DisplayName", out _));
        }

        [Fact]
        public async Task Business_rule_failures_carry_their_error_code()
        {
            var response = await factory.ClientFor(Ada).PostAsJsonAsync("/api/users",
                new CreateUserRequest("someone@sentinel.dev", "Someone", "Mars/Base"));

            Assert.Equal(HttpStatusCode.BadRequest, response.StatusCode);
            using var problem = JsonDocument.Parse(await
response.Content.ReadAsStringAsync());
            Assert.Equal("User.UnknownTimeZone",
problem.RootElement.GetProperty("code").GetString());
        }

        [Fact]
        public async Task Creating_a_user_returns_201_with_a_location()
```

```
            {
                var response = await factory.ClientFor(Ada).PostAsJsonAsync("/api/users",
                    new CreateUserRequest("kemi.ade@sentinel.dev", "Kemi Ade", "Africa/Lagos"));

                Assert.Equal(HttpStatusCode.Created, response.StatusCode);
                var created = await response.Content.ReadFromJsonAsync<CreatedResponse>();
                Assert.Equal($"/api/users/{created!.Id}",
response.Headers.Location!.OriginalString);
            }

            [Fact]
            public async Task Unknown_ids_are_404()
            {
                var response = await
factory.ClientFor(Ada).GetAsync($"/api/users/{Guid.NewGuid()}");

                Assert.Equal(HttpStatusCode.NotFound, response.StatusCode);
            }

            // ---- Page access over HTTP ----

            [Fact]
            public async Task Anonymous_navigation_is_the_public_pages()
            {
                var menu = await factory.ClientFor(null).GetFromJsonAsync<List<NavigationNodeDto>>
("/api/me/navigation");

                Assert.Equal(["/help", "/status"], menu!.Select(n => n.Path));
            }

            [Fact]
            public async Task The_guard_endpoint_explains_a_denial()
            {
                var verdict = await factory.ClientFor(Chen).GetFromJsonAsync<PageAccessDto>
("/api/me/page-access?path=/admin/users");

                Assert.False(verdict!.IsAllowed);
                Assert.Equal("ParentDenied", verdict.Reason);
            }

            // ---- The wire format for dates (SUP-4172, Chapter 22) ----

            [Fact]
            public async Task Every_timestamp_on_the_wire_is_utc_with_a_z()
            {
                var json = await factory.ClientFor(Dana).GetStringAsync("/api/me/security");

                var timestamps = TimestampProperty().Matches(json).Select(m =>
m.Groups["value"].Value).ToList();

                Assert.NotEmpty(timestamps);
                Assert.All(timestamps, t => Assert.EndsWith("Z", t));
            }

            [Fact]
            public async Task Acknowledging_a_finding_over_http_returns_204_and_lowers_the_risk()
            {
                var client = factory.ClientFor(Dana);
                var before = await client.GetFromJsonAsync<AccountSecurityStatusDto>
("/api/me/security");
                var finding = before!.Findings.First(f => f.Kind == "NewDevice");

                var response = await
client.PostAsync($"/api/me/security/findings/{Uri.EscapeDataString(finding.Key)}/acknowledge",
null);

                Assert.Equal(HttpStatusCode.NoContent, response.StatusCode);
                var after = await client.GetFromJsonAsync<AccountSecurityStatusDto>
("/api/me/security");
                Assert.DoesNotContain(after!.Findings, f => f.Key == finding.Key);
            }
```

```
            [GeneratedRegex("\"(occurredAtUtc|assessedAtUtc)\":\"(?<value>[^\"]+)\"")]
            private static partial Regex TimestampProperty();
        }
```

`Every_timestamp_on_the_wire_is_utc_with_a_z` is the test that would have prevented Chapter 22. It reads the raw JSON, not a deserialised object, because deserialising would hide the very thing it is checking.

13.4 Architecture tests

`tests/Sentinel.Architecture.Tests/DependencyRuleTests.cs`

```
        using System.Reflection;
        using Sentinel.Application.Abstractions.Messaging;

        namespace Sentinel.Architecture.Tests;

        /// <summary>
        /// The dependency rule, enforced by the build. A diagram in a wiki can be
        /// ignored; a red test cannot. No NetArchTest package: reading referenced
        /// assemblies and walking types is all it takes.
        /// </summary>
        public class DependencyRuleTests
        {
            private static readonly Assembly Domain =
typeof(Sentinel.Domain.Common.Result).Assembly;
            private static readonly Assembly Contracts =
typeof(Sentinel.Contracts.PageDto).Assembly;
            private static readonly Assembly Application = typeof(IDispatcher).Assembly;
            private static readonly Assembly Infrastructure =
typeof(Sentinel.Infrastructure.DependencyInjection).Assembly;
            private static readonly Assembly Api = typeof(Program).Assembly;

            private static IEnumerable<string> ReferencedNames(Assembly assembly) =>
                assembly.GetReferencedAssemblies().Select(a => a.Name!);

            private static bool IsFramework(string name) =>
                name is "netstandard" or "mscorlib" || name.StartsWith("System",
StringComparison.Ordinal);

            [Fact]
            public void Domain_references_nothing_but_the_base_class_library()
            {
                var offenders = ReferencedNames(Domain).Where(n => !IsFramework(n)).ToList();

                Assert.Empty(offenders);
            }

            [Fact]
            public void Contracts_reference_nothing_but_the_base_class_library()
            {
                Assert.DoesNotContain(ReferencedNames(Contracts), n => !IsFramework(n));
            }

            [Fact]
            public void Application_does_not_know_about_infrastructure_or_the_web()
            {
                var names = ReferencedNames(Application).ToList();

                Assert.DoesNotContain("Sentinel.Infrastructure", names);
                Assert.DoesNotContain("Sentinel.Api", names);
                Assert.DoesNotContain(names, n => n.StartsWith("Microsoft.AspNetCore",
StringComparison.Ordinal));
                Assert.DoesNotContain(names, n =>
n.StartsWith("Microsoft.EntityFrameworkCore.SqlServer", StringComparison.Ordinal));
            }
```

```
            [Fact]
            public void Infrastructure_does_not_know_about_the_web()
            {
                Assert.DoesNotContain("Sentinel.Api", ReferencedNames(Infrastructure));
            }

            [Fact]
            public void Handlers_are_internal_so_only_the_dispatcher_can_call_them()
            {
                var publicHandlers = Application.GetTypes()
                    .Where(t => t.IsClass && !t.IsAbstract)
                    .Where(t => t.GetInterfaces().Any(i => i.IsGenericType &&
i.GetGenericTypeDefinition() == typeof(IRequestHandler<,>)))
                    .Where(t => t.IsPublic)
                    .Select(t => t.FullName)
                    .ToList();

                Assert.Empty(publicHandlers);
            }

            [Fact]
            public void Every_request_has_exactly_one_handler()
            {
                var requestTypes = Application.GetTypes()
                    .Where(t => t is { IsClass: true, IsAbstract: false })
                    .Where(t => t.GetInterfaces().Any(i => i.IsGenericType &&
i.GetGenericTypeDefinition() == typeof(IRequest<>)))
                    .ToList();

                var handledTypes = Application.GetTypes()
                    .SelectMany(t => t.GetInterfaces())
                    .Where(i => i.IsGenericType && i.GetGenericTypeDefinition() ==
typeof(IRequestHandler<,>))
                    .Select(i => i.GetGenericArguments()[0])
                    .ToList();

                Assert.NotEmpty(requestTypes);
                Assert.All(requestTypes, r => Assert.Equal(1, handledTypes.Count(h => h == r)));
            }

            [Fact]
            public void Requests_live_in_a_folder_of_their_own_named_after_the_feature()
            {
                // Vertical slices: Sentinel.Application.<Feature>.<UseCase>.<Request>.
                var misplaced = Application.GetTypes()
                    .Where(t => t is { IsClass: true, IsAbstract: false })
                    .Where(t => t.GetInterfaces().Any(i => i.IsGenericType &&
i.GetGenericTypeDefinition() == typeof(IRequest<>)))
                    .Where(t => t.Namespace!.Split('.').Length != 4)
                    .Select(t => t.FullName)
                    .ToList();

                Assert.Empty(misplaced);
            }

            [Fact]
            public void The_api_references_the_layers_it_composes()
            {
                var names = ReferencedNames(Api).ToList();

                Assert.Contains("Sentinel.Application", names);
                Assert.Contains("Sentinel.Infrastructure", names);
            }
        }
```

A diagram in a wiki can be ignored. A red build cannot. These tests also check two conventions: handlers are `internal` (only the dispatcher calls them) and every request has exactly one handler (a missing handler is otherwise a run-time exception on first use).

13.5 What not to test

Do not test EF Core, ASP.NET or the framework's JSON serialiser. Test *your* decisions on top of them: that the converter makes dates UTC, not that EF can save a row. Do not write a test per getter. Do write a test for every decision you would have to explain to a reviewer.

Part III · The Angular Front End

Chapter 14 · Angular and TypeScript for the Test

The prep guide promises no trick questions about JavaScript and says the Angular scenario needs only basic reading and writing. “Basic” under a 25-minute timer means *automatic*. This chapter is the minimum you should be able to write without looking anything up.

14.1 The ES2015+ you will actually use

```
        // const by default; let when it changes; never var.
        const findings = status.findings;
        let flaggedCount = 0;

        // Arrow functions, and the array methods that replace most loops.
        const high = findings.filter((f) => f.severity === 'High');
        const keys = findings.map((f) => f.key);
        const anyHigh = findings.some((f) => f.severity === 'High');
        const total = signIns.reduce((sum, s) => sum + (s.succeeded ? 0 : 1), 0);

        // Sorting: sort() mutates. Copy first, always.
        const newestFirst = [...signIns].sort((a, b) =>
b.occurredAtUtc.localeCompare(a.occurredAtUtc));

        // Destructuring and defaults.
        const { riskLevel, findings: items = [] } = status;
        function format(value: string, { locale = 'en-GB', showZone = true } = {}) { /* ... */ }

        // Spread to copy and override (the builders in the specs do this).
        const changed = { ...finding, severity: 'High' };

        // Template literals.
        const label = `Review ${count} ${count === 1 ? 'item' : 'items'}`;

        // Optional chaining and nullish coalescing: different from || for 0 and ''.
        const zone = user?.timeZoneId ?? 'UTC';

        // Map and Set for lookups and "seen before".
        const seenCountries = new Set<string>();
        const byId = new Map(signIns.map((s) => [s.id, s]));
```

Two traps worth knowing because they cause real bugs, not because anyone will quiz you on them:

- `sort()` without a comparator sorts as **strings**: `[10, 9, 1].sort()` is `[1, 10, 9]`.
- `new Date('2026-10-24T08:00:00')` with no `Z` is **local time**. Chapter 18 is about this.

14.2 TypeScript in one table

You want	Write
A shape for API data	`interface SignInAttempt { id: string; succeeded: boolean; }`
A closed set of strings	`type RiskLevel = 'None' | 'Low' | 'Medium' | 'High';`
One of several states	a discriminated union: `{ kind: 'loading' } | { kind: 'loaded'; status: X }`
A lookup keyed by a union	`const titles: Record<FindingKind, string> = { ... }`
"May be missing"	`string | null` (from JSON) or an optional `?:` property
Narrowing	`if (state.kind === 'loaded') { state.status }`

14.3 Angular in one page

Concept	Modern syntax (Angular 17+)	What it is for
Component	`@Component({ selector, imports, template })` on a class	A piece of UI with its own template
Input	`readonly risk = input.required<RiskLevel>()`	Data **into** a child
Output	`readonly review = output<void>()` and `this.review.emit()`	Events **out of** a child
Interpolation	`{{ user.displayName }}`	Show a value
Property binding	`[riskLevel]="status.riskLevel"`	Set a child's input or a DOM property
Event binding	`(review)="scrollToFindings()"`	React to an output or DOM event
Two-way binding	`[(selected)]="email"`	Input + `selectedChange` output
Control flow	`@if`, `@for (x of xs; track x.id)`, `@switch`	Conditionals and loops in templates
Pipe	`{{ value | zonedDate: zone }}`	Format a value for display
Service	`@Injectable({ providedIn: 'root' })`	Shared logic or state
Injection	`private readonly api = inject(SecurityApi)`	Get a service
Signal	`count = signal(0)`, `count()`, `count.set(1)`, `computed(...)`	Reactive state
Observable	`http.get<T>(url).subscribe(...)`	Asynchronous streams (HTTP, events)

Appendix C translates every row into pre-17 syntax (`@Input()`, `*ngIf`, `NgModule`) in case the test environment is older.

14.4 RxJS: the six operators that cover most code

Operator	Use
`map`	Transform each value
`switchMap`	Start a new request for each value, cancelling the previous one
`catchError`	Turn an error into a value (an error state)
`debounceTime`	Wait for typing to pause
`distinctUntilChanged`	Ignore repeats
`takeUntilDestroyed`	Unsubscribe when the component goes away

14.5 Using AI in the Angular scenario, properly

The third scenario *encourages* AI and asks you to cite it. Use it for what it is good at (syntax you have forgotten, an `Intl` option name, a test skeleton) and keep the decisions yours. Cite it the same way as any source:

```
        // Relative time formatting adapted from an AI assistant's suggestion
        // (ChatGPT, 2026-10-25, prompt: "Intl.RelativeTimeFormat days/hours ago").
        // Reviewed and changed: "now" is a parameter so it can be tested.
```

What a reviewer cannot tell from AI-written code is whether *you* understood it. Your comments are where you show that you did.

Chapter 15 · The Angular Workspace and the Core Layer

15.1 Creating the workspace

```
mkdir client; cd client
npx @angular/cli@22 new sentinel-web --style=css --ssr=false --skip-git --routing `
  --package-manager=npm --ai-config=none --interactive=false
cd sentinel-web
```

Angular 22 creates a *standalone*, *zoneless* application tested with *Vitest*. Standalone means no `NgModule`: each component lists what it uses in its own `imports`. Zoneless means change detection is driven by signals and events rather than by Zone.js patching every timer; in practice it means "keep state in signals and the screen stays in step".

File names have lost their suffixes in recent versions (`app.ts` rather than `app.component.ts`, class `App` rather than `AppComponent`). This book follows the current style guide.

15.2 Talking to the API through a proxy

The API runs on port 5080 and the dev server on 4200. Rather than enable CORS on the API, the dev server forwards `/api` to it:

`client/sentinel-web/proxy.conf.json`

```
{
  "/api": {
    "target": "http://localhost:5080",
    "secure": false,
    "changeOrigin": true
  }
}
```

and `angular.json` points `serve` at that file (`"options": { "proxyConfig": "proxy.conf.json" }`). The browser only ever sees one origin, exactly as it will in production behind a reverse proxy.

> **Watch out —** If you write JSON files from Windows PowerShell 5 with `Set-Content -Encoding utf8`, they get a byte-order mark, and the Angular CLI then reports `proxy.conf.json contains parse errors: [1, 1] InvalidSymbol`. That happened while writing this chapter. The cure is to save without a BOM. Error positions like `[1, 1]` are a strong hint that the problem is invisible.

15.3 The models

`client/sentinel-web/src/app/core/models/api-models.ts`

```
// Hand-written mirrors of src/Sentinel.Contracts. Keep the two in step:
// a property renamed in C# and not here compiles fine on both sides and
```

```
// fails silently at run time, as `undefined`.
//
// Timestamps arrive as ISO 8601 strings in UTC ("2026-10-24T08:00:00Z").
// They are deliberately typed `string`, not `Date`: JSON has no date type,
// and pretending otherwise hides the moment where parsing happens. Parse
// them with parseApiTimestamp() in core/time, never with `new Date()`.

/** An ISO 8601 UTC timestamp exactly as the API sent it. */
export type UtcTimestamp = string;

export type RiskLevel = 'None' | 'Low' | 'Medium' | 'High';
export type FindingKind = 'BruteForce' | 'ImpossibleTravel' | 'NewCountry' | 'NewDevice';

export interface CurrentUser {
  id: string;
  email: string;
  displayName: string;
  timeZoneId: string;
  permissions: string[];
}

export interface DemoUser {
  email: string;
  displayName: string;
  description: string;
}

export interface NavigationNode {
  path: string;
  title: string;
  children: NavigationNode[];
}

export type AccessReason =
  | 'PublicPage'
  | 'AuthenticatedUser'
  | 'PermissionGranted'
  | 'NotAuthenticated'
  | 'UserSuspended'
  | 'MissingPermission'
  | 'ExplicitlyDenied'
  | 'ParentDenied'
  | 'OrphanedPage'
  | 'CyclicHierarchy';

export interface PageAccess {
  path: string;
  title: string;
  parentPath: string | null;
  requiredPermission: string | null;
  isAllowed: boolean;
  reason: AccessReason;
}

export interface SignInAttempt {
  id: string;
  occurredAtUtc: UtcTimestamp;
  ipAddress: string;
  countryCode: string;
  city: string;
  device: string;
  succeeded: boolean;
  failureReason: string | null;
  isFlagged: boolean;
}

export interface SecurityFinding {
  key: string;
  kind: FindingKind;
  severity: RiskLevel;
  occurredAtUtc: UtcTimestamp;
  summary: string;
```

```
  attemptIds: string[];
}

export interface AccountSecurityStatus {
  riskLevel: RiskLevel;
  isCompromiseSuspected: boolean;
  assessedAtUtc: UtcTimestamp;
  timeZoneId: string;
  findings: SecurityFinding[];
  recentSignIns: SignInAttempt[];
}

export interface PagedResponse<T> {
  items: T[];
  page: number;
  pageSize: number;
  totalCount: number;
}

export interface UserSummary {
  id: string;
  email: string;
  displayName: string;
  status: 'Active' | 'Suspended';
  roles: string[];
}

export interface Role {
  id: string;
  name: string;
  description: string;
  permissions: string[];
  userCount: number;
}
```

Timestamps are typed `string`, on purpose. JSON has no date type, and declaring `occurredAtUtc: Date` would be a lie that the compiler believes: the value at run time would still be a string, and the moment it is parsed would be hidden. Here it is visible, and done in exactly one function (Chapter 18).

15.4 Errors, once

client/sentinel-web/src/app/core/http/api-error.ts

```
import { HttpErrorResponse } from '@angular/common/http';

/**
 * Turns whatever an HTTP call threw into one sentence a user can read.
 *
 * The API answers failures with RFC 9457 problem details
 * (`{ title, detail, status, errors? }`), but a failure can also come from
 * the network (status 0) or from a proxy that knows nothing about problem
 * details. Every component shows errors through this function, so all
 * three cases read the same way everywhere.
 */
export function describeHttpError(error: unknown): string {
  if (!(error instanceof HttpErrorResponse)) {
    return 'Something went wrong. Please try again.';
  }

  if (error.status === 0) {
    return 'Cannot reach the Sentinel API. Is it running on http://localhost:5080?';
  }

  if (error.status === 401) {
    return 'You are not signed in. Choose a user from the switcher at the top of the
page.';
```

```
          }

          if (error.status === 403) {
            return 'You do not have permission to do that.';
          }

          const problem = error.error as { detail?: string; title?: string; errors?:
Record<string, string[]> } | null;
          if (problem?.errors) {
            return Object.values(problem.errors).flat().join(' ');
          }

          return problem?.detail ?? problem?.title ?? `The server answered ${error.status}
${error.statusText}.`;
        }
```

A failed HTTP call can be a network failure (status 0), an authentication problem, a validation problem with one message per field, or a proxy's HTML error page. Every component shows errors through this function, so all of them read the same way.

15.5 Identity on every request

`client/sentinel-web/src/app/core/http/demo-user.interceptor.ts`

```
        import { HttpInterceptorFn } from '@angular/common/http';
        import { inject } from '@angular/core';
        import { Session } from '../session/session';

        /** The header the API's development-only authentication scheme reads. */
        export const DEMO_USER_HEADER = 'X-Demo-User';

        /**
         * Adds the chosen demo user to every API request. In production this
         * interceptor would attach a bearer token instead; nothing else in the
         * application would change, because nothing else knows how identity travels.
         */
        export const demoUserInterceptor: HttpInterceptorFn = (request, next) => {
          const email = inject(Session).signedInEmail();

          if (!email || !request.url.startsWith('/api/')) {
            return next(request);
          }

          return next(request.clone({ setHeaders: { [DEMO_USER_HEADER]: email } }));
        };
```

A functional interceptor (`HttpInterceptorFn`) adds the header to calls to our own API only. The `startsWith('/api/')` check is the edge case: the identity must never leak to another origin, and there is a test for it.

15.6 The session

`client/sentinel-web/src/app/core/session/session.ts`

```
        import { HttpClient } from '@angular/common/http';
        import { Injectable, computed, inject, signal } from '@angular/core';
        import { Observable, catchError, of, tap } from 'rxjs';
        import { CurrentUser, DemoUser, NavigationNode } from '../models/api-models';

        const STORAGE_KEY = 'sentinel.demoUser';
```

```ts
/**
 * Who is signed in, and what they may see. This is the one piece of state
 * shared across the whole application, so it lives in a root service and
 * is exposed as read-only signals. Components read it; only this class
 * writes it.
 */
@Injectable({ providedIn: 'root' })
export class Session {
  private readonly http = inject(HttpClient);

  private readonly email = signal<string | null>(readStoredEmail());
  private readonly user = signal<CurrentUser | null>(null);
  private readonly menu = signal<NavigationNode[]>([]);

  /** The email sent in the demo header, or null when signed out. */
  readonly signedInEmail = this.email.asReadonly();
  readonly currentUser = this.user.asReadonly();
  readonly navigation = this.menu.asReadonly();
  readonly isSignedIn = computed(() => this.user() !== null);

  /**
   * The zone to show times in when a screen has no better information.
   * The user's profile zone, not the browser's: a Lagos user on a work
   * trip to London still thinks in Lagos time on their own account page.
   */
  readonly timeZone = computed(() => this.user()?.timeZoneId ?? 'UTC');

  getDemoUsers(): Observable<DemoUser[]> {
    return this.http.get<DemoUser[]>('/api/dev/demo-users');
  }

  /** Switches user (or signs out with null) and reloads everything that depends on
identity. */
  signInAs(email: string | null): Observable<unknown> {
    this.email.set(email);
    writeStoredEmail(email);
    return this.refresh();
  }

  /** Reloads the profile and the menu for whoever is signed in now. */
  refresh(): Observable<unknown> {
    const email = this.email();
    this.loadNavigation().subscribe();

    if (!email) {
      this.user.set(null);
      return of(null);
    }

    return this.http.get<CurrentUser>('/api/me').pipe(
      tap((user) => this.user.set(user)),
      catchError(() => {
        // The stored user no longer exists (database reset). Fall back to
        // signed out rather than leaving a half-signed-in shell.
        this.email.set(null);
        writeStoredEmail(null);
        this.user.set(null);
        return of(null);
      }),
    );
  }

  private loadNavigation(): Observable<NavigationNode[]> {
    return this.http.get<NavigationNode[]>('/api/me/navigation').pipe(
      tap((nodes) => this.menu.set(nodes)),
      catchError(() => {
        this.menu.set([]);
        return of([]);
      }),
    );
  }
```

```
}

// localStorage can throw (private windows, disabled storage). A failure to
// remember the user must never stop the application from starting.
function readStoredEmail(): string | null {
  try {
    return localStorage.getItem(STORAGE_KEY);
  } catch {
    return null;
  }
}

function writeStoredEmail(email: string | null): void {
  try {
    if (email) {
      localStorage.setItem(STORAGE_KEY, email);
    } else {
      localStorage.removeItem(STORAGE_KEY);
    }
  } catch {
    // Nothing useful to do; the session still works for this tab.
  }
}
```

The one piece of application-wide state lives in a root service, exposed as **read-only** signals (`asReadonly()`). Components read; only `Session` writes. That is the interface-segregation principle in a front end, and it makes the fakes in tests tiny (Chapter 20).

`localStorage` access is wrapped in `try/catch`: it can throw in private windows and with storage disabled, and a failure to *remember* a user must never stop the application from *starting*.

15.7 Wiring it together

`client/sentinel-web/src/app/app.config.ts`

```
import { provideHttpClient, withInterceptors } from '@angular/common/http';
import { ApplicationConfig, provideBrowserGlobalErrorListeners } from '@angular/core';
import { provideRouter, withComponentInputBinding, withRouterConfig } from
'@angular/router';
import { routes } from './app.routes';
import { demoUserInterceptor } from './core/http/demo-user.interceptor';

export const appConfig: ApplicationConfig = {
  providers: [
    provideBrowserGlobalErrorListeners(),
    provideHttpClient(withInterceptors([demoUserInterceptor])),
    provideRouter(
      routes,
      // Route data and query parameters arrive as component inputs, so a
      // page declares `readonly reason = input<string>()` instead of
      // injecting ActivatedRoute and subscribing.
      withComponentInputBinding(),
      withRouterConfig({ onSameUrlNavigation: 'reload' }),
    ),
  ],
};
```

`withComponentInputBinding()` lets route data and query parameters arrive as component inputs. `onSameUrlNavigation: 'reload'` lets the shell re-run the guard for the current page when you switch user (Chapter 16).

15.8 Global styles

client/sentinel-web/src/styles.css

```
        /* Design tokens. Components use these variables, never raw colours, so a
           theme change is one edit here. */
        :root {
          --bg: #f6f7f9;
          --surface: #ffffff;
          --border: #dfe3e8;
          --hover: #eef1f5;
          --text: #1d2733;
          --muted: #5d6b7a;
          --accent: #1f5fbf;
          --accent-bg: #e6eefb;
          --header-bg: #14213d;
          --header-fg: #ffffff;
          --ok: #1e7b45;
          --ok-bg: #e7f5ec;
          --info: #1f5fbf;
          --info-bg: #e9f0fb;
          --warning: #b26a00;
          --warning-bg: #fff4e0;
          --danger: #b42318;
          --danger-bg: #fdecea;
          --mono: ui-monospace, "Cascadia Code", Consolas, monospace;

          color-scheme: light;
          font-family: system-ui, -apple-system, "Segoe UI", Roboto, sans-serif;
          color: var(--text);
          background: var(--bg);
        }

        body { margin: 0; }
        h1 { font-size: 1.5rem; margin: 0 0 .75rem; }
        code { font-family: var(--mono); font-size: .85em; }
        a { color: var(--accent); }

        button {
          font: inherit;
          padding: .4rem .85rem;
          border-radius: 6px;
          border: 1px solid var(--border);
          background: var(--surface);
          cursor: pointer;
        }
        button:hover:not(:disabled) { background: var(--hover); }
        button:disabled { opacity: .6; cursor: progress; }
        button.primary { background: var(--accent); border-color: var(--accent); color: #fff; }
        button.primary:hover { background: #184e9e; }

        input, select { font: inherit; padding: .35rem .5rem; border: 1px solid var(--border);
border-radius: 6px; background: var(--surface); }

        :focus-visible { outline: 3px solid var(--accent); outline-offset: 2px; }
```

No component library: plain CSS with design tokens. Components refer to `var(--danger)` rather than `#b42318`, so the palette lives in one place.

Chapter 16 · The Shell, Navigation and the Route Guard

This chapter connects the second scenario (page authorisation) to the front end: the menu shows only what the server says you may open, and the router refuses everything else.

16.1 The shell

`client/sentinel-web/src/app/app.ts`

```
import { ChangeDetectionStrategy, Component, inject, signal } from '@angular/core';
import { Router, RouterOutlet } from '@angular/router';
import { DemoUser } from './core/models/api-models';
import { Session } from './core/session/session';
import { NavTree } from './layout/nav-tree';
import { UserSwitcher } from './layout/user-switcher';

/** The shell: header, menu, and the outlet the pages render into. */
@Component({
  selector: 'app-root',
  imports: [RouterOutlet, NavTree, UserSwitcher],
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <header class="top">
      <a class="brand" href="/">Sentinel</a>
      <app-user-switcher [users]="demoUsers()" [selected]="session.signedInEmail()"
(selectedChange)="switchUser($event)" />
    </header>
    <div class="body">
      <nav aria-label="Main">
        <app-nav-tree [nodes]="session.navigation()" />
      </nav>
      <main>
        <router-outlet />
      </main>
    </div>
  `,
  styles: `
    :host { display: grid; grid-template-rows: auto 1fr; min-height: 100vh; }
    .top { display: flex; justify-content: space-between; align-items: center; padding:
.6rem 1.25rem;
           background: var(--header-bg); color: var(--header-fg); }
    .brand { font-weight: 700; letter-spacing: .04em; color: inherit; text-decoration:
none; }
    .body { display: grid; grid-template-columns: 15rem 1fr; }
    nav { padding: 1rem .75rem; border-right: 1px solid var(--border); background: var(--
surface); }
    main { padding: 1.25rem 1.75rem; }
    @media (max-width: 760px) { .body { grid-template-columns: 1fr; } nav { border-right:
none; border-bottom: 1px solid var(--border); } }
  `,
})
export class App {
  protected readonly session = inject(Session);
  private readonly router = inject(Router);
  protected readonly demoUsers = signal<DemoUser[]>([]);

  constructor() {
    this.session.getDemoUsers().subscribe({
      next: (users) => this.demoUsers.set(users),
      error: () => this.demoUsers.set([]), // the API is down; pages will say so
    });
    this.session.refresh().subscribe();
```

```
  }

  protected switchUser(email: string | null): void {
    this.session.signInAs(email).subscribe(() => {
      // Re-run the guard for the page we are on: the new user may not be
      // allowed to see it. onSameUrlNavigation: 'reload' (app.config.ts)
      // and runGuardsAndResolvers: 'always' (app.routes.ts) make this work.
      this.router.navigateByUrl(this.router.url);
    });
  }
}
```

The shell owns two things: the list of demo users and the reaction to switching. It passes data down to two children and listens to one output. After a switch it re-navigates to the current URL, which, together with `runGuardsAndResolvers: 'always'` on the routes, re-runs the guard: the new user may not be allowed to stay where the old one was.

16.2 The user switcher: two-way binding by hand

`client/sentinel-web/src/app/layout/user-switcher.ts`

```
import { ChangeDetectionStrategy, Component, input, output } from '@angular/core';
import { DemoUser } from '../core/models/api-models';

/**
 * A select box of demo users. It supports two-way binding:
 *
 *   <app-user-switcher [users]="users" [(selected)]="email" />
 *
 * `[(selected)]` is sugar for `[selected]="email" (selectedChange)="email = $event"`;
 * Angular wires it up because the output is named after the input plus
 * "Change". (Angular 17.2+ also offers `model()`, which declares both in one
 * line. The explicit pair is shown here because it is what every version of
 * Angular understands, including whatever version the test environment has.)
 */
@Component({
  selector: 'app-user-switcher',
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <label>
      <span>Signed in as</span>
      <select [value]="selected() ?? ''" (change)="onChange($event)">
        <option value="">Nobody (signed out)</option>
        @for (user of users(); track user.email) {
          <option [value]="user.email" [selected]="user.email === selected()">
            {{ user.displayName }} · {{ user.description }}
          </option>
        }
      </select>
    </label>
  `,
  styles: `
    label { display: flex; gap: .5rem; align-items: center; font-size: .85rem; }
    select { max-width: 22rem; }
  `,
})
export class UserSwitcher {
  readonly users = input<DemoUser[]>([]);
  readonly selected = input<string | null>(null);
  readonly selectedChange = output<string | null>();

  protected onChange(event: Event): void {
    const value = (event.target as HTMLSelectElement).value;
    this.selectedChange.emit(value === '' ? null : value);
```

```
  }
}
```

The input `selected` and the output `selectedChange` form a pair that Angular recognises: a parent may write `[(selected)]="email"`. The shell uses the long form, `[selected]` and `(selectedChange)`, because it wants to *do* something on change, not just store the value.

16.3 The menu: a recursive component

`client/sentinel-web/src/app/layout/nav-tree.ts`

```
import { ChangeDetectionStrategy, Component, input } from '@angular/core';
import { RouterLink, RouterLinkActive } from '@angular/router';
import { NavigationNode } from '../core/models/api-models';

/**
 * The menu, drawn from the server's navigation tree. Recursive: a node's
 * children are another <app-nav-tree>. The component has no idea who is
 * signed in or what they may see; the server already filtered the tree.
 */
@Component({
  selector: 'app-nav-tree',
  // No need to import NavTree into itself: a standalone component can
  // always use its own selector, which is what makes recursion work.
  imports: [RouterLink, RouterLinkActive],
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <ul [class.nested]="depth() > 0">
      @for (node of nodes(); track node.path) {
        <li>
          <a [routerLink]="node.path"
             routerLinkActive="active"
             [routerLinkActiveOptions]="{ exact: true }">{{ node.title }}</a>
          @if (node.children.length > 0) {
            <app-nav-tree [nodes]="node.children" [depth]="depth() + 1" />
          }
        </li>
      }
    </ul>
  `,
  styles: `
    ul { list-style: none; margin: 0; padding: 0; }
    ul.nested { padding-left: .9rem; border-left: 1px solid var(--border); margin-left:
.4rem; }
    a { display: block; padding: .3rem .5rem; border-radius: 4px; color: inherit; text-
decoration: none; }
    a:hover { background: var(--hover); }
    a.active { background: var(--accent-bg); color: var(--accent); font-weight: 600; }
  `,
})
export class NavTree {
  readonly nodes = input.required<NavigationNode[]>();
  readonly depth = input(0);
}
```

The component renders itself for children. A standalone component can always use its own selector, so it does not list itself in `imports`. The `depth` input is passed down so the nested lists can be indented.

16.4 The routes

`client/sentinel-web/src/app/app.routes.ts`

```
        import { Route, Routes } from '@angular/router';
        import { pageAccessGuard } from './core/access/page-access.guard';
        import { DashboardPage } from './features/pages/dashboard-page';
        import { ForbiddenPage } from './features/pages/forbidden-page';
        import { PlaceholderPage } from './features/pages/placeholder-page';
        import { AccountSecurityPage } from './features/security/account-security-page';

        /**
         * Every route here has a page with the same path in the server's page tree,
         * and the guard asks the server about that path. Adding a route without
         * adding the page sends users to /forbidden ("Unknown"), which is the safe
         * failure: a page nobody configured is a page nobody may see.
         */
        function guarded(path: string, route: Omit<Route, 'path'>): Route {
          return { path, canActivate: [pageAccessGuard], runGuardsAndResolvers: 'always', ...route
};
        }

        function placeholder(path: string, title: string): Route {
          return guarded(path, { component: PlaceholderPage, data: { title } });
        }

        export const routes: Routes = [
          guarded('', { component: DashboardPage, pathMatch: 'full' }),
          placeholder('account', 'My account'),
          guarded('account/security', { component: AccountSecurityPage }),

          placeholder('reports', 'Reports'),
          placeholder('reports/finance', 'Finance'),
          placeholder('reports/finance/export', 'Ledger export'),
          placeholder('reports/operations', 'Operations'),
          placeholder('billing', 'Billing'),
          placeholder('support', 'Support desk'),

          // Admin screens are lazy: users who cannot open them never download them.
          placeholder('admin', 'Administration'),
          guarded('admin/users', { loadComponent: () => import('./features/admin/users-
page').then((m) => m.UsersPage) }),
          guarded('admin/roles', { loadComponent: () => import('./features/admin/roles-
page').then((m) => m.RolesPage) }),
          guarded('admin/access-explorer', {
            loadComponent: () => import('./features/admin/access-explorer-page').then((m) =>
m.AccessExplorerPage),
          }),

          placeholder('help', 'Help centre'),
          placeholder('status', 'System status'),

          { path: 'forbidden', component: ForbiddenPage },
          { path: '**', component: ForbiddenPage, data: { reason: 'Unknown' } },
        ];
```

Every route except `/forbidden` and the wildcard is guarded. The two helper functions are worth the lines: without them, twelve routes would repeat `canActivate: [pageAccessGuard], runGuardsAndResolvers: 'always'`, and the thirteenth would forget it. The admin screens are lazy-loaded, so users who cannot open them never download them.

16.5 The guard

`client/sentinel-web/src/app/core/access/page-access.guard.ts`

```
        import { HttpClient } from '@angular/common/http';
        import { inject } from '@angular/core';
        import { CanActivateFn, Router } from '@angular/router';
        import { Observable, catchError, map, of } from 'rxjs';
```

```ts
import { PageAccess } from '../models/api-models';

/**
 * Asks the API whether the signed-in user may open the page they are
 * navigating to, and sends them to /forbidden (with the reason) if not.
 *
 * Why ask the server on every navigation instead of checking the menu we
 * already have? Because the menu is only a *view* of the rules. A typed URL,
 * a bookmark or a stale tab must be judged by the rules themselves, and
 * the rules live in one place: PageAccessEvaluator, on the server.
 *
 * The guard protects the user experience, not the data. The API checks
 * permissions again on every data request, so a user who bypasses the
 * guard gets a blank page, never someone else's data.
 */
export const pageAccessGuard: CanActivateFn = (_route, state) => {
  const http = inject(HttpClient);
  const router = inject(Router);

  // The router's URL includes query string and fragment; pages are judged by path.
  const path = pathOnly(state.url);

  return http.get<PageAccess>('/api/me/page-access', { params: { path } }).pipe(
    map((verdict) =>
      verdict.isAllowed
        ? true
        : router.createUrlTree(['/forbidden'], { queryParams: { page: verdict.path,
reason: verdict.reason } }),
    ),
    catchError((): Observable<boolean | ReturnType<Router['createUrlTree']>> =>
      // 404 (no such page) or the API is down. Fail closed: an unknown
      // page is not a reason to let someone in.
      of(router.createUrlTree(['/forbidden'], { queryParams: { page: path, reason:
'Unknown' } })),
    ),
  );
};

/** "/reports/finance?tab=2#top" -> "/reports/finance" */
export function pathOnly(url: string): string {
  const end = url.search(/[?#]/);
  const path = end === -1 ? url : url.slice(0, end);
  return path === '' ? '/' : path;
}
```

Why ask the server on every navigation instead of checking the menu the client already holds? Because the menu is a *view* of the rules, not the rules. A typed URL, a bookmark or a tab left open while an administrator revoked a role must all be judged by the rules themselves, and the rules live in one place.

And the guard is user experience, not security. The comment says so: the API checks permissions again on every data request, so a user who bypasses the guard (it is JavaScript running on their machine) gets an empty page, never someone else's data.

Three edge cases are handled and tested:

1. Denied → redirect to `/forbidden` with the page and the reason.
2. Unknown page or API down → **fail closed**, reason `Unknown`.
3. The router's URL includes `?query` and `#fragment`; access is decided by the path alone (`pathOnly`).

16.6 Explaining a denial

`client/sentinel-web/src/app/core/access/access-reasons.ts`

```
import { AccessReason } from '../models/api-models';

/**
 * Plain-English explanations of the evaluator's reasons, for the people who
 * hit them. Written for a non-engineer: no "permission strings", no
 * "parents", just what happened and what to do next.
 */
const explanations: Record<AccessReason | 'Unknown', string> = {
  PublicPage: 'This page is open to everyone.',
  AuthenticatedUser: 'Every signed-in user can open this page.',
  PermissionGranted: 'One of your roles gives you access to this page.',
  NotAuthenticated: 'You need to sign in to see this page.',
  UserSuspended: 'Your account is suspended, so only public pages are available. Contact
your administrator.',
  MissingPermission: 'None of your roles includes this page. Ask your administrator if you
need it.',
  ExplicitlyDenied: 'Your administrator has specifically removed your access to this
page.',
  ParentDenied: 'This page sits inside a section you do not have access to.',
  OrphanedPage: 'This page is misconfigured. Please report it to support.',
  CyclicHierarchy: 'This page is misconfigured. Please report it to support.',
  Unknown: 'This page does not exist, or we could not check your access to it right now.',
};

export function explainAccess(reason: string | null | undefined): string {
  return explanations[(reason ?? 'Unknown') as AccessReason] ?? explanations.Unknown;
}
```

client/sentinel-web/src/app/features/pages/forbidden-page.ts

```
import { ChangeDetectionStrategy, Component, computed, input } from '@angular/core';
import { RouterLink } from '@angular/router';
import { explainAccess } from '../../core/access/access-reasons';

/**
 * Where the guard sends people it turns away. `page` and `reason` are query
 * parameters, bound to inputs by withComponentInputBinding().
 */
@Component({
  selector: 'app-forbidden-page',
  imports: [RouterLink],
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    <h1>You can't open this page</h1>
    @if (page()) {
      <p><code>{{ page() }}</code></p>
    }
    <p>{{ explanation() }}</p>
    <p><a routerLink="/">Back to the dashboard</a></p>
  `,
})
export class ForbiddenPage {
  readonly page = input<string>();
  readonly reason = input<string>();

  protected readonly explanation = computed(() => explainAccess(this.reason()));
}
```

These sentences are written for a non-engineer: no “permission strings”, no “parent nodes”. Writing them is the same skill as the customer reply in Chapter 24, and the same reviewer instinct applies: *say what happened and what to do next.*

client/sentinel-web/src/app/features/pages/placeholder-page.ts

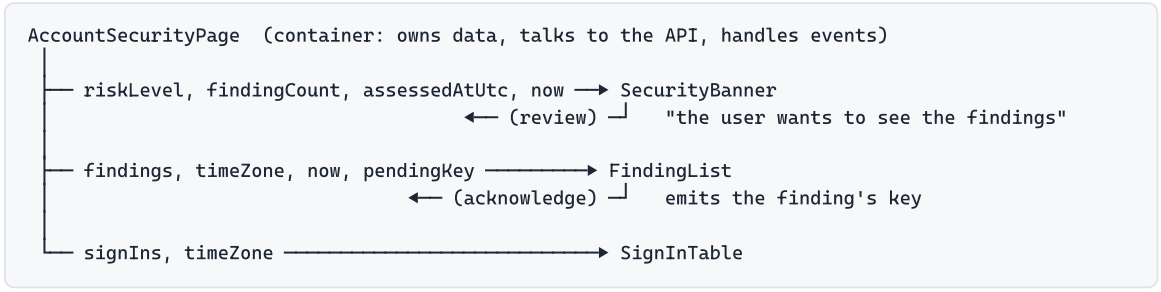
```
        import { ChangeDetectionStrategy, Component, inject, input } from '@angular/core';
        import { Session } from '../../core/session/session';

        /**
         * Stands in for the screens this book does not build (reports, billing...).
         * They exist so the page tree and the guard have something real to protect.
         * `title` arrives from the route's `data` through component input binding.
         */
        @Component({
          selector: 'app-placeholder-page',
          changeDetection: ChangeDetectionStrategy.OnPush,
          template: `
            <h1>{{ title() }}</h1>
            <p>
              @if (session.currentUser(); as user) {
                {{ user.displayName }}, you can see this page because the access rules allow it.
              } @else {
                This page is public.
              }
            </p>
            <p class="muted">The real screen is out of scope for this book. Try the <em>Account
security</em> page or the <em>Administration</em> section.</p>
          `,
          styles: `.muted { color: var(--muted); }`,
        })
        export class PlaceholderPage {
          protected readonly session = inject(Session);
          readonly title = input('Sentinel');
        }
```

Chapter 17 · Passing Data Between Components: the Account Security Page

"How do you pass information between components?" is the first Angular-specific criterion on the scenario's list. This chapter answers it with a real page, then lists every mechanism Angular offers and when each is the right one.

17.1 The component tree



Data flows down through inputs. Intentions flow up through outputs. Only the container has side effects. If you remember one sentence from this part of the book, make it that one. It is what a reviewer is checking when they ask how you pass information, and it is what makes each child trivial to test and reuse: `SecurityBanner` appears on the dashboard and the admin explorer too, with a different reaction to `(review)` each time, and was never changed to allow it.

17.2 The presentational children

`client/sentinel-web/src/app/features/security/risk.ts`

```
        import { FindingKind, RiskLevel } from '../../core/models/api-models';

        export type Tone = 'ok' | 'info' | 'warning' | 'danger';

        export interface RiskPresentation {
          tone: Tone;
          headline: string;
          advice: string;
        }

        /**
         * What each risk level means to the person reading it. The words are the
         * product here: "High" tells a user nothing, "Someone may be using your
         * account" tells them to act.
         */
        const presentations: Record<RiskLevel, RiskPresentation> = {
          None: {
            tone: 'ok',
            headline: 'No suspicious activity',
            advice: 'Your recent sign-ins all look like you.',
          },
          Low: {
            tone: 'info',
            headline: 'Something new to review',
            advice: 'We noticed a sign-in we have not seen before. If it was you, mark it so we
stop asking.',
          },
          Medium: {
```

```
            tone: 'warning',
            headline: 'Your account may be at risk',
            advice: 'Review the activity below. If you do not recognise it, change your password
now.',
          },
          High: {
            tone: 'danger',
            headline: 'Someone may be using your account',
            advice: 'Change your password now and sign out of all other sessions. Then review the
activity below.',
          },
        };

        export function presentRisk(level: RiskLevel | null | undefined): RiskPresentation {
          // An unexpected value from a newer API is shown as the most cautious
          // known level rather than as "all clear". Fail towards telling the user.
          return presentations[level as RiskLevel] ?? presentations.Medium;
        }

        const findingTitles: Record<FindingKind, string> = {
          BruteForce: 'Repeated failed sign-ins',
          ImpossibleTravel: 'Sign-ins too far apart to be one person',
          NewCountry: 'Sign-in from a new country',
          NewDevice: 'Sign-in from a new device',
        };

        export function findingTitle(kind: FindingKind | string): string {
          return findingTitles[kind as FindingKind] ?? 'Unusual activity';
        }

        export function severityTone(level: RiskLevel | string): Tone {
          return presentRisk(level as RiskLevel).tone;
        }
```

The words are the product. “High” tells a user nothing; “Someone may be using your account” tells them to act. Note the edge case: an unexpected level from a newer API is shown as a *warning*, never as all clear. When in doubt, fail towards telling the user.

`client/sentinel-web/src/app/features/security/security-banner.ts`

```
        import { ChangeDetectionStrategy, Component, computed, input, output } from
'@angular/core';
        import { RiskLevel } from '../../core/models/api-models';
        import { RelativeTimePipe, UtcTimePipe } from '../../core/time/time-pipes';
        import { presentRisk } from './risk';

        /**
         * The verdict at the top of the page. Presentational: everything it shows
         * arrives through inputs, and the one thing it can do (ask to review)
         * leaves through an output. It does not know where the data came from or
         * what "review" will do, which is exactly what makes it reusable on the
         * admin screen and trivial to test.
         */
        @Component({
          selector: 'app-security-banner',
          imports: [RelativeTimePipe, UtcTimePipe],
          changeDetection: ChangeDetectionStrategy.OnPush,
          template: `
            <section class="banner" [class]="'tone-' + presentation().tone" role="status" aria-
live="polite">
              <div class="banner-text">
                <h2>{{ presentation().headline }}</h2>
                <p>{{ presentation().advice }}</p>
                <p class="assessed" [title]="assessedAtUtc() | utcTime">
                  Checked {{ assessedAtUtc() | relativeTime: now() }}
                </p>
              </div>
              @if (findingCount() > 0) {
```

```
                <button type="button" class="primary" (click)="review.emit()">
                  Review {{ findingCount() }} {{ findingCount() === 1 ? 'item' : 'items' }}
                </button>
              }
            </section>
          `,
          styles: `
            .banner { display: flex; gap: 1rem; align-items: center; justify-content: space-
between;
                      padding: 1rem 1.25rem; border-radius: 8px; border-left: 6px solid; }
            .banner h2 { margin: 0 0 .25rem; font-size: 1.15rem; }
            .banner p { margin: .15rem 0; }
            .assessed { font-size: .8rem; opacity: .75; }
            .tone-ok { background: var(--ok-bg); border-color: var(--ok); }
            .tone-info { background: var(--info-bg); border-color: var(--info); }
            .tone-warning { background: var(--warning-bg); border-color: var(--warning); }
            .tone-danger { background: var(--danger-bg); border-color: var(--danger); }
          `,
        })
        export class SecurityBanner {
          readonly riskLevel = input.required<RiskLevel>();
          readonly findingCount = input(0);
          readonly assessedAtUtc = input.required<string>();
          readonly now = input.required<Date>();

          /** The user wants to see the findings. The parent decides what that means. */
          readonly review = output<void>();

          protected readonly presentation = computed(() => presentRisk(this.riskLevel()));
        }
```

- `input.required<RiskLevel>()`: the parent *must* bind it; forgetting is a compile error, not a blank banner.
- `input(0)`: optional, with a default.
- `output<void>()`: an event with no payload. The banner does not know what "review" will do, and should not.
- `computed(...)`: derived state, recalculated only when `riskLevel` changes.
- `ChangeDetectionStrategy.OnPush`: re-render only when an input or a signal it reads changes. With signal inputs this is the natural default.
- `role="status"` and `aria-live="polite"`: a screen reader announces the verdict when it changes.

client/sentinel-web/src/app/features/security/finding-list.ts

```
        import { ChangeDetectionStrategy, Component, input, output } from '@angular/core';
        import { SecurityFinding } from '../../core/models/api-models';
        import { RelativeTimePipe, UtcTimePipe, ZonedDatePipe } from '../../core/time/time-pipes';
        import { findingTitle, severityTone } from './risk';

        /**
         * The findings, one card each, with a "This was me" button. Clicking it
         * does not call the API; it emits the finding's key and the page decides.
         * `pendingKey` comes back down as an input so the button can show progress
         * and refuse a double click while the request is in flight.
         */
        @Component({
          selector: 'app-finding-list',
          imports: [ZonedDatePipe, RelativeTimePipe, UtcTimePipe],
          changeDetection: ChangeDetectionStrategy.OnPush,
          template: `
            @for (finding of findings(); track finding.key) {
              <article class="finding" [class]="'tone-' + tone(finding.severity)">
                <header>
                  <span class="severity">{{ finding.severity }}</span>
                  <h3>{{ title(finding.kind) }}</h3>
```

```
                </header>
                <p>{{ finding.summary }}</p>
                <p class="when">
                  <time [attr.datetime]="finding.occurredAtUtc" [title]="finding.occurredAtUtc |
utcTime">
                    {{ finding.occurredAtUtc | zonedDate: timeZone() }}
                  </time>
                  · {{ finding.occurredAtUtc | relativeTime: now() }}
                </p>
                <div class="actions">
                  <button type="button" (click)="acknowledge.emit(finding.key)"
[disabled]="pendingKey() !== null">
                    {{ pendingKey() === finding.key ? 'Saving…' : 'This was me' }}
                  </button>
                  <a href="/help">This wasn't me</a>
                </div>
              </article>
            } @empty {
              <p class="empty">Nothing to review.</p>
            }
          `,
          styles: `
            :host { display: grid; gap: .75rem; }
            .finding { border: 1px solid var(--border); border-left: 5px solid; border-radius:
6px; padding: .75rem 1rem; background: var(--surface); }
            .finding header { display: flex; gap: .5rem; align-items: baseline; }
            .finding h3 { margin: 0; font-size: 1rem; }
            .severity { font-size: .7rem; font-weight: 700; text-transform: uppercase; letter-
spacing: .05em; }
            .when { font-size: .85rem; color: var(--muted); }
            .actions { display: flex; gap: 1rem; align-items: center; }
            .tone-info { border-left-color: var(--info); }
            .tone-warning { border-left-color: var(--warning); }
            .tone-danger { border-left-color: var(--danger); }
            .empty { color: var(--muted); }
          `,
        })
        export class FindingList {
          readonly findings = input.required<SecurityFinding[]>();
          readonly timeZone = input.required<string>();
          readonly now = input.required<Date>();
          readonly pendingKey = input<string | null>(null);

          readonly acknowledge = output<string>();

          protected readonly title = findingTitle;
          protected readonly tone = severityTone;
        }
```

The list emits the **key**, not the whole finding and not an index. A key is stable; an index changes when the list does, which is how "I clicked *This was me* on one card and another one disappeared" bugs are born. `track finding.key` in the `@for` tells Angular the same thing about identity, so cards are not recreated on every reload.

`pendingKey` travels *down* after the event went *up*: the container decides that a request is in flight, and the child reflects it by disabling every button. That prevents double submission without the child knowing anything about HTTP.

client/sentinel-web/src/app/features/security/sign-in-table.ts

```
        import { ChangeDetectionStrategy, Component, input } from '@angular/core';
        import { SignInAttempt } from '../../core/models/api-models';
        import { UtcTimePipe, ZonedDatePipe } from '../../core/time/time-pipes';

        /** Recent sign-ins. Rows the findings cite as evidence are highlighted. */
        @Component({
```

```
          selector: 'app-sign-in-table',
          imports: [ZonedDatePipe, UtcTimePipe],
          changeDetection: ChangeDetectionStrategy.OnPush,
          template: `
            @if (signIns().length === 0) {
              <p class="empty">No sign-ins yet.</p>
            } @else {
              <table>
                <caption>Times shown in {{ timeZone() }}</caption>
                <thead>
                  <tr><th scope="col">When</th><th scope="col">Result</th><th
scope="col">Where</th><th scope="col">Device</th><th scope="col">IP address</th></tr>
                </thead>
                <tbody>
                  @for (signIn of signIns(); track signIn.id) {
                    <tr [class.flagged]="signIn.isFlagged">
                      <td>
                        <time [attr.datetime]="signIn.occurredAtUtc" [title]="signIn.occurredAtUtc
| utcTime">
                          {{ signIn.occurredAtUtc | zonedDate: timeZone() }}
                        </time>
                        @if (signIn.isFlagged) { <span class="flag" title="Part of a finding
above">⚑</span> }
                      </td>
                      <td>
                        @if (signIn.succeeded) { <span class="ok">Signed in</span> }
                        @else { <span class="failed">Failed · {{
describeFailure(signIn.failureReason) }}</span> }
                      </td>
                      <td>{{ signIn.countryCode === 'ZZ' ? 'Unknown' : signIn.city + ', ' +
signIn.countryCode }}</td>
                      <td>{{ signIn.device }}</td>
                      <td><code>{{ signIn.ipAddress }}</code></td>
                    </tr>
                  }
                </tbody>
              </table>
            }
          `,
          styles: `
            table { width: 100%; border-collapse: collapse; background: var(--surface); }
            caption { text-align: left; color: var(--muted); font-size: .85rem; padding: .25rem 0;
}
            th, td { text-align: left; padding: .45rem .6rem; border-bottom: 1px solid var(--
border); font-size: .9rem; }
            tr.flagged { background: var(--danger-bg); }
            .flag { color: var(--danger); margin-left: .35rem; }
            .ok { color: var(--ok); }
            .failed { color: var(--danger); }
            .empty { color: var(--muted); }
          `,
        })
        export class SignInTable {
          readonly signIns = input.required<SignInAttempt[]>();
          readonly timeZone = input.required<string>();

          protected describeFailure(reason: string | null): string {
            switch (reason) {
              case 'WrongPassword': return 'wrong password';
              case 'WrongSecondFactor': return 'wrong code';
              case 'AccountLocked': return 'account locked';
              case 'AccountSuspended': return 'account suspended';
              default: return 'unknown reason';
            }
          }
        }
```

Edge cases visible here: no sign-ins at all, unknown location (`ZZ`), and every failure reason including one we do not recognise. `<time datetime>` keeps the machine-readable UTC value next to the human one, and the tooltip shows UTC for support conversations.

17.3 The container

client/sentinel-web/src/app/features/security/security-api.ts

```
        import { HttpClient } from '@angular/common/http';
        import { Injectable, inject } from '@angular/core';
        import { Observable } from 'rxjs';
        import { AccountSecurityStatus } from '../../core/models/api-models';

        /** The account security endpoints, and nothing else. Components never build URLs. */
        @Injectable({ providedIn: 'root' })
        export class SecurityApi {
          private readonly http = inject(HttpClient);

          getStatus(): Observable<AccountSecurityStatus> {
            return this.http.get<AccountSecurityStatus>('/api/me/security');
          }

          /** "That was me." The key goes in the URL, so it must be escaped: keys contain ':'. */
          acknowledge(findingKey: string): Observable<void> {
            return this.http.post<void>
(`/api/me/security/findings/${encodeURIComponent(findingKey)}/acknowledge`, null);
          }
        }
```

client/sentinel-web/src/app/features/security/account-security-page.ts

```
        import { ChangeDetectionStrategy, Component, ElementRef, inject, signal, viewChild } from
'@angular/core';
        import { takeUntilDestroyed, toObservable } from '@angular/core/rxjs-interop';
        import { BehaviorSubject, Observable, catchError, combineLatest, map, of, startWith,
switchMap } from 'rxjs';
        import { describeHttpError } from '../../core/http/api-error';
        import { AccountSecurityStatus } from '../../core/models/api-models';
        import { Session } from '../../core/session/session';
        import { Clock } from '../../core/time/clock';
        import { FindingList } from './finding-list';
        import { SecurityApi } from './security-api';
        import { SecurityBanner } from './security-banner';
        import { SignInTable } from './sign-in-table';

        /**
         * Every state the page can be in. A union instead of three booleans
         * (isLoading, hasError, data?) means "loading AND error" cannot be
         * represented, so the template cannot accidentally render it.
         */
        type PageState =
          | { kind: 'signed-out' }
          | { kind: 'loading' }
          | { kind: 'error'; message: string }
          | { kind: 'loaded'; status: AccountSecurityStatus };

        /**
         * The account security page. A container: it owns the data and the side
         * effects, and hands slices of the data to three presentational children.
         *
         *   AccountSecurityPage ── status, now ──▶ SecurityBanner ── (review) ──▶ scroll to
findings
         *                       ── findings ─────▶ FindingList    ── (acknowledge) ──▶ POST, then
reload
         *                       ── signIns ──────▶ SignInTable
         *
         * Data goes down through inputs; intentions come up through outputs; only
         * this component talks to the API.
         */
        @Component({
```

```
          selector: 'app-account-security-page',
          imports: [SecurityBanner, FindingList, SignInTable],
          changeDetection: ChangeDetectionStrategy.OnPush,
          template: `
            <h1>Account security</h1>

            @switch (state().kind) {
              @case ('signed-out') {
                <p>Sign in to see your account's security.</p>
              }
              @case ('loading') {
                <p aria-busy="true">Checking your recent activity…</p>
              }
              @case ('error') {
                <div class="error" role="alert">
                  <p>{{ errorMessage() }}</p>
                  <button type="button" (click)="reload()">Try again</button>
                </div>
              }
              @case ('loaded') {
                @if (status(); as status) {
                  <app-security-banner
                    [riskLevel]="status.riskLevel"
                    [findingCount]="status.findings.length"
                    [assessedAtUtc]="status.assessedAtUtc"
                    [now]="clock.now()"
                    (review)="scrollToFindings()" />

                  @if (actionError()) {
                    <p class="error" role="alert">{{ actionError() }}</p>
                  }

                  <h2 #findingsHeading tabindex="-1">What we noticed</h2>
                  <app-finding-list
                    [findings]="status.findings"
                    [timeZone]="status.timeZoneId"
                    [now]="clock.now()"
                    [pendingKey]="pendingKey()"
                    (acknowledge)="acknowledge($event)" />

                  <h2>Recent sign-ins</h2>
                  <app-sign-in-table [signIns]="status.recentSignIns"
[timeZone]="status.timeZoneId" />
                }
              }
            }
          `,
          styles: `
            :host { display: grid; gap: 1rem; max-width: 64rem; }
            h2 { margin: .5rem 0 0; font-size: 1.1rem; }
            .error { color: var(--danger); }
          `,
        })
        export class AccountSecurityPage {
          private readonly api = inject(SecurityApi);
          private readonly session = inject(Session);
          protected readonly clock = inject(Clock);

          private readonly findingsHeading = viewChild<ElementRef<HTMLElement>>
('findingsHeading');
          private readonly reload$ = new BehaviorSubject<void>(undefined);

          protected readonly state = signal<PageState>({ kind: 'loading' });
          protected readonly pendingKey = signal<string | null>(null);
          protected readonly actionError = signal<string | null>(null);

          constructor() {
            // Reload whenever the signed-in user changes (the demo switcher) or a
            // reload is requested. switchMap cancels a slow request for the
            // previous user, so a late response can never overwrite a newer one.
            combineLatest([toObservable(this.session.currentUser), this.reload$])
              .pipe(
```

```
                switchMap(([user]) => (user ? this.load() : of<PageState>({ kind: 'signed-out'
})),
                takeUntilDestroyed(),
              )
              .subscribe((state) => this.state.set(state));
          }

          protected status(): AccountSecurityStatus | null {
            const state = this.state();
            return state.kind === 'loaded' ? state.status : null;
          }

          protected errorMessage(): string {
            const state = this.state();
            return state.kind === 'error' ? state.message : '';
          }

          protected reload(): void {
            this.reload$.next();
          }

          protected acknowledge(findingKey: string): void {
            if (this.pendingKey() !== null) {
              return; // one at a time; the buttons are disabled too, this is belt and braces
            }

            this.pendingKey.set(findingKey);
            this.actionError.set(null);

            this.api.acknowledge(findingKey).subscribe({
              next: () => {
                this.pendingKey.set(null);
                // Reload rather than removing the card locally: the risk level and
                // the flagged rows depend on every finding, and only the server
                // knows the new answer.
                this.reload();
              },
              error: (error: unknown) => {
                this.pendingKey.set(null);
                this.actionError.set(describeHttpError(error));
              },
            });
          }

          protected scrollToFindings(): void {
            const heading = this.findingsHeading()?.nativeElement;
            heading?.scrollIntoView({ behavior: 'smooth', block: 'start' });
            heading?.focus({ preventScroll: true }); // keyboard and screen-reader users land
there too
          }

          private load(): Observable<PageState> {
            return this.api.getStatus().pipe(
              map((status): PageState => ({ kind: 'loaded', status })),
              // Keep showing the old data while refreshing after an acknowledge,
              // instead of flashing "loading" over a page the user is reading.
              startWith<PageState>(this.state().kind === 'loaded' ? this.state() : { kind:
'loading' }),
              catchError((error: unknown) => of<PageState>({ kind: 'error', message:
describeHttpError(error) })),
            );
          }
        }
```

Four decisions in this class are worth being able to explain in an interview:

A union type for page state. Three booleans (`isLoading`, `hasError`, `data`) allow eight combinations, of which four are nonsense (“loading and failed and loaded”). A union with four members allows exactly the four real states, and `@switch (state().kind)` renders exactly one of them.

`switchMap` on user changes. The demo switcher can change user while a request is in flight. `switchMap` cancels the previous request, so a slow response for Ada can never overwrite a fast one for Tunde. Without it, the page could show one user's findings under another user's name.

Reload after acknowledging, rather than removing the card locally. The risk level and the flagged rows depend on all findings together; only the server knows the new answer. Optimistic updates are a fine technique, but not for a security verdict.

`startWith` the current state when refreshing. After an acknowledgement the page keeps showing the old data until the new data arrives, instead of flashing "loading" under the user's cursor.

17.4 Every way to pass information, and when to use each

Mechanism	Direction	Use it when	In Sentinel
`input()` / `@Input()`	parent → child	The child displays or depends on data	Every presentational component
`output()` / `@Output()` + `EventEmitter`	child → parent	The child reports an intention	`review`, `acknowledge`, `selectedChange`
Two-way `[(x)]` (`x` + `xChange`, or `model()`)	both	A child edits a value the parent owns	`UserSwitcher`
A shared service holding signals or a `BehaviorSubject`	any → any	State many unrelated components need	`Session`, `Clock`
Router inputs (`withComponentInputBinding`)	URL → page	State that belongs in the address bar	`userId` on the access explorer; `reason` on `/forbidden`
`viewChild()` / `@ViewChild`	parent reads child	The parent must call into its own template (focus, scroll)	`findingsHeading`
Content projection `<ng-content>`	parent → child markup	A child wraps markup it does not own	Not needed here

A reasonable rule for the test: inputs and outputs first; a service when three or more components need the same state; the URL when the state should survive a refresh or be shareable.

17.5 The edge-case checklist for this page

Before calling a page like this done, check each line:

- [ ] Signed out
- [ ] Loading
- [ ] API unreachable (status 0) and a retry
- [ ] Server error with a problem-details message
- [ ] No findings ("Nothing to review", and the banner offers no button)
- [ ] One finding (singular wording) and several (plural)
- [ ] An unknown risk level or finding kind from a newer server
- [ ] An action that fails (the card stays; the error is shown; buttons re-enable)
- [ ] Double-click on an action
- [ ] The user changes while a request is in flight

- [ ] Dates: zone, daylight saving, “just now” vs clock skew (Chapter 18)

Each of these has a test in Chapter 20.

Chapter 18 · Displaying and Formatting Dates

“How do you display and format dates?” is on the Angular scenario’s list for a reason: it is where competent code is most often confidently wrong. This chapter gives you three rules, the code that implements them, and the failure each rule prevents.

18.1 Three rules

1. **The wire format is UTC with an explicit designator**: `2026-10-24T08:00:00Z`. Never a bare `2026-10-24T08:00:00`.
2. **Convert to a zone exactly once, at display time.** Store, compare, sort and transmit instants; turn them into wall-clock time only in the template.
3. **Display in the user’s chosen zone** (their profile’s IANA zone), not the browser’s. A Lagos employee on a trip to London still thinks about their own account in Lagos time; a support agent in London reading a Lagos user’s history wants to see what the *user* saw.

18.2 The implementation

`client/sentinel-web/src/app/core/time/timestamps.ts`

```
        // Everything about turning the API's timestamps into words on a screen.
        //
        // Three rules, each of which is a bug report if broken:
        //   1. The wire format is UTC with an explicit designator ("...Z" or "+01:00").
        //   2. A timestamp is converted to a zone exactly once: at display time.
        //   3. The zone is the user's profile zone (an IANA name such as
        //      "Africa/Lagos"), not whatever the browser happens to be set to.
        //
        // These are plain functions, not an Angular service, so they can be tested
        // without TestBed and reused by pipes, components and specs alike.

        /** "Z", "+01:00", "-0500" at the end of an ISO string. */
        const ZONE_DESIGNATOR = /(Z|[+-]\d{2}:?\d{2})$/i;
        const ISO_DATE_TIME = /^\d{4}-\d{2}-\d{2}T\d{2}:\d{2}/;

        /**
         * Parses an API timestamp, refusing any string that does not say which zone
         * it is in.
         *
         * `new Date("2026-10-24T08:00:00")` is legal JavaScript and silently means
         * 08:00 *in the browser's zone*. When the server forgets its "Z", that
         * reading is wrong by the user's UTC offset and nothing anywhere fails. This
         * function makes that mistake loud instead: it returns null and logs, and
         * the UI shows "time unavailable" rather than a confident wrong answer.
         * (Chapter 22 is the incident that taught us this.)
         */
        export function parseApiTimestamp(value: string | null | undefined): Date | null {
          if (!value || !ISO_DATE_TIME.test(value)) {
            return null;
          }

          if (!ZONE_DESIGNATOR.test(value)) {
            console.error(`[time] Refusing to guess the zone of "${value}". The API must send UTC
with a "Z".`);
            return null;
          }

          const parsed = new Date(value);
          return Number.isNaN(parsed.getTime()) ? null : parsed;
```

```
        }

        /** True if `zone` is an IANA time zone this browser knows. */
        export function isValidTimeZone(zone: string | null | undefined): boolean {
          if (!zone) {
            return false;
          }
          try {
            new Intl.DateTimeFormat('en-GB', { timeZone: zone });
            return true;
          } catch {
            return false;
          }
        }

        export interface ZonedFormatOptions {
          /** BCP 47 locale for the words and the order of the parts. Defaults to en-GB (day
before month). */
          locale?: string;
          /** Include the zone abbreviation ("WAT", "GMT+1"). Defaults to true. */
          showZone?: boolean;
          /** "medium": 24 Oct 2026, 09:00 WAT. "short": 24/10/2026, 09:00. */
          style?: 'medium' | 'short';
        }

        /**
         * Formats an API timestamp for a person in `timeZone`.
         *
         *   formatInZone('2026-10-24T08:00:00Z', 'Africa/Lagos')  -> "24 Oct 2026, 09:00 WAT"
         *   formatInZone('2026-10-24T08:00:00Z', 'Asia/Singapore') -> "24 Oct 2026, 16:00 GMT+8"
         *
         * Abbreviations come from the browser's ICU data and vary: Lagos gets
         * "WAT" in some engines and "GMT+1" in others. Tests should therefore
         * assert on the time, not the abbreviation.
         *
         * An unknown zone falls back to UTC (and says so) rather than throwing,
         * because one bad profile value must not blank the whole page.
         */
        export function formatInZone(value: string | null | undefined, timeZone: string, options:
ZonedFormatOptions = {}): string {
          const date = parseApiTimestamp(value);
          if (!date) {
            return 'time unavailable';
          }

          const zone = isValidTimeZone(timeZone) ? timeZone : 'UTC';
          const { locale = 'en-GB', showZone = true, style = 'medium' } = options;

          const formatter = new Intl.DateTimeFormat(locale, {
            timeZone: zone,
            day: style === 'medium' ? 'numeric' : '2-digit',
            month: style === 'medium' ? 'short' : '2-digit',
            year: 'numeric',
            hour: '2-digit',
            minute: '2-digit',
            hourCycle: 'h23',
            ...(showZone ? { timeZoneName: 'short' as const } : {}),
          });

          return formatter.format(date);
        }

        /** The UTC form of a timestamp, for tooltips and support conversations: "2026-10-24
08:00:00 UTC". */
        export function formatUtc(value: string | null | undefined): string {
          const date = parseApiTimestamp(value);
          return date ? `${date.toISOString().slice(0, 19).replace('T', ' ')} UTC` : 'time
unavailable';
        }

        const RELATIVE_UNITS: [Intl.RelativeTimeFormatUnit, number][] = [
          ['year', 365 * 24 * 60 * 60],
```

```
  ['month', 30 * 24 * 60 * 60],
  ['week', 7 * 24 * 60 * 60],
  ['day', 24 * 60 * 60],
  ['hour', 60 * 60],
  ['minute', 60],
];

/**
 * "3 hours ago", "yesterday", "just now". Relative to `now`, which is a
 * parameter rather than `Date.now()` so the function is deterministic and
 * a test can pin the clock.
 *
 * Times slightly in the future (clock skew between server and browser)
 * read as "just now" instead of the confusing "in 2 minutes".
 */
export function formatRelative(value: string | null | undefined, now: Date, locale = 'en-GB'): string {
  const date = parseApiTimestamp(value);
  if (!date) {
    return 'time unavailable';
  }

  const secondsAgo = Math.round((now.getTime() - date.getTime()) / 1000);
  if (secondsAgo < 60) {
    return 'just now';
  }

  const formatter = new Intl.RelativeTimeFormat(locale, { numeric: 'auto' });
  for (const [unit, secondsInUnit] of RELATIVE_UNITS) {
    if (secondsAgo >= secondsInUnit) {
      return formatter.format(-Math.floor(secondsAgo / secondsInUnit), unit);
    }
  }

  return 'just now'; // unreachable: the loop ends at minutes
}
```

Strict parsing

`parseApiTimestamp` refuses any string without `Z` or an offset. JavaScript would happily parse it, *as local time*, and be wrong by the user's offset with no error anywhere. Refusing makes the UI say "time unavailable" and logs an error: a loud, visible failure instead of a quiet wrong one. Chapter 22 is the incident that justified this function.

Formatting with `Intl`, not `DatePipe`

	Angular `DatePipe`	**`Intl.DateTimeFormat`**
Zone argument	An offset: `'+0100'`	An IANA name: `'Africa/Lagos'`
Daylight saving	You must know the offset for each date	Correct for every date, past or future
Locale data	Must be registered per locale	Built into the browser
Available in templates	Yes	Through a pipe (we write one)

`DatePipe` with a fixed offset gets London wrong for half of every year: London is UTC+0 in January and UTC+1 in July. A test pins this: `applies daylight saving for the date being shown, not for today`.

Two more decisions in `formatInZone`:

- **`hourCycle: 'h23'`**, so `en-GB` shows `09:00`, never `9:00 am`, and the format is stable for tests.

- **An unknown zone falls back to UTC** and still renders. One bad profile value must not blank the page.

Relative time with an injectable “now”

`formatRelative(value, now)` takes `now` as a parameter. That makes it a pure function (same input, same output) and therefore testable with a pinned clock, which is how `5 minutes ago`, `yesterday` and `3 months ago` are each tested exactly. Times slightly in the future (the browser’s clock behind the server’s) read as “just now” rather than “in 2 minutes”.

18.3 The pipes

`client/sentinel-web/src/app/core/time/time-pipes.ts`

```
        import { Pipe, PipeTransform } from '@angular/core';
        import { formatInZone, formatRelative, formatUtc, ZonedFormatOptions } from
'./timestamps';

        /**
         * `{{ signIn.occurredAtUtc | zonedDate: timeZone }}` -> "24 Oct 2026, 09:00 WAT"
         *
         * Why not Angular's built-in DatePipe? DatePipe's timezone argument takes an
         * offset ("+0100"), not a zone name, so it cannot know that London is +0000
         * in January and +0100 in July. Intl.DateTimeFormat takes the IANA name and
         * gets daylight saving right for every date, past or future.
         */
        @Pipe({ name: 'zonedDate' })
        export class ZonedDatePipe implements PipeTransform {
          transform(value: string | null | undefined, timeZone: string, options?:
ZonedFormatOptions): string {
            return formatInZone(value, timeZone, options);
          }
        }

        /**
         * `{{ finding.occurredAtUtc | relativeTime: now }}` -> "3 hours ago"
         *
         * A pure pipe only re-runs when its inputs change, so "now" is an argument:
         * the page passes a signal that ticks once a minute, and the text stays
         * current without the pipe being impure (impure pipes run on every change
         * detection, which is wasteful for a table of fifty rows).
         */
        @Pipe({ name: 'relativeTime' })
        export class RelativeTimePipe implements PipeTransform {
          transform(value: string | null | undefined, now: Date): string {
            return formatRelative(value, now);
          }
        }

        /** `{{ value | utcTime }}` -> "2026-10-24 08:00:00 UTC". For tooltips. */
        @Pipe({ name: 'utcTime' })
        export class UtcTimePipe implements PipeTransform {
          transform(value: string | null | undefined): string {
            return formatUtc(value);
          }
        }
```

Pipes are pure by default: Angular re-runs them only when an argument changes. `relativeTime` takes `now` as an argument, so passing a “now” that ticks once a minute keeps “3 minutes ago” current without making the pipe impure (an impure pipe runs on every change detection, for every row).

18.4 The clock

client/sentinel-web/src/app/core/time/clock.ts

```ts
import { DestroyRef, Injectable, inject, signal } from '@angular/core';

/**
 * "Now", as a signal that ticks once a minute. Components that show
 * relative times read it and pass it down; tests replace the whole service
 * with a fixed time.
 */
@Injectable({ providedIn: 'root' })
export class Clock {
  private readonly current = signal(new Date());
  readonly now = this.current.asReadonly();

  constructor() {
    const timer = setInterval(() => this.current.set(new Date()), 60_000);
    inject(DestroyRef).onDestroy(() => clearInterval(timer));
  }
}
```

Tests replace this service with a fixed signal. Nothing in the application calls `new Date()` for “now” except here.

18.5 Pitfalls checklist

- [ ] `new Date('...')` on a string without `Z`: local time.
- [ ] `new Date(2026, 9, 24)`: months are zero-based; this is **October**.
- [ ] `date.getHours()`: the **browser’s** zone, not the user’s.
- [ ] `toLocaleString()` with no options: format varies by browser and machine.
- [ ] A fixed offset for a zone with daylight saving.
- [ ] Comparing formatted strings instead of instants.
- [ ] Sorting ISO strings: fine **only** if they are all UTC with the same format.
- [ ] Zone abbreviations (`WAT`, `GMT+1`) differ between engines: test the time, not the abbreviation.
- [ ] Relative time computed from a `Date.now()` you cannot control in tests.

Chapter 19 · The Dashboard and the Admin Screens

These screens are not on the skill test's critical path, but each one demonstrates a pattern you are likely to need.

19.1 The dashboard: reusing a child with a different parent

`client/sentinel-web/src/app/features/pages/dashboard-page.ts`

```
import { ChangeDetectionStrategy, Component, inject, signal } from '@angular/core';
import { takeUntilDestroyed, toObservable } from '@angular/core/rxjs-interop';
import { Router } from '@angular/router';
import { catchError, filter, of, switchMap } from 'rxjs';
import { AccountSecurityStatus } from '../../core/models/api-models';
import { Session } from '../../core/session/session';
import { Clock } from '../../core/time/clock';
import { SecurityApi } from '../security/security-api';
import { SecurityBanner } from '../security/security-banner';

/**
 * The landing page. It reuses SecurityBanner with a different reaction to
 * (review): here it navigates, on the security page it scrolls. Same child,
 * different parent, different behaviour, and the child never changed.
 */
@Component({
  selector: 'app-dashboard-page',
  imports: [SecurityBanner],
  changeDetection: ChangeDetectionStrategy.OnPush,
  template: `
    @if (session.currentUser(); as user) {
      <h1>Good to see you, {{ user.displayName }}</h1>
      @if (status(); as status) {
        <app-security-banner
          [riskLevel]="status.riskLevel"
          [findingCount]="status.findings.length"
          [assessedAtUtc]="status.assessedAtUtc"
          [now]="clock.now()"
          (review)="openSecurityPage()" />
      }
    } @else {
      <h1>Welcome to Sentinel</h1>
      <p>Choose a user from the switcher in the top right to sign in.</p>
    }
  `,
  styles: `:host { display: grid; gap: 1rem; max-width: 64rem; }`,
})
export class DashboardPage {
  protected readonly session = inject(Session);
  protected readonly clock = inject(Clock);
  private readonly api = inject(SecurityApi);
  private readonly router = inject(Router);

  protected readonly status = signal<AccountSecurityStatus | null>(null);

  constructor() {
    toObservable(this.session.currentUser)
      .pipe(
        filter((user) => user !== null),
        // The banner is a nicety here; if it fails, the dashboard still renders.
        switchMap(() => this.api.getStatus().pipe(catchError(() => of(null)))),
        takeUntilDestroyed(),
      )
      .subscribe((status) => this.status.set(status));
  }
```

```
          protected openSecurityPage(): void {
            this.router.navigateByUrl('/account/security');
          }
        }
```

The same `SecurityBanner` as on the security page. Here `(review)` navigates; there it scrolls. The child is unchanged, which is the payoff of keeping it presentational. Note also the deliberate difference in error handling: on the dashboard the banner is a nicety, so a failure hides it rather than showing an error.

19.2 The admin API service

`client/sentinel-web/src/app/features/admin/admin-api.ts`

```
        import { HttpClient, HttpParams } from '@angular/common/http';
        import { Injectable, inject } from '@angular/core';
        import { Observable } from 'rxjs';
        import { AccountSecurityStatus, PageAccess, PagedResponse, Role, UserSummary } from
'../../core/models/api-models';

        @Injectable({ providedIn: 'root' })
        export class AdminApi {
          private readonly http = inject(HttpClient);

          getUsers(search: string, page = 1, pageSize = 25):
Observable<PagedResponse<UserSummary>> {
            let params = new HttpParams().set('page', page).set('pageSize', pageSize);
            if (search.trim()) {
              params = params.set('search', search.trim());
            }
            return this.http.get<PagedResponse<UserSummary>>('/api/users', { params });
          }

          getRoles(): Observable<Role[]> {
            return this.http.get<Role[]>('/api/roles');
          }

          getPageAccess(userId: string): Observable<PageAccess[]> {
            return this.http.get<PageAccess[]>(`/api/users/${encodeURIComponent(userId)}/page-
access`);
          }

          getUserSecurity(userId: string): Observable<AccountSecurityStatus> {
            return this.http.get<AccountSecurityStatus>
(`/api/users/${encodeURIComponent(userId)}/security`);
          }
        }
```

`HttpParams` builds query strings safely, and every id that goes into a path is passed through `encodeURIComponent`.

19.3 Search as you type

`client/sentinel-web/src/app/features/admin/users-page.ts`

```
        import { ChangeDetectionStrategy, Component, inject, signal } from '@angular/core';
        import { takeUntilDestroyed } from '@angular/core/rxjs-interop';
        import { RouterLink } from '@angular/router';
        import { Subject, catchError, debounceTime, distinctUntilChanged, map, of, startWith,
```

```
switchMap } from 'rxjs';
        import { describeHttpError } from '../../core/http/api-error';
        import { UserSummary } from '../../core/models/api-models';
        import { AdminApi } from './admin-api';

        /**
         * The user register with search-as-you-type. The RxJS pipeline is the
         * classic one and worth knowing by heart:
         *
         *   keystrokes -> debounceTime (wait for a pause)
         *              -> distinctUntilChanged (ignore "ab" -> "abc" -> "ab")
         *              -> switchMap (cancel the previous search when a new one starts)
         */
        @Component({
          selector: 'app-users-page',
          imports: [RouterLink],
          changeDetection: ChangeDetectionStrategy.OnPush,
          template: `
            <h1>Users</h1>
            <label class="search">
              Search
              <input type="search" placeholder="Name, or a full email address"
(input)="search$.next($any($event.target).value)" />
            </label>

            @if (error()) {
              <p class="error" role="alert">{{ error() }}</p>
            } @else {
              <table>
                <thead><tr><th>Name</th><th>Email</th><th>Roles</th><th>Status</th><th></th></tr>
</thead>
                <tbody>
                  @for (user of users(); track user.id) {
                    <tr [class.suspended]="user.status === 'Suspended'">
                      <td>{{ user.displayName }}</td>
                      <td>{{ user.email }}</td>
                      <td>{{ user.roles.join(', ') || '—' }}</td>
                      <td>{{ user.status }}</td>
                      <td><a routerLink="/admin/access-explorer" [queryParams]="{ userId: user.id
}">Explain access</a></td>
                    </tr>
                  } @empty {
                    <tr><td colspan="5" class="muted">No users match.</td></tr>
                  }
                </tbody>
              </table>
              <p class="muted">{{ total() }} {{ total() === 1 ? 'user' : 'users' }}</p>
            }
          `,
          styles: `
            .search { display: flex; gap: .5rem; align-items: center; margin-bottom: .75rem; }
            .search input { width: 20rem; }
            table { width: 100%; border-collapse: collapse; background: var(--surface); }
            th, td { text-align: left; padding: .45rem .6rem; border-bottom: 1px solid var(--
border); }
            tr.suspended td { color: var(--muted); }
            .muted { color: var(--muted); }
            .error { color: var(--danger); }
          `,
        })
        export class UsersPage {
          private readonly api = inject(AdminApi);

          protected readonly search$ = new Subject<string>();
          protected readonly users = signal<UserSummary[]>([]);
          protected readonly total = signal(0);
          protected readonly error = signal<string | null>(null);

          constructor() {
            this.search$
              .pipe(
                debounceTime(250),
```

```
                map((term) => term.trim()),
                distinctUntilChanged(),
                startWith(''),
                switchMap((term) =>
                  this.api.getUsers(term).pipe(
                    map((page) => ({ page, error: null as string | null })),
                    catchError((error: unknown) => of({ page: null, error:
describeHttpError(error) })),
                  ),
                ),
                takeUntilDestroyed(),
              )
              .subscribe(({ page, error }) => {
                this.error.set(error);
                this.users.set(page?.items ?? []);
                this.total.set(page?.totalCount ?? 0);
              });
          }
        }
```

The pipeline in the class comment is worth learning as a unit: `debounceTime` → `distinctUntilChanged` → `switchMap`. Each operator prevents a specific bug: a request per keystroke, a repeated request when the user types and deletes a character, and an old slow response overwriting a newer one.

19.4 The access explorer: the second scenario, made visible

`client/sentinel-web/src/app/features/admin/access-explorer-page.ts`

```
        import { ChangeDetectionStrategy, Component, computed, effect, inject, input, signal }
from '@angular/core';
        import { Router } from '@angular/router';
        import { forkJoin } from 'rxjs';
        import { explainAccess } from '../../core/access/access-reasons';
        import { describeHttpError } from '../../core/http/api-error';
        import { AccountSecurityStatus, PageAccess, UserSummary } from '../../core/models/api-
models';
        import { Clock } from '../../core/time/clock';
        import { SecurityBanner } from '../security/security-banner';
        import { AdminApi } from './admin-api';

        /**
         * "Why can't Ada see the finance report?" answered on one screen: every
         * page, the verdict for one user, and the reason in words. Support staff
         * use this instead of reading role definitions and doing the logic in
         * their heads, which is how wrong answers reach customers.
         */
        @Component({
          selector: 'app-access-explorer-page',
          imports: [SecurityBanner],
          changeDetection: ChangeDetectionStrategy.OnPush,
          template: `
            <h1>Access explorer</h1>

            <label class="picker">
              User
              <select [value]="userId() ?? ''" (change)="choose($any($event.target).value)">
                <option value="">Choose a user…</option>
                @for (user of users(); track user.id) {
                  <option [value]="user.id" [selected]="user.id === userId()">{{ user.displayName
}} ({{ user.roles.join(', ') || 'no roles' }})</option>
                }
              </select>
            </label>

            @if (error()) {
              <p class="error" role="alert">{{ error() }}</p>
```

```
            }

            @if (security(); as status) {
              <app-security-banner
                [riskLevel]="status.riskLevel"
                [findingCount]="0"
                [assessedAtUtc]="status.assessedAtUtc"
                [now]="clock.now()" />
            }

            @if (rows().length > 0) {
              <p class="summary">{{ allowedCount() }} of {{ rows().length }} pages allowed.</p>
              <table>
                <thead><tr><th>Page</th><th>Requires</th><th>Verdict</th><th>Why</th></tr></thead>
                <tbody>
                  @for (row of rows(); track row.path) {
                    <tr [class.denied]="!row.isAllowed">
                      <td [style.padding-left.rem]="0.6 + depth(row.path) * 1.1"><code>{{ row.path
}}</code> {{ row.title }}</td>
                      <td>@if (row.requiredPermission) { <code>{{ row.requiredPermission }}</code>
} @else { <span class="muted">—</span> }</td>
                      <td>{{ row.isAllowed ? 'Allowed' : 'Denied' }}</td>
                      <td><span class="reason">{{ row.reason }}</span> {{ explain(row.reason) }}
</td>
                    </tr>
                  }
                </tbody>
              </table>
            }
          `,
          styles: `
            :host { display: grid; gap: 1rem; }
            .picker { display: flex; gap: .5rem; align-items: center; }
            table { width: 100%; border-collapse: collapse; background: var(--surface); }
            th, td { text-align: left; padding: .4rem .6rem; border-bottom: 1px solid var(--
border); font-size: .9rem; }
            tr.denied td { color: var(--muted); }
            .reason { font-family: var(--mono); font-size: .75rem; padding: .05rem .3rem; border-
radius: 3px; background: var(--hover); }
            .muted { color: var(--muted); }
            .error { color: var(--danger); }
            .summary { margin: 0; }
          `,
        })
        export class AccessExplorerPage {
          private readonly api = inject(AdminApi);
          private readonly router = inject(Router);
          protected readonly clock = inject(Clock);

          /** From the ?userId= query parameter, so a link from the users page lands here pre-
selected. */
          readonly userId = input<string>();

          protected readonly users = signal<UserSummary[]>([]);
          protected readonly rows = signal<PageAccess[]>([]);
          protected readonly security = signal<AccountSecurityStatus | null>(null);
          protected readonly error = signal<string | null>(null);
          protected readonly allowedCount = computed(() => this.rows().filter((r) =>
r.isAllowed).length);
          protected readonly explain = explainAccess;

          constructor() {
            this.api.getUsers('', 1, 100).subscribe({
              next: (page) => this.users.set(page.items),
              error: (error: unknown) => this.error.set(describeHttpError(error)),
            });

            // Runs whenever the userId input changes, including the first time.
            effect(() => {
              const id = this.userId();
              this.rows.set([]);
              this.security.set(null);
```

```
            if (id) {
              this.load(id);
            }
          });
        }

        protected choose(userId: string): void {
          // Put the choice in the URL, so the screen can be bookmarked and shared
          // with a colleague in a ticket. The input above picks it up from there.
          this.router.navigate([], { queryParams: { userId: userId || null } });
        }

        /** "/reports/finance" -> 2: used to indent the table into a tree. */
        protected depth(path: string): number {
          return path === '/' ? 0 : path.split('/').length - 1;
        }

        private load(userId: string): void {
          this.error.set(null);
          forkJoin({ access: this.api.getPageAccess(userId), security:
this.api.getUserSecurity(userId) }).subscribe({
            next: ({ access, security }) => {
              this.rows.set(access);
              this.security.set(security);
            },
            error: (error: unknown) => this.error.set(describeHttpError(error)),
          });
        }
      }
```

This screen answers the ticket “why can’t Ada see the finance report?” in one place: every page, the verdict, and the reason in words. The selected user lives in the URL (`?userId=`), so the screen can be bookmarked or pasted into a ticket. The `effect` reloads whenever that input changes, including the first time.

`forkJoin` runs the two requests in parallel and continues when both have finished; if either fails, the error handler runs once.

client/sentinel-web/src/app/features/admin/roles-page.ts

```
      import { ChangeDetectionStrategy, Component, inject, signal } from '@angular/core';
      import { describeHttpError } from '../../core/http/api-error';
      import { Role } from '../../core/models/api-models';
      import { AdminApi } from './admin-api';

      @Component({
        selector: 'app-roles-page',
        changeDetection: ChangeDetectionStrategy.OnPush,
        template: `
          <h1>Roles</h1>
          @if (error()) {
            <p class="error" role="alert">{{ error() }}</p>
          }
          <table>
            <thead><tr><th>Role</th><th>Grants</th><th>Users</th></tr></thead>
            <tbody>
              @for (role of roles(); track role.id) {
                <tr>
                  <td><strong>{{ role.name }}</strong><br /><span class="muted">{{
role.description }}</span></td>
                  <td>
                    @for (permission of role.permissions; track permission) {
                      <code class="permission" [class.wildcard]="permission.endsWith('*')">{{
permission }}</code>
                    }
                  </td>
                  <td>{{ role.userCount }}</td>
```

```
                </tr>
              }
            </tbody>
          </table>
        `,
        styles: `
          table { width: 100%; border-collapse: collapse; background: var(--surface); }
          th, td { text-align: left; padding: .5rem .6rem; border-bottom: 1px solid var(--
border); vertical-align: top; }
          .permission { display: inline-block; margin: 0 .3rem .3rem 0; padding: .05rem .35rem;
border-radius: 3px; background: var(--hover); }
          .permission.wildcard { background: var(--warning-bg); }
          .muted { color: var(--muted); font-size: .85rem; }
          .error { color: var(--danger); }
        `,
      })
      export class RolesPage {
        protected readonly roles = signal<Role[]>([]);
        protected readonly error = signal<string | null>(null);

        constructor() {
          inject(AdminApi).getRoles().subscribe({
            next: (roles) => this.roles.set(roles),
            error: (error: unknown) => this.error.set(describeHttpError(error)),
          });
        }
      }
```

Chapter 20 · Testing the Angular Application

```
cd client/sentinel-web
npx ng test --watch=false     # 62 tests, about 7 seconds
npx ng test                   # watch mode while you work
```

Angular 22 runs specs with **Vitest** in a jsdom environment. `describe`, `it`, `expect` and `vi` are globals. If your test environment uses Jasmine and Karma instead, the only differences you will meet are `vi.fn()` → `jasmine.createSpy()` and `vi.spyOn` → `spyOn`.

20.1 Pure functions first

The cheapest, fastest tests are of functions with no Angular in them:

`client/sentinel-web/src/app/core/time/timestamps.spec.ts`

```
import { formatInZone, formatRelative, formatUtc, isValidTimeZone, parseApiTimestamp }
from './timestamps';

describe('parseApiTimestamp', () => {
  it('parses UTC with a Z', () => {
    expect(parseApiTimestamp('2026-10-24T08:00:00Z')?.toISOString()).toBe('2026-10-
24T08:00:00.000Z');
  });

  it('parses an explicit offset', () => {
    expect(parseApiTimestamp('2026-10-24T09:00:00+01:00')?.toISOString()).toBe('2026-10-
24T08:00:00.000Z');
  });

  it('parses fractional seconds, which .NET emits', () => {
    expect(parseApiTimestamp('2026-10-24T08:00:00.1234567Z')).not.toBeNull();
  });

  it('refuses a timestamp with no zone instead of guessing local time', () => {
    // This is the string SUP-4172 was made of. new Date() would accept it.
    const error = vi.spyOn(console, 'error').mockImplementation(() => undefined);

    expect(parseApiTimestamp('2026-10-24T08:00:00')).toBeNull();
    expect(error).toHaveBeenCalledOnce();

    error.mockRestore();
  });

  it.each([null, undefined, '', 'yesterday', '24/10/2026 08:00', '2026-13-45T99:99:99Z'])
('returns null for %s', (value) => {
    expect(parseApiTimestamp(value as string | null | undefined)).toBeNull();
  });
});

describe('formatInZone', () => {
  const eightUtc = '2026-10-24T08:00:00Z';

  it('shows the same instant in different zones', () => {
    expect(formatInZone(eightUtc, 'Africa/Lagos', { showZone: false })).toBe('24 Oct 2026,
09:00');
    expect(formatInZone(eightUtc, 'Asia/Singapore', { showZone: false })).toBe('24 Oct
2026, 16:00');
    expect(formatInZone(eightUtc, 'America/New_York', { showZone: false })).toBe('24 Oct
2026, 04:00');
  });
```

```
          it('crosses the date line correctly', () => {
            expect(formatInZone('2026-10-24T23:30:00Z', 'Asia/Singapore', { showZone: false
})).toBe('25 Oct 2026, 07:30');
          });

          it('applies daylight saving for the date being shown, not for today', () => {
            // London is UTC+1 in July and UTC+0 in January.
            expect(formatInZone('2026-07-01T12:00:00Z', 'Europe/London', { showZone: false
})).toBe('1 Jul 2026, 13:00');
            expect(formatInZone('2026-01-01T12:00:00Z', 'Europe/London', { showZone: false
})).toBe('1 Jan 2026, 12:00');
          });

          it('includes a zone name by default', () => {
            // Engines disagree on the abbreviation (WAT vs GMT+1), so only check one is there.
            expect(formatInZone(eightUtc, 'Africa/Lagos')).toMatch(/09:00 \S+/);
          });

          it('falls back to UTC for an unknown zone instead of throwing', () => {
            expect(formatInZone(eightUtc, 'Mars/Olympus_Mons', { showZone: false })).toBe('24 Oct
2026, 08:00');
          });

          it('says so when there is no usable time', () => {
            expect(formatInZone(null, 'Africa/Lagos')).toBe('time unavailable');
          });

          it('supports a short numeric style', () => {
            expect(formatInZone(eightUtc, 'Africa/Lagos', { style: 'short', showZone: false
})).toBe('24/10/2026, 09:00');
          });
        });

        describe('formatUtc', () => {
          it('drops the milliseconds and says UTC', () => {
            expect(formatUtc('2026-10-24T08:00:00.123Z')).toBe('2026-10-24 08:00:00 UTC');
          });
        });

        describe('formatRelative', () => {
          const now = new Date('2026-10-25T12:00:00Z');

          it.each([
            ['2026-10-25T11:59:30Z', 'just now'],
            ['2026-10-25T12:01:00Z', 'just now'], // slightly in the future: clock skew
            ['2026-10-25T11:55:00Z', '5 minutes ago'],
            ['2026-10-25T09:00:00Z', '3 hours ago'],
            ['2026-10-24T10:00:00Z', 'yesterday'],
            ['2026-10-20T12:00:00Z', '5 days ago'],
            ['2026-10-11T12:00:00Z', '2 weeks ago'],
            ['2026-07-25T12:00:00Z', '3 months ago'],
          ])('%s is "%s"', (value, expected) => {
            expect(formatRelative(value, now)).toBe(expected);
          });
        });

        describe('isValidTimeZone', () => {
          it.each([['Africa/Lagos', true], ['UTC', true], ['Lagos', false], ['', false], [null,
false]])('%s -> %s', (zone, expected) => {
            expect(isValidTimeZone(zone)).toBe(expected);
          });
        });
```

`it.each` turns a table of examples into a test per row, exactly like xUnit's `[Theory]`. The relative-time table tests both sides of every unit boundary.

`client/sentinel-web/src/app/features/security/risk.spec.ts`

```
        import { RiskLevel } from '../../core/models/api-models';
        import { findingTitle, presentRisk } from './risk';

        describe('presentRisk', () => {
          it.each<[RiskLevel, string]>([
            ['None', 'ok'],
            ['Low', 'info'],
            ['Medium', 'warning'],
            ['High', 'danger'],
          ])('%s is shown as %s', (level, tone) => {
            expect(presentRisk(level).tone).toBe(tone);
          });

          it('treats an unknown level as a warning, never as all clear', () => {
            expect(presentRisk('Catastrophic' as RiskLevel).tone).toBe('warning');
            expect(presentRisk(undefined).tone).toBe('warning');
          });
        });

        describe('findingTitle', () => {
          it('names every known kind and copes with new ones', () => {
            expect(findingTitle('ImpossibleTravel')).toBe('Sign-ins too far apart to be one
person');
            expect(findingTitle('SomethingNew')).toBe('Unusual activity');
          });
        });
```

20.2 Presentational components: set inputs, read the DOM, catch outputs

client/sentinel-web/src/app/features/security/test-data.ts

```
        import { AccountSecurityStatus, SecurityFinding, SignInAttempt } from
'../../core/models/api-models';

        // Builders for specs. Each takes overrides, so a test states only what
        // matters to it: `aFinding({ severity: 'High' })`.

        export const NOW = new Date('2026-10-25T12:00:00Z');

        export function aFinding(overrides: Partial<SecurityFinding> = {}): SecurityFinding {
          return {
            key: 'new-country:abc',
            kind: 'NewCountry',
            severity: 'Medium',
            occurredAtUtc: '2026-10-25T09:00:00Z',
            summary: 'First sign-in from London, GB.',
            attemptIds: ['a1'],
            ...overrides,
          };
        }

        export function aSignIn(overrides: Partial<SignInAttempt> = {}): SignInAttempt {
          return {
            id: 'a1',
            occurredAtUtc: '2026-10-25T09:00:00Z',
            ipAddress: '81.2.69.17',
            countryCode: 'GB',
            city: 'London',
            device: 'Safari on iPhone',
            succeeded: true,
            failureReason: null,
            isFlagged: false,
            ...overrides,
          };
        }
```

```
        export function aStatus(overrides: Partial<AccountSecurityStatus> = {}):
AccountSecurityStatus {
          return {
            riskLevel: 'Medium',
            isCompromiseSuspected: true,
            assessedAtUtc: '2026-10-25T11:58:00Z',
            timeZoneId: 'Africa/Lagos',
            findings: [aFinding()],
            recentSignIns: [aSignIn({ isFlagged: true })],
            ...overrides,
          };
        }
```

client/sentinel-web/src/app/features/security/security-banner.spec.ts

```
        import { ComponentFixture, TestBed } from '@angular/core/testing';
        import { RiskLevel } from '../../core/models/api-models';
        import { SecurityBanner } from './security-banner';
        import { NOW } from './test-data';

        describe('SecurityBanner', () => {
          let fixture: ComponentFixture<SecurityBanner>;
          let element: HTMLElement;

          // Inputs are set the way a parent template would set them, through the
          // component ref, so the test exercises the real input() declarations.
          function render(riskLevel: RiskLevel, findingCount: number): void {
            fixture.componentRef.setInput('riskLevel', riskLevel);
            fixture.componentRef.setInput('findingCount', findingCount);
            fixture.componentRef.setInput('assessedAtUtc', '2026-10-25T11:55:00Z');
            fixture.componentRef.setInput('now', NOW);
            fixture.detectChanges();
          }

          beforeEach(() => {
            fixture = TestBed.createComponent(SecurityBanner);
            element = fixture.nativeElement as HTMLElement;
          });

          it('says all clear and offers nothing to review when risk is None', () => {
            render('None', 0);

            expect(element.querySelector('h2')?.textContent).toContain('No suspicious activity');
            expect(element.querySelector('button')).toBeNull();
            expect(element.querySelector('section')?.classList).toContain('tone-ok');
          });

          it('tells the user to act when risk is High', () => {
            render('High', 3);

            expect(element.querySelector('h2')?.textContent).toContain('Someone may be using your
account');
            expect(element.querySelector('section')?.classList).toContain('tone-danger');
            expect(element.querySelector('section')?.classList).toContain('banner'); // static
class survives [class]
          });

          it('pluralises the review button', () => {
            render('Medium', 1);
            expect(element.querySelector('button')?.textContent?.trim()).toBe('Review 1 item');

            render('Medium', 2);
            expect(element.querySelector('button')?.textContent?.trim()).toBe('Review 2 items');
          });

          it('shows when the check ran, relative to now', () => {
            render('None', 0);
```

```
            expect(element.querySelector('.assessed')?.textContent).toContain('5 minutes ago');
            expect(element.querySelector('.assessed')?.getAttribute('title')).toBe('2026-10-25
11:55:00 UTC');
          });

          it('emits review when the button is clicked', () => {
            render('High', 2);
            const review = vi.fn();
            fixture.componentInstance.review.subscribe(review);

            element.querySelector<HTMLButtonElement>('button')!.click();

            expect(review).toHaveBeenCalledOnce();
          });
        });
```

`fixture.componentRef.setInput(name, value)` sets inputs the way a parent template would, so the test exercises the real `input()` declarations rather than poking fields. Outputs are observed by subscribing to them.

`client/sentinel-web/src/app/features/security/finding-list.spec.ts`

```
        import { ComponentFixture, TestBed } from '@angular/core/testing';
        import { SecurityFinding } from '../../core/models/api-models';
        import { FindingList } from './finding-list';
        import { NOW, aFinding } from './test-data';

        describe('FindingList', () => {
          let fixture: ComponentFixture<FindingList>;
          let element: HTMLElement;

          function render(findings: SecurityFinding[], pendingKey: string | null = null): void {
            fixture.componentRef.setInput('findings', findings);
            fixture.componentRef.setInput('timeZone', 'Africa/Lagos');
            fixture.componentRef.setInput('now', NOW);
            fixture.componentRef.setInput('pendingKey', pendingKey);
            fixture.detectChanges();
          }

          beforeEach(() => {
            fixture = TestBed.createComponent(FindingList);
            element = fixture.nativeElement as HTMLElement;
          });

          it('shows an empty state', () => {
            render([]);

            expect(element.textContent).toContain('Nothing to review.');
          });

          it('shows each finding with its time in the user zone', () => {
            render([aFinding({ occurredAtUtc: '2026-10-25T09:00:00Z' })]);

            const card = element.querySelector('article')!;
            expect(card.querySelector('h3')?.textContent).toBe('Sign-in from a new country');
            expect(card.querySelector('time')?.textContent).toContain('25 Oct 2026, 10:00'); //
Lagos is UTC+1
            expect(card.querySelector('time')?.getAttribute('datetime')).toBe('2026-10-
25T09:00:00Z');
            expect(card.textContent).toContain('3 hours ago');
          });

          it('emits the key of the finding the user acknowledges', () => {
            render([aFinding({ key: 'first' }), aFinding({ key: 'second' })]);
            const acknowledged: string[] = [];
            fixture.componentInstance.acknowledge.subscribe((key) => acknowledged.push(key));

            element.querySelectorAll<HTMLButtonElement>('article button')[1].click();
```

```
            expect(acknowledged).toEqual(['second']);
          });

          it('disables every button while one acknowledgement is in flight', () => {
            render([aFinding({ key: 'first' }), aFinding({ key: 'second' })], 'first');

            const buttons = [...element.querySelectorAll<HTMLButtonElement>('article button')];
            expect(buttons.every((b) => b.disabled)).toBe(true);
            expect(buttons[0].textContent?.trim()).toBe('Saving…');
            expect(buttons[1].textContent?.trim()).toBe('This was me');
          });
        });
```

20.3 The container against a fake back end

client/sentinel-web/src/app/features/security/account-security-page.spec.ts

```
        import { provideHttpClient } from '@angular/common/http';
        import { HttpTestingController, provideHttpClientTesting } from
'@angular/common/http/testing';
        import { signal } from '@angular/core';
        import { ComponentFixture, TestBed } from '@angular/core/testing';
        import { CurrentUser } from '../../core/models/api-models';
        import { Session } from '../../core/session/session';
        import { Clock } from '../../core/time/clock';
        import { AccountSecurityPage } from './account-security-page';
        import { NOW, aFinding, aStatus } from './test-data';

        /**
         * The container, tested with its real children and a fake back end. These
         * tests read like the user's experience: the page loads, shows a verdict,
         * and reacts to a click. HttpTestingController lets each test decide what
         * the server answers, including failures that are hard to cause for real.
         */
        describe('AccountSecurityPage', () => {
          let fixture: ComponentFixture<AccountSecurityPage>;
          let http: HttpTestingController;
          let element: HTMLElement;
          const currentUser = signal<CurrentUser | null>(null);

          const ada: CurrentUser = { id: 'u1', email: 'ada@sentinel.dev', displayName: 'Ada',
timeZoneId: 'Africa/Lagos', permissions: [] };

          beforeEach(() => {
            currentUser.set(ada);
            TestBed.configureTestingModule({
              providers: [
                provideHttpClient(),
                provideHttpClientTesting(),
                // Only the parts of Session the page uses. A fake this small is a
                // sign the page depends on very little, which is the point.
                { provide: Session, useValue: { currentUser: currentUser.asReadonly() } },
                { provide: Clock, useValue: { now: signal(NOW).asReadonly() } },
              ],
            });

            http = TestBed.inject(HttpTestingController);
            fixture = TestBed.createComponent(AccountSecurityPage);
            element = fixture.nativeElement as HTMLElement;
          });

          afterEach(() => http.verify()); // no request left unanswered or unexpected

          async function settle(): Promise<void> {
            fixture.detectChanges();
            await fixture.whenStable();
```

```
            fixture.detectChanges();
          }

          it('shows loading, then the verdict, the findings and the sign-ins', async () => {
            await settle();
            expect(element.textContent).toContain('Checking your recent activity');

            http.expectOne('/api/me/security').flush(aStatus({ riskLevel: 'High' }));
            await settle();

            expect(element.querySelector('app-security-banner
h2')?.textContent).toContain('Someone may be using your account');
            expect(element.querySelectorAll('app-finding-list article')).toHaveLength(1);
            expect(element.querySelectorAll('app-sign-in-table tbody
tr.flagged')).toHaveLength(1);
          });

          it('acknowledges a finding, then reloads the verdict from the server', async () => {
            await settle();
            http.expectOne('/api/me/security').flush(aStatus({ findings: [aFinding({ key: 'new-
country:abc' })] }));
            await settle();

            element.querySelector<HTMLButtonElement>('app-finding-list button')!.click();
            await settle();

            const post = http.expectOne('/api/me/security/findings/new-
country%3Aabc/acknowledge');
            expect(post.request.method).toBe('POST');
            post.flush(null);
            await settle();

            http.expectOne('/api/me/security').flush(aStatus({ riskLevel: 'None', findings: []
}));
            await settle();

            expect(element.querySelector('app-security-banner h2')?.textContent).toContain('No
suspicious activity');
            expect(element.textContent).toContain('Nothing to review.');
          });

          it('keeps the page and shows the reason when an acknowledgement fails', async () => {
            await settle();
            http.expectOne('/api/me/security').flush(aStatus());
            await settle();

            element.querySelector<HTMLButtonElement>('app-finding-list button')!.click();
            await settle();
            http.expectOne((r) => r.url.endsWith('/acknowledge')).flush(
              { title: 'Finding.NotFound', detail: 'That finding does not exist or no longer
applies.' },
              { status: 404, statusText: 'Not Found' },
            );
            await settle();

            expect(element.querySelector('[role="alert"]')?.textContent).toContain('no longer
applies');
            expect(element.querySelectorAll('app-finding-list article')).toHaveLength(1);
            expect(element.querySelector<HTMLButtonElement>('app-finding-list
button')!.disabled).toBe(false);
          });

          it('offers a retry when the API cannot be reached', async () => {
            await settle();
            http.expectOne('/api/me/security').error(new ProgressEvent('error'), { status: 0 });
            await settle();

            expect(element.querySelector('[role="alert"]')?.textContent).toContain('Cannot reach
the Sentinel API');

            element.querySelector<HTMLButtonElement>('[role="alert"] button')!.click();
            await settle();
```

```
            http.expectOne('/api/me/security').flush(aStatus({ riskLevel: 'None', findings: [] }));
            await settle();

            expect(element.querySelector('app-security-banner')).not.toBeNull();
          });

          it('reloads for the new user when the signed-in user changes, and asks nothing when signed out', async () => {
            await settle();
            http.expectOne('/api/me/security').flush(aStatus());
            await settle();

            currentUser.set(null);
            await settle();

            http.expectNone('/api/me/security');
            expect(element.textContent).toContain("Sign in to see your account's security.");
          });
        });
```

`HttpTestingController` lets each test decide what the server answers, including the failures that are hard to cause on demand: a 404 on acknowledge, a network error. `http.verify()` in `afterEach` fails the test if a request was made and not answered, or answered and not made.

The `Session` fake provides only `currentUser`. A fake that small is a design signal: the page depends on very little of the session, which is what read-only signals were for.

20.4 The guard and the interceptor

`client/sentinel-web/src/app/core/access/page-access.guard.spec.ts`

```
        import { provideHttpClient } from '@angular/common/http';
        import { HttpTestingController, provideHttpClientTesting } from '@angular/common/http/testing';
        import { TestBed } from '@angular/core/testing';
        import { ActivatedRouteSnapshot, Router, RouterStateSnapshot, UrlTree, provideRouter } from '@angular/router';
        import { Observable, firstValueFrom } from 'rxjs';
        import { PageAccess } from '../models/api-models';
        import { pageAccessGuard, pathOnly } from './page-access.guard';

        describe('pageAccessGuard', () => {
          let http: HttpTestingController;
          let router: Router;

          beforeEach(() => {
            TestBed.configureTestingModule({
              providers: [provideRouter([]), provideHttpClient(), provideHttpClientTesting()],
            });
            http = TestBed.inject(HttpTestingController);
            router = TestBed.inject(Router);
          });

          afterEach(() => http.verify());

          function runGuard(url: string): Promise<boolean | UrlTree> {
            const result = TestBed.runInInjectionContext(() =>
              pageAccessGuard({} as ActivatedRouteSnapshot, { url } as RouterStateSnapshot),
            ) as Observable<boolean | UrlTree>;
            return firstValueFrom(result);
          }

          function verdict(overrides: Partial<PageAccess>): PageAccess {
            return { path: '/reports', title: 'Reports', parentPath: '/', requiredPermission:
```

```
'reports.view', isAllowed: true, reason: 'PermissionGranted', ...overrides };
          }

          it('lets the navigation through when the server allows it', async () => {
            const result = runGuard('/reports');

            http.expectOne((r) => r.url === '/api/me/page-access' && r.params.get('path') ===
'/reports').flush(verdict({}));

            expect(await result).toBe(true);
          });

          it('redirects to /forbidden with the reason when the server denies it', async () => {
            const result = runGuard('/admin/users');

            http.expectOne((r) => r.url === '/api/me/page-access').flush(verdict({ path:
'/admin/users', isAllowed: false, reason: 'ParentDenied' }));

            expect(router.serializeUrl((await result) as UrlTree)).toBe('/forbidden?
page=%2Fadmin%2Fusers&reason=ParentDenied');
          });

          it('fails closed when the page is unknown or the API is down', async () => {
            const result = runGuard('/nowhere');

            http.expectOne((r) => r.url === '/api/me/page-access').flush({ title: 'Page.NotFound'
}, { status: 404, statusText: 'Not Found' });

            expect(router.serializeUrl((await result) as UrlTree)).toBe('/forbidden?
page=%2Fnowhere&reason=Unknown');
          });

          it('judges the path, not the query string', async () => {
            const result = runGuard('/reports?tab=2#top');

            http.expectOne((r) => r.params.get('path') === '/reports').flush(verdict({}));

            expect(await result).toBe(true);
          });
        });

        describe('pathOnly', () => {
          it.each([
            ['/reports/finance?tab=2', '/reports/finance'],
            ['/reports#top', '/reports'],
            ['/', '/'],
            ['', '/'],
          ])('%s -> %s', (url, expected) => {
            expect(pathOnly(url)).toBe(expected);
          });
        });
```

A functional guard is tested by calling it inside `TestBed.runInInjectionContext`, so that its `inject()` calls work.

client/sentinel-web/src/app/core/http/demo-user.interceptor.spec.ts

```
        import { HttpClient, provideHttpClient, withInterceptors } from '@angular/common/http';
        import { HttpTestingController, provideHttpClientTesting } from
'@angular/common/http/testing';
        import { signal } from '@angular/core';
        import { TestBed } from '@angular/core/testing';
        import { Session } from '../session/session';
        import { DEMO_USER_HEADER, demoUserInterceptor } from './demo-user.interceptor';

        describe('demoUserInterceptor', () => {
          const email = signal<string | null>(null);
          let http: HttpClient;
```

```
          let backend: HttpTestingController;

          beforeEach(() => {
            TestBed.configureTestingModule({
              providers: [
                provideHttpClient(withInterceptors([demoUserInterceptor])),
                provideHttpClientTesting(),
                { provide: Session, useValue: { signedInEmail: email.asReadonly() } },
              ],
            });
            http = TestBed.inject(HttpClient);
            backend = TestBed.inject(HttpTestingController);
          });

          afterEach(() => backend.verify());

          it('adds the header to API calls when someone is signed in', () => {
            email.set('ada@sentinel.dev');

            http.get('/api/me').subscribe();


expect(backend.expectOne('/api/me').request.headers.get(DEMO_USER_HEADER)).toBe('ada@sentinel.dev'
);
          });

          it('sends nothing when signed out', () => {
            email.set(null);

            http.get('/api/me').subscribe();


expect(backend.expectOne('/api/me').request.headers.has(DEMO_USER_HEADER)).toBe(false);
          });

          it('never leaks the identity to other origins', () => {
            email.set('ada@sentinel.dev');

            http.get('https://example.com/api/thing').subscribe();


expect(backend.expectOne('https://example.com/api/thing').request.headers.has(DEMO_USER_HEADER)).t
oBe(false);
          });
        });
```

20.5 What to test in 25 minutes

In the Angular scenario you will not write 62 tests. Write the two or three that prove the risky parts: the date formatting in a non-UTC zone, the empty state, and the output your parent depends on. A reviewer reading `it('shows the time in the user zone, not the browser zone')` learns more about you than from any amount of coverage.

Part IV · Debugging with Limited Information

Chapter 21 · A Method for Debugging with Limited Information

The first scenario gives you a bug report from customer support and tells you up front that you will not have everything you need. That is not a trick; it is every real bug report. This chapter is a method you can run under a 35-minute timer, without AI, and that leaves a written trail a reviewer can follow.

21.1 The five questions the reviewer is asking

From the prep guide's list, rephrased as what they will look for in your submission:

1. **Gathering:** did you reproduce it, read the relevant code path, and use evidence (logs, data, the debugger) before deciding?
2. **Research:** when you looked something up, was it the right thing, from a good source, and did you cite it?
3. **Documenting:** could a colleague pick up your investigation from what you wrote?
4. **To engineers:** is the root cause precise, the fix justified, and the gap that let it ship named?
5. **To non-engineers:** does the customer-facing note answer *their* question, in *their* words?

21.2 The loop



Read, and separate facts from assumptions

Copy the ticket into your log verbatim. Then make three columns: what you **know** (and how), what you **assume**, what you **don't know**. Support tickets mix observation with interpretation: "the times are wrong" is an observation; "someone is in my account" is an interpretation. Keep them apart, because the interpretation may be a second, separate problem.

Write the questions you would ask

You cannot ask the customer during the test. Write the questions anyway, phrased so support could forward them unchanged:

- What time zone are you in, and what is your computer's clock set to?
- Roughly when did you first notice? Has it always been like this?
- Does it happen for every entry, or only some?
- What exactly did you see? A screenshot, or the text copied?

These show the reviewer you know which facts are missing, and often the act of writing them tells you where to look.

Reproduce before you theorise

A bug you can reproduce is half fixed. A bug you cannot reproduce is one where every fix is a guess. Reproduce at the **lowest level that still shows the problem**: if the API response is wrong, you do not need the browser; if one function's output is wrong, you do not need the API. `curl` and the `.http` file are faster than clicking.

Hypothesise, then try to kill each hypothesis

Write each hypothesis with the evidence for and against it and the *cheapest test that could rule it out*. Run the cheapest first. A hypothesis you cannot test is not yet useful.

Bisect when you know it used to work

If the ticket hints at a recent change ("since Tuesday's deploy"), the history is evidence. `git log --oneline -- <path>` lists changes to the code involved; `git bisect` finds the commit that broke it in a handful of steps:

```
git bisect start
git bisect bad                # this commit is broken
git bisect good <older-sha>   # this one was fine
# build/test, then: git bisect good | git bisect bad ... until it names the commit
git bisect reset
```

Fix the cause, then prove it with a test that fails first

Write the regression test **before** the fix, run it, and watch it fail for the reason you expect. A test that has never failed has not proved anything. Then fix, and watch it pass.

21.3 Tools for gathering evidence in .NET

Tool	Use	Example
The debugger	Inspect values where you suspect the problem	A breakpoint in a handler; watch `occurredAtUtc.Kind`
Structured logs	What happened, in what order, how long	`LoggingBehavior`: `GetAccountSecurityStatusQuery succeeded in 12.3 ms`
EF Core's SQL log	What query actually ran	`"Microsoft.EntityFrameworkCore.Database.Command": "Information"` in appsettings
`ToQueryString()`	The SQL for one LINQ query, without running it	`query.ToQueryString()` in the debugger's Immediate window
`curl` / `.http`	The exact bytes on the wire	`curl -s .../api/me/security -H "X-Demo-User: ..."`
Browser dev tools	What the client received and did	Network tab → Response; Console for logged errors
`git log`, `git blame`, `git bisect`	When and why the code changed	`git log -p -- src/.../SentinelDbContext.cs`
A unit test	A tiny, repeatable reproduction	Call the function with the customer's data

When the scenario's environment has no debugger, `Console.WriteLine` is a perfectly respectable tool. Print the input and output of the function you suspect, with labels, and delete the lines before you submit.

21.4 Research, and citing it

You may search; you must cite. Good habits:

- Search for the **precise** symptom: `DateTimeKind Unspecified System.Text.Json no Z`, not `dates wrong .NET`.
- Prefer **primary sources**: Microsoft Learn, the Angular docs, MDN, the library's GitHub issues. A Stack Overflow answer is a pointer to them, not a substitute.
- Record each source in your log with one line on what it told you. Copied code gets a link in a comment.

```
        // EF Core value converter approach, per Microsoft Learn: "Value Conversions",
        // https://learn.microsoft.com/ef/core/modeling/value-conversions
```

21.5 A 35-minute timetable

Minutes	Activity	Output
0–3	Read; copy the ticket; facts / assumptions / unknowns; questions for the reporter	Log sections 1–3
3–10	Reproduce at the lowest level	Log section 6
10–22	Hypotheses, cheapest test first; find the cause	Log sections 4–5, 7
22–25	Regression test (red), fix (green)	Code
25–35	Engineer write-up; customer reply	Two short documents

At minute 22, if you have not found the cause, stop investigating. Write up what you know, what you ruled out and how, and what you would do next. That is a complete, professional answer to "the bug is not found yet", and it scores on four of the five criteria. A late, unexplained fix scores on one.

21.6 The templates

They are in `docs/templates/`. Copy them at the start of each lab.

`docs/templates/investigation-log.md`

```
# Investigation log — <ticket id>: <one-line symptom in the reporter's words>

> Keep this open in a second window from the first minute. Write as you go,
> not afterwards: timestamps and dead ends are the point. A reviewer reads
> this to see *how* you think, not only what you found.

## 1. What we were told (verbatim)

<paste the ticket. Do not paraphrase; your paraphrase is already a hypothesis.>

## 2. What we know, what we assume, what we don't know

| Known (with source) | Assumed (to verify) | Unknown (to ask or find) |
|---|---|---|
| | | |

## 3. Questions for the reporter

Written so that support can forward them unchanged to a customer.

1.
2.

## 4. Timeline

| Time | Action | Result | Next |
|---|---|---|---|
| 00:00 | Read ticket | | |

## 5. Hypotheses

| # | Hypothesis | Evidence for | Evidence against | Test | Status |
|---|---|---|---|---|---|
| H1 | | | | | open / ruled out / confirmed |

## 6. Reproduction
```

```
Exact steps, data and environment that make the problem appear every time.

## 7. Root cause

One paragraph. Name the file and line. Explain *why* the code did what it did,
not just *what* it did.

## 8. Fix and verification

- Change:
- Regression test:
- How I verified it outside the test:

## 9. Sources consulted

Every page you read, with a link and one line on what it told you.
(Research is welcome; uncited research is not.)

-

## 10. Follow-ups (not done today)

-
```

Chapter 22 · Lab 1 — SUP-4172: “My sign-in history shows the wrong times”

Do the lab before reading past §22.1. Set a 35-minute timer, no AI.

22.1 Setting up

```
git switch lab/SUP-4172-sign-in-times
dotnet ef database drop --project src/Sentinel.Infrastructure -f   # fresh, reproducible data
dotnet run --project src/Sentinel.Api
# second terminal
cd client/sentinel-web; npm start
```

The ticket is `TICKET.md` at the root of the branch:

docs/labs/SUP-4172-ticket.md

```
# SUP-4172 — "My sign-in history shows the wrong times"

**Lab 1 for Chapter 22.** Everything below is exactly what support forwarded.
Nothing has been investigated yet. Start a copy of
`docs/templates/investigation-log.md` before you touch the code.

---

**From:** Kemi (Customer Support, Tier 1)
**Priority:** Normal
**Customer:** Bola Adeyemi (bola.adeyemi@sentinel.dev), Finance Manager

> Hi team, customer says the times on the Account security page are wrong.
> Quote from her email: *"I always sign in at 9 when I get to the office and
> the page says 8:00. It also says I signed in from London at 8:40 on Friday.
> I have never been to London and at 8:40 I was on the bus. Is someone else
> in my account??"* She is quite worried because of the red banner. I tried it as myself and the times look fine to me?? Not sure
> if related but we deployed the "EF Core housekeeping" change on Tuesday.
> Screenshot attached (not attached).
>
> Thanks, Kemi

---

## What you have

- This branch, which reproduces the problem. Run the API and the Angular app
  as in the README and sign in as Bola.
- `git log` — the history is real; read it like an investigator.
- No access to the customer, but you can write the questions you would send.

## What a good answer contains

1. An investigation log: hypotheses, how each was tested, sources cited.
2. A fix with a regression test that fails before and passes after.
3. A write-up for engineers (`docs/templates/engineering-write-up.md`).
4. A reply Kemi can send to Bola (`docs/templates/customer-update.md`).

Time box: 35 minutes, the length of the real scenario. Then read Chapter 22.
```

Stop here, start your log, and work the problem. Then read on and compare.

22.2 Reading the ticket

Known	Assumed	Unknown
Bola says her 09:00 sign-in shows as 08:00	That she signs in at 09:00 Lagos time	Bola's computer's time zone
The page shows a London sign-in she denies	That the two observations have the same cause	Whether she was in London
Kemi's own times "look fine"	That Kemi's account has the same kind of data	Kemi's time zone (London? UTC?)
An "EF Core housekeeping" change went out on Tuesday	That it is related	What that change touched

Two observations are bundled into one ticket, and they must be separated:

1. **The times are an hour early.** A display problem.
2. **A sign-in from London she did not make.** Possibly a real compromise, possibly a symptom of (1). Do not dismiss it: if it is real, it is the more urgent of the two.

Questions for Bola, via Kemi:

1. What time zone is your computer set to? (Settings → Time & language.)
2. Is it every sign-in that looks an hour early, or only some?
3. Were you anywhere outside Nigeria last Friday, or using a VPN?

The fact that "Kemi's times look fine" is the most useful sentence in the ticket. An off-by-one-hour error that appears for one person and not another smells of time zones, and Lagos is UTC+1.

22.3 Reproducing

Sign in as Bola. The page shows her ordinary Lagos sign-in at **08:00**. The seed data puts it at 09:00 Lagos time, which is 08:00 UTC. So the page is showing the UTC wall-clock time and labelling it as Lagos.

Go one level down. Is the server wrong, or the browser?

```
curl -s http://localhost:5080/api/me/security -H "X-Demo-User: bola.adeyemi@sentinel.dev"
```

```
"assessedAtUtc":"2026-10-25T16:35:41.5652577Z",
...
"occurredAtUtc":"2026-10-23T08:00:00",
```

There it is, and it is worth slowing down to see it: `assessedAtUtc` ends in `Z`, and `occurredAtUtc` does **not**. Both are `DateTime` properties on the same DTO. One has a zone designator and one does not.

22.4 Hypotheses

#	Hypothesis	Cheapest test	Result
H1	The seed data is wrong	Query the database directly	Ruled out: `SignInAttempts.OccurredAtUtc` is `08:00` (UTC, correct)
H2	The client formats in the wrong zone	Call `formatInZone('2026-10-23T08:00:00Z', 'Africa/Lagos')`	Ruled out: returns 09:00
H3	The client parses a zone-less string as local time	`new Date('2026-10-23T08:00:00').toISOString()` in a Lagos browser	**Confirmed**: `07:00:00.000Z`, one hour early
H4	The server sends some timestamps without `Z`	Compare the two fields in the raw JSON	**Confirmed**: only values read from the database lack `Z`

H3 explains the symptom; H4 explains H3. Why do values from the database lack the `Z` while `assessedAtUtc` (from `IClock`) has it?

`System.Text.Json` writes a `DateTime` according to its `Kind`: `Utc` gets a `Z`, `Local` gets an offset, and `Unspecified` gets nothing. A value created by `DateTime.UtcNow` has `Kind = Utc`. A value read from a SQL Server `datetime2` column has `Kind = Unspecified`, because the column has no notion of a zone. Something used to stamp `Utc` on values read from the database, and now does not.

22.5 Finding the change

The ticket mentions an "EF Core housekeeping" deploy. The history agrees:

```
git log --oneline -5
git show HEAD~1 --stat
git show HEAD~1 -- src/Sentinel.Infrastructure/Persistence/SentinelDbContext.cs
```

```
-    protected override void ConfigureConventions(ModelConfigurationBuilder
configurationBuilder)
-    {
-        configurationBuilder.Properties<DateTime>().HaveConversion<UtcDateTimeConverter>
();
-    }
```

The convention looked redundant ("datetime2 already round-trips the value") and it was removed. It round-trips the *value*; it never round-tripped the *kind*. `UtcDateTimeConverter.cs` is still in the repository, now referenced by nothing, which is itself a clue worth noticing in any investigation: unused code is often the ghost of a removed fix.

The same commit simplified the client's parser to `new Date(value)`, which is why the server's change became a visible symptom instead of an error.

Sources consulted (cite these in your log):

- Microsoft Learn, *DateTime.Kind* and *DateTime and DateTimeOffset in System.Text.Json*: serialisation depends on `Kind`.

- Microsoft Learn, *EF Core Value Conversions*: `ConfigureConventions` with `HaveConversion` applies a converter to every property of a type.
- MDN, *Date.parse()*: date-time strings without an offset are interpreted as local time.

22.6 The regression tests, then the fix

Write the tests first and watch them fail on the lab branch. At the wire:

```
        [Fact]
        public async Task Every_timestamp_on_the_wire_is_utc_with_a_z()
        {
            var json = await factory.ClientFor(Dana).GetStringAsync("/api/me/security");

            var timestamps = TimestampProperty().Matches(json).Select(m =>
m.Groups["value"].Value).ToList();

            Assert.NotEmpty(timestamps);
            Assert.All(timestamps, t => Assert.EndsWith("Z", t));
        }
```

And at the persistence boundary, closer to the cause:

```
        [Fact]
        public async Task Sign_in_times_come_back_from_the_database_as_utc()
        {
            await using var host = await TestHost.CreateAsync(seed: true);

            var status = (await host.Send(new GetAccountSecurityStatusQuery(await
host.UserIdAsync("ada.okafor@sentinel.dev")))).Value;

            Assert.All(status.RecentSignIns, s => Assert.Equal(DateTimeKind.Utc,
s.OccurredAtUtc.Kind));
        }
```

Both fail. Restore the convention (the fix, as it stands on `main`):

`src/Sentinel.Infrastructure/Persistence/SentinelDbContext.cs`

```
        using Microsoft.EntityFrameworkCore;
        using Sentinel.Application.Abstractions.Ports;
        using Sentinel.Domain.Pages;
        using Sentinel.Domain.Roles;
        using Sentinel.Domain.Security;
        using Sentinel.Domain.Users;
        using Sentinel.Infrastructure.Persistence.Converters;

        namespace Sentinel.Infrastructure.Persistence;

        public sealed class SentinelDbContext(DbContextOptions<SentinelDbContext> options)
            : DbContext(options), IApplicationDbContext
        {
            public DbSet<User> Users => Set<User>();
            public DbSet<Role> Roles => Set<Role>();
            public DbSet<Page> Pages => Set<Page>();
            public DbSet<SignInAttempt> SignInAttempts => Set<SignInAttempt>();
            public DbSet<FindingAcknowledgement> FindingAcknowledgements =>
Set<FindingAcknowledgement>();

            protected override void OnModelCreating(ModelBuilder modelBuilder) =>
                modelBuilder.ApplyConfigurationsFromAssembly(typeof(SentinelDbContext).Assembly);

            protected override void ConfigureConventions(ModelConfigurationBuilder
configurationBuilder)
```

```
            {
                // Every DateTime in this model is UTC. SQL Server's datetime2 has no
                // notion of a zone, so without this line every value read back has
                // DateTimeKind.Unspecified, serialises without its 'Z', and is then
                // read by every browser on earth as local time. Chapter 22 is the
                // post-mortem of deleting this line.
                configurationBuilder.Properties<DateTime>().HaveConversion<UtcDateTimeConverter>
();
            }
        }
```

Both pass. Then **defence in depth** on the client: a browser that receives a zone-less timestamp should say so rather than guess. That is the strict `parseApiTimestamp` from Chapter 18, with its own test:

```
        it('refuses a timestamp with no zone instead of guessing local time', () => {
          const error = vi.spyOn(console, 'error').mockImplementation(() => undefined);

          expect(parseApiTimestamp('2026-10-24T08:00:00')).toBeNull();
          expect(error).toHaveBeenCalledOnce();

          error.mockRestore();
        });
```

Is the client change part of the fix or scope creep? Argue it in your write-up either way; what matters is that you made the choice on purpose. The argument for it: the server bug was invisible for days because the client silently "handled" bad data. A client that refuses bad data turns the next such bug into an error on first use.

22.7 And the London sign-in?

With the times fixed, Bola's history shows Lagos at 09:00 and London at 09:40, both successful, from a Windows laptop and an iPhone respectively. That is a genuine *impossible travel* finding. The display bug did not create it; it only made it look stranger. Your reply to support must say both things clearly: *the times were wrong, we've fixed that; separately, the London sign-in is real and Bola should change her password now.* Merging them into "it was a bug, all fine" would be the most harmful possible outcome of this ticket.

22.8 A follow-up the investigation uncovered

While reproducing, `Sentinel.http` showed that `POST /api/sign-ins` with a zone-less `occurredAt` is accepted and interpreted as **server-local** time. On a server in Lagos, `09:00` becomes `08:00Z`; on a server in UTC, it stays `09:00Z`. The contract says a zone is required. Record it as a follow-up (reject zone-less timestamps with a 400, via a custom `JsonConverter<DateTimeOffset>`), not as part of today's fix. Knowing what *not* to fix in an incident is part of the skill.

22.9 The write-ups

The engineer write-up and the customer reply for this lab are in Chapter 24, as worked examples.

Chapter 23 · Lab 2 — SUP-4219: “Security page says all clear, but I got a lockout email”

Do the lab first. 35 minutes, no AI.

```
git switch lab/SUP-4219-all-clear
dotnet ef database drop --project src/Sentinel.Infrastructure -f
dotnet run --project src/Sentinel.Api
```

`docs/labs/SUP-4219-ticket.md`

```
# SUP-4219 — "Security page says all clear, but I got a lockout email"

**Lab 2 for Chapter 23.** Exactly what support forwarded; nothing has been
investigated. Start a copy of `docs/templates/investigation-log.md` first.

---

**From:** Ifeanyi (Customer Support, Tier 2)
**Priority:** High
**Customer:** Tunde Bakare (tunde.bakare@sentinel.dev), Operations

> Tunde got the "unusual sign-in activity" email from the identity provider
> yesterday evening and someone in his team says they saw a login from
> Amsterdam. But when he opens Account security it's green, "No suspicious
> activity". I checked two other users who were in the same phishing
> campaign (Chen and Bola) and theirs show red/amber properly, so the
> detector seems to work?
>
> He uses the same account for his integration scripts, if that matters.
> Can someone look urgently, we don't know if the account is compromised.

---

## What you have

- This branch, which reproduces the problem (sign in as Tunde).
- The API, the database, and the logs (`LoggingBehavior` writes one line per request).
- `src/Sentinel.Api/Sentinel.http` for poking the API directly.

## What a good answer contains

1. An investigation log with hypotheses ruled in and out, and sources cited.
2. The fix, with a regression test that fails on this branch and passes after.
3. An answer to the question support actually asked: *is Tunde's account compromised?*
4. A write-up for engineers and a message support can send to Tunde.

Time box: 35 minutes. Then read Chapter 23.
```

23.1 Reading the ticket

Known	Assumed	Unknown
The identity provider emailed Tunde about unusual activity	That the email was about real activity	When exactly
A colleague "saw a login from Amsterdam"	That it is in Sentinel's data	Where they saw it
Tunde's page is green	That green means "no findings"	—
Chen's and Bola's pages work	That the detector is fine	What is different about Tunde
Tunde runs integration scripts on his account	That it does not matter	—

The support agent has done something valuable: **a comparison**. Two other users in the same campaign show findings, so "the detector is broken" is unlikely. The question becomes *what is different about Tunde?* and the ticket answers it in passing: integration scripts.

Support's real question is not "why is the page green?" but **"is Tunde's account compromised?"** Answer that first, whatever the code does.

23.2 Reproducing, and the first clue

```
curl -s http://localhost:5080/api/me/security -H "X-Demo-User: tunde.bakare@sentinel.dev"
```

```
{"riskLevel":"None","isCompromiseSuspected":false, ... "recentSignIns":
[{"occurredAtUtc":"2026-10-18T23:37:33Z", ...
```

The risk is `None`, as reported. But look at `recentSignIns[0]`: the "most recent" sign-in is **a week old**, and Tunde's scripts sign in every four hours. The page is not looking at recent history at all.

Check the data directly, to separate "the data is missing" from "the code is not reading it":

```
SELECT COUNT(*), MIN(OccurredAtUtc), MAX(OccurredAtUtc)
FROM SignInAttempts a JOIN Users u ON u.Id = a.UserId
WHERE u.Email = 'tunde.bakare@sentinel.dev';
-- about 550 rows, from 90 days ago to a few hours ago

SELECT OccurredAtUtc, Succeeded, IpAddress FROM SignInAttempts a JOIN Users u ON u.Id =
a.UserId
WHERE u.Email = 'tunde.bakare@sentinel.dev' AND a.IpAddress LIKE '145.53.%' ORDER BY
OccurredAtUtc;
-- 7 failures and 1 success from Amsterdam, yesterday evening
```

So the attack **is** in the database. Answer for support, already: *yes, treat Tunde's account as compromised*, a successful sign-in from Amsterdam followed seven failures. That message can go out now, before the bug is fixed.

23.3 Hypotheses

#	Hypothesis	Test	Result
H1	The detector misses this pattern	Unit test: Tunde-like history into `CompromiseDetector.Assess`	Ruled out: High
H2	Findings are acknowledged	`SELECT * FROM FindingAcknowledgements` for Tunde	Ruled out: none
H3	The query loads the wrong attempts	Log the min/max of what `AccountAssessor` loads	**Confirmed**: oldest 500 of ~550

The difference between Tunde and the others: **about 550 attempts in 90 days**. Chen and Bola have about 45. `AccountAssessor` loads at most `MaxHistory = 500`.

23.4 The cause

```
        var history = await db.SignInAttempts
            .AsNoTracking()
            .Where(a => a.UserId == userId && a.OccurredAtUtc >= since)
            .OrderBy(a => a.OccurredAtUtc)      // oldest first...
            .Take(MaxHistory)                   // ...then keep 500: the NEWEST are dropped
            .ToListAsync(cancellationToken);
```

`OrderBy` then `Take` keeps the *oldest* 500. For a user with fewer than 500 attempts the order is irrelevant, which is why every other account worked and why no test failed. The commit that introduced it had a sensible-sounding reason ("the detector walks history oldest-first, so let the database sort it"). The detector does want chronological order, and it sorts its input itself (Chapter 8 said so). The query's order matters for one thing only: **which rows survive the `Take`**.

You can confirm without a debugger:

```
        Console.WriteLine(query.ToQueryString());
        // ... ORDER BY [s].[OccurredAtUtc] OFFSET 0 ROWS FETCH NEXT @__p_1 ROWS ONLY
```

23.5 The regression test, then the fix

```
        [Fact]
        public async Task When_history_exceeds_the_cap_the_newest_attempts_are_assessed()
        {
            await using var host = await TestHost.CreateAsync(seed: true);
            var tunde = await host.UserIdAsync("tunde.bakare@sentinel.dev");
            var total = await host.Query(db => db.SignInAttempts.CountAsync(a => a.UserId ==
tunde));
            Assert.True(total > AccountAssessor.MaxHistory, "The seed data must exceed the cap for
this test to mean anything.");

            var status = (await host.Send(new GetAccountSecurityStatusQuery(tunde))).Value;

            Assert.Contains(status.Findings, f => f.Kind == "BruteForce" && f.Severity == "High");
        }
```

Note the guard assertion: a test about a cap is meaningless if the data does not exceed the cap, and a future change to the seed data could make it pass vacuously. It fails on the lab branch. The fix is one word, and the comment that now sits above it on `main`:

`src/Sentinel.Application/Security/AccountAssessor.cs (lines 40–48)`

```
            // Newest first, THEN take: when the ceiling bites, what we drop must
            // be the oldest baseline, never the most recent evidence. The order
            // of these two calls is the whole of Chapter 23.
            var history = await db.SignInAttempts
                .AsNoTracking()
                .Where(a => a.UserId == userId && a.OccurredAtUtc >= since)
                .OrderByDescending(a => a.OccurredAtUtc)
                .Take(MaxHistory)
                .ToListAsync(cancellationToken);
```

23.6 What made this bug possible

- **A truncation without a test at the limit.** Every cap (`Take`, page size, `MaxHistory`) is a boundary, and boundaries get tests on both sides.
- **A comment that described the wrong concern.** The order was justified by the detector's needs, which the detector already met itself.
- **Seed data that did not exercise the cap.** Tunde exists *because* of this incident; heavy users are real users.

23.7 Follow-ups

- Should a user with more than 500 attempts in 90 days be assessed on 500? Dropping the oldest baseline can make an old country look "new". Options: aggregate the baseline in SQL (distinct countries and devices) and load only the last 30 days in full. Record it; do not do it in the incident.
- Alert on accounts whose sign-in rate looks automated: they are the ones most likely to have shared credentials.

Chapter 24 · Writing It Up for Engineers and for Everyone Else

Three of the five criteria in the debugging scenario are about writing. This chapter shows what good looks like, using both labs, and gives you the habits that produce it in ten minutes.

24.1 Two audiences, two questions

	Engineers	**Non-engineers**
Their question	*What broke, why, and will it happen again?*	*Am I affected, am I safe, what do I do?*
Lead with	The cause, in one sentence	The answer to their worry
Vocabulary	Precise: file, line, type, `Kind`	Plain: "times were shown an hour early"
Length	As long as the cause needs	As short as possible
Must include	Why it was not caught; follow-ups	What to do, if anything; how to report it again
Must not include	Blame	Jargon, stack traces, speculation

24.2 Worked example: SUP-4172 for engineers

> **SUP-4172 — Sign-in times from the database serialised without a zone and displayed one UTC offset early**
>
> **Audience:** engineering · **Status:** fixed on `main` · **Severity:** S3 (display; no data corrupted)
>
> **Summary.** Commit "EF Core housekeeping" removed the `ConfigureConventions` override that applied `UtcDateTimeConverter` to every `DateTime`. Values read from `datetime2` then had `DateTimeKind.Unspecified`, `System.Text.Json` serialised them without `Z`, and browsers parsed them as local time. Users east of UTC saw times early by their offset (Lagos: one hour); users west of UTC saw them late; users in UTC saw no change, which is why support could not reproduce it.
>
> **Impact.** Every timestamp read from the database, on every screen, since Tuesday's deploy. Stored data is correct; only display was wrong. Values created in memory (`assessedAtUtc`) were unaffected, which made the bug look intermittent.
>
> **Root cause.** `SentinelDbContext.ConfigureConventions` was deleted as apparently redundant. `datetime2` round-trips the value but not the kind. The client's `new Date(value)` silently interpreted the zone-less strings.
>
> **Why it was not caught.** No test asserted the wire format of timestamps or the `Kind` of values read back; the client accepted zone-less input.
>
> **Fix.** Restored the convention. Added `Every_timestamp_on_the_wire_is_utc_with_a_z` (API) and `Sign_in_times_come_back_from_the_database_as_utc` (persistence), both red before and green after. Hardened `parseApiTimestamp` to reject timestamps without a designator (defence in depth: the next such bug fails loudly).

> **Separately:** the London sign-in on Bola Adeyemi's account is a genuine impossible-travel finding and has been passed to security.
>
> **Follow-ups.** Reject zone-less `occurredAt` on `POST /api/sign-ins` (today it is read as server-local time) · add a comment to the convention so it is not "tidied" again.
>
> **References.** Microsoft Learn: *Value Conversions*; *DateTime.Kind*; *System.Text.Json DateTime and DateTimeOffset support*. MDN: *Date.parse()*.

Notice: the summary is three sentences and contains the whole story. The impact section says what was *not* affected. "Why it was not caught" describes the system, not the person who made the commit.

24.3 Worked example: SUP-4172 for the customer

> Hi Bola,
>
> **What happened.** A change we made on Tuesday caused the times on your Account security page to show one hour early. Your 9:00 sign-ins were displayed as 8:00. That is fixed: the times now show correctly in Lagos time.
>
> **Is your account safe?** The time problem did not affect your account's security. **However, the sign-in from London last Friday is real**: someone signed in to your account from London forty minutes after you signed in from Lagos. Please change your password now, and then choose *This wasn't me* on that entry so our security team can follow up.
>
> **If you see anything else unusual,** reply to this email with a screenshot and the time you saw it.
>
> Thank you for reporting this: the detail that your sign-in "always" happens at 9 is what let us find the problem quickly.
>
> Kemi, Customer Support

What makes this good: it answers her actual fear first ("is someone in my account?"), does not hide the bad news behind the good, gives one clear action, and thanks her for the specific detail that helped.

24.4 Worked example: SUP-4219, both audiences

For support (sent first, before the fix):

> Ifeanyi, short answer: **yes, treat Tunde's account as compromised.** Our data shows seven failed sign-ins from Amsterdam yesterday at about 21:05 UTC, followed by a successful one. Please ask him to change his password and to rotate the credentials his integration scripts use. The green status on his security page is a bug on our side, which we are fixing now; it affects only accounts with very many sign-ins.

For engineers:

> **SUP-4219 — Assessment loaded the oldest 500 sign-ins instead of the newest**
>
> **Summary.** `AccountAssessor` ordered history ascending before `Take(500)`, so for users with more than 500 attempts in 90 days the most recent attempts were never assessed. A real brute-force-then-success on Tunde Bakare's account showed as "No suspicious activity".
>
> **Impact.** Users with more than 500 sign-ins in 90 days, i.e. accounts used by automation. For them, findings from roughly the last week were invisible.

> **Root cause.** Commit "Let the database sort sign-in history chronologically" changed `OrderByDescending` to `OrderBy`, reasoning that the detector wants oldest-first. The detector sorts its input itself; the query order only decides which rows survive `Take`.
>
> **Why it was not caught.** No test exercised the cap; seed users had fewer than 500 attempts.
>
> **Fix.** Restored `OrderByDescending` with a comment explaining why the order matters. Added `When_history_exceeds_the_cap_the_newest_attempts_are_assessed` with a guard assertion that the seed data exceeds the cap. Added a heavy-usage user to the seed data.
>
> **Follow-ups.** Compute the baseline (seen countries/devices) in SQL so the cap never drops baseline either · flag automated sign-in patterns.

24.5 Habits that make this fast

- **Write the log as you go**; the write-up is then mostly copying.
- **Summary first**, in three sentences: what broke, why, what changed.
- **Say what you don't know.** "We have not checked other environments" is professional; silence is not.
- **Name the gap, not the person.** "No test asserted…" rather than "X forgot…".
- **For customers: answer the fear, then the facts, then the action.**
- **Read the customer reply aloud.** If a sentence needs an engineer to explain it, rewrite it.

24.6 The templates

`docs/templates/engineering-write-up.md`

```
# <ticket id> — <precise technical title>

**Audience:** engineers. **Status:** fixed in <commit/branch> · **Severity:** <S1–S4>

## Summary

Three sentences: what broke, why, what we changed.

## Impact

Who was affected, since when, and how we know (logs, queries, reports).
Say plainly what we do *not* know.

## Root cause

The chain of events, from the code that was wrong to the symptom the user
saw. Include the line(s) and, where it helps, a minimal snippet.

## Why it was not caught

The missing test, the misleading name, the assumption that did not hold.
Blameless: describe the system, not the person.

## The fix

What changed and why this change rather than the alternatives you
considered (list them in one line each).

## Verification

Tests added, manual checks performed, data checked after deploy.

## Follow-ups
```

```
| Action | Owner | Ticket |
|---|---|---|
| | | |

## References

Links to documentation, issues and articles used, one line each on why.
```

docs/templates/customer-update.md

```
# Update for <customer / support agent> — <ticket id>

> **Audience:** non-engineers. No class names, no stack traces, no jargon.
> Read it aloud: if a sentence needs an engineer to explain it, rewrite it.

Hi <name>,

**What happened.** <One or two sentences in the customer's terms.
"The times on your sign-in history were shown an hour early.">

**Were you affected, and is your account safe?** <Answer the question they
are actually worried about, first and directly.>

**What we've done.** <The fix, in outcome terms. "Times now show correctly
in your own time zone.">

**Anything you need to do.** <Usually "Nothing." If there is something,
make it a numbered list of steps.>

**If you see it again.** <What to send us: a screenshot, the time it
happened and your time zone.>

Thanks for reporting it. Your report included <the detail that helped>,
which is what let us find it quickly.

<signature>
```

Part V · Rehearsal

Chapter 25 · Katas and Timed Drills

Reading about a timed test does not prepare you for one. This chapter is a practice schedule: four exercises, each against the clock, each scored with a rubric built from the test's own criteria.

25.1 The exercises

#	Exercise	Time	Where	Skill
1	Page access kata	40 min	`katas/PageAccessKata`	Scenario 2
2	Access log kata	25 min	`katas/AccessLogKata`	Fundamentals: parsing, dictionaries, edge cases
3	Lab 1 and Lab 2	35 min each	`lab/*` branches	Scenario 1
4	Security page drill	25 min	`drill/angular-security-start`	Scenario 3

Do each at least twice, a few days apart. The second attempt is where the habits form.

25.2 Kata 1: which pages may each user open?

`katas/PageAccessKata/README.md`

```
# Kata 1 — Which pages may each user open? (40 minutes)

A practice problem in the *shape* of the skill test's second scenario. It is
not the test's problem (that is confidential and unknown to this book); it
exercises the same muscles: modelling, iteration, dictionaries, edge cases,
naming, tests and comments for a reviewer.

## The brief

You are given users and pages.

- A **user** has a name and a list of role names.
- A **page** has a path (`/reports/finance`), an optional parent path, and a
  list of role names that may open it.

Return, for every user, the paths of the pages they may open, sorted
alphabetically.

Rules:

1. A page with **no required roles** may be opened by any user.
2. A page with required roles may be opened by a user holding **any one** of them.
3. A user may only open a page if they may also open its **parent** (and so on, up to the
root).
4. The role `admin` may open every page that exists.
5. Role names are **case-insensitive**. Paths are compared case-insensitively and
   without a trailing slash; return them as given in the page list.

Edge cases the reviewer will look for (decide, implement, and comment):

- a page whose parent does not exist;
- two pages with the same path;
- a user with no roles, or a `null` role list;
```

```
        - a parent cycle (`/a` → `/b` → `/a`);
        - an empty list of users or pages.

        ## Working it

        ```powershell
        # Your attempt: implement Starter/StarterPageAccessChecker.cs, then
        dotnet test katas/Katas.slnx --filter "Category=Starter"

        # The model answer's tests (always green):
        dotnet test katas/Katas.slnx --filter "Category!=Starter"
        ```

        Set a 40-minute timer. Write a comment at the top of your class listing the
        decisions you made about each edge case — that comment is worth as much as
        the code. Then compare with `Solution/SolutionPageAccessChecker.cs` and
        Chapter 25.
```

`katas/PageAccessKata/Model.cs`

```
        namespace PageAccessKata;

        public sealed record KataUser(string Name, IReadOnlyList<string>? Roles);

        public sealed record KataPage(string Path, string? ParentPath, IReadOnlyList<string>?
RequiredRoles);

        public interface IPageAccessChecker
        {
            /// <summary>User name -> accessible page paths, sorted alphabetically.</summary>
            IReadOnlyDictionary<string, IReadOnlyList<string>> AccessiblePagesByUser(
                IEnumerable<KataUser> users,
                IEnumerable<KataPage> pages);
        }
```

The tests, shared by your starter and the model answer:

`katas/PageAccessKata.Tests/PageAccessCheckerTests.cs`

```
        using PageAccessKata.Solution;
        using PageAccessKata.Starter;

        namespace PageAccessKata.Tests;

        /// <summary>
        /// One set of tests, run against two implementations. Your starter must
        /// pass exactly what the model answer passes.
        /// </summary>
        public abstract class PageAccessCheckerTests
        {
            protected abstract IPageAccessChecker CreateChecker();

            private static readonly KataPage[] Site =
            [
                new("/", null, []),
                new("/reports", "/", ["analyst", "manager"]),
                new("/reports/finance", "/reports", ["manager"]),
                new("/admin", "/", ["it"]),
                new("/help", null, null),
            ];

            private IReadOnlyList<string> PagesFor(KataUser user, IEnumerable<KataPage>? pages =
null) =>
                CreateChecker().AccessiblePagesByUser([user], pages ?? Site)[user.Name];

            [Fact]
```

```
            public void Pages_without_required_roles_are_open_to_everyone()
            {
                Assert.Equal(["/", "/help"], PagesFor(new KataUser("guest", [])));
            }

            [Fact]
            public void Any_one_required_role_is_enough()
            {
                Assert.Equal(["/", "/help", "/reports"], PagesFor(new KataUser("ana",
["analyst"])));
            }

            [Fact]
            public void A_child_needs_its_parent()
            {
                var pages = new KataPage[] { new("/", null, []), new("/reports", "/",
["analyst"]), new("/reports/finance", "/reports", ["manager"]) };

                // manager without analyst: finance is allowed by role but sits behind /reports.
                Assert.Equal(["/"], PagesFor(new KataUser("mo", ["manager"]), pages));
            }

            [Fact]
            public void Admin_sees_every_page()
            {
                Assert.Equal(["/", "/admin", "/help", "/reports", "/reports/finance"],
PagesFor(new KataUser("root", ["admin"])));
            }

            [Fact]
            public void Role_names_and_paths_ignore_case_and_trailing_slashes()
            {
                var pages = new KataPage[] { new("/", null, []), new("/Reports/", "/",
["Analyst"]), new("/reports/finance", "/REPORTS", [" analyst "]) };

                Assert.Equal(["/", "/Reports/", "/reports/finance"], PagesFor(new KataUser("ana",
["ANALYST"]), pages));
            }

            [Fact]
            public void Null_role_list_is_treated_as_no_roles()
            {
                Assert.Equal(["/", "/help"], PagesFor(new KataUser("nobody", null)));
            }

            [Fact]
            public void Missing_parent_denies_the_page_even_to_admin()
            {
                var pages = new KataPage[] { new("/orphan", "/gone", []) };

                Assert.Empty(PagesFor(new KataUser("root", ["admin"]), pages));
            }

            [Fact]
            public void A_parent_cycle_denies_every_page_on_it()
            {
                var pages = new KataPage[] { new("/a", "/b", []), new("/b", "/a", []), new("/c",
"/a", []) };

                Assert.Empty(PagesFor(new KataUser("root", ["admin"]), pages));
            }

            [Fact]
            public void The_first_definition_of_a_duplicated_path_wins()
            {
                var pages = new KataPage[] { new("/x", null, []), new("/X", null, ["it"]) };

                Assert.Equal(["/x"], PagesFor(new KataUser("guest", []), pages));
            }

            [Fact]
            public void Every_user_gets_an_entry_even_if_it_is_empty()
```

```
            {
                var result = CreateChecker().AccessiblePagesByUser([new KataUser("a", []), new
KataUser("b", ["it"])], []);

                Assert.Equal(2, result.Count);
                Assert.All(result.Values, Assert.Empty);
            }

            [Fact]
            public void No_users_means_an_empty_result()
            {
                Assert.Empty(CreateChecker().AccessiblePagesByUser([], Site));
            }
        }

        public sealed class SolutionTests : PageAccessCheckerTests
        {
            protected override IPageAccessChecker CreateChecker() => new
SolutionPageAccessChecker();
        }

        [Trait("Category", "Starter")]
        public sealed class StarterTests : PageAccessCheckerTests
        {
            protected override IPageAccessChecker CreateChecker() => new
StarterPageAccessChecker();
        }
```

Running one test class against two implementations is the Liskov substitution principle used as a teaching device: anything that satisfies `IPageAccessChecker` must pass the same tests.

Do the kata now, before reading the model answer.

`katas/PageAccessKata/Solution/SolutionPageAccessChecker.cs`

```
        namespace PageAccessKata.Solution;

        /// <summary>
        /// A model answer, written the way you would write it in the test: one
        /// public method, small private helpers, and the decisions stated up front.
        /// </summary>
        /// <remarks>
        /// Decisions (each is a choice the brief left open; a reviewer wants to see
        /// that you noticed it and chose on purpose):
        /// <list type="bullet">
        ///   <item>Missing parent: the page is inaccessible. Failing closed is the safe default
for access control.</item>
        ///   <item>Duplicate paths: the first definition wins and later ones are ignored.
Throwing would also be defensible; ignoring keeps one bad row from hiding every page.</item>
        ///   <item>Null role lists are treated as empty. Role names are trimmed and compared
ignoring case.</item>
        ///   <item>Parent cycles: every page on or below the cycle is inaccessible, even to admin
(it can never be reached by navigation).</item>
        ///   <item>Admin bypasses role checks but not structure: an orphan is still inaccessible
to admin.</item>
        ///   <item>Duplicate user names: the last one wins in the result dictionary. Names are
assumed unique.</item>
        /// </list>
        /// Complexity: O(users × pages × depth) in the worst case; each page's
        /// ancestry is resolved once per user thanks to memoisation.
        /// </remarks>
        public sealed class SolutionPageAccessChecker : IPageAccessChecker
        {
            private const string AdminRole = "admin";

            public IReadOnlyDictionary<string, IReadOnlyList<string>> AccessiblePagesByUser(
                IEnumerable<KataUser> users,
                IEnumerable<KataPage> pages)
```

```
            {
                ArgumentNullException.ThrowIfNull(users);
                ArgumentNullException.ThrowIfNull(pages);

                var pagesByPath = IndexPages(pages);
                var result = new Dictionary<string, IReadOnlyList<string>>();

                foreach (var user in users)
                {
                    var roles = new HashSet<string>(
                        (user.Roles ?? []).Where(r => !string.IsNullOrWhiteSpace(r)).Select(r =>
r.Trim()),
                        StringComparer.OrdinalIgnoreCase);

                    var memo = new Dictionary<string, bool>();
                    result[user.Name] = pagesByPath.Values
                        .Where(page => CanOpen(page, roles, pagesByPath, memo, visiting: []))
                        .Select(page => page.Path)
                        .Order(StringComparer.OrdinalIgnoreCase)
                        .ToList();
                }

                return result;
            }

            private static Dictionary<string, KataPage> IndexPages(IEnumerable<KataPage> pages)
            {
                var index = new Dictionary<string, KataPage>(StringComparer.OrdinalIgnoreCase);
                foreach (var page in pages)
                {
                    index.TryAdd(Normalise(page.Path), page); // first definition wins
                }
                return index;
            }

            private static bool CanOpen(
                KataPage page,
                HashSet<string> roles,
                Dictionary<string, KataPage> pagesByPath,
                Dictionary<string, bool> memo,
                HashSet<string> visiting)
            {
                var key = Normalise(page.Path);
                if (memo.TryGetValue(key, out var known))
                    return known;

                // Seen on the way up: we are walking in a circle.
                if (!visiting.Add(key))
                    return false;

                var allowed = ParentAllows(page, roles, pagesByPath, memo, visiting) &&
RolesAllow(page, roles);

                visiting.Remove(key);
                memo[key] = allowed;
                return allowed;
            }

            private static bool ParentAllows(
                KataPage page,
                HashSet<string> roles,
                Dictionary<string, KataPage> pagesByPath,
                Dictionary<string, bool> memo,
                HashSet<string> visiting)
            {
                if (string.IsNullOrWhiteSpace(page.ParentPath))
                    return true;

                return pagesByPath.TryGetValue(Normalise(page.ParentPath), out var parent)
                    && CanOpen(parent, roles, pagesByPath, memo, visiting);
            }
```

```
            private static bool RolesAllow(KataPage page, HashSet<string> roles)
            {
                var required = (page.RequiredRoles ?? []).Where(r =>
!string.IsNullOrWhiteSpace(r)).ToList();
                return required.Count == 0
                    || roles.Contains(AdminRole)
                    || required.Any(r => roles.Contains(r.Trim()));
            }

            /// <summary>"/Reports/" -> "/reports". The root "/" stays "/".</summary>
            private static string Normalise(string path)
            {
                var trimmed = path.Trim().ToLowerInvariant();
                return trimmed.Length > 1 ? trimmed.TrimEnd('/') : trimmed;
            }
        }
```

What to compare with your attempt:

- Is there a “Decisions” comment, and does it cover every edge case in the brief?
- Is the input indexed once (`Dictionary`) rather than searched in a loop?
- Are the rules in small named functions (`ParentAllows`, `RolesAllow`)?
- Is the cycle handled, and how? (`visiting` set.)
- Is anything hard-coded that should be a named constant? (`AdminRole`.)
- Did you add tests for decisions the provided tests did not cover?

25.3 Kata 2: summarise an access log

katas/AccessLogKata/README.md

```
        # Kata 2 — Summarise an access log (25 minutes)

        Warm-up practice for the fundamentals the prep guide lists: iterating,
        dictionaries, converting strings to numbers and dates, and handling input
        that is not what you expected.

        ## The brief

        Each line of the log looks like this:

        ```
        2026-10-24T08:00:00Z,ada,/reports/finance,ALLOW,142
        ```

        timestamp (ISO 8601, must include a zone), user, path, decision (`ALLOW` or
        `DENY`, any case), and the response time in milliseconds (a non-negative
        integer).

        Write `Summarise(IEnumerable<string> lines)` returning:

        - per user: allowed count, denied count, and **average** response time (ms, as a
`double`);
        - the path with the most denials (ties: alphabetical first; none: `null`);
        - the 1-based line numbers of every malformed line, which are otherwise ignored.

        Blank lines and lines starting with `#` are comments, not errors.
        User names are case-insensitive (`Ada` and `ada` are one user, reported in
        lower case).

        ```powershell
        dotnet test katas/Katas.slnx --filter "Category=Starter&FullyQualifiedName~AccessLog"
        ```
```

katas/AccessLogKata/Model.cs

```
namespace AccessLogKata;

public sealed record UserActivity(int Allowed, int Denied, double AverageResponseMs);

public sealed record LogSummary(
    IReadOnlyDictionary<string, UserActivity> ByUser,
    string? MostDeniedPath,
    IReadOnlyList<int> MalformedLines);

public interface IAccessLogSummariser
{
    LogSummary Summarise(IEnumerable<string> lines);
}
```

katas/AccessLogKata.Tests/AccessLogSummariserTests.cs

```
using AccessLogKata.Solution;
using AccessLogKata.Starter;

namespace AccessLogKata.Tests;

public abstract class AccessLogSummariserTests
{
    protected abstract IAccessLogSummariser Create();

    [Fact]
    public void Counts_and_averages_per_user()
    {
        var summary = Create().Summarise(
        [
            "2026-10-24T08:00:00Z,ada,/reports,ALLOW,100",
            "2026-10-24T08:01:00Z,Ada,/admin,DENY,50",
            "2026-10-24T08:02:00+01:00,bola,/reports,allow,30",
        ]);

        Assert.Equal(new UserActivity(1, 1, 75), summary.ByUser["ada"]);
        Assert.Equal(new UserActivity(1, 0, 30), summary.ByUser["bola"]);
    }

    [Fact]
    public void Most_denied_path_breaks_ties_alphabetically()
    {
        var summary = Create().Summarise(
        [
            "2026-10-24T08:00:00Z,a,/z,DENY,1",
            "2026-10-24T08:00:00Z,a,/b,DENY,1",
            "2026-10-24T08:00:00Z,a,/z,ALLOW,1",
        ]);

        Assert.Equal("/b", summary.MostDeniedPath);
    }

    [Fact]
    public void No_denials_means_no_most_denied_path()
    {
        Assert.Null(Create().Summarise(["2026-10-
24T08:00:00Z,a,/z,ALLOW,1"]).MostDeniedPath);
    }

    [Fact]
    public void Malformed_lines_are_reported_by_number_and_otherwise_ignored()
    {
        var summary = Create().Summarise(
        [
            "# header comment",
```

```
                    "2026-10-24T08:00:00Z,ada,/reports,ALLOW,100",
                    "2026-10-24T08:00:00,ada,/reports,ALLOW,100",   // 3: no zone
                    "not,a,log,line",                                // 4: four fields
                    "2026-10-24T08:00:00Z,ada,/reports,MAYBE,100",  // 5: bad decision
                    "2026-10-24T08:00:00Z,ada,/reports,ALLOW,-5",   // 6: negative
                    "2026-10-24T08:00:00Z,ada,/reports,ALLOW,1,000",// 7: six fields
                    "",
                    "2026-10-24T08:00:00Z,,/reports,ALLOW,1",       // 9: no user
                    "2026-10-24T08:00:00Z,ada,reports,ALLOW,1",     // 10: not a path
                ]);

                Assert.Equal([3, 4, 5, 6, 7, 9, 10], summary.MalformedLines);
                Assert.Equal(new UserActivity(1, 0, 100), Assert.Single(summary.ByUser).Value);
            }

            [Fact]
            public void An_empty_log_is_an_empty_summary()
            {
                var summary = Create().Summarise([]);

                Assert.Empty(summary.ByUser);
                Assert.Empty(summary.MalformedLines);
                Assert.Null(summary.MostDeniedPath);
            }
        }

        public sealed class SolutionTests : AccessLogSummariserTests
        {
            protected override IAccessLogSummariser Create() => new SolutionAccessLogSummariser();
        }

        [Trait("Category", "Starter")]
        public sealed class StarterTests : AccessLogSummariserTests
        {
            protected override IAccessLogSummariser Create() => new StarterAccessLogSummariser();
        }
```

katas/AccessLogKata/Solution/SolutionAccessLogSummariser.cs

```
        using System.Globalization;

        namespace AccessLogKata.Solution;

        /// <summary>
        /// Model answer. Parsing and aggregating are separate steps: a line either
        /// becomes a well-typed <see cref="Entry"/> or a line number in the error
        /// list, and the aggregation never sees a string.
        /// </summary>
        public sealed class SolutionAccessLogSummariser : IAccessLogSummariser
        {
            private sealed record Entry(DateTimeOffset At, string User, string Path, bool Allowed,
int ResponseMs);

            private sealed class Totals
            {
                public int Allowed;
                public int Denied;
                public long ResponseMsSum;
            }

            public LogSummary Summarise(IEnumerable<string> lines)
            {
                ArgumentNullException.ThrowIfNull(lines);

                var byUser = new Dictionary<string, Totals>();
                var deniesByPath = new Dictionary<string, int>(StringComparer.Ordinal);
                var malformed = new List<int>();

                var lineNumber = 0;
```

```
                foreach (var line in lines)
                {
                    lineNumber++;
                    if (string.IsNullOrWhiteSpace(line) || line.TrimStart().StartsWith('#'))
                        continue;

                    if (TryParse(line) is not { } entry)
                    {
                        malformed.Add(lineNumber);
                        continue;
                    }

                    if (!byUser.TryGetValue(entry.User, out var totals))
                        byUser[entry.User] = totals = new Totals();

                    totals.ResponseMsSum += entry.ResponseMs; // long: a busy day of ints can
overflow int
                    if (entry.Allowed)
                    {
                        totals.Allowed++;
                    }
                    else
                    {
                        totals.Denied++;
                        deniesByPath[entry.Path] = deniesByPath.GetValueOrDefault(entry.Path) + 1;
                    }
                }

                var mostDenied = deniesByPath
                    .OrderByDescending(kv => kv.Value)
                    .ThenBy(kv => kv.Key, StringComparer.Ordinal)
                    .Select(kv => kv.Key)
                    .FirstOrDefault();

                var activity = byUser.ToDictionary(
                    kv => kv.Key,
                    kv => new UserActivity(kv.Value.Allowed, kv.Value.Denied,
                        (double)kv.Value.ResponseMsSum / (kv.Value.Allowed + kv.Value.Denied)));

                return new LogSummary(activity, mostDenied, malformed);
            }

            private static Entry? TryParse(string line)
            {
                var parts = line.Split(',');
                if (parts.Length != 5)
                    return null;

                var (timestamp, user, path, decision, response) = (parts[0].Trim(),
parts[1].Trim(), parts[2].Trim(), parts[3].Trim(), parts[4].Trim());

                // A timestamp without a zone is ambiguous; refuse it rather than guess.
                if (!timestamp.EndsWith('Z') && !HasOffset(timestamp))
                    return null;

                if (!DateTimeOffset.TryParse(timestamp, CultureInfo.InvariantCulture,
DateTimeStyles.None, out var at))
                    return null;

                if (user.Length == 0 || !path.StartsWith('/'))
                    return null;

                bool allowed;
                if (decision.Equals("ALLOW", StringComparison.OrdinalIgnoreCase)) allowed = true;
                else if (decision.Equals("DENY", StringComparison.OrdinalIgnoreCase)) allowed =
false;
                else return null;

                // NumberStyles.None: no sign, no spaces, no thousands separators: "-5" and
"1,000" are malformed.
                if (!int.TryParse(response, NumberStyles.None, CultureInfo.InvariantCulture, out
var ms))
```

```
                    return null;

                return new Entry(at, user.ToLowerInvariant(), path, allowed, ms);
            }

            private static bool HasOffset(string timestamp) =>
                timestamp.Length > 6 && (timestamp[^6] == '+' || timestamp[^6] == '-') &&
timestamp[^3] == ':';
        }
```

The patterns to take away: **parse into a typed record or reject**, so the aggregation never sees a string; `NumberStyles.None` and `CultureInfo.InvariantCulture` so that `-5`, `1,000` and a French machine's decimal comma do not surprise you; `long` for sums that can outgrow `int`; and `GetValueOrDefault(key) + 1` as the idiomatic dictionary counter.

25.4 The Angular drill

```
git switch drill/angular-security-start
cd client/sentinel-web
npx ng test --watch=false     # 12 failures: your target
```

`docs/labs/DRILL.md`

```
# Drill — the account security page, against the clock (25 minutes)

The container (`account-security-page.ts`), the API service, the time
utilities and every spec are intact. The three presentational components
have been emptied down to their inputs and outputs.

1. Start a 25-minute timer.
2. `cd client/sentinel-web && npx ng test --watch=false` — note what fails.
3. Rebuild `security-banner.ts`, `finding-list.ts`, `sign-in-table.ts`.
4. Run the app and look at Tunde (High), Dana (Medium) and Emeka (None).
5. Compare with `git diff drill/angular-security-start main -- client/`.

Chapter 25 lists what a reviewer looks for. Score yourself honestly.
```

25.5 The rubric

Score each attempt honestly, 0 to 2 per line (0 absent, 1 partly, 2 clearly). The lines are the prep guide's criteria, made concrete.

Coding (katas, drill)

Criterion	2 points looks like
Core requirements	Every rule in the brief is implemented and visibly traceable to the code
Edge cases	Each listed edge case has a decision, a comment and a test
Names	A reader could guess each function's body from its name
Structure	One entry point; helpers of 5–20 lines; no magic strings or numbers
Tests	At least three tests beyond those provided, each for a decision
Comments	A "Decisions"/trade-offs comment; no comments that restate the code
(Angular) Passing data	Inputs down, outputs up, one container with the side effects
(Angular) Dates	Parsed strictly, formatted in the user's zone, relative time testable

Debugging (labs)

Criterion	2 points looks like
Gathering	Reproduced at the lowest level; evidence before conclusions
Research	Precise searches, primary sources, every source cited
Documenting	A log a colleague could continue from
To engineers	Cause, fix, why missed, follow-ups, in under a page
To non-engineers	Answers the reader's worry first; one clear action; no jargon

Fourteen or more out of sixteen on the coding rubric, and nine or more out of ten on the debugging one, and you are ready.

Chapter 26 · Test Day

26.1 The day before

- [ ] Open the Qualified practice environment for ten minutes (§1.3). Note the Angular version.
- [ ] Do Kata 1 once more, fast. Do not learn anything new.
- [ ] Re-read §21.5 (the debugging timetable) and the rubric in §25.5.
- [ ] Decide which scenario you will do first. Many people prefer the one they fear most, while fresh.
- [ ] Set the 47-hour alarm the prep guide recommends when you start the 48-hour window.

26.2 Before each scenario

- [ ] Note whether **AI is allowed**. Two scenarios: no, and it disqualifies you. One: yes, with citation.
- [ ] Start your own timer alongside the platform's, with your timetable's checkpoints.
- [ ] Open a scratch file for notes; you will paste from it into your submission.

26.3 Inside the authorisation scenario (40 min)

1. Requirements as a numbered comment.
2. Edge cases as a table in the comment, each with a decision.
3. Index the input; one function per rule; a cycle guard if there is a hierarchy.
4. Run the provided tests early and often.
5. A test per edge-case decision.
6. Names, constants, the "Decisions" comment. If something advanced is beyond the time, use what you know and **say so in a comment**.

26.4 Inside the debugging scenario (35 min)

1. Copy the report; facts, assumptions, unknowns; questions for the reporter.
2. Reproduce at the lowest level.
3. Hypotheses, cheapest test first. Cite every source.
4. At minute 22: found it → test, fix; not found → write up what you ruled out.
5. Ten minutes for two write-ups. Protect this time.

26.5 Inside the Angular scenario (25 min)

1. Sketch the component tree in a comment: container, children, inputs, outputs.
2. Get correct data on screen first.
3. Dates: strict parsing, the user's zone, `Intl`, relative time with a passed-in "now".
4. Edge cases: empty, loading, error, unknown values, singular/plural.
5. If you use AI, cite it in a comment and say what you changed.

26.6 Citing, in practice

```
// Two-pointer sliding window adapted from:
// https://learn.microsoft.com/... (or the Stack Overflow answer's permalink)
```

```
        // Suggested by an AI assistant (2026-10-26) for Intl.RelativeTimeFormat units;
        // changed to take `now` as a parameter for testability.
```

26.7 After

Woven promises detailed feedback whatever the outcome. Read it against the rubric in §25.5, find the two lowest lines, and practise those. Whatever the result, you will have built, tested and debugged a real application, and that stays with you.

Appendix A · C# Fundamentals for the Test

The prep guide lists fundamentals to review. Here each one is, in C#, with an example from Sentinel's domain.

A.1 Iteration

```
        // for: when you need the index
        for (var i = 0; i < attempts.Count; i++) { var attempt = attempts[i]; }

        // foreach: the default
        foreach (var attempt in attempts) { if (!attempt.Succeeded) failures++; }

        // LINQ: when the loop computes a value
        var failures = attempts.Count(a => !a.Succeeded);
        var countries = attempts.Where(a => a.Succeeded).Select(a =>
a.Location.CountryCode).Distinct().ToList();
        var newestFirst = attempts.OrderByDescending(a => a.OccurredAtUtc).ToList();
        var byUser = attempts.GroupBy(a => a.UserId).ToDictionary(g => g.Key, g => g.Count());

        // while: when the end condition is not a count (see MaxInAnyWindow)
        while (burst[end].OccurredAtUtc - burst[start].OccurredAtUtc > window) start++;
```

LINQ is lazy: a query runs when it is enumerated (`ToList`, `foreach`, `Count`). Enumerating it twice runs it twice. Call `ToList()` once when you will use the result more than once.

A.2 Inspecting values

```
        Console.WriteLine($"[debug] user={user.Name} roles={string.Join(",", user.Roles ?? [])}");
```

In xUnit, output from tests goes through `ITestOutputHelper` (constructor-injected) rather than `Console`. Remove debug output before submitting.

A.3 Classes and objects

```
        public sealed class Role                    // a class with behaviour and identity
        {
            private readonly List<string> _permissions = [];
            public Role(string name) => Name = name;  // constructor
            public string Name { get; }               // get-only property
            public IReadOnlyList<string> Permissions => _permissions;
            public void Grant(string p) => _permissions.Add(p);   // a method
        }

        var role = new Role("Auditor");               // initialise
        role.Grant("reports.view");                   // call a method
        Console.WriteLine(role.Name);                 // get a property

        public sealed record KataPage(string Path, string? ParentPath, IReadOnlyList<string>?
RequiredRoles);
        var page = new KataPage("/reports", "/", ["analyst"]);   // records: value equality,
concise
        var moved = page with { ParentPath = "/home" };           // non-destructive copy
```

A.4 Collections

Need	Type	Key operations
Fixed sequence	`T[]`	index, `Length`
Growable list	`List<T>`	`Add`, `Remove`, `Count`, index
Lookup by key	`Dictionary<TKey, TValue>`	`TryGetValue`, `TryAdd`, `GetValueOrDefault`, `ContainsKey`
Membership	`HashSet<T>`	`Add` (returns false if present), `Contains`
Read-only view	`IReadOnlyList<T>`, `IReadOnlyDictionary<,>`	expose, don't mutate

Case-insensitive keys: `new Dictionary<string, Page>(StringComparer.OrdinalIgnoreCase)`. Counting idiom: `counts[key] = counts.GetValueOrDefault(key) + 1;`

A.5 Converting between types

```
        int.TryParse("142", NumberStyles.None, CultureInfo.InvariantCulture, out var ms);   //
never int.Parse on input
        DateTimeOffset.TryParse("2026-10-24T08:00:00Z", CultureInfo.InvariantCulture,
DateTimeStyles.None, out var at);
        Enum.TryParse<PermissionEffect>("deny", ignoreCase: true, out var effect);           //
also accepts "1"! check Enum.IsDefined
        var asText = ms.ToString(CultureInfo.InvariantCulture);
        var utc = at.UtcDateTime;                                                            //
DateTimeOffset -> DateTime (Kind=Utc)
```

Always pass `CultureInfo.InvariantCulture` when parsing machine data. A machine set to French parses `1,5` as one and a half.

A.6 Null handling

```
        var roles = user.Roles ?? [];                       // default for null
        var city = attempt?.Location?.City ?? "Unknown";    // null-conditional chain
        ArgumentNullException.ThrowIfNull(pages);           // programming errors fail fast
        if (page.ParentId is { } parentId) { ... }          // pattern: non-null and bind
```

Appendix B · TypeScript and JavaScript Fundamentals for the Test

B.1 Iteration

```
        for (let i = 0; i < signIns.length; i++) { const s = signIns[i]; }
        for (const s of signIns) { if (!s.succeeded) failures++; }     // for...of: values
        for (const key in record) { }                                   // for...in: keys of an
object (rarely what you want)

        const failed = signIns.filter((s) => !s.succeeded);
        const ids = signIns.map((s) => s.id);
        const firstFlagged = signIns.find((s) => s.isFlagged);          // undefined if none
        const count = signIns.reduce((n, s) => n + (s.isFlagged ? 1 : 0), 0);
```

B.2 Inspecting values

```
        console.log('[security] status', status);          // objects are expandable in dev tools
        console.table(status.recentSignIns);               // arrays of objects as a table
```

B.3 Classes and objects

```
        class FindingCounter {
          private readonly counts = new Map<string, number>();
          constructor(private readonly label: string) {}      // parameter property
          add(kind: string): void { this.counts.set(kind, (this.counts.get(kind) ?? 0) + 1); }
          get total(): number { return [...this.counts.values()].reduce((a, b) => a + b, 0); }
        }

        const counter = new FindingCounter('today');
        counter.add('NewDevice');
        console.log(counter.total);

        const finding = { key: 'x', severity: 'High' };       // object literal
        finding.severity = 'Low';                              // set a property
        const copy = { ...finding, key: 'y' };                 // copy with a change
```

B.4 Arrays, maps, objects as dictionaries

Need	Use
Ordered list	`T[]` (`push`, `length`, `includes`, `indexOf`)
Lookup by key	`Map<K, V>` (`get`, `set`, `has`), or `Record<string, V>` for fixed keys
Membership	`Set<T>` (`add`, `has`)

B.5 Converting between types

```
        Number('142');                // 142; Number('') is 0, Number('abc') is NaN
        parseInt('142px', 10);        // 142 (stops at the first non-digit)
```

```
Number.isNaN(value);          // the only reliable NaN check
String(42); `${42}`;          // to string
JSON.parse(text);             // string -> object (throws on bad JSON)
new Date('2026-10-24T08:00:00Z');   // ALWAYS with a Z or an offset
date.toISOString();           // Date -> UTC string with Z
```

B.6 Equality and truthiness

Use `===` and `!==`. `0`, `''`, `null`, `undefined`, `NaN` and `false` are falsy: `count || 10` replaces a real `0`; `count ?? 10` does not.

Appendix C · The Same Code in Older Angular

If the test environment runs an Angular older than 17, translate as follows. Everything in this book works in Angular 22 with either column.

Modern (this book)	Classic (Angular 2–16)
`readonly risk = input.required<RiskLevel>();`	`@Input({ required: true }) risk!: RiskLevel;` (`required` from v16; earlier, just `@Input() risk!: RiskLevel;`)
`readonly count = input(0);`	`@Input() count = 0;`
`readonly review = output<void>();`	`@Output() review = new EventEmitter<void>();`
`this.review.emit()`	`this.review.emit()` (same)
reading an input: `this.risk()`	reading an input: `this.risk`
`computed(() => presentRisk(this.risk()))`	a getter: `get presentation() { return presentRisk(this.risk); }` or `ngOnChanges`
`signal(x)`, `.set()`, `()`	a field, or a `BehaviorSubject` with the `async` pipe
`@if (x) { ... } @else { ... }`	`<ng-container *ngIf="x; else other">...</ng-container><ng-template #other>...</ng-template>`
`@for (f of findings; track f.key) { } @empty { }`	`<div *ngFor="let f of findings; trackBy: trackByKey">` and a separate `*ngIf="!findings.length"`
`@switch (kind) { @case ('a') { } }`	`<ng-container [ngSwitch]="kind"><div *ngSwitchCase="'a'">`
`inject(SecurityApi)`	`constructor(private readonly api: SecurityApi) {}`
standalone `imports: [FindingList]`	declare components in an `NgModule`'s `declarations`
`viewChild<ElementRef>('ref')`	`@ViewChild('ref') ref?: ElementRef;`
`takeUntilDestroyed()`	a `destroy$ = new Subject<void>()`, `takeUntil(this.destroy$)`, and `ngOnDestroy`
functional guard `CanActivateFn`	a class implementing `CanActivate` (functional guards from v14.2)
`withInterceptors([fn])`	a class implementing `HttpInterceptor`, provided with `HTTP_INTERCEPTORS`
`withComponentInputBinding()`	`ActivatedRoute.queryParamMap.subscribe(...)`
Vitest `vi.fn()`	Jasmine `jasmine.createSpy()`

The container from Chapter 17, in classic form, is a useful exercise: write it once and the translation becomes automatic.

Appendix D · Troubleshooting

Every entry below happened while this book was written.

Symptom	Cause	Fix
`NU1903: Package 'System.Security.Cryptography.Pkcs' 9.0.11 has a known high severity vulnerability`	A transitive dependency of the SQL Server provider, with warnings as errors	Pin it centrally with `CentralPackageTransitivePinningEna` (§4.4)
`NU1301: ... No such host is known (api.nuget.org)`	A transient network failure	Retry; check proxy settings
`Invalid object name 'Users'` on start-up	The API was run with `--no-build` before the migration was added, so the old assembly had no migrations	Build, then run
`CS8188: An expression tree may not contain a throw-expression`	`throw` inside a value converter lambda	Move the check into a static method (§
`CS8620` nullability mismatch on `HasConversion` for a nullable value object	The generic overload's nullability	Cast to the non-generic `ValueConvert` (`PageConfiguration`)
`npm error notarget No matching version found` during `ng new`	A package published moments earlier, not yet on the mirror	Run `npm install` again in the workspa
`proxy.conf.json contains parse errors: [1, 1] InvalidSymbol`	A UTF-8 byte-order mark from PowerShell 5's `Set-Content -Encoding utf8`	Save without BOM
A path argument becomes `C:/Program Files/Git/account/security`	Git Bash converts arguments that look like POSIX paths	`export MSYS_NO_PATHCONV=1`
Demo data differs between runs	`string.GetHashCode()` is randomised per process	Seed `Random` from stable input (§11.4)
A role's new permission is not saved	No `ValueComparer` on the converted list	Add one (`PermissionListComparer`)
Times on screen are off by the user's UTC offset	A `DateTime` with `Kind = Unspecified` serialised without `Z`	Chapter 22
A heavy user's security page is green during an attack	`OrderBy` + `Take` kept the oldest rows	Chapter 23
Tests in Api.Tests still hit SQL Server	Only `DbContextOptions<T>` removed; EF Core 9+ also registers `IDbContextOptionsConfiguration<T>`	Remove both (§13.3)

Resetting everything

```
dotnet ef database drop --project src/Sentinel.Infrastructure -f
git clean -xdf -e node_modules     # removes bin/obj and build output
dotnet build; dotnet test
```